

SKRIPSI

PEMBANGUNAN PERANGKAT LUNAK PERMAINAN
COLORED QUEENS DAN PERBANDINGAN SOLVER
BACKTRACKING DENGAN *PARTICLE SWARM*
OPTIMIZATION



Arlo Dante Hananvyasa

NPM: 6182201010

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS SAINS
UNIVERSITAS KATOLIK PARAHYANGAN
2026

UNDERGRADUATE THESIS

**DEVELOPMENT OF A COLORED QUEENS GAME
APPLICATION AND COMPARATIVE ANALYSIS OF
BACKTRACKING AND PARTICLE SWARM OPTIMIZATION
SOLVERS**



Arlo Dante Hananvyasa

NPM: 6182201010

**DEPARTMENT OF INFORMATICS
FACULTY OF SCIENCES
PARAHYANGAN CATHOLIC UNIVERSITY
2026**

ABSTRAK

Colored Queens adalah *puzzle* logika berbasis papan yang dipopulerkan oleh platform LinkedIn, di mana pemain harus menempatkan tepat satu menteri pada setiap baris, kolom, dan sektor warna pada papan berukuran $n \times n$, tanpa ada dua menteri yang saling bertetangga dalam delapan arah. Permasalahan ini merupakan variasi dari N-Queens klasik dengan kendala tambahan berupa sektor warna ireguler dan aturan *adjacency* lokal, sehingga tidak dapat diselesaikan dengan teknik konstruktif standar dan secara alami dapat dimodelkan sebagai *Constraint Satisfaction Problem* (CSP).

Tugas akhir ini mengimplementasikan dan membandingkan dua pendekatan algoritmik untuk menyelesaikan *Colored Queens*: *backtracking* yang diperkuat dengan *forward checking* dan *Arc Consistency 3* (AC-3) sebagai pendekatan deterministik dan lengkap, serta *Particle Swarm Optimization* (PSO) diskrit sebagai pendekatan *metaheuristic*. Selain kedua solver tersebut, dikembangkan pula sebuah aplikasi permainan berbasis *desktop* menggunakan Java Swing yang mengintegrasikan solver *backtracking* sebagai mesin untuk fitur *hint* dan *auto-solve*. Pengujian dilakukan terhadap 1.502 *puzzle* yang mencakup ukuran papan 7×7 hingga 30×30 , dengan dataset yang diperoleh dari *playqueensgame.com* melalui *web scraping*.

Solver *backtracking* dengan FC+AC-3 berhasil menyelesaikan seluruh 1.500 *puzzle* ukuran standar (7×7 hingga 12×12) dengan tingkat keberhasilan 100% dan waktu eksekusi rata-rata kurang dari 2 ms. Pada papan berukuran besar (20×20 dan 30×30), algoritma ini sebagai algoritma yang bersifat lengkap pada prinsipnya tetap akan menemukan solusi, namun waktu komputasi yang diperlukan tidak praktis, mengindikasikan bahwa *overhead* propagasi AC-3 pada setiap simpul pencarian mengkompensasi manfaat pemangkasan domain pada skala tersebut. Solver PSO diskrit menunjukkan penurunan tingkat keberhasilan yang tajam seiring bertambahnya ukuran papan, dari 99,20% pada papan 7×7 hingga hanya 5,20% pada papan 12×12 dengan konfigurasi parameter terbaik, dan gagal sepenuhnya pada ukuran besar. Analisis sensitivitas parameter menunjukkan bahwa jumlah partikel merupakan faktor yang paling berpengaruh terhadap kualitas solusi, meskipun peningkatannya mengalami *diminishing returns* pada ukuran papan yang lebih besar. Secara keseluruhan, hasil ini mengonfirmasi bahwa untuk CSP berkendala ketat seperti *Colored Queens*, pendekatan berbasis propagasi kendala terbukti jauh lebih efektif dibandingkan pendekatan *metaheuristic*.

Kata-kata kunci: *Colored Queens*, *constraint satisfaction problem*, *backtracking*, *forward checking*, AC-3, *particle swarm optimization*, *puzzle* logika.

ABSTRACT

Colored Queens is a logic-based board puzzle popularized by the LinkedIn platform, in which the player must place exactly one queen in each row, column, and colored region of an $n \times n$ board, with no two queens adjacent in any of eight directions. The problem is a variant of the classical N-Queens problem featuring irregular colored regions and a local adjacency constraint, making it unsolvable by standard constructive techniques and naturally suited for modeling as a Constraint Satisfaction Problem (CSP).

This thesis implements and compares two algorithmic approaches to solving *Colored Queens*: backtracking augmented with forward checking and Arc Consistency 3 (AC-3) as a deterministic and complete approach, and discrete Particle Swarm Optimization (PSO) as a metaheuristic approach. In addition to the two solvers, a desktop application was developed using Java Swing, integrating the backtracking solver as the engine for hint and auto-solve features. Testing was conducted on 1,502 puzzles spanning board sizes from 7×7 to 30×30 , with the dataset obtained from *playqueensgame.com* via web scraping.

The backtracking solver with FC+AC-3 successfully solved all 1,500 standard-size puzzles (7×7 to 12×12) with a 100% success rate and an average execution time under 2 ms. On large boards (20×20 and 30×30), the algorithm is theoretically guaranteed to find a solution by virtue of being a complete algorithm, but the required computation time is impractical, indicating that the AC-3 propagation overhead at each search node outweighs the benefit of domain pruning at that scale. The discrete PSO solver exhibited a sharp decline in success rate as board size increased, from 99.20% on 7×7 boards to only 5.20% on 12×12 under the best parameter configuration, and failed entirely on larger boards. Parameter sensitivity analysis revealed that swarm size was the most influential factor on solution quality, though its effect exhibited diminishing returns on larger board sizes. Overall, these results confirm that for tightly-constrained CSPs such as *Colored Queens*, constraint propagation-based approaches are far more effective than metaheuristic ones.

Keywords: Colored Queens, constraint satisfaction problem, backtracking, forward checking, AC-3, particle swarm optimization, logic puzzle.

DAFTAR ISI

DAFTAR ISI	ix
DAFTAR GAMBAR	xi
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Tujuan	3
1.4 Batasan Masalah	3
1.5 Metodologi	4
1.6 Sistematika Pembahasan	4
2 DASAR TEORI	7
2.1 Permasalahan N-Queens	7
2.1.1 Definisi dan Representasi	7
2.1.2 Aturan Konflik Queen	8
2.1.3 Kompleksitas dan Relevansi	9
2.2 Colored Queens Problem	11
2.2.1 Definisi dan Aturan Permainan	11
2.2.2 Kompleksitas dan Struktur Masalah	12
2.2.3 Posisi dan Relevansi Penelitian	13
2.3 Constraint Satisfaction Problem	14
2.3.1 Definisi dan Komponen CSP	14
2.3.2 Teknik Representasi	15
2.3.3 Pemodelan Colored Queens sebagai CSP	15
2.4 Teknik Penyelesaian CSP	16
2.4.1 Algoritma Sistematis	16
2.4.2 Metaheuristic	22
3 ANALISIS	31
3.1 Analisis Masalah	31
3.2 Pemodelan Permasalahan <i>Colored Queens</i> sebagai CSP	33
3.2.1 Representasi Papan	33
3.2.2 Definisi Variabel	33
3.2.3 Definisi Domain	34
3.2.4 Definisi <i>Constraint</i>	34
3.2.5 Representasi Solusi	35
3.3 Analisis Algoritma <i>Backtracking</i> dengan <i>Forward Checking</i> dan AC-3	35
3.3.1 Alur Utama Algoritma <i>Backtracking</i>	36
3.3.2 <i>Forward Checking</i>	37
3.3.3 Fungsi <i>Revise</i>	38
3.3.4 Propagasi AC-3	40
3.3.5 Studi Kasus: Penyelesaian Papan 7×7	41

3.4	Analisis Algoritma <i>Particle Swarm Optimization</i>	46
3.4.1	Fungsi <i>Fitness</i>	47
3.4.2	Pembaruan Kecepatan dan Posisi Partikel	48
3.4.3	Alur Utama Algoritma PSO	50
3.4.4	Studi Kasus: Pencarian Solusi dengan PSO pada Papan 7×7	52
4	PERANCANGAN	59
4.1	Perancangan Perangkat Lunak	59
4.1.1	Desain Kelas	59
4.1.2	Antarmuka Pengguna	74
4.1.3	Teknologi yang Digunakan	80
5	IMPLEMENTASI DAN PENGUJIAN	83
5.1	Implementasi	83
5.1.1	Implementasi Solver <i>Backtracking</i> dengan <i>Forward Checking</i> dan AC-3	83
5.1.2	Implementasi Solver PSO Diskrit	85
5.1.3	Implementasi Antarmuka Pengguna	87
5.1.4	Implementasi Kerangka Pengujian	93
5.2	Skenario dan Metode Pengujian	94
5.2.1	Dataset Puzzle	94
5.2.2	Parameter Pengujian	94
5.2.3	Skenario Pengujian	94
5.2.4	Lingkungan Pengujian	96
5.3	Hasil Pengujian	96
5.3.1	Hasil Pengujian <i>Backtracking</i>	96
5.3.2	Hasil Pengujian Variasi Parameter PSO	97
5.3.3	Hasil Perbandingan <i>Backtracking</i> dan PSO	99
5.4	Analisis dan Pembahasan	100
5.4.1	Analisis Skalabilitas <i>Backtracking</i>	101
5.4.2	Analisis Pengaruh Parameter PSO	101
5.4.3	Analisis Perbandingan Kedua Pendekatan	103
6	KESIMPULAN DAN SARAN	107
6.1	Kesimpulan	107
6.2	Saran	108
	DAFTAR REFERENSI	111
A	KODE PROGRAM	113
A.1	Kelas Inti	113
A.2	Solver	119
A.2.1	Solver <i>Backtracking</i> dengan <i>Forward Checking</i> dan AC-3	119
A.2.2	Solver PSO Diskrit	123
A.3	Antarmuka Pengguna	129

DAFTAR GAMBAR

1.1	Contoh papan permainan <i>Colored Queens</i> berukuran 7×7 : (a) konfigurasi awal dengan tujuh sektor warna berbentuk ireguler; (b) konfigurasi solusi valid yang memenuhi seluruh kendala permainan.	2
2.1	Contoh aturan penyerangan bidak menteri pada permainan catur. Silang merah menandakan kotak yang terserang oleh bidak menteri.	7
2.2	Contoh solusi valid permasalahan 8-Queens. Panah hijau menandakan jangkauan serangan salah satu menteri; tidak ada menteri lain yang berada pada baris, kolom, ataupun diagonal yang sama. ¹	9
2.3	Contoh papan permainan <i>Colored Queens</i> berukuran 8×8 dengan 8 sektor warna. Setiap sektor membentuk wilayah terhubung, dan solusi menempatkan tepat satu menteri per baris, kolom, dan sektor warna tanpa ada dua menteri yang bertetangga. ²	11
2.4	Dampak kendala <i>adjacency</i> pada papan berukuran kecil. Penempatan satu menteri pada papan 7×7 mengeliminasi delapan sel tetangga (ditandai dengan tanda X merah), yang setara dengan sekitar 16% dari total 49 sel papan.	13
2.5	Contoh propagasi AC-3 pada <i>constraint graph</i> dengan lima variabel. Sebelum propagasi (a), domain variabel masih lebar. Setelah AC-3 diterapkan (b), nilai-nilai yang tidak memiliki <i>support</i> dieliminasi, menghasilkan domain yang jauh lebih kecil. ³ . .	20
2.6	Ilustrasi perpindahan satu partikel berdasarkan <i>velocity</i> baru yang dipengaruhi oleh <i>personal best</i> dan <i>global best</i> . ⁴	25
2.7	Diagram alir algoritma PSO.	26
3.1	Alur kerja penelitian secara keseluruhan.	32
3.2	Diagram alur prosedur rekursif pencarian <i>backtracking</i>	36
3.3	Diagram alur prosedur <i>forward checking</i>	38
3.4	Diagram alur fungsi <i>revise</i>	39
3.5	Diagram alur prosedur propagasi AC-3.	40
3.6	Penempatan menteri pertama pada salah satu sel dari domain warna biru.	42
3.7	Hasil pemangkasan oleh <i>forward checking</i> setelah penempatan menteri pertama. .	42
3.8	Konsep dasar busur (<i>arc</i>) dalam AC-3.	42
3.9	Graf kendala lengkap dan antrean awal busur AC-3.	43
3.10	Pemrosesan busur Hijau→Kuning: sel Hijau(1,2) memiliki <i>support</i> pada Kuning(6,3) dan tetap valid.	43
3.11	Pemrosesan busur Hijau→Kuning: sel Hijau(5,2) dan Hijau(5,3) tidak memiliki <i>support</i> dan dipangkas.	44
3.12	Kondisi papan setelah propagasi AC-3 selesai.	44
3.13	Penempatan menteri kedua pada salah satu sel dari domain warna oranye.	44
3.14	<i>Domain wipeout</i> : seluruh sel pada domain warna kuning menjadi tidak valid. . . .	45
3.15	<i>Backtrack</i> : pembatalan penempatan menteri oranye yang menyebabkan kegagalan. .	45
3.16	Penempatan alternatif untuk warna oranye setelah <i>backtrack</i>	45
3.17	Solusi akhir papan 7×7 yang memenuhi seluruh kendala <i>Colored Queens</i>	46
3.18	Diagram alur pembaruan kecepatan dan posisi partikel PSO diskrit.	48
3.19	Diagram alur utama algoritma PSO diskrit.	51

3.20	Pasangan <i>adjacency</i> pertama: menteri pada (3,1) dan (4,1) bertetangga vertikal. .	54
3.21	Pasangan <i>adjacency</i> kedua: menteri pada (4,1) dan (4,2) bertetangga horizontal. .	54
3.22	Pasangan <i>adjacency</i> ketiga: menteri pada (3,1) dan (4,2) bertetangga diagonal. . .	55
3.23	Pasangan <i>attacking</i> pertama: menteri pada (4,2) dan (6,2) berada pada kolom yang sama.	55
3.24	Pasangan <i>attacking</i> kedua: menteri pada (4,1) dan (4,2) berada pada baris yang sama. Pasangan ini juga merupakan pelanggaran <i>adjacency</i>	55
3.25	Pasangan <i>attacking</i> ketiga: menteri pada (4,1) dan (4,4) berada pada baris yang sama.	56
3.26	Pasangan <i>attacking</i> keempat: menteri pada (3,1) dan (4,1) berada pada kolom yang sama. Pasangan ini juga merupakan pelanggaran <i>adjacency</i>	56
3.27	Pasangan <i>attacking</i> kelima: menteri pada (4,2) dan (4,4) berada pada baris yang sama.	56
4.1	Diagram kelas sistem yang menunjukkan hubungan antara kelas data, kelas <i>solver</i> , dan kelas antarmuka.	59
4.2	Alur interaksi pengguna pada aplikasi <i>Colored Queens</i>	74
4.3	Rancangan halaman utama (<i>Homepage</i>).	75
4.4	Rancangan halaman pemilihan level (<i>Level Selector</i>).	76
4.5	Rancangan halaman permainan (<i>Game Window</i>).	76
4.6	Tampilan papan setelah solver menerapkan solusi lengkap.	77
4.7	Tampilan peringatan pelanggaran kendala <i>adjacency</i>	77
4.8	Tampilan peringatan pelanggaran kendala kesamaan warna.	78
4.9	Tampilan peringatan pelanggaran kendala baris.	78
4.10	Tampilan peringatan pelanggaran kendala kolom.	79
4.11	Tampilan fitur <i>hint</i> : sorotan posisi yang benar.	79
4.12	Tampilan fitur <i>hint</i> : sorotan posisi salah dan posisi benar.	79
4.13	Rancangan tampilan kemenangan.	80
5.1	Tangkapan layar halaman utama (<i>Homepage</i>).	89
5.2	Tangkapan layar halaman pemilihan level.	90
5.3	Tangkapan layar halaman permainan utama.	90
5.4	Tangkapan layar respons sistem terhadap pelanggaran penempatan dua bidak menteri pada warna yang sama.	90
5.5	Tangkapan layar respons sistem terhadap pelanggaran penempatan dua bidak menteri pada baris yang sama.	91
5.6	Tangkapan layar respons sistem terhadap pelanggaran penempatan dua bidak menteri pada kolom yang sama.	91
5.7	Tangkapan layar respons sistem terhadap pelanggaran penempatan dua bidak menteri yang bersebelahan.	91
5.8	Tangkapan layar bantuan dari sistem ketika belum ada bidak menteri yang ditempatkan pada sektor warna tersebut.	92
5.9	Tangkapan layar bantuan dari sistem ketika ada bidak menteri yang ditempatkan pada sektor warna tersebut	92
5.10	Tangkapan layar respons dari sistem ketika pemain berhasil menyelesaikan <i>puzzle</i>	92

BAB 1

PENDAHULUAN

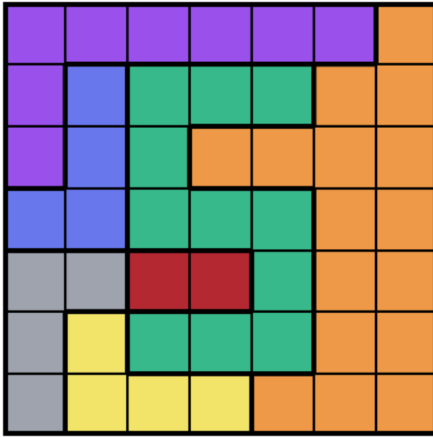
1.1 Latar Belakang

Permasalahan *N-Queens* merupakan salah satu permasalahan klasik dalam ilmu komputer dan matematika diskrit yang telah dipelajari secara ekstensif selama lebih dari satu setengah abad [1]. Dalam bentuk standarnya, permasalahan ini meminta penempatan n buah bidak menteri pada papan berukuran $n \times n$ sedemikian rupa sehingga tidak ada dua menteri yang saling menyerang secara horizontal, vertikal, maupun diagonal. Meskipun terkesan sederhana, permasalahan ini memiliki relevansi yang jauh melampaui konteks permainan catur. Berbagai kajian telah menunjukkan bahwa *N-Queens* dapat dijadikan model untuk permasalahan nyata seperti penjadwalan, alokasi sumber daya, dan desain sirkuit terpadu, sehingga menjadikannya salah satu tolok ukur klasik dalam penelitian algoritma pencarian dan pemodelan berbasis kendala [1].

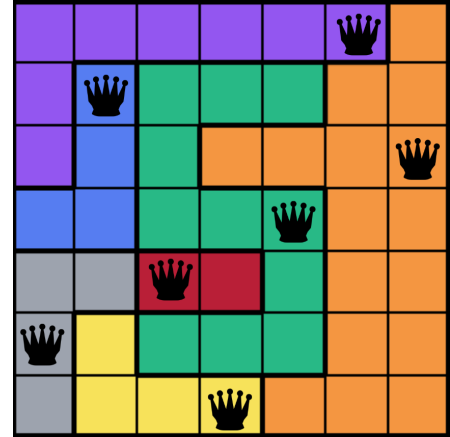
Seiring perkembangan penelitian, bermunculan berbagai variasi dari *N-Queens* yang memperkenalkan kendala tambahan sehingga meningkatkan kompleksitas komputasional secara signifikan. Gent, Jefferson, dan Nightingale [2] membuktikan bahwa varian *N-Queens Completion*, yaitu menyelesaikan konfigurasi parsial yang sudah diberikan sebelumnya, tergolong NP-Complete dan #P-Complete. Temuan ini mengonfirmasi bahwa variasi-variasi *N-Queens* dengan kendala tambahan atau struktur papan yang tidak beraturan memiliki kompleksitas teoritis yang setara dengan permasalahan-permasalahan sulit klasik dalam ilmu komputer, sehingga tidak dapat diselesaikan oleh konstruksi aritmatika sederhana dan memerlukan pendekatan algoritmik berbasis pencarian.

Salah satu variasi yang menarik perhatian luas belakangan ini adalah *Colored Queens*, sebuah puzzle logika berbasis papan yang dipopulerkan melalui permainan *Queens* pada platform LinkedIn [3]. Secara struktural, permainan ini memiliki kedekatan dengan *Star Battle*, puzzle penempatan yang pertama kali dirancang oleh Hans Eendebak untuk *World Puzzle Championship* tahun 2003 [4]. Dalam *Colored Queens*, sebuah papan berukuran $n \times n$ dibagi menjadi n buah sektor berwarna dengan bentuk yang umumnya tidak beraturan. Tujuan permainan adalah menempatkan tepat satu menteri pada setiap sektor, setiap baris, dan setiap kolom, dengan tambahan bahwa tidak ada dua menteri yang boleh bertetangga secara langsung dalam delapan arah, sebagaimana diilustrasikan pada Gambar 1.1. Kendala adjacency yang bersifat lokal ini, dikombinasikan dengan sektor warna yang berbentuk ireguler, menciptakan ruang pencarian yang secara struktural jauh berbeda dari *N-Queens* klasik dan tidak dapat diselesaikan dengan teknik konstruktif standar.

Dari sudut pandang akademis, *Colored Queens* merupakan topik yang relatif baru dan masih sangat kurang dikaji secara formal. Sejauh tinjauan pustaka yang dilakukan, satu-satunya publikasi ilmiah yang secara langsung memformulasikan permainan LinkedIn *Queens* adalah karya Mata



(a) Konfigurasi awal



(b) Konfigurasi solusi valid

Gambar 1.1: Contoh papan permainan *Colored Queens* berukuran 7×7 : (a) konfigurasi awal dengan tujuh sektor warna berbentuk ireguler; (b) konfigurasi solusi valid yang memenuhi seluruh kendala permainan.

1 Ali dan Mencia [5], yang mengajukan formulasi *Quadratic Unconstrained Binary Optimization*
 2 (QUBO) untuk menyelesaikannya pada perangkat keras kuantum. Belum terdapat kajian yang
 3 memodelkan *Colored Queens* secara eksplisit sebagai *Constraint Satisfaction Problem* (CSP) dan
 4 menyelesaikannya menggunakan teknik-teknik CSP klasik, maupun kajian yang membandingkan
 5 pendekatan deterministik dengan pendekatan *metaheuristic* pada permasalahan ini.

6 Untuk mengisi celah tersebut, tugas akhir ini mengimplementasikan dan membandingkan dua
 7 pendekatan algoritmik yang berbeda paradigma. Pendekatan pertama adalah *backtracking* yang
 8 diperkuat dengan *forward checking* dan *Arc Consistency 3* (AC-3). *Backtracking* membangun solusi
 9 secara inkremental dan mundur ketika menemui jalan buntu, sementara *forward checking* dan AC-3
 10 berfungsi sebagai teknik propagasi kendala yang memangkas domain variabel secara dini sehingga
 11 konfigurasi tidak valid dapat dieliminasi sebelum sempat dieksplorasi [6]. Kombinasi ketiga teknik
 12 ini merupakan pendekatan standar yang telah terbukti efektif untuk berbagai permasalahan CSP
 13 dan bersifat *complete*, artinya selalu menemukan solusi jika solusi tersebut ada [7].

14 Pendekatan kedua adalah *Particle Swarm Optimization* (PSO) yang diadaptasi untuk domain
 15 diskrit. PSO adalah algoritma *metaheuristic* berbasis populasi yang diperkenalkan oleh Kennedy
 16 dan Eberhart pada tahun 1995, terinspirasi dari perilaku kolektif kawanan burung dalam mencari
 17 makanan [8]. Tidak seperti *backtracking* yang menjamin kelengkapan solusi, PSO menjelajahi ruang
 18 pencarian secara paralel melalui sekumpulan partikel yang bergerak menuju solusi terbaik yang
 19 ditemukan secara kolektif. Pendekatan ini berpotensi lebih efisien pada instansi berukuran besar,
 20 namun tidak menjamin ditemukannya solusi valid. Perbandingan antara kedua pendekatan ini
 21 diharapkan memberikan wawasan mengenai *trade-off* antara kelengkapan solusi, efisiensi waktu,
 22 dan skalabilitas pada berbagai ukuran papan.

23 Selain mengimplementasikan kedua solver tersebut, tugas akhir ini juga membangun sebuah apli-
 24 kasi permainan *Colored Queens* berbasis *desktop* menggunakan Java Swing yang mengintegrasikan
 25 solver *backtracking* untuk fitur *hint* dan *auto-solve*. Aplikasi ini bertujuan memberikan pengalam-
 26 an bermain yang interaktif sekaligus menjadi wahana demonstrasi konkret dari hasil penelitian.
 27 Dengan demikian, tugas akhir ini memberikan dua kontribusi utama, yaitu formalisasi *Colored*

- 1 *Queens* sebagai CSP beserta analisis komparatif dua pendekatan algoritmik yang berbeda, serta
- 2 pembangunan perangkat lunak permainan yang mengintegrasikan solver sebagai fitur interaktif.

3 1.2 Rumusan Masalah

4 Rumusan masalah dari tugas akhir ini adalah sebagai berikut:

- 5 • Bagaimana cara membangun perangkat lunak permainan *Colored Queens*?
- 6 • Bagaimana membangun solusi permainan *Colored Queens* menggunakan teknik *Backtracking*
- 7 dan *Particle Swarm Optimization* (PSO)?
- 8 • Bagaimana membangun solver untuk permainan colored queen yang mengimplementasikan
- 9 *Backtracking* dan PSO yang dapat diintegrasikan dengan perangkat lunak yang dibangun?
- 10 • Bagaimana kinerja dari solver yang dibangun dalam mencari solusi permainan *Colored Queens*?

11 1.3 Tujuan

12 Selanjutnya, tujuan dari tugas akhir ini dapat dirumuskan sebagai berikut:

- 13 • Membangun perangkat lunak permainan *Colored Queens*.
- 14 • Mempelajari cara membangun solusi permainan Colored Queen menggunakan teknik *Back-*
- 15 *tracking* dan *Particle Swarm Optimization*.
- 16 • Membangun solver untuk permainan colored queen yang mengimplementasikan *Backtracking*
- 17 dan PSO yang dapat diintegrasikan dengan perangkat lunak yang dibangun.
- 18 • Melakukan pegujian untuk mengukur kinerja dari solver yang dibangun dalam mencari solusi
- 19 permainan *Colored Queens*

20 1.4 Batasan Masalah

21 Untuk menjaga fokus dan ruang lingkup penelitian, berikut adalah batasan-batasan yang diterapkan

22 dalam tugas akhir ini:

- 23 1. Permainan yang menjadi objek penelitian adalah *Colored Queens* dengan aturan sebagaimana
- 24 didefinisikan pada Bagian 2.2.1, yaitu: satu menteri per sektor warna, satu menteri per baris,
- 25 satu menteri per kolom, dan tidak ada dua menteri yang bertetangga secara langsung dalam
- 26 delapan arah.
- 27 2. Ukuran papan yang digunakan dalam pengujian berkisar dari 7×7 hingga 30×30 . Papan
- 28 berukuran lebih kecil dari 7×7 tidak diuji karena kompleksitasnya terlalu rendah untuk
- 29 menunjukkan perbedaan kinerja antaralgoritma, sedangkan papan berukuran lebih besar dari
- 30 30×30 tidak diuji karena keterbatasan ketersediaan data.
- 31 3. Dataset puzzle bersumber dari situs *playqueensgame.com* dan diperoleh melalui proses *web*
- 32 *scraping*. Validitas dan keunikan solusi setiap puzzle bergantung pada sumber data tersebut.
- 33 4. Algoritma penyelesaian yang diimplementasikan dan dibandingkan terbatas pada dua pende-
- 34 katan, yaitu *backtracking* dengan *forward checking* dan AC-3 sebagai pendekatan deterministik,
- 35 serta *Particle Swarm Optimization* (PSO) diskrit sebagai pendekatan *metaheuristic*. Algoritma
- 36 penyelesaian lainnya tidak dibahas.

5. Aplikasi permainan dibangun menggunakan Java Swing (JFrame) sebagai *desktop application*. Aspek antarmuka dan estetika visual bukan menjadi fokus utama penelitian; tampilan aplikasi hanya dipercantik secukupnya agar lebih nyaman dilihat dan digunakan.
6. Solver yang diintegrasikan ke dalam aplikasi permainan hanya solver *backtracking*. Solver PSO digunakan secara terpisah untuk keperluan eksperimen dan perbandingan.
7. Pengujian kinerja dilakukan pada satu lingkungan perangkat keras yang konsisten. Hasil pengujian dapat bervariasi pada perangkat keras yang berbeda.

1.5 Metodologi

Metodologi yang digunakan dalam penelitian ini terdiri atas tahapan-tahapan berikut:

1. **Studi literatur.** Melakukan kajian terhadap teori dan penelitian terdahulu yang berkaitan dengan permasalahan N-Queens dan variannya, pemodelan *Constraint Satisfaction Problem* (CSP), algoritma *backtracking* beserta teknik pemangkasan (*forward checking* dan AC-3), serta algoritma *Particle Swarm Optimization* dan adaptasinya untuk domain diskrit.
2. **Pengumpulan data.** Mengumpulkan dataset puzzle *Colored Queens* dari situs *playqueen-game.com* melalui *web scraping* menggunakan *Selenium WebDriver*. Dataset mencakup 250 level untuk setiap ukuran papan standar (7×7 hingga 12×12) serta masing-masing 1 level untuk ukuran 20×20 dan 30×30 , sehingga totalnya berjumlah 1.502 puzzle.
3. **Pemodelan permasalahan.** Memodelkan permasalahan *Colored Queens* sebagai *Constraint Satisfaction Problem* dengan mendefinisikan variabel, domain, dan kendala yang sesuai dengan aturan permainan.
4. **Perancangan dan implementasi algoritma.** Merancang dan mengimplementasikan dua solver secara terpisah, yaitu solver *backtracking* dengan *forward checking* dan AC-3, serta solver PSO diskrit dengan topologi *neighborhood* dan fungsi *fitness* berbasis penalti pelanggaran kendala.
5. **Perancangan dan implementasi aplikasi.** Merancang dan mengimplementasikan aplikasi permainan *Colored Queens* berbasis Java Swing yang mengintegrasikan solver *backtracking* untuk fitur *hint* dan *auto-solve*, serta menyediakan antarmuka interaktif bagi pengguna.
6. **Pengujian dan analisis.** Menjalankan kedua solver pada seluruh dataset dengan berbagai konfigurasi parameter, kemudian menganalisis dan membandingkan kinerja berdasarkan metrik waktu eksekusi, tingkat keberhasilan, jumlah langkah/iterasi, dan nilai *fitness* akhir.
7. **Penarikan kesimpulan.** Menyusun kesimpulan berdasarkan hasil analisis perbandingan serta mengidentifikasi kelebihan, keterbatasan, dan arah pengembangan dari masing-masing pendekatan.

1.6 Sistematika Pembahasan

Tugas akhir ini disusun dalam enam bab dengan sistematika sebagai berikut:

1. **Bab 1 Pendahuluan.** Memuat latar belakang permasalahan, rumusan masalah, tujuan penelitian, batasan masalah, metodologi, dan sistematika pembahasan.
2. **Bab 2 Dasar Teori.** Membahas teori-teori yang mendasari penelitian, meliputi permasalahan N-Queens dan variannya, *Colored Queens Problem*, *Constraint Satisfaction Problem* (CSP),

serta teknik penyelesaian CSP yang mencakup *backtracking*, *forward checking*, AC-3, dan *Particle Swarm Optimization* beserta adaptasinya untuk domain diskrit.

3. **Bab 3 Analisis.** Menyajikan analisis masalah, pemodelan permasalahan *Colored Queens* sebagai CSP, serta analisis algoritmik mendalam terhadap kedua pendekatan solver, yaitu *backtracking* dengan *forward checking* dan AC-3 serta *Particle Swarm Optimization* diskrit, beserta studi kasus untuk kedua pendekatan.
4. **Bab 4 Perancangan.** Membahas perancangan perangkat lunak, meliputi desain kelas dengan pseudocode untuk setiap komponen, perancangan antarmuka pengguna, serta teknologi yang digunakan dalam pembangunan perangkat lunak.
5. **Bab 5 Implementasi dan Pengujian.** Membahas implementasi perangkat lunak dan kedua solver, skenario serta metode pengujian, hasil pengujian, dan analisis pembahasan terhadap kinerja kedua pendekatan pada seluruh dataset.
6. **Bab 6 Kesimpulan dan Saran.** Memuat kesimpulan yang menjawab rumusan masalah berdasarkan hasil analisis, serta saran untuk pengembangan lebih lanjut.

BAB 2

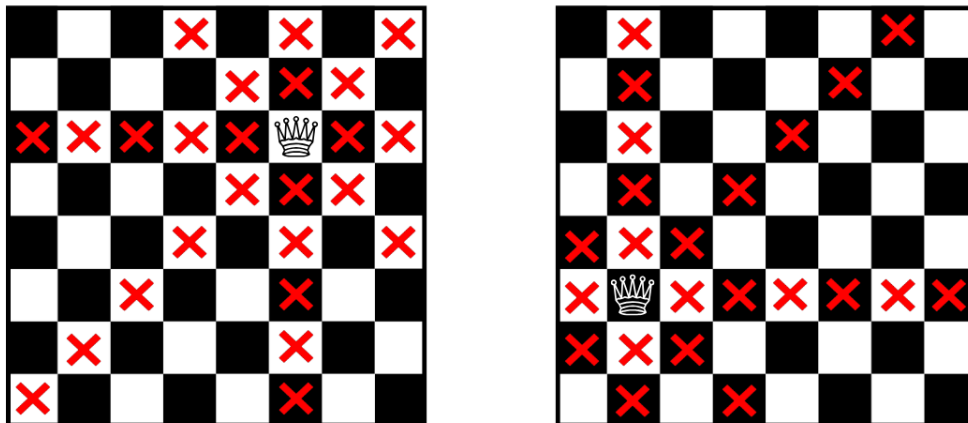
DASAR TEORI

2.1 Permasalahan N-Queens

2.1.1 Definisi dan Representasi

Permasalahan N-Queens merupakan salah satu permasalahan klasik dalam bidang ilmu komputer dan matematika diskrit dengan sejarah panjang yang didokumentasikan secara komprehensif dalam survei yang dilakukan oleh Bell dan Stevens [1]. Permasalahan ini pertama kali diperkenalkan oleh Max Bezzel pada tahun 1848 dalam majalah catur Jerman *Berliner Schachzeitung*. Formulasi awal yang diajukan Bezzel hanya membahas kasus papan berukuran 8×8 , dan baru pada tahun 1850 Franz Nauck mempublikasikan seluruh 92 solusi untuk kasus tersebut sekaligus memperumum permasalahan ke papan berukuran $n \times n$.

Pada bentuk umumnya, diberikan sebuah papan catur berukuran $n \times n$ dan tugasnya adalah menempatkan n buah bidak menteri sedemikian rupa sehingga tidak ada dua menteri yang saling menyerang. Sebagaimana dalam aturan permainan catur, sebuah menteri dapat bergerak secara bebas dalam arah horizontal, vertikal, ataupun diagonal dengan jarak tak terbatas, sehingga setiap penempatan menteri harus mempertimbangkan seluruh arah serangan tersebut seperti yang tertera pada Gambar 2.1.



Gambar 2.1: Contoh aturan penyerangan bidak menteri pada permainan catur. Silang merah menandakan kotak yang terserang oleh bidak menteri.

Dalam praktiknya, papan dapat direpresentasikan menggunakan beberapa cara. Representasi paling intuitif adalah matriks biner berukuran $n \times n$, di mana nilai 1 menandakan keberadaan menteri dan nilai 0 menandakan kotak kosong. Namun, representasi ini tidak efisien dari sudut pandang komputasi karena memungkinkan konfigurasi yang secara trivial tidak valid, seperti dua menteri pada baris yang sama. Oleh sebab itu, representasi yang lebih umum digunakan dalam literatur adalah representasi berbasis permutasi, yaitu sebuah array satu dimensi $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ dengan π_i menyatakan kolom tempat menteri pada baris ke- i ditempatkan [9]. Karena π merupakan permutasi dari himpunan $\{1, 2, \dots, n\}$, representasi ini secara implisit menjamin tidak adanya konflik baris maupun konflik kolom, sehingga ruang pencarian yang perlu dieksplorasi berkurang dari n^n menjadi $n!$ konfigurasi.

Dengan demikian, permasalahan N-Queens pada dasarnya dapat dirumuskan sebagai pencarian permutasi π yang memenuhi kondisi bahwa tidak ada dua menteri yang berbagi diagonal yang sama. Secara formal, π merupakan solusi valid jika dan hanya jika untuk setiap pasangan indeks $i \neq j$ berlaku $|i - j| \neq |\pi_i - \pi_j|$ [9]. Formulasi ringkas ini menjadi titik berangkat bagi pemodelan permasalahan dalam kerangka kerja yang lebih formal, yang akan dibahas pada Bagian 2.3.

2.1.2 Aturan Konflik Queen

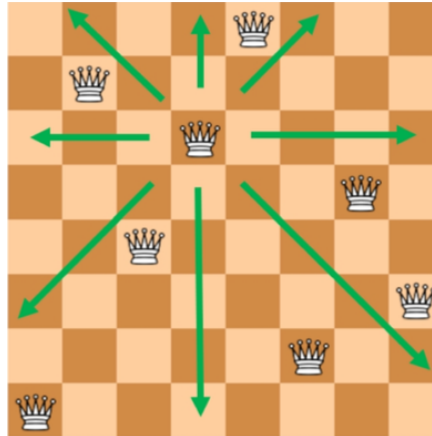
Aturan pergerakan bidak menteri pada permainan catur memunculkan tiga jenis konflik yang harus dihindari secara bersamaan dalam permasalahan N-Queens. Ketiga jenis konflik tersebut bersesuaian dengan tiga arah pergerakan menteri, yaitu horizontal, vertikal, dan diagonal [1].

Konflik horizontal terjadi apabila terdapat dua menteri yang ditempatkan pada baris yang sama. Mengingat sebuah menteri dapat bergerak bebas sepanjang barisnya, kedua menteri tersebut akan saling menyerang tanpa memandang jarak di antara keduanya. Sebaliknya, konflik vertikal terjadi apabila dua menteri menempati kolom yang sama. Kedua jenis konflik ini mudah diidentifikasi karena hanya melibatkan kesetaraan indeks baris atau kolom.

Konflik diagonal memerlukan analisis yang sedikit lebih dalam. Sebuah menteri dapat menyerang sepanjang dua jenis diagonal, yaitu diagonal utama yang membentang dari kiri atas ke kanan bawah, serta diagonal anti yang membentang dari kanan atas ke kiri bawah. Untuk dua menteri yang ditempatkan pada posisi (i, π_i) dan (j, π_j) , keduanya berada pada diagonal yang sama apabila selisih baris dan selisih kolomnya memiliki nilai absolut yang setara, yaitu $|i - j| = |\pi_i - \pi_j|$. Karakterisasi aritmatika ini memanfaatkan sifat geometris papan, di mana seluruh kotak pada diagonal utama memiliki nilai $i - \pi_i$ yang konstan, sedangkan seluruh kotak pada diagonal anti memiliki nilai $i + \pi_i$ yang konstan [6].

Berdasarkan ketiga jenis konflik di atas, permasalahan N-Queens dapat dirumuskan secara formal sebagai pencarian penempatan n menteri pada papan $n \times n$ yang memenuhi tiga kendala berikut secara bersamaan, yaitu tidak ada dua menteri yang berbagi baris yang sama, tidak ada dua menteri yang berbagi kolom yang sama, dan tidak ada dua menteri yang berbagi diagonal yang sama [6]. Sebagaimana telah dibahas pada Bagian 2.1.1, penggunaan representasi permutasi $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ secara implisit menghilangkan dua kendala pertama, sehingga hanya kendala diagonal yang perlu diperiksa secara eksplisit selama proses pencarian solusi. Sebagai ilustrasi, Gambar 2.2 menampilkan salah satu solusi valid untuk papan 8-Queens, di mana visualisasi jangkauan serangan salah satu menteri menunjukkan bahwa tidak terdapat menteri lain yang

- 1 berbagi baris, kolom, ataupun diagonal yang sama, sehingga konfigurasi tersebut memenuhi seluruh
- 2 kendala yang telah didefinisikan.



Gambar 2.2: Contoh solusi valid permasalahan 8-Queens. Panah hijau menandakan jangkauan serangan salah satu menteri; tidak ada menteri lain yang berada pada baris, kolom, ataupun diagonal yang sama.¹

- 3 Interaksi antarkendala inilah yang membuat permasalahan N-Queens menarik dari sudut pandang
- 4 komputasi. Pelanggaran terhadap salah satu kendala mengakibatkan konfigurasi menjadi tidak valid,
- 5 dan kendala-kendala tersebut tidak dapat diperiksa secara terpisah karena keputusan penempatan
- 6 satu menteri secara langsung memengaruhi domain posisi valid bagi menteri-menteri lainnya. Sifat
- 7 saling-bergantung antarkendala ini menjadi dasar bagi pemodelan permasalahan menggunakan
- 8 kerangka kerja *Constraint Satisfaction Problem* yang akan dibahas pada Bagian 2.3.

9 2.1.3 Kompleksitas dan Relevansi

- 10 Dari perspektif komputasi, permasalahan N-Queens menarik karena memiliki struktur yang seder-
- 11 hana namun ruang solusinya sangat besar dan tumbuh secara eksponensial. Apabila tidak ada
- 12 asumsi tambahan yang digunakan, setiap menteri memiliki n^2 kemungkinan posisi pada papan,
- 13 menghasilkan ruang pencarian awal sebesar $\binom{n^2}{n}$, yang berarti jumlah cara memilih n posisi dari n^2
- 14 kemungkinan posisi pada papan. Penggunaan representasi permutasi sebagaimana dijelaskan pada
- 15 Bagian 2.1.1 mempersempit ruang ini menjadi $n!$, namun pertumbuhannya tetap superpolynomial.
- 16 Sebagai gambaran, untuk $n = 14$ terdapat $14! \approx 8,7 \times 10^{10}$ kemungkinan permutasi yang harus
- 17 dievaluasi, jumlah yang jauh melampaui kemampuan komputasi *brute-force* konvensional pada
- 18 perangkat keras umum.

- 19 Meskipun ruang pencarian tumbuh secara eksponensial, perlu ditekankan bahwa permasalahan
- 20 N-Queens dalam formulasi dasarnya bukanlah permasalahan yang sulit secara teoretis. Hoffman,
- 21 Loessi, dan Moore [10] sebagaimana dirangkum dalam survei Bell dan Stevens [1] menunjukkan
- 22 bahwa terdapat konstruksi aritmatika eksplisit yang dapat menghasilkan satu solusi N-Queens
- 23 untuk setiap $n \geq 4$ dalam waktu linier $O(n)$ tanpa perlu melakukan pencarian sama sekali. Dengan
- 24 demikian, persoalan keputusan “apakah solusi N-Queens ada untuk papan $n \times n$ ” dapat dijawab

¹Diambil dari <https://www.researchgate.net/publication/372415823/figure/fig3/AS:114312812131343730/1702956395727/N-Queen-problem-explanation-with-8-queens-in-a-chessboard-of-8.png>, diakses ulang pada 1 Juni 2026

1 dalam waktu konstan, dengan solusi yang selalu ada kecuali untuk $n = 2$ dan $n = 3$.

2 Karakteristik kompleksitas N-Queens telah menjadi topik diskusi yang panjang dalam komunitas
3 *Artificial Intelligence*, khususnya terkait kelayakannya sebagai *benchmark* algoritma pencarian.
4 Gent, Jefferson, dan Nightingale [2] memberikan analisis komprehensif terhadap persoalan ini
5 dan menunjukkan bahwa permasalahan N-Queens dalam bentuk dasarnya tidak ideal sebagai
6 tolok ukur kesulitan algoritma, mengingat solusinya dapat dikonstruksi secara langsung tanpa
7 proses pencarian. Sebaliknya, mereka membuktikan bahwa varian *N-Queens Completion*, yaitu
8 permasalahan menyelesaikan konfigurasi parsial yang telah diberikan sebelumnya, tergolong NP-
9 Complete dan #P-Complete. *NP-Complete* adalah kelas permasalahan keputusan (ya/tidak) yang
10 dianggap paling sulit dalam kelas NP, yaitu permasalahan yang solusinya dapat diverifikasi dalam
11 waktu polinomial namun belum diketahui dapat diselesaikan dalam waktu polinomial. *#P-Complete*
12 adalah versi penghitungan dari NP-Complete, yang tidak hanya menentukan apakah solusi ada,
13 tetapi juga menghitung *berapa banyak* solusi yang ada. Permasalahan dalam kelas ini diyakini
14 tidak dapat diselesaikan secara efisien pada perangkat komputasi konvensional [11]. Hasil ini
15 mengindikasikan bahwa varian-varian N-Queens dengan kendala tambahan atau struktur papan
16 ireguler memiliki kompleksitas teoritis yang setara dengan permasalahan-permasalahan sulit klasik
17 dalam ilmu komputer.

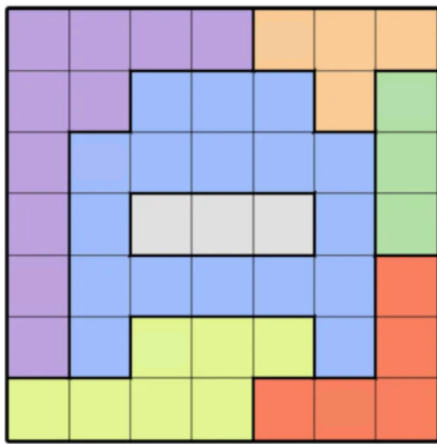
18 Permasalahan *Colored Queens* yang menjadi fokus tugas akhir ini memiliki struktur kendala
19 yang jauh lebih kompleks dibandingkan N-Queens klasik, termasuk pembagian sektor warna dan
20 kendala *adjacency* yang tidak dapat diselesaikan dengan konstruksi aritmatika sederhana. Dengan
21 kata lain, meskipun teknik konstruktif tersedia untuk N-Queens dasar, teknik tersebut tidak
22 dapat digeneralisasi ke varian-varian dengan kendala ireguler, yaitu kendala yang strukturnya
23 bergantung pada konfigurasi papan tertentu sehingga bervariasi antarpermasalahan. Sebagai contoh,
24 sektor warna pada *Colored Queens* memiliki bentuk dan ukuran yang berbeda-beda pada setiap
25 papan, sehingga tidak dapat ditangani oleh formula tunggal yang berlaku universal sebagaimana
26 kendala baris, kolom, dan diagonal pada N-Queens klasik. Hal inilah yang menjadikan pendekatan
27 algoritmik berbasis pencarian, baik yang bersifat sistematis maupun *metaheuristic*, tetap relevan
28 sebagai kerangka penyelesaian untuk permasalahan *Colored Queens* pada tugas akhir ini.

29 Selain relevansi pada bentuk dasarnya, variasi-variasi N-Queens dengan kendala tambah-
30 an—seperti yang akan dibahas pada Bagian 2.2—memperluas penerapan permasalahan ini ke
31 domain yang lebih spesifik. Penambahan kendala spasial seperti kendala *adjacency* atau pembagian
32 wilayah merepresentasikan tipe permasalahan yang muncul ketika objek-objek yang ditempatkan
33 tidak hanya memiliki jangkauan konflik linear tetapi juga interaksi lokal yang kompleks. Sebagai
34 contoh, pada permasalahan penempatan komponen sirkuit, dua komponen tidak hanya tidak boleh
35 berada pada jalur listrik yang sama, tetapi juga harus menjaga jarak fisik minimum agar tidak
36 terjadi interferensi. Tipe kendala berlapis semacam ini banyak ditemui pada permasalahan tata letak
37 (*layout problem*), penjadwalan dengan kendala kedekatan waktu, serta permasalahan pewarnaan
38 graf dengan kendala geometris. Dengan demikian, studi terhadap varian N-Queens yang memiliki
39 kendala tambahan bukan sekadar eksplorasi teoretis, melainkan juga memberikan wawasan yang
40 dapat ditransfer ke permasalahan praktis dengan struktur kendala serupa.

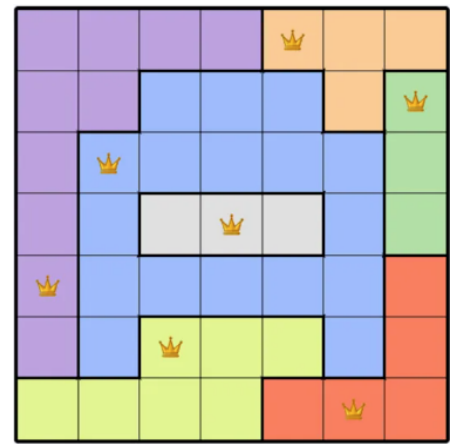
2.2 Colored Queens Problem

2.2.1 Definisi dan Aturan Permainan

Permasalahan *Colored Queens* merupakan sebuah *puzzle* logika berbasis papan yang dipopulerkan melalui permainan *Queens* pada platform LinkedIn [3]. Secara struktural, permasalahan ini memiliki tingkat kesamaan yang tinggi dengan jenis *puzzle* penempatan objek yang dikenal sebagai *Star Battle*, yang pertama kali dirancang oleh Hans Eendebak untuk *World Puzzle Championship* tahun 2003 [4]. Dalam varian yang dibahas pada tugas akhir ini, permainan menggunakan papan berukuran $n \times n$ yang dibagi menjadi n buah sektor berwarna (*colored regions*) dengan bentuk yang umumnya tidak beraturan (*irregular*). Setiap sektor warna membentuk satu wilayah yang terhubung (*connected region*) pada papan, artinya seluruh sel dalam satu sektor dapat dijangkau dari sel lainnya melalui tetangga horizontal atau vertikal tanpa melewati sel dari sektor lain. Contoh papan permainan beserta solusinya diperlihatkan pada Gambar 2.3.



(a) Papan awal



(b) Papan yang telah diselesaikan

Gambar 2.3: Contoh papan permainan *Colored Queens* berukuran 8×8 dengan 8 sektor warna. Setiap sektor membentuk wilayah terhubung, dan solusi menempatkan tepat satu menteri per baris, kolom, dan sektor warna tanpa ada dua menteri yang bertetangga.²

Tujuan permainan adalah menempatkan tepat n buah bidak menteri pada papan sedemikian rupa sehingga seluruh aturan berikut terpenuhi secara bersamaan:

1. Setiap baris memuat tepat satu menteri.
2. Setiap kolom memuat tepat satu menteri.
3. Setiap sektor warna memuat tepat satu menteri.
4. Tidak ada dua menteri yang menempati sel-sel yang bertetangga (*adjacent*), termasuk tetangga diagonal. Secara formal, dua sel (r_1, c_1) dan (r_2, c_2) dikatakan bertetangga apabila $|r_1 - r_2| \leq 1$ dan $|c_1 - c_2| \leq 1$.

Perlu dicatat bahwa, berbeda dengan permasalahan N-Queens klasik yang dibahas pada Bagian 2.1, bidak menteri dalam *Colored Queens* tidak menyerang sepanjang diagonal. Serangan hanya berlaku dalam arah horizontal dan vertikal, sehingga kendala diagonal digantikan oleh kendala *adjacency* yang bersifat lokal (hanya memengaruhi delapan sel di sekeliling menteri). Perbedaan ini

²Diambil dari <https://tech.yahoo.com/puzzles/connections/articles/today-queens-737-linkedin-answer-131012952.html>, diakses ulang pada 3 Juni 2026

mengubah karakteristik ruang pencarian secara fundamental, karena kendala *adjacency* tidak dapat direduksi menjadi perbandingan aritmatika sederhana seperti kendala diagonal pada N-Queens.

Selain itu, keberadaan sektor warna yang berbentuk ireguler menambah dimensi kendala yang tidak dimiliki oleh N-Queens. Pada N-Queens, satu-satunya entitas pembatas adalah baris, kolom, dan diagonal yang semuanya memiliki struktur geometris teratur. Pada *Colored Queens*, sektor warna memiliki bentuk yang bervariasi antarpapan dan tidak mengikuti pola geometris yang dapat diprediksi, sehingga setiap instansi permainan memerlukan analisis kendala yang berbeda.

2.2.2 Kompleksitas dan Struktur Masalah

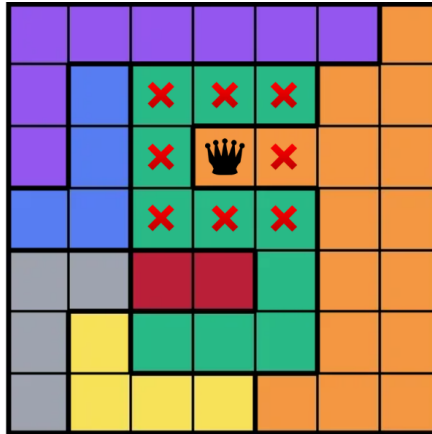
Permasalahan *Colored Queens* memiliki struktur kendala berlapis (*multi-layered constraints*) yang membedakannya secara fundamental dari N-Queens klasik. Sebagaimana disusun oleh Mata Ali dan Mencia [5], permainan ini merupakan varian dari N-Queens yang melonggarkan kendala diagonal namun menambahkan kendala sektor warna dan *adjacency*. Kombinasi kendala-kendala tersebut menghasilkan interaksi yang lebih kompleks dibandingkan N-Queens, karena setiap keputusan penempatan menteri secara simultan memengaruhi tiga domain pembatas yang berbeda, yaitu baris/kolom, sektor warna, dan lingkungan tetangga.

Lapisan pertama adalah kendala baris dan kolom, yang secara struktural identik dengan N-Queens dan membatasi setiap baris serta kolom untuk memuat tepat satu menteri. Lapisan kedua adalah kendala sektor warna, yang mengharuskan setiap sektor memiliki tepat satu menteri. Berbeda dengan kendala baris dan kolom yang bersifat reguler dan simetris, sektor warna memiliki bentuk ireguler yang bervariasi antarpapan. Implikasinya, ukuran domain untuk setiap sektor (jumlah sel yang tersedia) tidak seragam, dan beberapa sektor mungkin memiliki domain yang jauh lebih kecil dibandingkan sektor lainnya. Hal ini menciptakan ketidakseimbangan dalam proses pencarian, di mana penempatan menteri pada sektor berukuran kecil memiliki dampak propagasi yang berbeda dibandingkan penempatan pada sektor berukuran besar.

Lapisan ketiga adalah kendala *adjacency*, yang melarang dua menteri menempati sel-sel yang bertetangga. Kendala ini bersifat lokal dan simetris, namun efeknya bersifat global karena penempatan satu menteri secara langsung mengeliminasi hingga delapan sel tambahan dari domain menteri-menteri lainnya. Pada papan berukuran kecil ($n \leq 8$), eliminasi ini sangat signifikan secara proporsional terhadap total sel yang tersedia. Gambar 2.4 memperlihatkan dampak penempatan satu menteri pada papan 7×7 , di mana delapan sel di sekitarnya langsung dieliminasi dari domain variabel-variabel lain. Eliminasi sebanyak ini setara dengan sekitar 16% dari total sel papan, sehingga ruang pencarian menyempit secara signifikan hanya dari satu penugasan.

Interaksi antara ketiga lapisan kendala tersebut menciptakan beberapa tantangan komputasi. Pertama, tidak ada representasi permutasi sederhana sebagaimana pada N-Queens, karena variabel terkait erat dengan sektor warna yang bentuknya tidak teratur. Kedua, kendala *adjacency* tidak dapat diekspresikan sebagai ketidaksamaan aritmatika global (seperti $|i - j| \neq |\pi_i - \pi_j|$ pada N-Queens), melainkan harus diperiksa secara lokal untuk setiap pasangan sel. Ketiga, bentuk ireguler sektor warna menyebabkan *constraint graph* yang dihasilkan tidak memiliki struktur simetris yang dapat dieksploitasi oleh teknik konstruktif.

Karakteristik kendala berlapis ini juga memiliki implikasi terhadap bentuk *fitness landscape* yang dihadapi oleh pendekatan *metaheuristic*. Karena setiap keputusan penempatan menteri secara



Gambar 2.4: Dampak kendala *adjacency* pada papan berukuran kecil. Penempatan satu menteri pada papan 7×7 mengeliminasi delapan sel tetangga (ditandai dengan tanda X merah), yang setara dengan sekitar 16% dari total 49 sel papan.

simultan memengaruhi tiga domain pembatas yang berbeda, perubahan pada satu variabel dapat secara drastis mengubah jumlah pelanggaran pada variabel lain. Hal ini menghasilkan *landscape* yang *rugged* dengan banyak *local optima*, di mana lembah-lembah sempit menuju solusi valid dikelilingi oleh plateau pelanggaran yang luas [20]

Dari sudut pandang kompleksitas, meskipun belum terdapat pembuktian formal bahwa *Colored Queens* termasuk dalam kelas NP-Complete, struktur permasalahannya memiliki kemiripan dengan *N-Queens Completion* yang telah terbukti NP-Complete [2]. Keduanya melibatkan kendala tambahan yang merusak keteraturan geometris N-Queens dasar, sehingga teknik konstruktif tidak dapat diterapkan. Kompleksitas inilah yang menjadikan permasalahan ini menarik untuk diselesaikan secara algoritmik menggunakan pendekatan berbasis pencarian.

2.2.3 Posisi dan Relevansi Penelitian

Permasalahan *Colored Queens* merupakan topik yang relatif baru dalam literatur akademis. Meskipun permainan ini telah menarik perhatian luas sejak diperkenalkan oleh LinkedIn sebagai *puzzle* harian [3], kajian ilmiah formal terhadap permasalahan ini masih sangat terbatas. Sejauh tinjauan pustaka yang dilakukan, satu-satunya publikasi akademis yang secara langsung memformulasikan permainan LinkedIn Queens adalah karya Mata Ali dan Mencia [5], yang mengusulkan formulasi *Quadratic Unconstrained Binary Optimization* (QUBO) untuk menyelesaikan generalisasi permainan tersebut pada perangkat keras kuantum. Pendekatan-pendekatan lain yang tersedia di komunitas pengembang umumnya bersifat informal dan tidak dipublikasikan melalui kanal akademis, seperti implementasi *backtracking* sederhana, formulasi *integer programming*, serta penggunaan *SAT/SMT solver*.

Celah penelitian (*research gap*) yang teridentifikasi dari tinjauan ini adalah sebagai berikut. Pertama, belum terdapat kajian yang memodelkan *Colored Queens* secara eksplisit sebagai *Constraint Satisfaction Problem* (CSP) dan menyelesaikannya menggunakan teknik-teknik klasik CSP seperti *backtracking* dengan *forward checking* dan *arc consistency*. Kedua, belum ditemukan penelitian yang membandingkan pendekatan sistematis (*complete*) dengan pendekatan *metaheuristic* untuk permasalahan ini. Ketiga, analisis empiris mengenai pengaruh ukuran papan terhadap performa

1 algoritma penyelesaian juga belum dilakukan secara formal.

2 Tugas akhir ini bertujuan untuk mengisi celah tersebut dengan memberikan dua kontribusi
3 utama. Kontribusi pertama adalah pemodelan *Colored Queens* sebagai CSP dan penyelesaiannya
4 menggunakan *backtracking* yang diperkuat dengan *forward checking* dan *Arc Consistency* (AC-3).
5 Kontribusi kedua adalah penerapan *Particle Swarm Optimization* (PSO) yang diadaptasi untuk
6 ruang pencarian diskrit sebagai pendekatan alternatif. Perbandingan antara kedua pendekatan
7 tersebut diharapkan memberikan wawasan mengenai *trade-off* antara kelengkapan solusi dan efisiensi
8 waktu, serta perilaku masing-masing algoritma pada papan berukuran kecil hingga besar.

9 2.3 Constraint Satisfaction Problem

10 2.3.1 Definisi dan Komponen CSP

11 *Constraint Satisfaction Problem* (CSP) merupakan kerangka kerja formal untuk merepresentasikan
12 dan menyelesaikan permasalahan yang melibatkan sekumpulan variabel dengan batasan-batasan
13 yang harus dipenuhi secara bersamaan. Secara formal, sebuah CSP didefinisikan sebagai tripel
14 $\mathcal{P} = \langle X, D, C \rangle$ [6] di mana:

- 15 • $X = \{x_1, x_2, \dots, x_n\}$ adalah himpunan variabel yang nilainya harus ditentukan.
- 16 • $D = \{D_1, D_2, \dots, D_n\}$ adalah himpunan domain, di mana D_i menyatakan himpunan nilai
17 yang dapat diambil oleh variabel x_i .
- 18 • $C = \{C_1, C_2, \dots, C_m\}$ adalah himpunan kendala, di mana setiap kendala C_j mendefinisikan
19 kombinasi nilai yang diperbolehkan untuk suatu subhimpunan variabel tertentu.

20 Secara konseptual, ketiga komponen tersebut menjawab tiga pertanyaan dasar dalam suatu
21 permasalahan. *Variabel* menyatakan “apa yang perlu ditentukan”, yang berarti suatu entitas yang
22 nilainya belum diketahui dan harus dicari oleh algoritma. *Domain* menyatakan “nilai apa saja yang
23 mungkin” dari sebuah himpunan kandidat nilai yang valid untuk masing-masing variabel. *Kendala*
24 (*constraint*) menyatakan “aturan apa yang harus dipatuhi”, atau batasan yang membatasi kombinasi
25 nilai yang dapat dipilih, baik pada satu variabel maupun di antara beberapa variabel sekaligus.
26 Sebagai contoh konkret, pada permasalahan pewarnaan peta, variabel adalah wilayah-wilayah yang
27 harus diwarnai, domain adalah himpunan warna yang tersedia, dan kendala adalah aturan bahwa
28 dua wilayah yang bertetangga tidak boleh memiliki warna yang sama.

29 Sebuah *assignment* adalah pemetaan dari satu atau lebih variabel ke nilai-nilai dalam domainnya.
30 *Assignment* disebut *consistent* apabila tidak melanggar kendala manapun, dan disebut *complete*
31 apabila seluruh variabel telah diberi nilai. Solusi dari sebuah CSP adalah *complete assignment* yang
32 bersifat *consistent*, yaitu seluruh variabel telah diberi nilai dan seluruh kendala terpenuhi [6].

33 Kajian formal mengenai CSP berawal dari karya Montanari pada tahun 1974 yang memperke-
34 nalkan kerangka jaringan kendala (*constraint network*), dan dilanjutkan oleh Mackworth pada tahun
35 1977 yang mengembangkan algoritma *arc consistency* sebagai teknik dasar untuk mereduksi ruang
36 pencarian [12]. Sejak saat itu, CSP telah menjadi paradigma fundamental dalam *Artificial Intelli-*
37 *gence* dan digunakan secara luas untuk memodelkan berbagai permasalahan seperti penjadwalan,
38 pewarnaan graf, dan penyelesaian *puzzle* [6].

39 Perkembangan CSP sebagai paradigma yang berdiri sendiri dalam *Artificial Intelligence* didorong
40 oleh kebutuhan untuk menyatukan berbagai pendekatan ad hoc yang sebelumnya digunakan untuk

menyelesaikan permasalahan yang melibatkan kendala. Sebelum formalisasi CSP, permasalahan seperti perencanaan, penjadwalan, dan penyelesaian *puzzle* ditangani menggunakan teknik-teknik khusus yang sulit digeneralisasi antardomain [13]. CSP menyediakan abstraksi yang memungkinkan permasalahan-permasalahan tersebut dimodelkan menggunakan komponen-komponen yang sama (variabel, domain, dan kendala), sehingga algoritma-algoritma penyelesaian yang dikembangkan dapat digunakan secara lintas domain tanpa modifikasi struktural yang signifikan.

Salah satu kekuatan utama paradigma CSP terletak pada pemisahan antara model permasalahan dan algoritma penyelesaian, yaitu pendekatan deklaratif (*declarative modeling*). Pemodel hanya perlu mendefinisikan *apa* yang dianggap sebagai solusi valid, yaitu seluruh kendala yang harus dipenuhi, tanpa perlu menetapkan *bagaimana* solusi tersebut harus dicari. Sebagai ilustrasi, pada permasalahan N-Queens, pendekatan deklaratif cukup menyatakan aturan “tidak boleh ada dua menteri pada baris, kolom, atau diagonal yang sama”; algoritma generik kemudian secara otomatis mencari konfigurasi yang memenuhi aturan tersebut. Sebaliknya, pendekatan prosedural mengharuskan pemodel merancang langkah-langkah pencarian secara eksplisit, misalnya: “letakkan menteri pada baris pertama, kolom pertama; jika konflik, geser ke kolom berikutnya; jika seluruh kolom habis, mundur ke baris sebelumnya”, dan seterusnya. Pendekatan deklaratif memisahkan logika permasalahan dari strategi pencarian, sehingga algoritma yang sama dapat digunakan untuk memodelkan permasalahan yang sangat berbeda (penjadwalan, pewarnaan peta, *puzzle*) tanpa perlu menulis ulang prosedur pencarian. Kemudahan inilah yang menjadi salah satu alasan utama mengapa CSP banyak diadopsi sebagai paradigma pemodelan permasalahan kombinatorik dalam praktik industri [13].

2.3.2 Teknik Representasi

Sebuah CSP dapat direpresentasikan secara visual menggunakan *constraint graph*, yaitu sebuah graf di mana setiap simpul merepresentasikan variabel dan setiap sisi menghubungkan dua variabel yang terlibat dalam suatu kendala bersama [6]. Representasi ini berguna untuk menganalisis struktur ketergantungan antarvariabel serta menentukan strategi penyelesaian yang tepat.

Kendala dalam CSP dapat diklasifikasikan berdasarkan jumlah variabel yang terlibat. Kendala *unary* melibatkan satu variabel dan membatasi domain variabel tersebut secara langsung, misalnya $x_1 \neq 3$. Kendala *binary* melibatkan dua variabel dan membatasi kombinasi nilai yang diperbolehkan di antara keduanya, misalnya $x_1 \neq x_2$. Dalam praktiknya, sebagian besar CSP dapat ditransformasikan ke dalam bentuk yang hanya mengandung kendala *binary* tanpa kehilangan informasi [6], dan banyak algoritma penyelesaian CSP dirancang secara khusus untuk menangani kendala jenis ini.

Struktur *constraint graph* memiliki implikasi langsung terhadap kompleksitas penyelesaian. Apabila graf memiliki struktur pohon (*tree-structured*), CSP dapat diselesaikan dalam waktu polinomial. Sebaliknya, graf yang padat (*dense*) dengan banyak sisi mengindikasikan interaksi kendala yang tinggi dan umumnya memerlukan teknik penyelesaian yang lebih canggih [6].

2.3.3 Pemodelan Colored Queens sebagai CSP

Permasalahan *Colored Queens* dapat dimodelkan secara natural sebagai CSP dengan mendefinisikan komponen-komponen $\langle X, D, C \rangle$ sebagai berikut.

Himpunan variabel $X = \{x_1, x_2, \dots, x_n\}$ merepresentasikan n buah sektor warna pada papan, di mana setiap variabel x_i bersesuaian dengan sektor warna ke- i . Nilai yang diberikan pada x_i menyatakan posisi sel tempat menteri ditempatkan dalam sektor tersebut.

Domain D_i untuk setiap variabel x_i adalah himpunan seluruh sel yang termasuk dalam sektor warna ke- i . Secara formal, apabila sektor warna ke- i terdiri dari sel-sel $\{(r_1, c_1), (r_2, c_2), \dots, (r_k, c_k)\}$, maka $D_i = \{(r_1, c_1), (r_2, c_2), \dots, (r_k, c_k)\}$. Perlu dicatat bahwa ukuran domain bervariasi antarsektor karena bentuk sektor yang ireguler, sehingga $|D_i| \neq |D_j|$ pada umumnya.

Himpunan kendala C terdiri dari empat jenis kendala yang bersesuaian dengan aturan permainan:

1. **Kendala baris:** untuk setiap pasangan variabel x_i, x_j dengan $i \neq j$, apabila $x_i = (r_i, c_i)$ dan $x_j = (r_j, c_j)$, maka $r_i \neq r_j$.
2. **Kendala kolom:** untuk setiap pasangan variabel x_i, x_j dengan $i \neq j$, berlaku $c_i \neq c_j$.
3. **Kendala sektor warna:** dipenuhi secara implisit oleh definisi variabel, karena setiap variabel hanya dapat mengambil nilai dari sektornya sendiri.
4. **Kendala adjacency:** untuk setiap pasangan variabel x_i, x_j dengan $i \neq j$, apabila $x_i = (r_i, c_i)$ dan $x_j = (r_j, c_j)$, maka $|r_i - r_j| > 1$ atau $|c_i - c_j| > 1$.

Seluruh kendala di atas bersifat *binary*, yaitu melibatkan tepat dua variabel, sehingga permasalahan *Colored Queens* menghasilkan *constraint graph* yang bersifat *complete*—setiap pasangan variabel terhubung oleh setidaknya satu kendala. Karakteristik ini mengindikasikan tingkat interaksi kendala yang tinggi dan memperkuat argumen bahwa teknik propagasi kendala diperlukan untuk mereduksi ruang pencarian secara efektif sebelum atau selama proses pencarian solusi. Teknik-teknik penyelesaian tersebut akan dibahas pada bagian berikutnya.

2.4 Teknik Penyelesaian CSP

Bagian ini membahas teknik-teknik yang digunakan untuk menyelesaikan CSP secara umum, yang kemudian akan diterapkan pada permasalahan *Colored Queens*. Teknik-teknik tersebut dikelompokkan menjadi dua kategori besar, yaitu algoritma sistematis yang menjamin kelengkapan pencarian, dan pendekatan *metaheuristic* yang bersifat probabilistik namun mampu menjelajahi ruang solusi yang sangat besar secara efisien.

2.4.1 Algoritma Sistematis

Algoritma sistematis untuk CSP adalah algoritma yang menjamin ditemukannya solusi apabila solusi tersebut ada, atau membuktikan ketiadaan solusi apabila tidak ada satu pun penugasan yang memenuhi seluruh kendala. Jaminan kelengkapan (*completeness*) ini membedakan algoritma sistematis dari pendekatan *metaheuristic* yang akan dibahas pada Bagian 2.4.2. Dalam konteks CSP, keluarga algoritma sistematis yang paling banyak digunakan dibangun di atas fondasi *backtracking search*, yang kemudian diperkuat oleh teknik-teknik propagasi kendala seperti *forward checking* dan *arc consistency* [6].

Backtracking Search

Backtracking search adalah algoritma pencarian berbasis *depth-first search* (DFS) yang membangun solusi CSP secara inkremental, satu variabel dalam satu langkah [6]. Pada setiap langkah, algoritma

memilih sebuah variabel yang belum diberi nilai, lalu mencoba setiap nilai dalam domain variabel tersebut secara berurutan. Apabila sebuah nilai tidak melanggar kendala apapun terhadap variabel-variabel yang telah ditetapkan sebelumnya, nilai tersebut ditetapkan dan algoritma melanjutkan ke variabel berikutnya secara rekursif. Apabila tidak ada nilai yang konsisten ditemukan untuk variabel saat ini, algoritma mundur (*backtrack*) ke variabel sebelumnya dan mencoba nilai lain dari domain variabel tersebut.

Pseudocode dasar *backtracking search* sebagaimana disajikan oleh Russell dan Norvig [6] diperlihatkan pada Listing 1.

Algorithm 1 Backtracking Search untuk CSP

```

1: function BACKTRACK(assignment, csp)
2:   if assignment lengkap then
3:     return assignment
4:   end if
5:   var ← SELECTUNASSIGNEDVARIABLE(csp)
6:   for setiap value dalam ORDERDOMAINVALUES(var, assignment, csp) do
7:     if value konsisten dengan assignment then
8:       Tambahkan {var = value} ke assignment
9:       result ← BACKTRACK(assignment, csp)
10:      if result ≠ failure then
11:        return result
12:      end if
13:      Hapus {var = value} dari assignment
14:    end if
15:  end for
16:  return failure
17: end function

```

Keunggulan utama *backtracking search* dibandingkan pencarian *brute-force* naif adalah pemangkasan ruang pencarian (*pruning*) yang terjadi secara otomatis ketika setiap kali penugasan parsial terdeteksi melanggar kendala, seluruh cabang pencarian di bawahnya dieliminasi tanpa perlu dieksplorasi. Secara formal, kompleksitas waktu terburuk dari *backtracking* pada CSP dengan n variabel dan ukuran domain maksimum d adalah $O(d^n)$ [13]. Meskipun tetap eksponensial, angka ini jauh lebih kecil dibandingkan pencarian lengkap yang mengevaluasi seluruh d^n kombinasi tanpa pemangkasan.

Namun, *backtracking* murni memiliki kerentanan terhadap fenomena *redundant search*, yaitu eksplorasi cabang-cabang pohon pencarian yang strukturnya secara esensial setara dengan cabang yang telah diuji sebelumnya namun dengan urutan penugasan yang berbeda [13]. Hal ini terjadi karena *backtracking* murni tidak mempertahankan ingatan mengenai alasan kegagalan suatu cabang, sehingga konflik yang sama dapat ditemukan kembali pada cabang-cabang pencarian yang berbeda. Berbagai teknik telah dikembangkan untuk mengatasi keterbatasan ini, di antaranya *conflict-directed backjumping* yang melompati beberapa level mundur secara langsung ke variabel yang menyebabkan kegagalan, serta *constraint learning* yang menyimpan kendala-kendala turunan yang ditemukan selama pencarian untuk mencegah eksplorasi ulang konfigurasi yang setara [13].

Pendekatan lain untuk mengurangi inefisiensi *backtracking* adalah dengan menambah kandungan informasi yang diperoleh dari setiap langkah penugasan. Teknik propagasi kendala, yang menjadi

fokus pembahasan dua bagian selanjutnya, mengikuti pendekatan ini yang alih-alih hanya memeriksa apakah penugasan saat ini konsisten, propagasi kendala juga menyebarkan implikasi penugasan tersebut ke variabel-variabel yang belum ditugaskan, memangkas nilai-nilai yang dipastikan tidak dapat menjadi bagian dari solusi. Dengan demikian, propagasi kendala mengubah *backtracking* dari algoritma pencarian murni menjadi algoritma pencarian yang dipadu dengan deduksi (*search with inference*) [7].

Forward Checking

Forward checking adalah teknik propagasi kendala yang diintegrasikan ke dalam *backtracking search* untuk mendeteksi kegagalan lebih awal [7]. Ide dasarnya adalah setiap kali sebuah variabel x_i diberi nilai v , algoritma segera memeriksa seluruh variabel yang belum diberi nilai dan terhubung dengan x_i melalui suatu kendala, lalu menghapus dari domain variabel-variabel tersebut semua nilai yang tidak konsisten dengan penugasan $x_i = v$. Proses ini dilakukan setelah penugasan dilakukan, sehingga inkonsistensi dapat dideteksi secepat mungkin dan tidak menunggu hingga variabel yang bersangkutan ditugaskan.

Pseudocode *forward checking* sebagaimana dirumuskan oleh Van Beek [7] diperlihatkan pada Listing 2..

Algorithm 2 Forward Checking

```

1: function FORWARDCHECK( $var, value, assignment, csp$ )
2:   for setiap variabel  $Y$  yang belum ditugaskan dan memiliki kendala dengan  $var$  do
3:     for setiap  $v \in D_Y$  do
4:       if  $v$  tidak konsisten dengan  $value$  berdasarkan kendala  $(var, Y)$  then
5:         Hapus  $v$  dari  $D_Y$ 
6:       end if
7:     end for
8:     if  $D_Y$  kosong then
9:       return failure
10:    end if
11:  end for
12:  return success
13: end function

```

Sebelum melanjutkan, perlu diperkenalkan terlebih dahulu konsep *domain wipeout*, yaitu kondisi di mana domain sebuah variabel menjadi kosong sepenuhnya setelah proses pemangkasan. Apabila *domain wipeout* terjadi, hal ini berarti tidak ada nilai yang dapat ditugaskan ke variabel tersebut tanpa melanggar setidaknya satu kendala, sehingga cabang pencarian saat ini dipastikan tidak dapat menghasilkan solusi valid. Deteksi *domain wipeout* merupakan sinyal bagi algoritma untuk segera melakukan *backtrack*, sehingga waktu komputasi tidak terbuang untuk mengeksplorasi cabang yang sudah pasti gagal. Konsep ini akan terus muncul pada teknik-teknik propagasi kendala yang dibahas pada bagian ini.

Keuntungan langsung dari *forward checking* adalah kemampuannya untuk mendeteksi *domain wipeout* jauh sebelum variabel yang bersangkutan diproses oleh rekursi. Apabila *domain wipeout* terdeteksi, algoritma dapat segera melakukan *backtrack* tanpa perlu mengeksplorasi cabang-cabang yang sudah pasti tidak akan menghasilkan solusi yang valid. Dengan demikian, *forward checking*

mengurangi kedalaman pohon pencarian yang perlu dijelajahi sebelum kegagalan ditemukan.

Namun, *forward checking* memiliki keterbatasan fundamental, di mana teknik ini hanya memeriksa konsistensi antara variabel yang baru ditugaskan dengan variabel-variabel yang belum ditugaskan, tanpa memeriksa konsistensi antarsesama variabel yang belum ditugaskan [6]. Konflik baru akan ditemukan ketika salah satu dari kedua variabel tersebut diproses, yang berpotensi membuang banyak langkah pencarian. Keterbatasan inilah yang diatasi oleh teknik *arc consistency* yang dibahas pada bagian berikutnya.

Arc Consistency (AC-3)

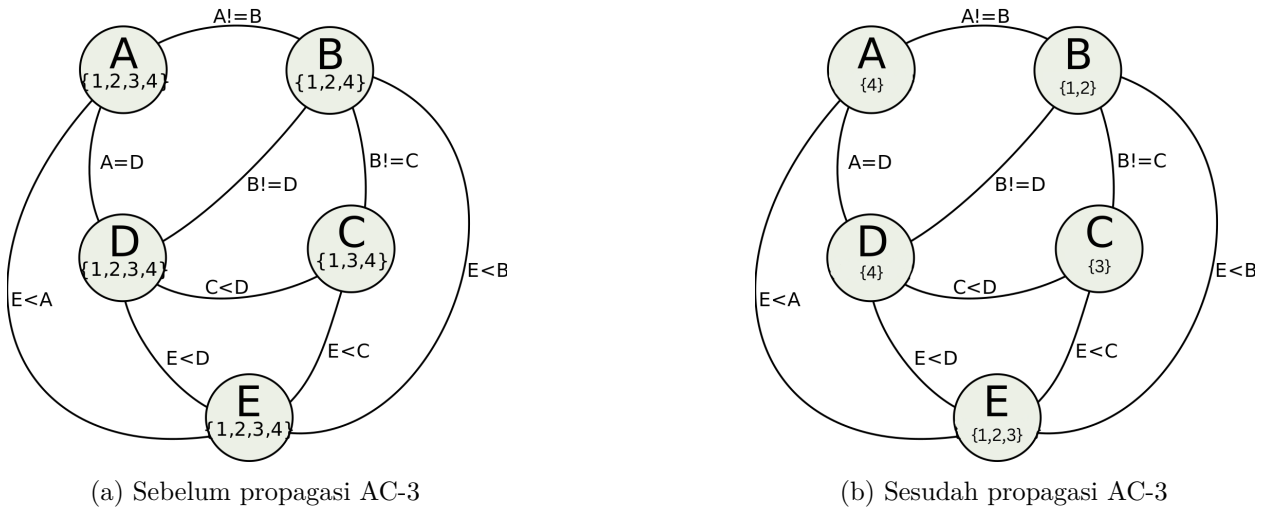
Sebelum membahas *arc consistency* secara formal, perlu diperkenalkan istilah *arc*. Dalam konteks CSP, sebuah *arc* merupakan pasangan terurut dua variabel (x_i, x_j) yang terhubung oleh suatu kendala. Istilah ini berasal dari representasi CSP sebagai *constraint graph*, di mana setiap variabel menjadi simpul dan setiap kendala biner menjadi sisi berarah antara dua simpul. Sebuah kendala biner antara x_i dan x_j menghasilkan dua *arc*, yaitu (x_i, x_j) yang merepresentasikan arah pemeriksaan dari x_i ke x_j , dan (x_j, x_i) untuk arah sebaliknya. Pemeriksaan pada arah-arrah ini dilakukan secara independen karena penghapusan nilai dari domain salah satu variabel dapat memengaruhi konsistensi variabel lain, namun tidak sebaliknya.

Sebelum membahas AC-3 secara spesifik, perlu diperkenalkan terlebih dahulu konsep umum tentang tingkat konsistensi (*level of consistency*) dalam CSP. Konsistensi sebuah CSP dapat didefinisikan pada beberapa tingkat, masing-masing memberikan jaminan yang berbeda terhadap kemungkinan keberadaan solusi [14]. Tingkat paling sederhana adalah *node consistency* (1-consistency), yaitu kondisi di mana setiap nilai dalam domain setiap variabel memenuhi seluruh kendala unary yang melibatkan variabel tersebut. Tingkat berikutnya adalah *arc consistency* (2-consistency), yang menjadi fokus pembahasan pada bagian ini. Tingkat yang lebih tinggi mencakup *path consistency* (3-consistency) yang memeriksa konsistensi antara tiga variabel sekaligus, dan secara umum *k-consistency* yang memeriksa konsistensi antara k variabel.

Secara umum, semakin tinggi tingkat konsistensi yang ditegakkan, semakin besar pemangkasan domain yang dapat dicapai, namun juga semakin besar *overhead* komputasional yang diperlukan. Pada praktik, *arc consistency* merupakan keseimbangan yang baik antara kekuatan pemangkasan dan efisiensi komputasional, sehingga menjadi pilihan standar untuk diintegrasikan dengan *backtracking search* [14]. *Forward checking* yang telah dibahas pada bagian sebelumnya dapat dipandang sebagai bentuk parsial dari *arc consistency* karena memeriksa konsistensi *arc* antara variabel yang baru ditugaskan dengan variabel-variabel lainnya, tetapi tidak memeriksa konsistensi *arc* antarsesama variabel yang belum ditugaskan. AC-3 melengkapi pemeriksaan ini dengan menegakkan konsistensi *arc* di seluruh CSP, sehingga memberikan pemangkasan yang jauh lebih agresif.

Arc consistency adalah properti konsistensi yang lebih kuat dibandingkan yang ditegakkan oleh *forward checking*. Sebuah *arc* (x_i, x_j) dalam CSP dikatakan *arc-consistent* apabila untuk setiap nilai $v \in D_i$, terdapat setidaknya satu nilai $w \in D_j$ sedemikian sehingga penugasan $x_i = v$ dan $x_j = w$ memenuhi semua kendala di antara x_i dan x_j [15]. Nilai w tersebut disebut sebagai *support* untuk v pada *arc* (x_i, x_j) . Apabila suatu nilai $v \in D_i$ tidak memiliki *support* pada D_j , maka v dapat dihapus dari D_i tanpa menghilangkan solusi valid manapun, karena v dipastikan tidak dapat menjadi bagian dari solusi selama domain x_j tetap seperti saat ini. Gambar 2.5 merupakan contoh

- 1 pemangkasan domain solusi yang dapat terjadi saat AC-3 dijalankan pada suatu permasalahan
- 2 CSP.



Gambar 2.5: Contoh propagasi AC-3 pada *constraint graph* dengan lima variabel. Sebelum propagasi (a), domain variabel masih lebar. Setelah AC-3 diterapkan (b), nilai-nilai yang tidak memiliki *support* dieliminasi, menghasilkan domain yang jauh lebih kecil.³

- 3 Properti *arc consistency* yang telah didefinisikan di atas berlaku untuk satu *arc* secara individual.
- 4 Apabila pemeriksaan ini diterapkan pada *seluruh* pasangan variabel dalam CSP, maka CSP tersebut
- 5 dikatakan memenuhi *arc consistency* secara menyeluruh, yaitu tidak ada satu pun nilai dalam
- 6 domain manapun yang tidak memiliki *support* pada variabel lain. Perbedaan utama dengan *forward*
- 7 *checking* terletak pada cakupan pemeriksaan, di mana *forward checking* hanya memeriksa konsistensi
- 8 antara variabel yang baru ditugaskan dengan variabel lain, sedangkan *arc consistency* memeriksa
- 9 konsistensi antara *seluruh* pasangan variabel, termasuk antarsesama variabel yang belum ditugaskan
- 10 [6].

- 11 Algoritma AC-3 (*Arc Consistency Algorithm #3*), yang diperkenalkan oleh Mackworth [15],
- 12 merupakan algoritma standar untuk menegakkan *arc consistency*. AC-3 mengelola sebuah antrean
- 13 Q yang berisi seluruh *arc* yang perlu diperiksa konsistensinya. Untuk setiap *arc* (x_i, x_j) yang
- 14 diambil dari antrean, fungsi REVISE memeriksa apakah ada nilai dalam D_i yang tidak memiliki
- 15 *support* di D_j ; nilai-nilai tanpa *support* dihapus dari D_i . Apabila D_i berubah (ada penghapusan),
- 16 maka seluruh *arc* (x_k, x_i) untuk setiap tetangga x_k dari x_i (selain x_j) ditambahkan kembali ke
- 17 antrean, karena penghapusan nilai dari D_i berpotensi membuat beberapa nilai di D_k kehilangan
- 18 *support*-nya. Proses berlanjut hingga antrean kosong (*arc consistency* tercapai) atau domain suatu
- 19 variabel menjadi kosong (inkonsistensi terdeteksi). Pseudocode AC-3 sebagaimana diperkenalkan
- 20 oleh Mackworth [15] diperlihatkan pada Listing 3.

- 21 Lebih luas lagi, AC-3 merupakan bagian dari keluarga algoritma *arc consistency* yang telah
- 22 dikembangkan sejak Mackworth pertama kali memformalkan konsep tersebut. Algoritma sebelumnya,
- 23 yaitu AC-1 dan AC-2, juga diperkenalkan oleh Mackworth pada makalah yang sama [15], namun
- 24 keduanya memiliki kompleksitas yang lebih buruk dibandingkan AC-3 karena tidak memanfaatkan
- 25 informasi tentang *arc* mana yang perlu diperiksa ulang setelah suatu pemangkasan. AC-1 secara

³Diambil dari https://www.boristhebrave.com/wp-content/uploads/2021/08/arc_consistency_5var_diagram.svg dan diadaptasi untuk menunjukkan hasil AC-3, diakses ulang pada 1 Juni 2026

1 naif memeriksa seluruh *arc* pada setiap iterasi, sedangkan AC-2 memerlukan struktur antrean yang
 2 lebih kompleks. AC-3 mencapai keseimbangan yang lebih baik dengan memelihara antrean tunggal
 3 yang berisi *arc* yang perlu diperiksa ulang. Pengembangan selanjutnya menghasilkan algoritma
 4 seperti AC-4 yang mencapai kompleksitas waktu optimal $O(cd^2)$ melalui pemeliharaan struktur data
 5 tambahan yang menyimpan informasi *support* secara eksplisit [16], serta varian-varian lain yang
 6 merupakan optimasi lebih lanjut. Pertimbangan praktis seperti kesederhanaan kode dan *overhead*
 7 per-operasi yang rendah pada akhirnya menjadikan AC-3 sebagai standar de facto untuk teknik
 8 propagasi *arc consistency* dalam implementasi nyata.

Algorithm 3 Algoritma AC-3

```

1: function AC3(csp)
2:    $Q \leftarrow$  seluruh busur  $(X_i, X_j)$  dalam csp
3:   while  $Q$  tidak kosong do
4:      $(X_i, X_j) \leftarrow \text{REMOVEFROMQUEUE}(Q)$ 
5:     if REVISE(csp,  $X_i, X_j$ ) then
6:       if  $D_{X_i}$  kosong then
7:         return false
8:       end if
9:       for setiap  $X_k$  tetangga  $X_i$ ,  $X_k \neq X_j$  do
10:        Tambahkan  $(X_k, X_i)$  ke  $Q$ 
11:      end for
12:    end if
13:  end while
14:  return true
15: end function

16: function REVISE(csp,  $X_i, X_j$ )
17:  revised  $\leftarrow$  false
18:  for setiap  $v \in D_{X_i}$  do
19:    if tidak ada nilai  $w \in D_{X_j}$  yang memenuhi kendala  $(X_i, X_j)$  then
20:      Hapus  $v$  dari  $D_{X_i}$ 
21:      revised  $\leftarrow$  true
22:    end if
23:  end for
24:  return revised
25: end function

```

9 Kompleksitas waktu AC-3 adalah $O(cd^3)$, di mana c adalah jumlah kendala (*arc*) dan d adalah
 10 ukuran maksimum domain [15]. Analisisnya adalah setiap *arc* dapat dimasukkan ke dalam antrean
 11 paling banyak d kali (karena setiap kali *arc* diproses ulang, setidaknya satu nilai telah dihapus
 12 dari domain terkait), dan setiap pemanggilan fungsi REVISE memerlukan waktu $O(d^2)$ untuk
 13 memeriksa setiap pasangan nilai. Meskipun terdapat algoritma yang lebih efisien seperti AC-4
 14 dengan kompleksitas optimal $O(cd^2)$ [16], AC-3 tetap menjadi pilihan yang paling banyak digunakan
 15 dalam praktik karena kesederhanaan implementasinya dan *overhead* per-operasi yang rendah.

16 Dalam konteks penyelesaian CSP berbasis *backtracking*, AC-3 digunakan sebagai teknik propagasi
 17 yang dijalankan setelah *forward checking* pada setiap langkah penugasan. Pendekatan gabungan ini,
 18 yang sering disebut *Maintaining Arc Consistency* (MAC), yang bekerja sebagai berikut [14]: setelah
 19 sebuah variabel diberi nilai dan *forward checking* menghapus nilai-nilai yang secara langsung tidak

konsisten dari domain variabel-variabel yang belum ditugaskan, AC-3 dijalankan pada seluruh *arc* di antara variabel-variabel yang belum ditugaskan untuk mendeteksi dan mengeliminasi inkonsistensi transitif yang tidak terdeteksi oleh *forward checking*. Apabila proses propagasi ini menghasilkan *domain wipeout* pada salah satu variabel, *backtrack* dilakukan segera. Kombinasi *backtracking* + *forward checking* + AC-3 inilah yang menjadi fondasi solver sistematis pada tugas akhir ini dan implementasinya akan dibahas secara rinci pada Bab 5.

2.4.2 Metaheuristic

Konsep Umum Metaheuristic

Metaheuristic adalah kerangka kerja algoritma tingkat tinggi yang dirancang untuk memandu proses pencarian solusi pada masalah optimasi yang ruang pencariannya terlalu besar atau terlalu kompleks untuk dijelajahi secara sistematis [17]. Istilah ini berasal dari gabungan kata Yunani *meta* (“di atas”) dan *heuriskein* (“menemukan”), yang secara harfiah berarti “strategi pencarian tingkat tinggi.” Berbeda dengan algoritma sistematis yang dibahas pada Bagian 2.4.1, *metaheuristic* bersifat *incomplete*, yang berarti tidak ada jaminan bahwa solusi optimal, atau bahkan solusi valid sekalipun, akan selalu ditemukan. Sebagai gantinya, *metaheuristic* menawarkan kemampuan untuk menavigasi ruang solusi yang sangat besar secara efisien, dengan harapan menemukan solusi yang cukup baik dalam waktu yang jauh lebih singkat dibandingkan pencarian eksak.

Karakteristik utama yang membedakan *metaheuristic* dari pencarian lokal sederhana (*simple local search*) adalah adanya mekanisme untuk menghindari jebakan *local optimum* [17]. Pencarian lokal sederhana, seperti *hill climbing*, rentan terjebak pada solusi yang optimal secara lokal tetapi bukan optimal secara global, karena algoritma berhenti begitu tidak ada tetangga yang lebih baik dari solusi saat ini. *Metaheuristic* mengatasi keterbatasan ini melalui berbagai mekanisme, antara lain penerimaan solusi yang sementara lebih buruk (seperti pada *Simulated Annealing*), penggunaan memori terhadap solusi-solusi yang telah dikunjungi (seperti pada *Tabu Search*), atau pemanfaatan populasi solusi yang berinteraksi satu sama lain (seperti pada *Genetic Algorithm* dan *Particle Swarm Optimization*).

Secara umum, *metaheuristic* dapat diklasifikasikan berdasarkan beberapa dimensi [17]:

- Berdasarkan jumlah solusi aktif yang dikelola.** Beberapa *metaheuristic* bekerja dengan satu solusi tunggal yang dimodifikasi secara iteratif, seperti *Simulated Annealing* dan *Tabu Search*. Pendekatan lain bekerja dengan sekumpulan solusi yang berevolusi secara paralel, seperti *Genetic Algorithm* (GA) dan *Particle Swarm Optimization* (PSO). Pendekatan berbasis populasi memiliki keunggulan natural dalam hal eksplorasi karena memelihara diversitas solusi di dalam populasi.
- Berdasarkan keseimbangan strategi pencarian.** Setiap *metaheuristic* harus menyeimbangkan dua kecenderungan yang saling melengkapi, yaitu *eksplorasi*, yang berarti penjelajahan area baru dalam ruang solusi, dan *eksploitasi*, yang merupakan intensifikasi pencarian di sekitar solusi-solusi terbaik yang telah ditemukan. Penekanan yang berlebihan pada eksplorasi menyebabkan pencarian menjadi acak dan tidak efisien, sementara penekanan yang berlebihan pada eksploitasi menyebabkan konvergensi prematur ke *local optimum*.
- Berdasarkan sumber inspirasi.** Banyak *metaheuristic* terinspirasi dari fenomena biologis atau fisika, seperti evolusi genetik (GA), perilaku kawanan (PSO), dan pendinginan logam

(*Simulated Annealing*). Sebagian lainnya dirancang murni berdasarkan pertimbangan matematis tanpa analogi alam. Pada akhirnya, efektivitas algoritma bergantung pada mekanisme komputasional yang mendasarinya, bukan pada kesetiaan terhadap analogi biologis.

Dalam konteks penyelesaian *Constraint Satisfaction Problem* (CSP), penggunaan *metaheuristic* memerlukan perubahan cara pandang dari “cari penugasan yang memenuhi seluruh kendala” menjadi “optimasi fungsi objektif yang mengukur kualitas suatu kandidat solusi” [18]. Dengan kata lain, CSP ditransformasikan menjadi *Constraint Optimization Problem* (COP) di mana fungsi *fitness* mengevaluasi seberapa dekat suatu kandidat dengan solusi valid. Fungsi *fitness* dapat diformulasikan dalam dua arah, yaitu sebagai fungsi yang *diminimalkan* (misalnya, jumlah pelanggaran kendala, di mana solusi valid bernilai nol) atau sebagai fungsi yang *dimaksimalkan* (misalnya, jumlah kendala yang dipenuhi, di mana solusi valid bernilai maksimum). Reformulasi ini memungkinkan penerapan *metaheuristic* yang pada awalnya dirancang untuk masalah optimasi kontinu ke permasalahan kombinatorik diskrit seperti *Colored Queens*, meskipun diperlukan adaptasi representasi solusi dan operator pencarian sebagaimana akan dibahas pada bagian berikutnya.

Konsekuensi dari reformulasi ini adalah bahwa algoritma *metaheuristic* memperlakukan permasalahan sebagai optimasi *black-box*, yang berarti fungsi *fitness* hanya mengomunikasikan kualitas suatu kandidat solusi secara agregat, tanpa memberikan informasi struktural mengenai kendala mana yang dilanggar atau bagaimana suatu perubahan pada satu variabel memengaruhi variabel lainnya secara eksplisit. Berbeda dengan algoritma sistematis yang secara eksplisit mengeksplorasi struktur kendala melalui propagasi, *metaheuristic* mengandalkan mekanisme pencarian populasi untuk menavigasi ruang solusi tanpa pengetahuan struktural tersebut [18].

Pemilihan antara pendekatan sistematis dan pendekatan *metaheuristic* untuk menyelesaikan suatu CSP melibatkan pengorbanan (*trade-off*) yang harus dipertimbangkan secara cermat [19]. Pendekatan sistematis menawarkan jaminan kelengkapan, di mana apabila solusi ada, algoritma akan menemukannya dalam waktu terbatas; apabila tidak ada solusi, algoritma akan membuktikan ketiadaan tersebut. Jaminan ini sangat berharga pada permasalahan dengan kendala keras (*hard constraints*) di mana solusi parsial atau solusi yang melanggar sebagian kendala tidak dapat diterima. Namun, jaminan kelengkapan tersebut datang dengan biaya kompleksitas waktu yang dapat menjadi eksponensial pada kasus terburuk.

Pendekatan *metaheuristic*, sebaliknya, mengorbankan jaminan kelengkapan demi efisiensi dan skalabilitas. Karena tidak melakukan pencarian sistematis, *metaheuristic* dapat menjelajahi sebagian dari ruang solusi yang sangat besar dalam waktu yang relatif singkat. Pendekatan ini sangat cocok untuk permasalahan dengan kendala lunak (*soft constraints*) di mana “solusi yang cukup baik” lebih diutamakan daripada “solusi yang sempurna,” atau untuk permasalahan dengan ruang solusi yang sangat besar sehingga pencarian sistematis menjadi tidak praktis [17]. Dalam konteks CSP, *metaheuristic* biasanya tidak dianggap sebagai pendekatan utama, namun dapat menjadi alternatif yang menarik ketika kombinasi propagasi kendala dan *backtracking* tidak mampu mencapai konvergensi dalam waktu yang dapat diterima.

Penting untuk dicatat bahwa hasil teoretis yang dikenal sebagai *No Free Lunch Theorem* [19] menyatakan bahwa tidak ada satu algoritma optimasi yang secara universal lebih unggul dibandingkan algoritma lainnya pada seluruh kelas permasalahan. Efektivitas suatu *metaheuristic* pada permasalahan tertentu bergantung pada seberapa baik mekanisme pencariannya selaras

dengan struktur permasalahan tersebut. Oleh karena itu, evaluasi empiris pada permasalahan spesifik—seperti yang dilakukan pada tugas akhir ini untuk *Colored Queens*—merupakan langkah esensial untuk menentukan apakah suatu *metaheuristic* merupakan pilihan yang tepat untuk konteks tersebut.

Particle Swarm Optimization (PSO)

Particle Swarm Optimization (PSO) adalah algoritma optimasi berbasis populasi yang diperkenalkan oleh Kennedy dan Eberhart pada tahun 1995 [8]. Algoritma ini terinspirasi dari perilaku sosial kawanan burung (*bird flocking*) atau gerombolan ikan (*fish schooling*) dalam mencari sumber makanan. Dalam PSO, setiap solusi kandidat disebut *particle* (partikel), dan kumpulan partikel-partikel tersebut disebut *swarm* (kawanan). Setiap partikel menavigasi ruang solusi dengan kecepatan (*velocity*) yang disesuaikan berdasarkan pengalaman individu dan pengalaman sosial dari kawanan.

PSO pada Ruang Kontinu. Dalam formulasi standar untuk ruang kontinu, setiap partikel i memiliki dua atribut utama, yaitu posisi $\vec{X}_i$ yang merepresentasikan solusi kandidat dalam ruang pencarian berdimensi d , dan *velocity* $\vec{V}_i$ yang menentukan arah dan jarak perpindahan partikel pada setiap iterasi. Selain itu, setiap partikel menyimpan *personal best* ($pBest_i$), yaitu posisi terbaik yang pernah dicapai oleh partikel tersebut selama proses pencarian, serta mengacu pada posisi terbaik yang dicapai oleh tetangga-tetangganya, yang disebut *neighborhood best* ($nBest$) pada topologi lokal atau *global best* ($gBest$) pada topologi global [8].

Pada setiap iterasi, *velocity* dan posisi setiap partikel diperbarui menggunakan Persamaan 2.1 dan 2.2 [8]:

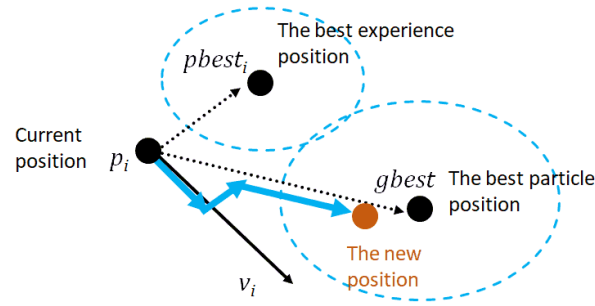
$$\vec{V}_i^{k+1} = \omega \cdot \vec{V}_i^k + c_1 \cdot r_1 \cdot (pBest_i - \vec{X}_i^k) + c_2 \cdot r_2 \cdot (nBest - \vec{X}_i^k) \quad (2.1)$$

$$\vec{X}_i^{k+1} = \vec{X}_i^k + \vec{V}_i^{k+1} \quad (2.2)$$

di mana $\vec{V}_i^k$ adalah *velocity* partikel i pada iterasi ke- k , $\vec{X}_i^k$ adalah posisi partikel i pada iterasi ke- k , ω adalah *inertia weight* yang mengontrol momentum partikel, c_1 adalah koefisien kognitif yang mengontrol pengaruh *personal best*, c_2 adalah koefisien sosial yang mengontrol pengaruh *neighborhood best*, serta r_1 dan r_2 adalah bilangan acak dalam rentang $[0, 1]$ yang memberikan komponen stokastik pada pencarian.

Persamaan 2.1 menggabungkan tiga komponen pergerakan. Pertama, *komponen inersia* ($\omega \cdot \vec{V}_i^k$), yaitu kecenderungan partikel untuk terus bergerak ke arah yang sama dengan langkah sebelumnya. Kedua, *komponen kognitif* ($c_1 \cdot r_1 \cdot (pBest_i - \vec{X}_i^k)$), yaitu tarikan menuju posisi terbaik yang pernah ditemukan oleh partikel itu sendiri. Ketiga, *komponen sosial* ($c_2 \cdot r_2 \cdot (nBest - \vec{X}_i^k)$), yaitu tarikan menuju posisi terbaik yang ditemukan oleh tetangga partikel. Interaksi ketiga komponen ini menghasilkan keseimbangan antara eksplorasi dan eksploitasi: komponen inersia mendorong eksplorasi area baru, sedangkan komponen kognitif dan sosial mendorong eksploitasi area yang menjanjikan. Ilustrasi dari proses ini ditunjukkan pada Gambar 2.6.

⁴Diambil dari <https://www.baeldung.com/wp-content/uploads/sites/4/2022/10/ps0-vectors.png>, diakses ulang pada 1 Juni 2026



Gambar 2.6: Ilustrasi perpindahan satu partikel berdasarkan *velocity* baru yang dipengaruhi oleh *personal best* dan *global best*.⁴

1 Topologi Kawan PSO

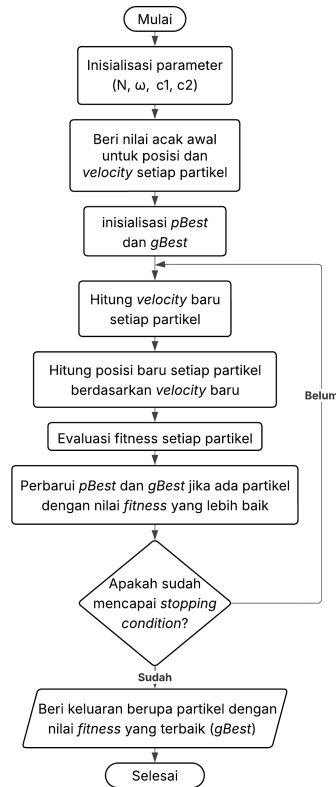
Pemilihan topologi *neighborhood* memiliki implikasi yang signifikan terhadap dinamika informasi dalam *swarm* [8]. Pada topologi *gBest*, informasi mengenai posisi terbaik menyebar secara instan ke seluruh kawanan, sehingga konvergensi terjadi dengan cepat. Namun, pola penyebaran informasi yang seragam ini juga berarti bahwa seluruh kawanan akan tertarik ke arah yang sama secara bersamaan, yang dapat menyebabkan kehilangan diversitas dan terjebak pada *local optimum* apabila *gBest* awal tidak terletak di daerah pencarian yang menjanjikan. Topologi *lBest*, sebaliknya, memperlambat penyebaran informasi dengan membatasi komunikasi pada *neighborhood* lokal. Akibatnya, beberapa subkawanan dapat menjelajahi area pencarian yang berbeda secara paralel sebelum informasi menyatu, sehingga meningkatkan kemungkinan menemukan *global optimum* pada permasalahan dengan banyak *local optima*.

Berbagai konfigurasi *neighborhood* telah diusulkan dalam literatur PSO. Topologi *ring* menempatkan partikel-partikel dalam susunan melingkar di mana setiap partikel hanya berkomunikasi dengan dua tetangga langsungnya. Topologi *star* mirip dengan *gBest* di mana semua partikel berkomunikasi dengan satu partikel pusat. Topologi *von Neumann* membentuk struktur kisi (*grid*) di mana setiap partikel berkomunikasi dengan tetangga di atas, bawah, kiri, dan kanannya [20]. Pilihan topologi yang tepat bergantung pada karakteristik permasalahan, di mana untuk permasalahan dengan ruang pencarian yang kompleks dan banyak *local optima*, topologi yang lebih jarang umumnya memberikan kinerja yang lebih baik karena memungkinkan eksplorasi yang lebih beragam.

Secara keseluruhan, alur kerja algoritma PSO dapat dirangkum dalam diagram alir pada Gambar 2.7. Proses ini terdiri atas dua fase utama, yaitu fase inisialisasi yang dijalankan satu kali pada awal eksekusi, dan fase iteratif yang berulang hingga *stopping condition* terpenuhi.

Berikut adalah penjelasan setiap tahap pada Gambar 2.7:

1. **Inisialisasi parameter.** Algoritma dimulai dengan menetapkan parameter-parameter utama, yaitu jumlah partikel N , bobot inersia ω , koefisien kognitif c_1 , dan koefisien sosial c_2 . Pemilihan nilai parameter ini sangat memengaruhi perilaku konvergensi algoritma, sehingga umumnya diatur berdasarkan rekomendasi literatur atau melalui eksperimen pendahuluan [8].
2. **Inisialisasi posisi dan *velocity* acak.** Posisi awal setiap partikel dibangkitkan secara acak dalam ruang pencarian, dan *velocity* awalnya juga diberi nilai acak. Inisialisasi acak ini memastikan partikel-partikel tersebar pada ruang pencarian, mendukung eksplorasi awal yang luas tanpa bias terhadap area tertentu.
3. **Inisialisasi *pBest* dan *gBest*.** Setelah posisi awal ditetapkan, *pBest* setiap partikel diini-



Gambar 2.7: Diagram alir algoritma PSO.

sialisasi sebagai posisi awal partikel tersebut, sedangkan $gBest$ diinisialisasi sebagai posisi terbaik di antara seluruh partikel berdasarkan evaluasi nilai *fitness* awal.

4. **Perhitungan *velocity* baru.** Fase iteratif dimulai dengan menghitung *velocity* baru setiap partikel menggunakan Persamaan 2.1. Perhitungan ini mempertimbangkan tiga komponen yang telah dijelaskan sebelumnya, yaitu komponen inersia, komponen kognitif, dan komponen sosial, dengan faktor acak yang menjaga sifat stokastik algoritma.
5. **Perhitungan posisi baru.** *Velocity* baru yang telah dihitung digunakan untuk memperbarui posisi setiap partikel menggunakan Persamaan 2.2. Posisi baru tersebut merepresentasikan solusi kandidat baru yang akan dievaluasi pada langkah berikutnya.
6. **Evaluasi *fitness*.** Nilai *fitness* setiap partikel dievaluasi menggunakan fungsi objektif yang spesifik untuk permasalahan yang sedang diselesaikan. Fungsi *fitness* mengukur kualitas solusi yang direpresentasikan oleh posisi partikel, di mana nilai yang lebih baik menandakan solusi yang lebih mendekati optimum.
7. **Pembaruan $pBest$ dan $gBest$.** Hasil evaluasi *fitness* dibandingkan dengan $pBest$ setiap partikel dan $gBest$ kawanan. Apabila terdapat partikel dengan nilai *fitness* yang lebih baik dari $pBest$ -nya, $pBest$ tersebut diperbarui dengan posisi saat ini. Demikian pula, apabila terdapat partikel dengan *fitness* yang lebih baik dari $gBest$, nilai $gBest$ diperbarui agar seluruh kawanan dapat mengikutinya pada iterasi berikutnya.
8. **Pengecekan *stopping condition*.** Pada akhir setiap iterasi, algoritma memeriksa apakah kondisi terminasi telah terpenuhi. Kondisi ini umumnya berupa tercapainya jumlah iterasi maksimum, ditemukannya solusi dengan *fitness* optimal, atau terdeteksinya stagnasi, yaitu

tidak adanya perbaikan signifikan selama sejumlah iterasi berturut-turut. Apabila kondisi belum terpenuhi, algoritma kembali ke langkah perhitungan *velocity* dan mengulang siklus iteratif.

9. **Output solusi terbaik.** Setelah *stopping condition* terpenuhi, algoritma mengembalikan partikel dengan nilai *fitness* terbaik, yaitu *gBest*, sebagai solusi keluaran.

Adaptasi PSO untuk Ruang Diskrit. PSO pada awalnya dirancang untuk masalah optimasi dengan ruang solusi kontinu, di mana operasi aritmetika seperti penjumlahan, pengurangan, dan perkalian skalar pada vektor posisi dan *velocity* memiliki interpretasi geometris yang jelas. Namun, pada masalah kombinatorik diskrit seperti *Colored Queens*, ruang solusi terdiri dari kombinasi indeks-indeks diskrit yang tidak dapat dijumlahkan atau dikalikan secara langsung [21].

Secara umum, adaptasi PSO untuk ruang diskrit memerlukan tiga modifikasi utama:

1. **Representasi posisi.** Posisi partikel direpresentasikan sebagai vektor indeks diskrit, bukan vektor bilangan real. Setiap dimensi vektor merepresentasikan pilihan dari sebuah domain diskrit yang berhingga. Adaptasi paling awal diperkenalkan oleh Kennedy dan Eberhart [22] melalui *Binary PSO* (BPSO), yang merepresentasikan posisi sebagai vektor bit dengan setiap dimensi bernilai 0 atau 1. Hu, Eberhart, dan Shi [21] kemudian memperluas pendekatan ini ke representasi permutasi untuk masalah N-Queens, dengan setiap dimensi vektor menyatakan indeks penempatan menteri.
2. **Reinterpretasi velocity.** *Velocity* tidak lagi berarti perpindahan aritmetika, melainkan probabilitas perubahan pada setiap dimensi. Komponen kognitif dan sosial dalam persamaan *velocity* dihitung berdasarkan perbedaan biner (*match/mismatch*) antara posisi saat ini dengan *pBest* dan *nBest*, bukan selisih numerik. Pada BPSO, *velocity* dihitung menggunakan persamaan standar PSO seperti pada Persamaan 2.1, kemudian ditransformasikan menjadi nilai probabilitas dalam rentang $[0, 1]$ menggunakan fungsi *sigmoid*. Pada adaptasi permutasi oleh Hu et al., *velocity* dinormalisasi ke rentang $[0, 1]$ dengan membaginya terhadap nilai maksimum pada partikel.
3. **Mekanisme pembaruan posisi.** Posisi diperbarui bukan dengan penjumlahan vektor, melainkan dengan mekanisme probabilistik, di mana untuk setiap dimensi, apabila bilangan acak lebih kecil dari *velocity* pada dimensi tersebut, posisi diubah; sebaliknya, posisi dipertahankan. Pada BPSO, perubahan tersebut berarti pembalikan nilai bit pada dimensi yang bersangkutan. Pada adaptasi permutasi, perubahan tersebut berarti penyesuaian posisi pada dimensi tersebut agar sesuai dengan posisi pada dimensi yang sama di *nBest*. Pendekatan ini mengubah interpretasi *velocity* dari “jarak perpindahan” pada ruang kontinu menjadi “kemungkinan untuk mengadopsi posisi terbaik yang diketahui” pada ruang diskrit.

Adaptasi-adaptasi ini memungkinkan PSO diterapkan pada permasalahan kombinatorik diskrit seperti *Colored Queens* tanpa mengubah struktur dasar algoritma. Detail spesifik mengenai bagaimana adaptasi ini diimplementasikan untuk *Colored Queens* akan dibahas pada Bab 5.

Parameter PSO dan Pengaruhnya pada Perilaku Algoritma. Kinerja PSO sangat bergantung pada pemilihan nilai parameter yang mengontrol perilaku partikel dan dinamika kawanan. Parameter-parameter tersebut tidak independen satu sama lain, melainkan saling berinteraksi dalam menentukan keseimbangan antara eksplorasi (penjelajahan area baru dalam ruang pencarian) dan

eksploitasi (intensifikasi pencarian di sekitar solusi terbaik yang telah ditemukan) [20]. Bagian ini menjabarkan parameter-parameter utama PSO dan dampaknya terhadap perilaku algoritma, yang menjadi landasan untuk evaluasi empiris terhadap pemilihan nilai parameter pada permasalahan spesifik.

Jumlah partikel (P) menentukan banyaknya solusi kandidat yang dievaluasi secara paralel pada setiap iterasi. Jumlah partikel yang besar meningkatkan kemampuan eksplorasi karena populasi yang lebih besar memiliki probabilitas lebih tinggi untuk menjangkau area-area berbeda dalam ruang pencarian. Pada ruang pencarian yang besar atau memiliki banyak *local optima*, populasi yang besar menjadi krusial untuk meningkatkan kemungkinan setidaknya satu partikel terinisialisasi di dekat solusi global [8]. Namun, peningkatan jumlah partikel memiliki konsekuensi komputasional yang signifikan, di mana setiap iterasi memerlukan pembaruan posisi dan evaluasi *fitness* untuk seluruh partikel, sehingga biaya per iterasi tumbuh secara linear terhadap P . Pada permasalahan dengan struktur *fitness landscape* yang menyulitkan konvergensi, peningkatan P juga dapat mengalami *diminishing returns*, di mana penambahan partikel tidak meningkatkan probabilitas keberhasilan secara proporsional terhadap biaya komputasi yang ditambahkan.

Bobot inersia (ω) mengontrol kontribusi kecepatan partikel pada iterasi sebelumnya terhadap perhitungan kecepatan baru, sebagaimana terlihat pada komponen pertama Persamaan 2.1. Nilai ω yang besar (mendekati 1) menyebabkan partikel cenderung mempertahankan momentumnya, sehingga partikel terus bergerak ke arah yang sama meskipun ada tarikan dari *pBest* dan *nBest*. Karakteristik ini mendukung eksplorasi karena memungkinkan partikel menjelajahi area-area baru tanpa terlalu cepat tertarik ke titik konvergensi, tetapi juga dapat menyebabkan partikel bergerak melampaui (*overshoot*) area yang menjanjikan tanpa sempat mengeksploitasinya. Sebaliknya, nilai ω yang kecil (mendekati 0) mengurangi pengaruh momentum, sehingga partikel lebih responsif terhadap tarikan komponen kognitif dan sosial. Hal ini mendukung eksploitasi karena partikel akan lebih cepat berkonvergensi ke area yang telah teridentifikasi sebagai menjanjikan, namun dengan risiko terjebak pada *local optimum* apabila konvergensi terjadi terlalu dini. Salah satu strategi yang lazim digunakan untuk menyeimbangkan dua perilaku tersebut adalah *linearly decreasing inertia*, yaitu menurunkan nilai ω secara bertahap selama iterasi sehingga algoritma mengeksplorasi luas pada fase awal dan mengeksploitasi intensif pada fase akhir [20].

Koefisien kognitif (c_1) dan *koefisien sosial* (c_2) menentukan kekuatan tarikan partikel terhadap *personal best* (pengalaman individu) dan *neighborhood best* (pengalaman kolektif), sebagaimana terdapat pada komponen kedua dan ketiga Persamaan 2.1. Nilai $c_1 > c_2$ menekankan eksplorasi individu di mana setiap partikel cenderung mengikuti pengalaman pribadinya, yang meningkatkan diversitas tetapi memperlambat konvergensi kolektif. Sebaliknya, $c_1 < c_2$ menekankan konvergensi sosial di mana seluruh kawanan tertarik pada area yang dianggap menjanjikan secara kolektif, yang mempercepat konvergensi tetapi meningkatkan risiko terjebak pada *local optimum* apabila pengalaman kolektif belum mencerminkan area solusi yang benar. Konvensi yang sering digunakan dalam literatur adalah $c_1 = c_2 = 2$, yang menyeimbangkan kedua komponen tersebut [8]. Efek c_1 dan c_2 pada perilaku partikel dapat bergantung pada mekanisme pembaruan posisi yang digunakan, sebagai contoh pada varian PSO yang menggunakan *nBest* sebagai satu-satunya target perpindahan posisi, c_2 memiliki pengaruh langsung yang lebih kuat dibandingkan c_1 yang pengaruhnya tersalur secara tidak langsung melalui besaran kecepatan.

Jumlah *neighborhood* (N_{nh}) pada topologi *lBest* menentukan jumlah kelompok partikel yang masing-masing memiliki *nBest* lokalnya. Parameter ini memiliki beberapa pengaruh terhadap dinamika kawanan. Pertama, jumlah *neighborhood* yang lebih kecil menghasilkan ukuran kelompok yang lebih besar, sehingga setiap *nBest* dipilih dari lebih banyak partikel dan cenderung memiliki nilai *fitness* yang lebih baik. Kedua, jumlah *nBest* yang lebih sedikit berarti lebih sedikit *attractor* dalam ruang pencarian, sehingga pencarian terkonsentrasi pada area yang lebih sempit dan terfokus. Sebaliknya, jumlah *neighborhood* yang besar menghasilkan ukuran kelompok yang kecil dengan banyak *nBest* yang berbeda, yang meningkatkan diversitas pencarian tetapi menurunkan kualitas individual setiap *nBest*. Pemilihan N_{nh} pada akhirnya mengontrol keseimbangan antara konsentrasi pencarian (sedikit *neighborhood* besar) dan diversifikasi pencarian (banyak *neighborhood* kecil), yang efektivitasnya bergantung pada karakteristik *fitness landscape* permasalahan yang sedang diselesaikan.

Bobot fungsi fitness (umumnya dinotasikan sebagai w_1 dan w_2 pada permasalahan dengan dua komponen pelanggaran) tidak termasuk parameter dinamika kawanan, namun memengaruhi perilaku algoritma melalui pembentukan *fitness landscape*. Kedua bobot menentukan kontribusi relatif setiap jenis pelanggaran terhadap nilai *fitness* keseluruhan, sehingga variasi bobot menghasilkan *fitness landscape* yang berbeda. Peningkatan salah satu bobot memperbesar gradien pada jenis pelanggaran yang bersangkutan, mendorong partikel untuk memprioritaskan pengurangan pelanggaran tersebut selama pencarian. Pemilihan bobot yang tidak seimbang dapat membantu pada kasus di mana satu jenis pelanggaran lebih informatif untuk arah pencarian dibandingkan yang lain. Namun, ketidakseimbangan yang berlebihan juga dapat mengarahkan pencarian ke konfigurasi yang bebas dari satu jenis pelanggaran tetapi masih memiliki pelanggaran jenis lain, sehingga tidak membantu konvergensi menuju solusi valid pada CSP dengan kendala keras di mana seluruh jenis pelanggaran harus dieliminasi secara absolut.

Kondisi Terminasi dan Stagnasi. Algoritma PSO bersifat iteratif dan secara prinsip dapat berjalan tanpa batas waktu, sehingga diperlukan kondisi terminasi eksplisit untuk menghentikan eksekusi. Pada implementasi praktis untuk penyelesaian CSP, terdapat tiga kondisi terminasi yang lazim digunakan secara bersamaan [20]. Kondisi pertama, dan yang paling diinginkan, adalah *tercapainya solusi valid*, yaitu ketika nilai *fitness* terbaik di antara seluruh partikel mencapai nol, yang menandakan tidak adanya pelanggaran kendala. Kondisi ini merepresentasikan keberhasilan algoritma dan menghasilkan keluaran berupa posisi partikel yang merepresentasikan solusi valid.

Kondisi kedua adalah *tercapainya batas iterasi maksimum*, yang berfungsi sebagai jaminan terminasi pada kasus di mana algoritma tidak berhasil menemukan solusi valid. Tanpa batas iterasi, algoritma berpotensi berjalan tanpa henti pada permasalahan yang sulit, sehingga batas ini menjadi mekanisme keselamatan komputasional yang penting. Nilai batas iterasi harus dipilih cukup besar agar algoritma memiliki kesempatan yang wajar untuk berkonvergensi, namun tidak terlalu besar sehingga waktu eksekusi pada kasus gagal tetap dalam batas yang dapat diterima.

Salah satu fenomena yang sering ditemui pada eksekusi PSO adalah stagnasi (*stagnation*), yaitu kondisi di mana *swarm* berhenti memberikan perbaikan signifikan terhadap nilai *fitness* terbaik meskipun iterasi terus berlangsung. Stagnasi umumnya terjadi akibat *premature convergence*, di mana seluruh partikel telah berkumpul di sekitar satu titik yang merupakan *local optimum*, sehingga partikel-partikel menjadi terlalu mirip satu sama lain. Akibatnya, perbedaan antara *pBest* dan

1 posisi saat ini, serta antara $nBest$ dan posisi saat ini, menjadi sangat kecil, dan komponen kognitif
2 maupun sosial pada Persamaan 2.1 tidak lagi memberikan dorongan pergerakan yang berarti. *Swarm*
3 pun terjebak pada area sempit tanpa kemampuan untuk keluar dari *basin of attraction* tersebut
4 [20].

5 Untuk mengatasi stagnasi, berbagai teknik telah diusulkan dalam literatur, antara lain reinisiali-
6 sasi sebagian partikel ketika stagnasi terdeteksi (*re-randomization*), penambahan *noise* pada *velocity*
7 untuk mempertahankan diversitas, dan penggunaan strategi parameter yang adaptif sebagaimana
8 telah dibahas pada bagian sebelumnya [20]. Pada implementasi praktis, deteksi stagnasi biasanya
9 dilakukan dengan menetapkan batas iterasi tanpa perbaikan; apabila tersebut tercapai, algoritma
10 melakukan tindakan adaptasi atau menghentikan eksekusi. Pemilihan batas stagnasi melibatkan
11 *trade-off* sederhana, di mana batas yang terlalu rendah menghentikan eksekusi prematur, sedangkan
12 batas yang terlalu tinggi membiarkan iterasi yang tidak produktif terus berjalan.

BAB 3

ANALISIS

3.1 Analisis Masalah

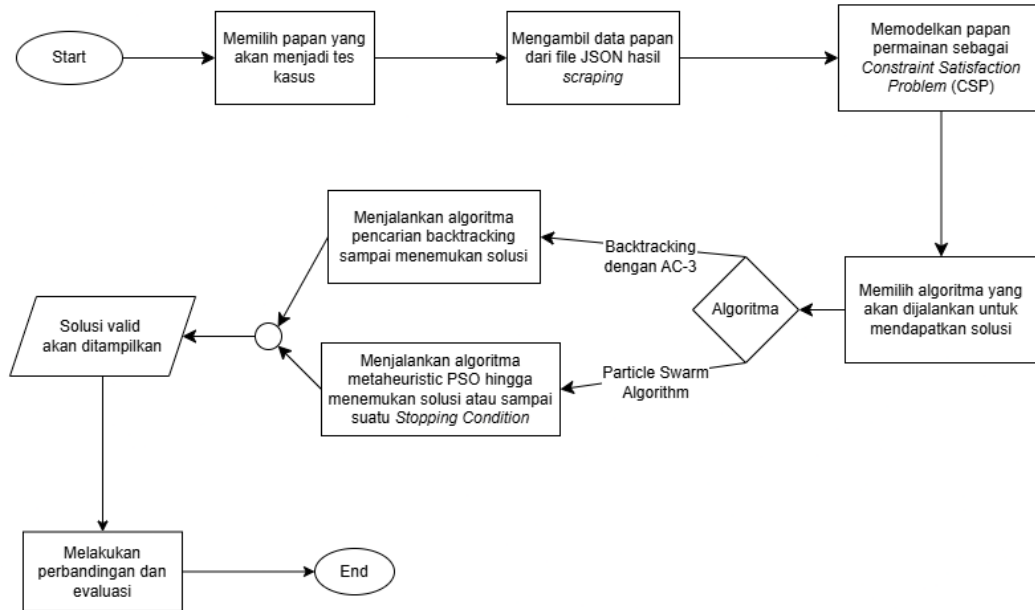
Tugas akhir ini bertujuan untuk merancang, mengimplementasikan, dan membandingkan dua pendekatan algoritmik dalam menyelesaikan permasalahan *Colored Queens*: pendekatan berbasis pencarian sistematis menggunakan *Backtracking* yang diperkuat dengan *Forward Checking* dan AC-3 sebagaimana dibahas pada Bagian 2.4.1, serta pendekatan berbasis *metaheuristic* menggunakan *Particle Swarm Optimization* (PSO) diskrit sebagaimana dibahas pada Bagian 2.4.2. Selain kedua *solver* tersebut, Tugas akhir ini juga mencakup perancangan dan implementasi sebuah aplikasi permainan berbasis antarmuka grafis yang berfungsi sebagai sarana bermain *Colored Queens* dan integrasi *solver*.

Pendekatan pertama adalah *Backtracking* yang diperkuat dengan *Forward Checking* dan AC-3 sebagaimana dibahas pada Bagian 2.4.1. Pendekatan ini bersifat sistematis dan lengkap (*complete*), sehingga selalu menemukan solusi apabila solusi tersebut ada. Pendekatan kedua adalah *Particle Swarm Optimization* (PSO) diskrit sebagaimana dibahas pada Bagian 2.4.2, yang merupakan *metaheuristic* berbasis populasi yang tidak menjamin kelengkapan solusi namun berpotensi lebih efisien pada ruang pencarian yang besar. Pemilihan kedua pendekatan ini didasarkan pada kontras paradigmanya, sehingga hasil perbandingan dapat memberikan gambaran yang jelas mengenai *trade-off* antara pencarian deterministik dan stokastik pada permasalahan *Colored Queens*.

Sistem yang dibangun terdiri atas tiga komponen utama. Pertama, sebuah *loader* yang bertugas membaca dan mengurai berkas puzzle dalam format JSON ke dalam struktur papan yang dapat diproses oleh *solver*. Kedua, dua buah *solver* yang mengimplementasikan kedua algoritma pencarian secara terpisah dan independen, sehingga keduanya dapat dijalankan dengan masukan yang sama untuk keperluan eksperimen. Ketiga, sebuah antarmuka pengguna yang dibangun menggunakan Java Swing dan JFrame, yang menampilkan papan permainan secara visual serta memungkinkan pengguna untuk berinteraksi dan memainkan *Colored Queens* secara langsung dan mencoba fitur *hint* dan *auto-solve*.

Mengingat aplikasi permainan memerlukan jaminan bahwa solusi selalu tersedia ketika pengguna menggunakan fitur *hint* atau *auto-solve*, *solver* yang diintegrasikan ke dalam aplikasi adalah *backtracking* dengan *forward checking* dan AC-3. Sifat *complete* dari pendekatan ini menjamin ditemukannya solusi apabila solusi tersebut ada, yang merupakan syarat fungsional yang tidak dapat dipenuhi oleh PSO. *Solver* tersebut dijalankan secara asinkron di latar belakang saat pengguna membuka sebuah puzzle, sehingga solusi tersedia tanpa perlu menyimpan seluruh solusi puzzle secara *hardcoded*. *Solver* PSO diskrit tidak diintegrasikan ke dalam aplikasi, melainkan digunakan

- 1 sebagai pembanding dalam kerangka eksperimen untuk dievaluasi dari segi waktu eksekusi dan
 2 tingkat keberhasilan. Alur kerja penelitian secara keseluruhan diilustrasikan pada Gambar 3.1.



Gambar 3.1: Alur kerja penelitian secara keseluruhan.

Berikut adalah penjelasan setiap proses pada Gambar 3.1:

1. **Memilih papan yang akan menjadi tes kasus.** Tahap pertama adalah menentukan papan puzzle yang akan digunakan sebagai masukan pengujian. Dataset mencakup papan berukuran 7×7 hingga 12×12 dengan masing-masing 250 level, serta papan berukuran 20×20 dan 30×30 dengan masing-masing 1 level. Rincian dataset disajikan pada Bagian 5.2.1.
2. **Mengambil data papan dari berkas hasil *scraping*.** Data puzzle diperoleh dari situs *playqueensgame.com* melalui proses *web scraping* otomatis. Untuk setiap level, posisi dan warna setiap sel pada papan diekstrak dari halaman puzzle, kemudian disimpan dalam berkas berformat JSON. Berkas tersebut menjadi sumber data untuk seluruh tahap berikutnya.
3. **Memodelkan papan permainan sebagai *Constraint Satisfaction Problem (CSP)*.** Data papan yang telah dibaca dimodelkan sebagai CSP dengan mendefinisikan variabel (sektor warna), domain (sel-sel dalam setiap sektor), dan kendala (baris, kolom, *adjacency*). Pemodelan CSP secara lengkap dibahas pada Bagian 3.2.
4. **Memilih algoritma yang akan dijalankan untuk mendapatkan solusi.** Pada tahap ini, salah satu dari dua algoritma dipilih untuk menyelesaikan puzzle. Dalam konteks eksperimen, kedua algoritma dijalankan pada puzzle yang sama untuk keperluan perbandingan.
5. **Menjalankan algoritma pencarian *backtracking* sampai menemukan solusi.** Apabila algoritma *backtracking* dengan *forward checking* dan AC-3 dipilih, *solver* melakukan pencarian sistematis dengan pemangkasan domain hingga solusi valid ditemukan. Sifat deterministik dan kelengkapan (*completeness*) algoritma ini menjamin ditemukannya solusi apabila solusi tersebut ada. Perancangan algoritma dibahas pada Bagian 3.3.
6. **Menjalankan algoritma *metaheuristic* PSO hingga menemukan solusi atau sampai suatu *stopping condition*.** Apabila algoritma PSO diskrit dipilih, *solver* mengeksplorasi

ruang solusi melalui sejumlah partikel yang berinteraksi dalam *neighborhood*. Pencarian berhenti apabila solusi valid ditemukan, batas iterasi tercapai, atau stagnasi terdeteksi. Perancangan algoritma dibahas pada Bagian 3.4.

7. **Solusi valid akan ditampilkan.** Solusi berupa pasangan koordinat penempatan menteri untuk setiap sektor warna dihasilkan oleh solver. Pada konteks aplikasi, solusi dirender secara visual pada papan permainan. Pada konteks eksperimen, solusi beserta metrik kinerja dicatat ke dalam berkas hasil untuk dianalisis lebih lanjut.

8. **Melakukan perbandingan dan evaluasi.** Metrik dari kedua *solver* diintegrasikan dan dibandingkan berdasarkan waktu eksekusi, tingkat keberhasilan, dan karakteristik pencarian pada berbagai ukuran papan dan konfigurasi parameter. Skenario pengujian dibahas pada Bagian 5.2 dan hasil analisis disajikan pada Bab 5.

3.2 Pemodelan Permasalahan *Colored Queens* sebagai CSP

Sebagaimana telah dibahas pada Bagian 2.3, sebuah *Constraint Satisfaction Problem* didefinisikan oleh tiga komponen utama: himpunan variabel, domain masing-masing variabel, dan himpunan kendala yang harus dipenuhi secara bersamaan. Bagian ini menjabarkan secara konkret bagaimana ketiga komponen tersebut didefinisikan dalam konteks permasalahan *Colored Queens*. Pemodelan formal ini menjadi landasan bagi kedua pendekatan penyelesaian yang dirancang pada Bagian 3.3 dan Bagian 3.4.

3.2.1 Representasi Papan

Papan permainan *Colored Queens* dimodelkan sebagai sebuah grid berukuran $n \times n$, di mana n menyatakan jumlah baris, jumlah kolom, sekaligus jumlah sektor warna yang ada pada papan. Setiap sel pada papan diidentifikasi oleh pasangan koordinat (*baris*, *kolom*) dengan indeks berbasis nol, sehingga sel pojok kiri atas memiliki koordinat $(0, 0)$ dan sel pojok kanan bawah memiliki koordinat $(n - 1, n - 1)$. Setiap sel memiliki atribut warna yang menentukan sektor mana sel tersebut termasuk.

Sektor warna didefinisikan sebagai himpunan sel-sel yang memiliki warna sama. Pada papan yang valid, terdapat tepat n sektor warna, dan setiap sel pada papan termasuk dalam tepat satu sektor warna. Bentuk dan luas sektor warna dapat berbeda-beda tergantung pada tata letak papan yang diberikan, sebagaimana telah diperlihatkan pada contoh papan di Bagian 2.2.1.

3.2.2 Definisi Variabel

Mengacu pada kerangka CSP yang dijelaskan pada Bagian 2.3, variabel dalam permasalahan *Colored Queens* didefinisikan sebagai sektor warna pada papan. Secara formal, diberikan himpunan warna $C = \{c_1, c_2, \dots, c_n\}$, maka himpunan variabel didefinisikan pada Persamaan 3.1.

$$X = \{X_{c_1}, X_{c_2}, \dots, X_{c_n}\} \quad (3.1)$$

di mana setiap variabel X_{c_i} merepresentasikan pilihan posisi penempatan menteri untuk sektor warna c_i .

Pemilihan sektor warna sebagai variabel, bukan baris atau kolom seperti pada N-Queens klasik, didasarkan pada struktur kendala *Colored Queens* itu sendiri. Aturan permainan mengharuskan tepat satu menteri ditempatkan pada setiap sektor warna (kendala *one queen per color*), sehingga setiap sektor warna secara alami berkorespondensi dengan satu variabel keputusan. Dengan pemodelan ini, himpunan variabel memiliki kardinalitas yang sama dengan ukuran papan n , dan setiap penugasan lengkap (*complete assignment*) secara otomatis memenuhi kendala jumlah menteri per warna.

3.2.3 Definisi Domain

Domain dari setiap variabel X_{c_i} adalah himpunan semua sel yang termasuk dalam sektor warna c_i , sebagaimana didefinisikan pada Persamaan 3.2.

$$D_{c_i} = \{(r, k) \mid \text{sel } (r, k) \text{ memiliki warna } c_i\} \quad (3.2)$$

Dengan demikian, ukuran domain setiap variabel dapat berbeda-beda tergantung pada bentuk dan luas sektor warna yang bersangkutan pada papan. Tidak seperti N-Queens klasik yang menggunakan representasi permutasi dengan domain seragam $\{1, 2, \dots, n\}$, domain pada *Colored Queens* bersifat heterogen dan bergantung pada tata letak papan yang diberikan.

Selama proses pencarian, status kevalidan setiap sel dalam domain bersifat dinamis. Sel dapat dipangkas dari domain (dianggap tidak lagi valid sebagai kandidat penempatan) oleh mekanisme seperti *forward checking* atau propagasi AC-3, dan dapat dipulihkan kembali ke domain ketika algoritma melakukan *backtrack*. Dengan demikian, domain efektif yang dipertimbangkan oleh algoritma pada suatu titik dalam pencarian merupakan himpunan bagian dari domain awalnya.

3.2.4 Definisi Constraint

Sesuai dengan aturan permainan *Colored Queens* yang dijabarkan pada Bagian 2.2.1, terdapat tiga jenis kendala yang harus dipenuhi secara bersamaan oleh setiap pasangan variabel (X_{c_i}, X_{c_j}) dengan $i \neq j$. Ketiga kendala ini bersifat biner dan membentuk *constraint graph* yang lengkap (*complete*), artinya setiap pasang variabel terhubung oleh kendala.

Kendala Baris. Dua menteri tidak boleh ditempatkan pada baris yang sama. Untuk dua sel (r_1, k_1) dan (r_2, k_2) yang ditugaskan kepada variabel X_{c_i} dan X_{c_j} , kendala ini dinyatakan pada Persamaan 3.3.

$$r_1 \neq r_2 \quad (3.3)$$

Kendala Kolom. Dua menteri tidak boleh ditempatkan pada kolom yang sama, sebagaimana dinyatakan pada Persamaan 3.4.

$$k_1 \neq k_2 \quad (3.4)$$

Kendala Adjacency. Berbeda dari permasalahan N-Queens klasik yang melarang konflik diagonal tak terbatas, *Colored Queens* hanya melarang penempatan dua menteri pada sel yang bertetangga secara langsung dalam delapan arah (horizontal, vertikal, dan diagonal). Dua sel (r_1, k_1) dan (r_2, k_2) dinyatakan bertetangga apabila memenuhi Persamaan 3.5.

$$|r_1 - r_2| \leq 1 \quad \text{dan} \quad |k_1 - k_2| \leq 1 \quad (3.5)$$

Perlu dicatat bahwa tidak ada larangan bagi dua menteri untuk berada pada diagonal yang sama, selama keduanya tidak bertetangga secara langsung. Hal ini merupakan perbedaan struktural yang signifikan dari N-Queens klasik dan menjadi karakteristik utama dari *Colored Queens* sebagaimana dibahas pada Bagian 2.2.1.

Ketiga kendala di atas dapat digabungkan menjadi sebuah definisi konflik tunggal antara dua sel sebagaimana dinyatakan pada Persamaan 3.6. Dua sel dinyatakan berkonflik apabila memenuhi salah satu dari kondisi pada persamaan tersebut.

$$\text{konflik}((r_1, k_1), (r_2, k_2)) \iff (r_1 = r_2) \vee (k_1 = k_2) \vee (|r_1 - r_2| \leq 1 \wedge |k_1 - k_2| \leq 1) \quad (3.6)$$

Pemeriksaan konflik melalui formulasi ini akan digunakan secara konsisten oleh kedua algoritma penyelesaian yang dirancang pada bagian-bagian berikutnya.

3.2.5 Representasi Solusi

Sebuah solusi valid dari permasalahan *Colored Queens* yang dimodelkan sebagai CSP adalah sebuah penugasan lengkap sebagaimana dinyatakan pada Persamaan 3.7.

$$A = \{X_{c_1} \mapsto s_1, X_{c_2} \mapsto s_2, \dots, X_{c_n} \mapsto s_n\} \quad (3.7)$$

di mana $s_i \in D_{c_i}$ untuk setiap i , dan tidak ada pasangan (s_i, s_j) dengan $i \neq j$ yang melanggar kendala konflik yang telah didefinisikan pada Persamaan 3.6.

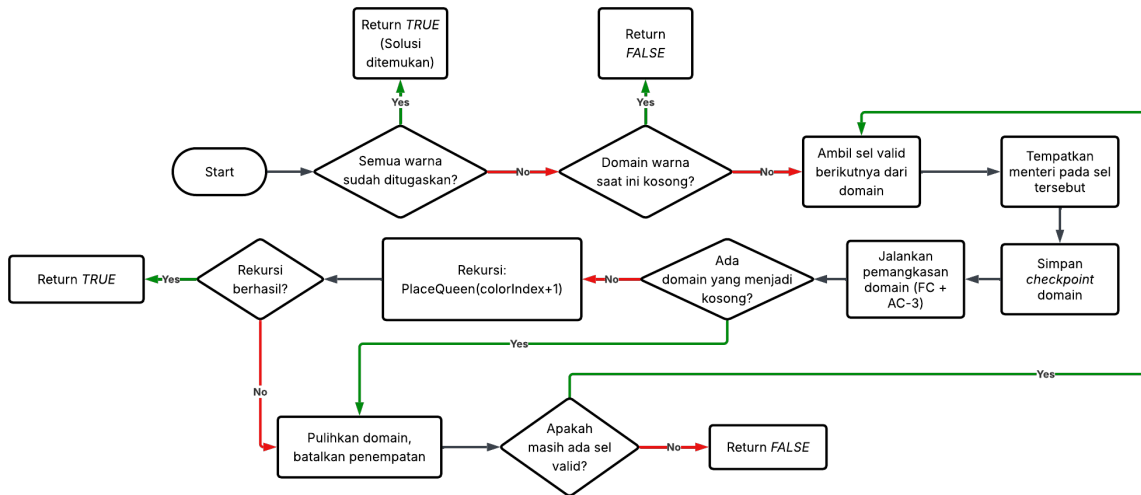
Penugasan lengkap inilah yang menjadi keluaran dari kedua algoritma yang dirancang dalam tugas akhir ini. Pada algoritma *backtracking*, penugasan dibangun secara inkremental melalui penambahan satu variabel pada satu waktu hingga seluruh variabel mendapat nilai. Pada algoritma PSO, setiap partikel pada setiap iterasi merepresentasikan satu kandidat penugasan lengkap, meskipun belum tentu memenuhi seluruh kendala.

3.3 Analisis Algoritma *Backtracking* dengan *Forward Checking* dan AC-3

Bagian ini menjelaskan analisis algoritmik dari pendekatan pertama yang digunakan untuk menyelesaikan permasalahan *Colored Queens*, yaitu *backtracking* sistematis yang diperkuat dengan dua mekanisme pemangkasan domain: *forward checking* dan AC-3. Landasan teori ketiga teknik tersebut telah dibahas pada Bagian 2.4.1 dan Bagian 2.4.1. Analisis disajikan melalui empat diagram alur yang merepresentasikan setiap komponen algoritma secara terpisah, yaitu alur utama rekursif pencarian, prosedur *forward checking*, fungsi *revise* sebagai inti dari konsistensi busur, dan prosedur propagasi AC-3 yang menjalankan propagasi secara iteratif. Rincian implementasi masing-masing komponen dibahas pada Bab 4.

3.3.1 Alur Utama Algoritma *Backtracking*

Alur utama algoritma *backtracking* mengimplementasikan pencarian rekursif *depth-first* yang menggunakan variabel warna satu per satu sesuai urutan yang telah ditentukan. Pada setiap pemanggilan rekursif, satu menteri ditempatkan untuk satu sektor warna, diikuti oleh pemangkasan domain dan pemanggilan rekursif berikutnya. Apabila pencarian pada suatu cabang gagal, algoritma membatalkan penugasan dan mencoba alternatif lain pada level yang sama. Keseluruhan alur diilustrasikan pada Gambar 3.2.



Gambar 3.2: Diagram alur prosedur rekursif pencarian *backtracking*.

Pengecekan Kondisi Terminasi

Setiap pemanggilan rekursif diawali dengan dua pemeriksaan kondisi terminasi. Pemeriksaan pertama, “Semua warna sudah ditugaskan?”, menentukan apakah *base case* pencarian telah tercapai. Kondisi ini terpenuhi ketika seluruh sektor warna telah memiliki menteri yang ditempatkan, sehingga prosedur mengembalikan nilai *true* yang menandakan ditemukannya solusi valid. Pemeriksaan kedua, “Domain warna saat ini kosong?”, memeriksa apakah variabel warna yang sedang diproses tidak memiliki sel valid yang tersisa. Apabila kondisi ini terpenuhi, pencarian pada cabang ini tidak dapat dilanjutkan dan prosedur mengembalikan *false* kepada pemanggilnya sebagai sinyal untuk melakukan *backtrack*.

Pemilihan dan Penempatan Menteri

Apabila kedua kondisi terminasi tidak terpenuhi, algoritma masuk ke dalam perulangan utama yang mengiterasi seluruh sel valid dalam domain variabel saat ini. Dari seluruh sel yang tersedia, algoritma memilih satu sel kandidat berikutnya dan menempatkan menteri pada sel tersebut secara tentatif, artinya penempatan ini dapat dibatalkan apabila pencarian pada cabang ini gagal. Penempatan menteri pada sel tertentu secara langsung membatasi pilihan yang tersedia bagi variabel-variabel yang belum ditugaskan, karena setiap sel yang berkonflik dengan posisi tersebut tidak lagi dapat digunakan.

1 **Penyimpanan *Checkpoint* dan Pemangkasan Domain**

2 Sebelum menjalankan pemangkasan, algoritma mencatat kondisi domain saat ini sebagai *checkpoint*.
3 Mekanisme ini memungkinkan seluruh perubahan domain yang dilakukan pada langkah berikutnya
4 untuk dibatalkan secara efisien apabila pencarian gagal, tanpa perlu menyalin ulang seluruh struktur
5 domain. Setelah *checkpoint* disimpan, algoritma menjalankan dua tahap pemangkasan domain secara
6 berurutan: *forward checking* yang akan dijabarkan pada Bagian 3.3.2, diikuti oleh propagasi AC-3
7 yang akan dijabarkan pada Bagian 3.3.4. Kedua tahap ini secara bersama-sama mengeliminasi sel-sel
8 yang sudah tidak konsisten dari domain seluruh variabel yang belum ditugaskan, mempersempit
9 ruang pencarian sebelum rekursi dilanjutkan.

10 **Evaluasi Pemangkasan dan Pemanggilan Rekursif**

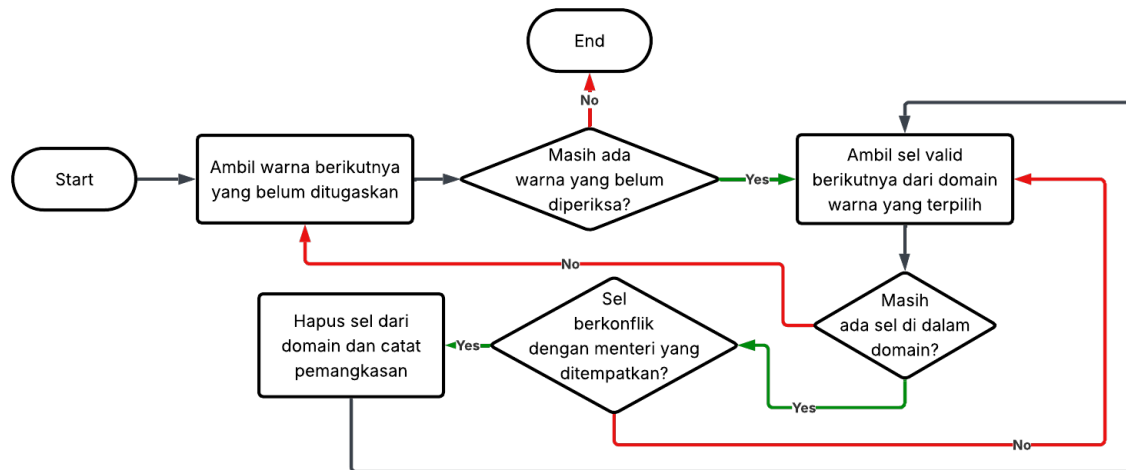
11 Setelah pemangkasan selesai, algoritma memeriksa apakah salah satu variabel yang belum ditugask-
12 an kehilangan seluruh kandidat selnya akibat pemangkasan. Apabila kondisi ini terjadi, cabang
13 pencarian ini dipastikan tidak dapat menghasilkan solusi valid, sehingga algoritma langsung melom-
14 pat ke proses pemulihan tanpa melanjutkan rekursi. Deteksi dini kegagalan ini merupakan salah
15 satu keunggulan utama kombinasi *forward checking* dan AC-3 dibandingkan *backtracking* murni.
16 Apabila tidak ada domain yang kosong, algoritma memanggil dirinya sendiri secara rekursif untuk
17 memproses variabel warna berikutnya. Apabila pemanggilan rekursif tersebut berhasil menemukan
18 solusi, nilai *true* diteruskan kembali ke pemanggil.

19 **Pemulihan dan Eksplorasi Sel Berikutnya**

20 Apabila pemanggilan rekursif gagal atau pemangkasan menyebabkan domain variabel lain menjadi
21 kosong, algoritma melakukan *backtrack* dengan memulihkan seluruh domain ke kondisi tepat sebelum
22 penempatan saat ini, menggunakan informasi yang tersimpan pada *checkpoint*. Penempatan menteri
23 yang gagal dibatalkan, dan sel tersebut dianggap tidak layak untuk iterasi saat ini. Algoritma
24 kemudian memeriksa apakah masih terdapat sel valid lain dalam domain yang belum dicoba.
25 Apabila masih ada, perulangan dilanjutkan untuk mencoba sel berikutnya. Apabila seluruh sel
26 telah dicoba tanpa keberhasilan, prosedur mengembalikan *false* kepada pemanggilnya, yang akan
27 memicu *backtrack* pada level rekursi di atasnya.

28 **3.3.2 *Forward Checking***

29 *Forward checking* merupakan tahap pemangkasan pertama yang dijalankan setiap kali seorang
30 menteri ditempatkan pada papan. Sebagaimana dibahas pada Bagian 2.4.1, mekanisme ini secara
31 reaktif mengeliminasi nilai-nilai domain yang tidak konsisten dengan penugasan yang baru saja
32 dilakukan, sehingga cabang pencarian yang pasti gagal dapat dihindari sedini mungkin. Dalam
33 konteks *Colored Queens*, *forward checking* memangkas seluruh sel yang berkonflik dengan menteri
34 yang baru ditempatkan dari domain variabel-variabel warna yang belum ditugaskan. Alur prosedur
35 ini diperlihatkan pada Gambar 3.3.



Gambar 3.3: Diagram alur prosedur *forward checking*.

1 Iterasi atas Warna yang Belum Ditugaskan

Prosedur dimulai dengan mengiterasi seluruh variabel warna yang belum ditugaskan, yaitu variabel-variabel yang belum memiliki menteri. Iterasi dilakukan secara terurut sesuai urutan pemrosesan variabel yang telah ditetapkan. Apabila seluruh variabel yang belum ditugaskan telah diperiksa, prosedur berakhir dan kendali dikembalikan ke alur utama *backtracking*.

6 Iterasi atas Sel dalam Domain Warna Terpilih

Untuk setiap variabel warna yang diperiksa, prosedur mengiterasi seluruh sel yang masih valid dalam domainnya. Sel yang sebelumnya telah dipangkas oleh penugasan-penugasan sebelumnya tidak diperiksa ulang. Apabila seluruh sel pada domain variabel saat ini telah diperiksa, alur beralih ke variabel warna berikutnya.

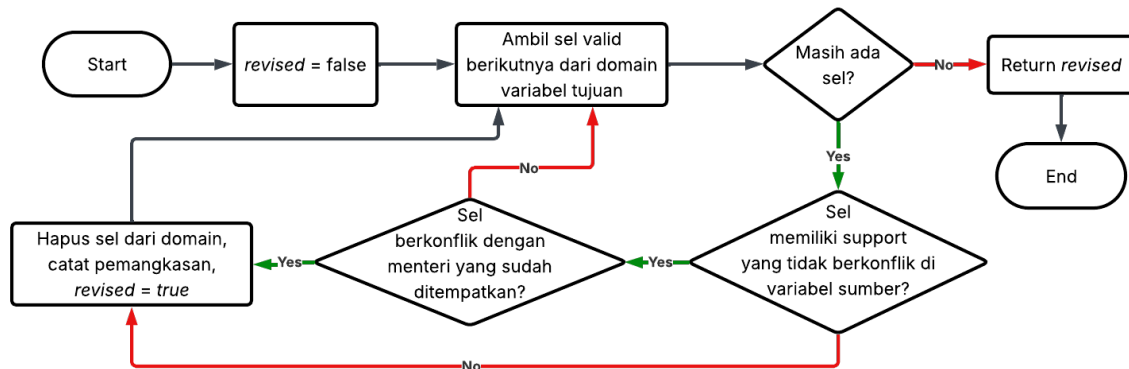
11 Pemeriksaan Konflik dan Pemangkasan Sel

Setiap sel yang diperiksa dievaluasi terhadap posisi menteri yang baru ditempatkan. Pemeriksaan konflik mengacu pada definisi kendala *Colored Queens* yang telah dijabarkan pada Bagian 3.2.4, yang mencakup kesamaan baris, kesamaan kolom, dan ketetangaan dalam delapan arah. Apabila sel tidak berkonflik, sel tersebut tetap berada dalam domain dan alur melanjutkan ke sel berikutnya. Apabila sel berkonflik, sel tersebut dihapus dari domain dan pemangkasan ini dicatat agar dapat dipulihkan apabila algoritma melakukan *backtrack* di kemudian hari.

18 3.3.3 Fungsi *Revise*

Fungsi *revise* merupakan inti dari mekanisme konsistensi busur AC-3. Sebagaimana telah dijabarkan pada Bagian 2.4.1, fungsi ini menerima sepasang variabel sebagai parameter, yaitu variabel sumber dan variabel tujuan, kemudian memeriksa apakah setiap nilai dalam domain variabel tujuan memiliki nilai pendamping yang konsisten pada variabel sumber. Nilai yang tidak memiliki pendamping yang konsisten dianggap tidak *arc-consistent* dan dipangkas dari domain. Dalam konteks *Colored Queens*,

1 fungsi ini juga memperhitungkan posisi menteri yang sudah ditempatkan pada variabel-variabel
 2 yang telah ditugaskan, sehingga setiap kandidat sel tidak hanya konsisten secara berpasangan,
 3 tetapi juga konsisten dengan keseluruhan penugasan parsial yang sedang dibangun. Alur fungsi ini
 4 diperlihatkan pada Gambar 3.4.



Gambar 3.4: Diagram alur fungsi *revise*.

5 Inisialisasi dan Iterasi atas Domain Variabel Tujuan

6 Fungsi dimulai dengan menetapkan penanda *revised* bernilai *false*. Penanda ini akan menjadi nilai
 7 kembalian fungsi yang menunjukkan apakah domain variabel tujuan mengalami perubahan akibat
 8 eksekusi. Selanjutnya, fungsi mengiterasi seluruh sel valid dalam domain variabel tujuan. Apabila
 9 seluruh sel telah diperiksa, fungsi mengembalikan nilai *revised* sebagai hasil akhir.

10 Pemeriksaan Keberadaan *Support* pada Variabel Sumber

11 Untuk setiap sel yang diperiksa pada variabel tujuan, fungsi memeriksa apakah terdapat setidaknya
 12 satu sel pada variabel sumber yang tidak berkonflik dengannya. Sebuah sel dinyatakan memiliki
 13 *support* apabila terdapat minimal satu pasangan nilai yang valid antara kedua variabel. Konflik
 14 antara sepasang sel mengacu pada definisi kendala yang telah dijabarkan pada Bagian 3.2.4. Apabila
 15 tidak ditemukan sel pendukung pada variabel sumber, sel pada variabel tujuan dipastikan tidak
 16 *arc-consistent* dan dipangkas dari domain.

17 Pemeriksaan Konflik dengan Menteri yang Sudah Ditempatkan

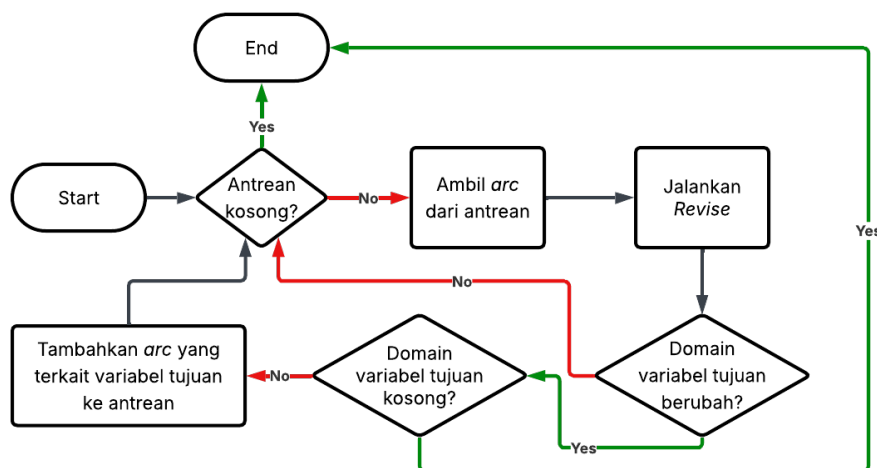
18 Apabila sel memiliki *support* dari variabel sumber, fungsi melanjutkan pemeriksaan terhadap
 19 seluruh menteri yang telah ditempatkan pada variabel-variabel sebelumnya. Langkah tambahan ini
 20 diperlukan karena AC-3 dalam konteks *Colored Queens* tidak hanya menjaga konsistensi antarvariabel
 21 yang belum ditugaskan, tetapi juga memastikan bahwa setiap kandidat sel tetap konsisten dengan
 22 keseluruhan penugasan parsial yang telah dibangun. Apabila sel berkonflik dengan salah satu
 23 menteri yang telah ditempatkan, sel tersebut dipangkas meskipun memiliki *support* pada variabel
 24 sumber.

1 Pemangkasan Sel Tanpa *Support*

2 Pada saat sel terbukti tidak memiliki *support* yang valid, baik karena tidak ditemukan sel pendamping
 3 pada variabel sumber maupun karena berkonflik dengan menteri yang telah ditempatkan, sel
 4 tersebut dihapus dari domain variabel tujuan. Pemangkasan ini dicatat agar dapat dipulihkan
 5 apabila algoritma melakukan *backtrack*, dan penanda *revised* diatur menjadi *true* untuk menandakan
 6 bahwa domain mengalami perubahan. Iterasi kemudian dilanjutkan ke sel berikutnya pada domain
 7 variabel tujuan.

8 3.3.4 Propagasi AC-3

9 Propagasi AC-3 merupakan tahap pemangkasan kedua yang dijalankan setelah *forward checking*
 10 selesai. Sebagaimana dijabarkan pada Bagian 2.4.1, AC-3 menegakkan konsistensi busur antarseluruh
 11 pasangan variabel yang belum ditugaskan, mengeksplorasi keterkaitan kendala antarvariabel yang
 12 tidak terdeteksi oleh *forward checking*. Propagasi dilakukan secara iteratif menggunakan antrean
 13 busur: setiap busur diambil, diperiksa konsistensinya melalui fungsi *revise* sebagaimana dibahas pada
 14 Bagian 3.3.3, dan apabila terjadi perubahan domain, busur-busur lain yang terkait ditambahkan ke
 15 antrean untuk dipropagasi lebih lanjut. Alur prosedur ini diperlihatkan pada Gambar 3.5.



Gambar 3.5: Diagram alur prosedur propagasi AC-3.

16 Pengecekan Antrean dan Pengambilan Busur

17 Prosedur dimulai dengan memeriksa apakah antrean busur masih berisi busur yang belum diproses.
 18 Antrean awal diinisialisasi dengan seluruh pasang busur antara variabel-variabel yang belum
 19 ditugaskan. Apabila antrean kosong, propagasi telah selesai dan seluruh busur dipastikan konsisten,
 20 sehingga prosedur berakhir. Apabila antrean masih berisi busur, satu busur diambil untuk diproses
 21 pada langkah berikutnya.

1 Pemanggilan Fungsi *Revise*

2 Busur yang diambil dari antrean diproses dengan menjalankan fungsi *revise* sebagaimana dijelaskan
3 pada Bagian 3.3.3. Fungsi ini memangkas sel-sel pada domain variabel tujuan yang tidak memiliki
4 *support* pada variabel sumber atau yang berkonflik dengan menteri yang telah ditempatkan, kemu-
5 dian mengembalikan status perubahan domain. Status ini menentukan langkah berikutnya pada
6 propagasi.

7 Evaluasi Hasil Pemangkasan

8 Apabila fungsi *revise* tidak menghasilkan perubahan pada domain variabel tujuan, dampak propagasi
9 terhenti pada busur tersebut, dan alur kembali memeriksa antrean. Apabila domain mengalami
10 perubahan, prosedur selanjutnya memeriksa apakah domain variabel tujuan menjadi kosong. Jika
11 demikian, hal ini menandakan terjadinya *domain wipeout*. Pada kondisi ini, propagasi dihentikan
12 dan sinyal kegagalan dikembalikan kepada alur utama *backtracking* agar dapat melakukan *backtrack*.

13 Pengembangan Antrean dengan Busur Terkait

14 Apabila domain variabel tujuan mengalami perubahan namun tidak menjadi kosong, dampak
15 pemangkasan tersebut dapat memengaruhi konsistensi busur lain yang melibatkan variabel tujuan
16 sebagai variabel sumber. Oleh karena itu, seluruh busur yang berbentuk (t, k) , di mana t adalah
17 variabel tujuan saat ini dan k adalah variabel lain yang belum ditugaskan, ditambahkan kembali ke
18 antrean. Mekanisme ini memastikan dampak pemangkasan terpropagasi secara transitif ke seluruh
19 variabel yang saling terhubung. Setelah penambahan selesai, alur kembali ke pemeriksaan antrean
20 untuk melanjutkan iterasi.

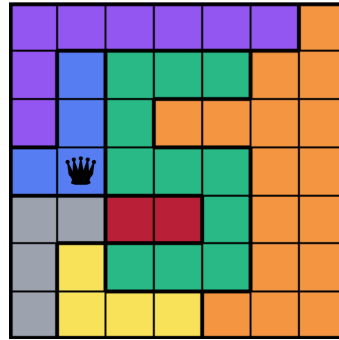
21 3.3.5 Studi Kasus: Penyelesaian Papan 7×7

22 Untuk memperjelas cara kerja algoritma *backtracking* yang diperkuat dengan *forward checking* dan
23 AC-3, bagian ini menyajikan studi kasus penyelesaian sebuah papan *Colored Queens* berukuran
24 7×7 yang terdiri atas tujuh sektor warna, yaitu ungu, biru, hijau, oranye, merah, abu-abu, dan
25 kuning. Studi kasus ini menelusuri proses penyelesaian dengan penekanan pada bagaimana kedua
26 mekanisme pemangkasan mengurangi ruang pencarian dan bagaimana *backtrack* terjadi ketika
27 cabang pencarian gagal.

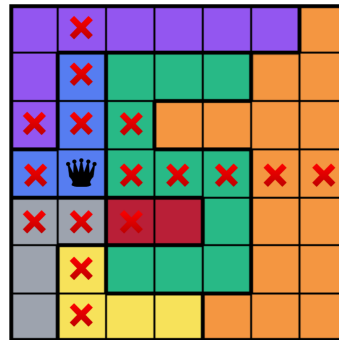
28 Penempatan Awal dan *Forward Checking*

29 Algoritma memulai pencarian dengan memproses variabel warna pertama, dalam contoh ini war-
30 na biru. Sebuah sel valid dipilih dari domain warna biru sebagai posisi penempatan menteri,
31 sebagaimana diperlihatkan pada Gambar 3.6.

32 Setelah menteri ditempatkan, *forward checking* dijalankan untuk memangkas sel-sel yang
33 berkonflik langsung dengan menteri tersebut dari domain seluruh variabel warna yang belum
34 ditugaskan. Hasil pemangkasan diperlihatkan pada Gambar 3.7: seluruh sel yang berada pada
35 baris atau kolom yang sama dengan menteri, serta sel-sel yang bertetangga secara langsung dalam
36 delapan arah, ditandai sebagai tidak valid (tanda X merah).



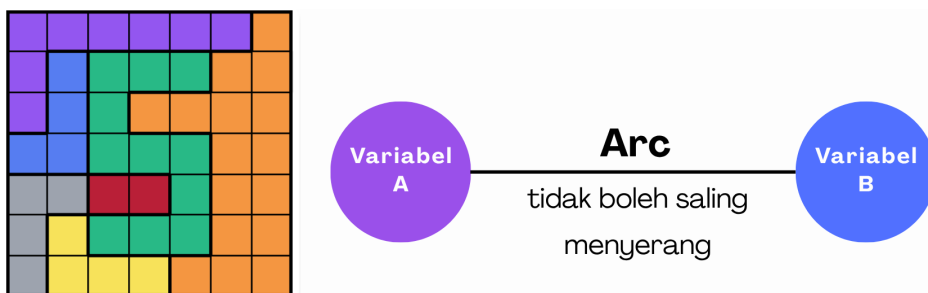
Gambar 3.6: Penempatan menteri pertama pada salah satu sel dari domain warna biru.



Gambar 3.7: Hasil pemangkasan oleh *forward checking* setelah penempatan menteri pertama.

1 Propagasi AC-3

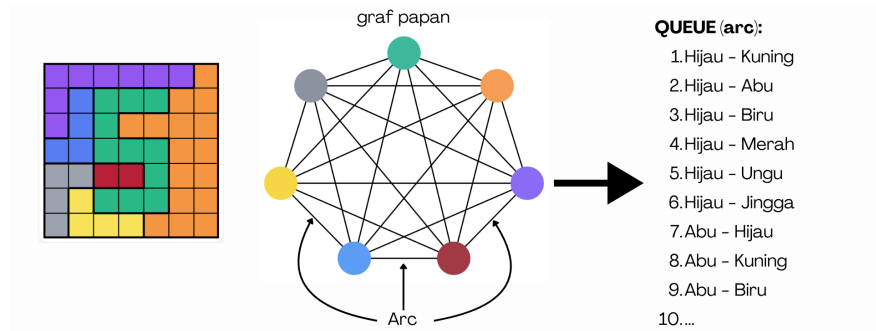
- 2 Setelah *forward checking* selesai, algoritma melanjutkan dengan propagasi AC-3 untuk menegakkan
- 3 konsistensi busur antarvariabel yang belum ditugaskan. Konsep dasar dari pemeriksaan satu busur
- 4 (*arc*) diilustrasikan pada Gambar 3.8: untuk setiap pasang variabel, AC-3 memastikan bahwa setiap
- 5 nilai dalam domain salah satu variabel memiliki minimal satu nilai pendamping yang konsisten
- 6 (*support*) pada variabel lainnya.



Gambar 3.8: Konsep dasar busur (*arc*) dalam AC-3.

- 7 Dalam konteks *Colored Queens*, setiap variabel warna saling terhubung dengan seluruh variabel
- 8 warna lainnya, membentuk graf kendala yang lengkap. Antrean awal AC-3 diinisialisasi dengan
- 9 seluruh pasang busur antara variabel-variabel yang belum ditugaskan, sebagaimana diperlihatkan
- 10 pada Gambar 3.9.

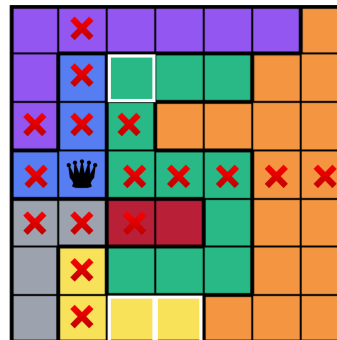
- 11 Sebagai ilustrasi pemrosesan satu busur, perhatikan busur Hijau→Kuning yang menjadi busur
- 12 pertama dalam antrean. Pada titik ini, domain warna hijau yang masih valid setelah *forward*
- 13 *checking* terdiri atas sel-sel $\{(1, 2), (1, 3), (1, 4), (4, 4), (5, 2), (5, 3), (5, 4)\}$, sementara domain warna



Gambar 3.9: Graf kendala lengkap dan antrean awal busur AC-3.

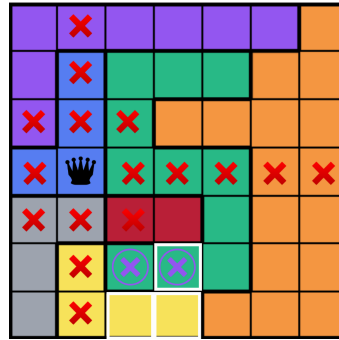
1 kuning terdiri atas $\{(6, 2), (6, 3)\}$. Algoritma memeriksa setiap sel valid pada domain warna hijau
 2 dan menentukan apakah terdapat setidaknya satu sel pada domain warna kuning yang tidak
 3 berkonflik dengannya.

4 Gambar 3.10 menunjukkan pemrosesan sel Hijau(1,2), yang ditandai dengan kotak putih pada
 5 papan. Sel ini berkonflik dengan Kuning(6,2) karena berada pada kolom yang sama, tetapi tidak
 6 berkonflik dengan Kuning(6,3) karena berada pada baris, kolom, dan ketetanggaan yang berbeda.
 7 Dengan demikian, Hijau(1,2) memiliki minimal satu *support* pada domain warna kuning, sehingga
 8 sel ini tetap dipertahankan dalam domain.

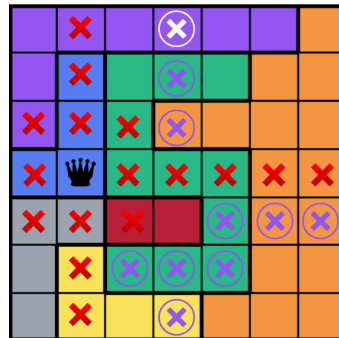
Gambar 3.10: Pemrosesan busur Hijau→Kuning: sel Hijau(1,2) memiliki *support* pada Kuning(6,3) dan tetap valid.

9 Sebaliknya, Gambar 3.11 menunjukkan pemrosesan sel Hijau(5,2) dan Hijau(5,3). Sel Hijau(5,2)
 10 berkonflik dengan Kuning(6,2) karena berada pada kolom yang sama, sekaligus berkonflik dengan
 11 Kuning(6,3) karena bertetangga secara diagonal langsung. Demikian pula, sel Hijau(5,3) berkonflik
 12 dengan Kuning(6,2) secara diagonal dan dengan Kuning(6,3) pada kolom yang sama. Karena tidak
 13 ada satu pun sel pada domain warna kuning yang dapat menjadi *support*, kedua sel hijau tersebut
 14 dipangkas dari domain dan ditandai dengan lingkaran ungu pada papan.

15 Propagasi serupa dilakukan untuk seluruh busur dalam antrean, dengan busur-busur baru
 16 ditambahkan setiap kali sebuah domain mengalami pemangkasan. Kondisi papan setelah seluruh
 17 propagasi AC-3 selesai diperlihatkan pada Gambar 3.12. Tanda X merah menunjukkan sel yang
 18 dipangkas oleh *forward checking*, sedangkan tanda X dengan lingkaran ungu menunjukkan sel
 19 tambahan yang dipangkas oleh AC-3. Penggabungan kedua mekanisme ini menghasilkan reduksi
 20 ruang pencarian yang lebih agresif dibandingkan menggunakan salah satunya saja.



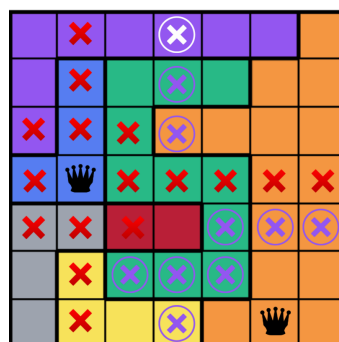
Gambar 3.11: Pemrosesan busur Hijau→Kuning: sel Hijau(5,2) dan Hijau(5,3) tidak memiliki *support* dan dipangkas.



Gambar 3.12: Kondisi papan setelah propagasi AC-3 selesai.

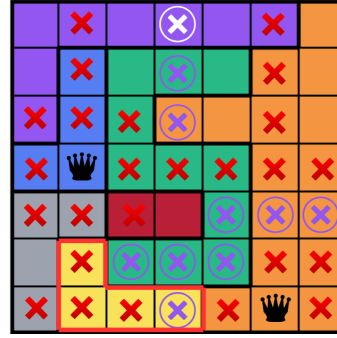
1 Penempatan Berikutnya dan Deteksi *Domain Wipeout*

- 2 Setelah pemangkasan selesai dan tidak ada domain yang kosong, algoritma melanjutkan pencarian
- 3 dengan memanggil dirinya sendiri secara rekursif untuk variabel warna berikutnya. Misalkan
- 4 algoritma menempatkan menteri pada salah satu sel dari domain warna oranye sebagaimana
- 5 ditunjukkan pada Gambar 3.13.



Gambar 3.13: Penempatan menteri kedua pada salah satu sel dari domain warna oranye.

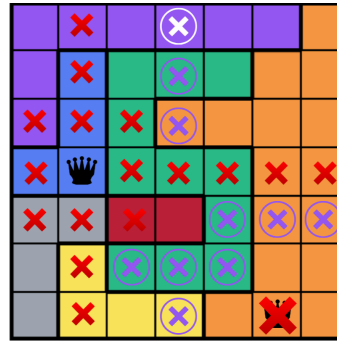
- 6 Setelah penempatan ini, *forward checking* dan AC-3 dijalankan kembali. Namun, kombinasi
- 7 penempatan ini ternyata menyebabkan seluruh sel pada domain warna kuning menjadi tidak valid,
- 8 sebagaimana diperlihatkan pada Gambar 3.14. Kondisi ini merupakan *domain wipeout*, di mana
- 9 tidak ada lagi nilai yang dapat ditugaskan kepada warna kuning, sehingga *backtrack* diperlukan.



Gambar 3.14: *Domain wipeout*: seluruh sel pada domain warna kuning menjadi tidak valid.

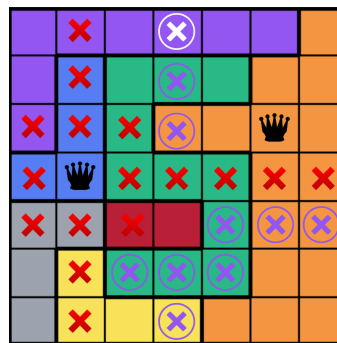
1 *Backtrack* dan Pemulihan Domain

- 2 Sebagai respons terhadap kegagalan, algoritma melakukan *backtrack* dengan membatalkan penem-
- 3 patan menteri oranye yang baru saja dilakukan, sebagaimana diperlihatkan pada Gambar 3.15.
- 4 Seluruh pemangkasan yang dilakukan akibat penempatan tersebut dipulihkan, mengembalikan
- 5 domain seluruh variabel ke kondisi tepat sebelum penempatan oranye dilakukan.



Gambar 3.15: *Backtrack*: pembatalan penempatan menteri oranye yang menyebabkan kegagalan.

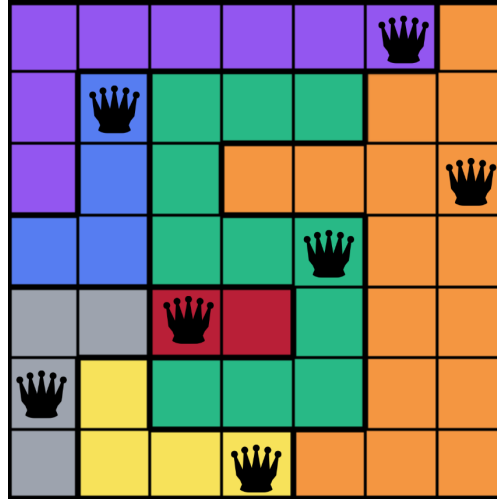
- 6 Algoritma kemudian mencoba sel berikutnya dari domain warna oranye yang masih valid,
- 7 sebagaimana diperlihatkan pada Gambar 3.16. Pemangkasan dijalankan kembali pada konfigurasi
- 8 baru ini, dan apabila tidak terjadi *domain wipeout*, pencarian dilanjutkan ke variabel warna
- 9 berikutnya.



Gambar 3.16: Penempatan alternatif untuk warna oranye setelah *backtrack*.

1 Solusi Akhir

2 Proses penempatan, pemangkasan, dan *backtrack* berulang hingga seluruh tujuh variabel warna
 3 mendapat penugasan yang konsisten. Solusi akhir untuk papan 7×7 diperlihatkan pada Gambar 3.17,
 4 di mana setiap sektor warna memiliki tepat satu menteri yang ditempatkan, tidak ada dua menteri
 5 yang berbagi baris atau kolom yang sama, dan tidak ada dua menteri yang bertetangga secara
 6 langsung.



Gambar 3.17: Solusi akhir papan 7×7 yang memenuhi seluruh kendala *Colored Queens*.

7 Studi kasus ini mengilustrasikan bagaimana sinergi antara *forward checking* dan AC-3 secara
 8 signifikan mengurangi ruang pencarian. *Forward checking* menangani konflik langsung dengan
 9 penugasan terbaru, sementara AC-3 menelusuri konsekuensi tidak langsung dari penugasan tersebut
 10 terhadap konsistensi seluruh pasangan variabel yang belum ditugaskan. Mekanisme *backtrack*
 11 memastikan bahwa kegagalan pada satu cabang pencarian dapat ditangani secara efisien dengan
 12 memulihkan domain ke kondisi sebelumnya, sehingga algoritma dapat mengeksplorasi cabang
 13 alternatif tanpa perlu menghitung ulang seluruh struktur domain.

14 3.4 Analisis Algoritma *Particle Swarm Optimization*

15 Bagian ini menjelaskan analisis algoritmik dari pendekatan kedua yang digunakan untuk menyelesa-
 16 ikan permasalahan *Colored Queens*, yaitu *Particle Swarm Optimization* (PSO) dalam varian diskrit.
 17 Berbeda dengan *backtracking* yang menjamin penemuan solusi melalui pencarian sistematis, PSO
 18 merupakan algoritma stokastik berbasis populasi yang mencari solusi melalui pergerakan kolektif
 19 partikel-partikel pada ruang pencarian. Landasan teori PSO klasik dan adaptasinya untuk ruang
 20 pencarian diskrit telah dibahas pada Bagian 2.4.2. Analisis disajikan melalui dua diagram alur,
 21 yaitu mekanisme pembaruan kecepatan dan posisi setiap partikel, kemudian alur utama algoritma
 22 yang menjalankan pembaruan tersebut secara iteratif hingga salah satu kondisi terminasi terca-
 23 pai. Diagram pembaruan partikel disajikan terlebih dahulu agar pembaca dapat memahami unit
 24 komputasi inti dari PSO sebelum melihat keseluruhan alur. Rincian implementasi masing-masing
 25 komponen dibahas pada Bab 4.

3.4.1 Fungsi *Fitness*

Berbeda dengan algoritma *backtracking* yang menjamin validitas solusi melalui pemeriksaan kendala secara eksplisit selama proses pencarian, PSO sebagai algoritma berbasis populasi tidak menggunakan kendala secara langsung untuk membatasi pergerakan partikel. Sebagai gantinya, kualitas suatu posisi partikel diukur secara numerik melalui sebuah fungsi *fitness*, yaitu fungsi yang memetakan setiap posisi partikel ke sebuah nilai real yang mencerminkan seberapa dekat posisi tersebut dengan solusi valid. Pendekatan ini sejalan dengan transformasi umum dari *Constraint Satisfaction Problem* (CSP) menjadi *Constraint Optimization Problem* (COP) sebagaimana telah dibahas pada Bagian 2.4.2: dengan reformulasi ini, permasalahan “mencari penugasan yang memenuhi seluruh kendala” diubah menjadi permasalahan “meminimalkan jumlah pelanggaran kendala,” yang dapat ditangani oleh *metaheuristic* berbasis optimasi seperti PSO. Posisi partikel yang merepresentasikan solusi valid bagi permasalahan *Colored Queens* berkorespondensi dengan nilai *fitness* sama dengan nol, dan algoritma berhenti ketika kondisi tersebut tercapai.

Fungsi *fitness* pada penelitian ini dirancang untuk mengukur dua jenis pelanggaran kendala yang dapat terjadi pada konfigurasi penempatan menteri, yang masing-masing bersesuaian dengan kendala yang telah didefinisikan pada Bagian 3.2.4. Pelanggaran pertama, yang disebut *attacking violation*, menghitung jumlah pasangan menteri yang ditempatkan pada baris atau kolom yang sama. Pelanggaran jenis ini menggabungkan kendala baris dan kendala kolom yang secara struktural setara: keduanya melarang dua menteri berbagi sebuah sumbu linear pada papan. Pelanggaran kedua, yang disebut *adjacency violation*, menghitung jumlah pasangan menteri yang menempati sel-sel yang bertetangga secara langsung dalam delapan arah. Kedua jenis pelanggaran ini diperhitungkan secara terpisah karena merepresentasikan dimensi konflik yang berbeda secara semantik: *attacking* mengukur konflik berbasis sumbu (*axis-based conflict*), sedangkan *adjacency* mengukur konflik berbasis kedekatan posisi (*proximity-based conflict*). Kendala sektor warna tidak perlu diperhitungkan dalam fungsi *fitness* karena telah dipenuhi secara implisit oleh representasi posisi partikel, di mana setiap indeks pada vektor posisi merepresentasikan pilihan sel dari domain satu sektor tertentu.

Kedua komponen pelanggaran tersebut kemudian dikombinasikan menjadi sebuah nilai *fitness* tunggal melalui penjumlahan berbobot, sebagaimana ditunjukkan pada Persamaan 3.8.

$$f(\vec{x}) = w_1 \cdot V_{attacking}(\vec{x}) + w_2 \cdot V_{adjacency}(\vec{x}) \quad (3.8)$$

di mana $\vec{x}$ adalah posisi partikel, $V_{attacking}(\vec{x})$ adalah jumlah pasangan menteri yang melanggar kendala *attacking*, $V_{adjacency}(\vec{x})$ adalah jumlah pasangan menteri yang melanggar kendala *adjacency*, serta w_1 dan w_2 adalah bobot positif yang menentukan kontribusi relatif setiap jenis pelanggaran terhadap nilai *fitness* keseluruhan. Pemilihan bobot mencerminkan prioritas relatif dalam penyelesaian: nilai w_1 yang lebih besar mendorong algoritma untuk memprioritaskan pengurangan pelanggaran *attacking*, sedangkan nilai w_2 yang lebih besar mendorong prioritas pada pengurangan pelanggaran *adjacency*. Konfigurasi *baseline* pada tugas akhir ini menggunakan $w_1 = w_2 = 1$, yang memberikan bobot setara pada kedua jenis pelanggaran dan menjadikan fungsi *fitness* setara dengan jumlah total pelanggaran kendala pada konfigurasi tersebut.

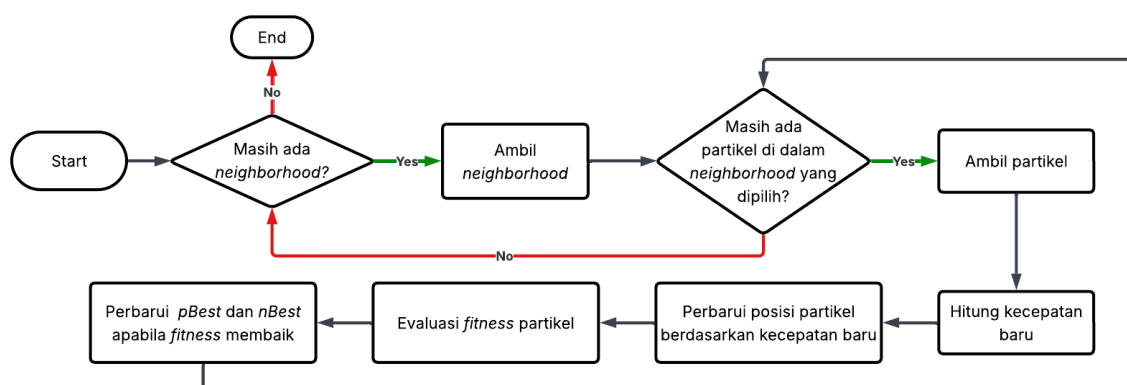
Penjumlahan berbobot dipilih sebagai mekanisme agregasi karena kesederhanaan interpretasinya

dan kompatibilitasnya dengan kerangka kerja optimasi PSO yang bersifat skalar. Fungsi *fitness* sebagai keluaran skalar memungkinkan perbandingan antarposisi partikel secara langsung melalui operator perbandingan numerik, yang diperlukan untuk pembaruan *pBest* dan *nBest* pada setiap iterasi. Alternatif lain seperti agregasi multiplikatif atau formulasi vektorial akan menghasilkan permasalahan multi-objektif yang memerlukan mekanisme penanganan tambahan, seperti konsep dominasi Pareto, yang menambah kompleksitas algoritma tanpa memberikan keuntungan signifikan pada konteks *Colored Queens* di mana kedua jenis pelanggaran sama-sama harus dieliminasi secara absolut pada solusi valid.

Perlu dicatat bahwa kedua komponen pelanggaran tidak bersifat saling-eksklusif. Sepasang menteri yang berada pada baris atau kolom yang sama dan secara bersamaan menempati sel-sel yang bertetangga akan dihitung sebagai pelanggaran pada kedua komponen secara independen. Sebagai contoh, dua menteri yang ditempatkan pada sel (r, c) dan $(r, c + 1)$ melanggar kendala baris (keduanya pada baris r) sekaligus kendala *adjacency* (keduanya bertetangga secara horizontal). Penghitungan ganda ini bukan suatu redundansi, melainkan keputusan desain yang disengaja: pasangan yang melanggar dua kendala sekaligus merepresentasikan konfigurasi yang lebih jauh dari solusi valid dibandingkan pasangan yang hanya melanggar satu kendala, sehingga sewajarnya menerima penalti yang lebih tinggi pada nilai *fitness*. Mekanisme ini menghasilkan gradien yang lebih curam pada konfigurasi yang sangat melanggar, yang membantu mengarahkan partikel keluar dari area pencarian yang tidak menjanjikan dengan lebih cepat.

3.4.2 Pembaruan Kecepatan dan Posisi Partikel

Mekanisme pembaruan kecepatan dan posisi merupakan inti komputasi PSO yang dijalankan pada setiap iterasi terhadap seluruh partikel dalam populasi. Pada varian diskrit yang digunakan dalam tugas akhir ini, kecepatan tidak diinterpretasikan sebagai perubahan posisi numerik, melainkan sebagai probabilitas adopsi posisi terbaik tetangga (*neighborhood best* atau *nBest*) pada setiap dimensi. Pendekatan ini mempertahankan kerangka kerja PSO klasik sekaligus mengakomodasi sifat diskrit dari ruang pencarian *Colored Queens*. Alur pembaruan partikel diperlihatkan pada Gambar 3.18.



Gambar 3.18: Diagram alur pembaruan kecepatan dan posisi partikel PSO diskrit.

1 Iterasi atas *Neighborhood*

2 Prosedur dimulai dengan mengiterasi seluruh kelompok *neighborhood* yang telah dibentuk pada
 3 tahap inisialisasi populasi. Setiap *neighborhood* adalah himpunan partikel yang berbagi informasi
 4 mengenai posisi terbaik kolektifnya (*nBest*), sehingga partikel-partikel dalam satu kelompok bergerak
 5 dipandu oleh referensi yang sama. Penggunaan topologi *neighborhood* dipilih atas pertimbangan
 6 yang dijabarkan pada Bagian 2.4.2. Apabila seluruh *neighborhood* telah diproses, prosedur berakhir.

7 Iterasi atas Partikel dalam *Neighborhood*

8 Untuk setiap *neighborhood* yang diambil, prosedur mengiterasi seluruh partikel yang menjadi
 9 anggotanya. Setiap partikel akan melalui rangkaian langkah pembaruan secara independen, tetapi
 10 semuanya merujuk pada *nBest* yang sama untuk kelompok tersebut. Apabila seluruh partikel dalam
 11 *neighborhood* saat ini telah diperbarui, alur kembali ke iterasi *neighborhood* berikutnya.

12 Penghitungan Kecepatan Baru

13 Kecepatan baru setiap partikel dihitung sebagai kombinasi linear dari tiga komponen, yaitu komponen
 14 inersia yang mempertahankan arah pergerakan sebelumnya, komponen kognitif yang menarik
 15 partikel menuju posisi terbaik historisnya (*personal best* atau *pBest*), dan komponen sosial yang
 16 menarik partikel menuju *nBest*. Untuk setiap partikel, kecepatan baru pada dimensi ke-*j* dihitung
 17 sebagaimana ditunjukkan pada Persamaan 3.9.

$$v_j^{(t+1)} = w \cdot v_j^{(t)} + c_1 \cdot r_1 \cdot \delta_{pBest,j} + c_2 \cdot r_2 \cdot \delta_{nBest,j} \quad (3.9)$$

18 di mana w adalah koefisien inersia, c_1 dan c_2 adalah koefisien akselerasi kognitif dan sosial, r_1
 19 dan r_2 adalah bilangan acak seragam pada interval $[0, 1)$ yang dibangkitkan secara independen
 20 untuk setiap dimensi, dan $\delta_{pBest,j}$ serta $\delta_{nBest,j}$ adalah indikator perbedaan posisi yang didefinisikan
 21 pada Persamaan 3.10.

$$\delta_{pBest,j} = \begin{cases} 1 & \text{jika } x_j^{(t)} \neq pBest_j \\ 0 & \text{jika } x_j^{(t)} = pBest_j \end{cases} \quad \delta_{nBest,j} = \begin{cases} 1 & \text{jika } x_j^{(t)} \neq nBest_j \\ 0 & \text{jika } x_j^{(t)} = nBest_j \end{cases} \quad (3.10)$$

22 Penggunaan indikator biner δ sebagai pengganti selisih aritmatika merupakan adaptasi kunci
 23 dari PSO kontinu ke ruang diskrit. Pada PSO kontinu, komponen kognitif dan sosial dihitung
 24 melalui selisih vektor posisi yang menghasilkan arah perpindahan dalam ruang Euclidean. Pada
 25 ruang diskrit berindeks, selisih aritmatika antara dua indeks sel tidak memiliki makna geometris:
 26 sel dengan indeks tertentu dan sel dengan indeks lain dalam domain suatu warna tidak memiliki
 27 hubungan spasial yang ditentukan oleh selisih indeksnya. Oleh sebab itu, informasi yang relevan
 28 hanyalah apakah posisi saat ini berbeda dari posisi referensi (*pBest* atau *nBest*), bukan seberapa
 29 jauh perbedaannya.

30 Setelah perhitungan, nilai kecepatan di-*clamp* pada interval $[0, 1]$ untuk mempertahankan
 31 interpretasinya sebagai probabilitas, sebagaimana ditunjukkan pada Persamaan 3.11.

$$v_j^{(t+1)} = \max(0, \min(1, v_j^{(t+1)})) \quad (3.11)$$

1 Pembaruan Posisi Partikel

2 Posisi baru partikel diturunkan dari kecepatan dengan interpretasi probabilistik. Untuk setiap
 3 dimensi j , sebuah bilangan acak $r \in [0, 1)$ dibangkitkan, kemudian dibandingkan dengan nilai
 4 kecepatan pada dimensi tersebut. Apabila $r < v_j$, posisi partikel pada dimensi tersebut diadopsi
 5 dari $nBest$. Apabila $r \geq v_j$, posisi tidak berubah. Secara formal, mekanisme pembaruan posisi ini
 6 dinyatakan pada Persamaan 3.12.

$$x_j^{(t+1)} = \begin{cases} nBest_j & \text{jika } r < v_j^{(t+1)} \\ x_j^{(t)} & \text{jika } r \geq v_j^{(t+1)} \end{cases} \quad (3.12)$$

7 Mekanisme ini secara sengaja hanya menggunakan posisi $nBest$ sebagai target perpindahan,
 8 meskipun $pBest$ turut memengaruhi probabilitas perpindahan melalui formula kecepatan. Pemi-
 9 sahan peran ini merupakan keputusan desain yang didasarkan pada karakteristik ruang pencarian
 10 diskrit. Pada PSO kontinu, posisi baru merupakan hasil penjumlahan vektor yang secara alami
 11 menginterpolasi antara $pBest$ dan $nBest$. Pada ruang diskrit, interpolasi semacam ini tidak terdefi-
 12 nisi: sebuah dimensi hanya dapat mengambil satu nilai indeks pada satu waktu, sehingga partikel
 13 harus memilih salah satu target, bukan menggabungkan keduanya. Apabila pembaruan posisi juga
 14 mempertimbangkan $pBest$ sebagai target alternatif, setiap dimensi akan menghadapi dua arah
 15 perpindahan yang dapat saling bertentangan, yang dalam ruang diskrit menghasilkan pergantian
 16 indeks yang tidak koheren alih-alih konvergensi bertahap. Dengan menjadikan $nBest$ sebagai
 17 satu-satunya target perpindahan, mekanisme pembaruan posisi menjadi lebih terarah, sementara
 18 pengaruh pengalaman individual partikel tetap terkodifikasi dalam besaran probabilitas melalui
 19 komponen $c_1 \cdot r_1 \cdot \delta_{pBest,j}$ pada formula kecepatan.

20 Evaluasi *Fitness* dan Pembaruan $pBest$ serta $nBest$

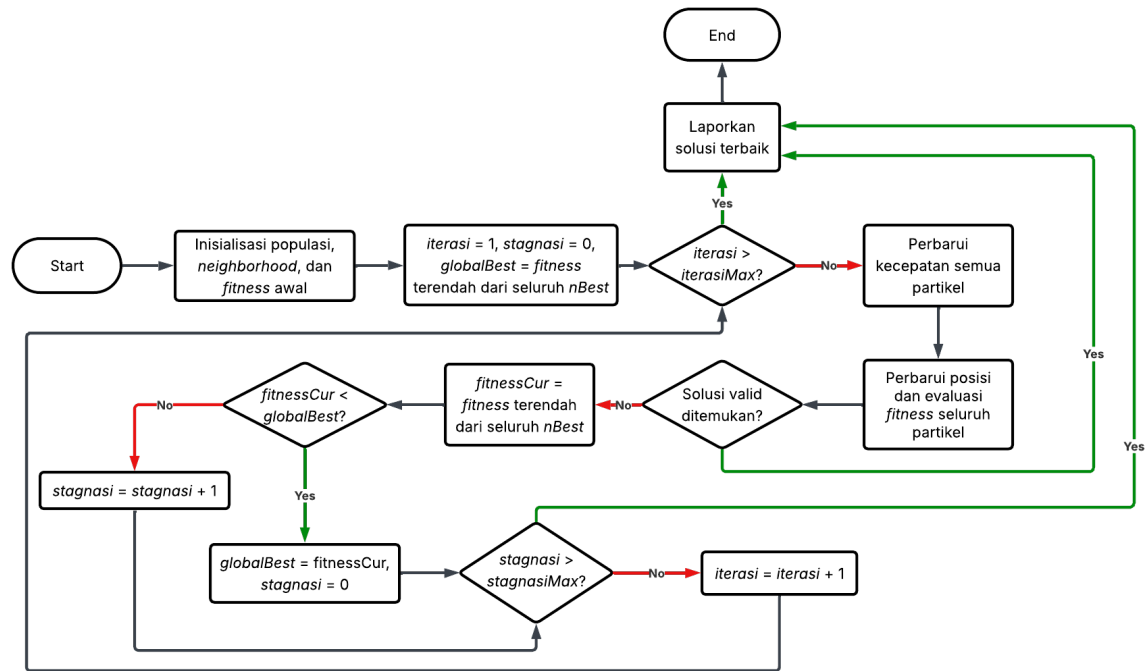
21 Setelah posisi partikel diperbarui, nilai *fitness* dievaluasi ulang berdasarkan fungsi *fitness* yang
 22 menghitung jumlah pelanggaran kendala sesuai definisi pada Bagian 3.2.4 dan formulasi pada
 23 Bagian 3.4.1. Nilai *fitness* ini kemudian dibandingkan dengan *fitness* dari $pBest$ partikel: apabila
 24 lebih baik, $pBest$ diperbarui dengan posisi saat ini. Selanjutnya, *fitness* juga dibandingkan dengan
 25 *fitness* dari $nBest$ *neighborhood*: apabila lebih baik, $nBest$ diperbarui agar partikel-partikel lain
 26 dalam kelompok yang sama dapat mengikutinya pada iterasi berikutnya.

27 3.4.3 Alur Utama Algoritma PSO

28 Alur utama algoritma PSO mengorkestrasi keseluruhan proses optimasi, mulai dari inisialisasi
 29 populasi hingga terminasi. Pada setiap iterasi, mekanisme pembaruan partikel yang telah dijabarkan
 30 pada Bagian 3.4.2 dijalankan terhadap seluruh partikel, diikuti oleh evaluasi terhadap tiga kondisi
 31 terminasi: ditemukannya solusi valid, tercapainya batas iterasi maksimum, atau terdeteksinya
 32 stagnasi. Keseluruhan alur diperlihatkan pada Gambar 3.19.

33 Inisialisasi Populasi dan Variabel Pelacakan

34 Algoritma dimulai dengan inisialisasi populasi, yaitu pembangkitan sejumlah partikel dengan posisi
 35 acak pada ruang pencarian dan kecepatan acak pada interval valid. Selanjutnya, partikel-partikel



Gambar 3.19: Diagram alur utama algoritma PSO diskrit.

tersebut dipartisi ke dalam beberapa *neighborhood* dengan ukuran yang relatif seimbang, dan nilai *fitness* awal seluruh partikel dievaluasi. Setelah inisialisasi populasi selesai, algoritma menetapkan pencacah iterasi pada nilai satu, pencacah stagnasi pada nilai nol, dan menyimpan *fitness* terendah di antara seluruh *nBest* sebagai nilai *fitness* global terbaik (*globalBest*) yang akan digunakan sebagai acuan deteksi stagnasi pada iterasi-iterasi berikutnya.

6 Pengecekan Batas Iterasi

Pada awal setiap iterasi, algoritma memeriksa apakah pencacah iterasi telah melampaui batas iterasi maksimum yang telah ditetapkan sebagai parameter. Apabila ya, pencarian dihentikan dan algoritma melaporkan partikel dengan *fitness* terendah di antara seluruh *nBest* sebagai solusi terbaik yang ditemukan, meskipun solusi tersebut belum tentu valid. Apabila batas belum tercapai, algoritma melanjutkan ke tahap pembaruan partikel.

12 Pembaruan Seluruh Partikel

Algoritma menjalankan dua tahap pembaruan secara berurutan terhadap seluruh partikel dalam populasi. Tahap pertama memperbarui kecepatan seluruh partikel berdasarkan *pBest* masing-masing dan *nBest* dari *neighborhood*-nya. Tahap kedua memperbarui posisi partikel berdasarkan kecepatan yang baru dihitung, mengevaluasi ulang *fitness*, serta memperbarui *pBest* dan *nBest* apabila ditemukan posisi yang lebih baik. Kedua tahap ini secara kolektif merupakan eksekusi penuh dari prosedur pembaruan partikel yang telah dijabarkan pada Bagian 3.4.2.

1 Pengecekan Solusi Valid

2 Setelah pembaruan selesai, algoritma memeriksa apakah salah satu $nBest$ memiliki nilai $fitness$
 3 sama dengan nol. Nilai $fitness$ nol menandakan tidak adanya pelanggaran kendala pada posisi
 4 tersebut, yang berarti partikel tersebut merepresentasikan solusi valid bagi permasalahan *Colored*
 5 *Queens*. Pemeriksaan dilakukan hanya terhadap $nBest$ dari setiap $neighborhood$, bukan terhadap
 6 seluruh partikel, karena partikel dengan $fitness$ terendah dalam setiap kelompok pasti merupakan
 7 $nBest$ saat ini. Apabila solusi valid ditemukan, algoritma berhenti dan melaporkan solusi tersebut.

8 Pembaruan $Fitness$ Global dan Pencacah Stagnasi

9 Apabila solusi valid belum ditemukan, algoritma menghitung $fitness$ terendah di antara seluruh
 10 $nBest$ saat ini ($fitnessCur$) dan membandingkannya dengan $globalBest$ yang tersimpan. Apabila
 11 $fitnessCur$ lebih rendah, hal ini menandakan bahwa populasi mengalami perbaikan, sehingga
 12 $globalBest$ diperbarui dan pencacah stagnasi direset menjadi nol. Apabila tidak terjadi perbaikan,
 13 pencacah stagnasi ditambahkan satu untuk menandai berlalunya satu iterasi tanpa kemajuan.

14 Pengecekan Stagnasi dan Lanjutan Iterasi

15 Setelah pencacah stagnasi diperbarui, algoritma memeriksa apakah nilainya telah melampaui batas
 16 stagnasi maksimum. Mekanisme ini menghentikan pencarian lebih awal pada kasus di mana populasi
 17 telah terjebak pada *local optimum* dan eksplorasi lebih lanjut tidak produktif. Justifikasi keberadaan
 18 dan pemilihan nilai batas stagnasi dijabarkan pada Bagian 2.4.2. Apabila batas tidak terlampaui,
 19 pencacah iterasi ditambahkan satu dan alur kembali ke pengecekan batas iterasi, melanjutkan siklus
 20 optimasi pada iterasi berikutnya.

21 3.4.4 Studi Kasus: Pencarian Solusi dengan PSO pada Papan 7×7

22 Sebagai pembanding terhadap pendekatan *backtracking* yang telah dijabarkan pada Bagian 3.3.5,
 23 bagian ini menyajikan studi kasus penerapan PSO pada papan 7×7 yang sama. Untuk menjaga
 24 kesederhanaan ilustrasi, contoh ini menggunakan populasi minimal yang terdiri atas dua partikel
 25 yang dikelompokkan dalam satu $neighborhood$. Pada implementasi nyata, jumlah partikel dan
 26 $neighborhood$ jauh lebih besar untuk meningkatkan keragaman eksplorasi.

27 Inisialisasi Populasi

28 Pada tahap inisialisasi, dua partikel dibangkitkan dengan posisi awal acak. Setiap posisi partikel
 29 direpresentasikan sebagai vektor berukuran tujuh, di mana setiap index pada vektor berkaitan
 30 dengan satu sektor warna. Nilai pada index ke- i menyatakan posisi sel terpilih dari domain warna
 31 ke- i . Posisi awal kedua partikel ditetapkan sebagai berikut:

- 32 • Partikel 1: $\{3, 1, 12, 8, 0, 2, 1\}$
- 33 • Partikel 2: $\{0, 3, 9, 2, 0, 3, 1\}$

34 Selain posisi, setiap partikel juga memiliki vektor kecepatan awal dengan nilai acak pada interval
 35 $[0, 1)$ untuk setiap index:

- 36 • Partikel 1: $v_i = [0.12, 0.85, 0.43, 0.67, 0.09, 0.91, 0.274]$
- 37 • Partikel 2: $v_i = [0.76, 0.21, 0.58, 0.04, 0.69, 0.47, 0.83]$

1 Perhitungan *Fitness* Awal dan Penetapan *nBest*

2 Nilai *fitness* awal setiap partikel dievaluasi berdasarkan fungsi *fitness* yang telah dijabarkan pada
3 Bagian 3.4.1. Dalam contoh ini, hasil evaluasi adalah:

- 4 • Partikel 1: $f = 14$
- 5 • Partikel 2: $f = 20$

6 Karena partikel 1 memiliki nilai *fitness* terendah, partikel 1 ditetapkan sebagai *neighborhood best*
7 (*nBest*) dengan nilai 14. Penetapan ini akan digunakan sebagai acuan untuk pembaruan kecepatan
8 dan posisi pada iterasi berikutnya. Selain itu, posisi awal masing-masing partikel juga ditetapkan
9 sebagai *personal best* (*pBest*) mereka.

10 Pembaruan Kecepatan

11 Pada iterasi pertama, kecepatan setiap partikel diperbarui menggunakan Persamaan 3.9. Sebagai
12 contoh ilustratif, misalkan parameter PSO ditetapkan sebagai $\omega = 0.7$, $c_1 = 0.5$, $c_2 = 0.5$,
13 dan bilangan acak yang dibangkitkan adalah $r_1 = 0.2$ serta $r_2 = 0.27$. Untuk index ke-0 dari
14 partikel 2, perhitungan kecepatan baru ditunjukkan pada Persamaan 3.13 dan menghasilkan nilai
15 $v_{i+1}[0] = 0.682$.

$$\begin{aligned}
 v_{i+1}[0] &= \underbrace{(\omega \cdot v_i[0])}_{\text{inersia}} + \underbrace{(r_1 \cdot c_1 \cdot \delta_{pBest,0})}_{\text{kognitif}} + \underbrace{(r_2 \cdot c_2 \cdot \delta_{nBest,0})}_{\text{sosial}} \\
 &= (0.7 \cdot 0.76) + (0.2 \cdot 0.5 \cdot 0) + (0.27 \cdot 0.5 \cdot 1) \\
 &= 0.532 + 0 + 0.15 = 0.682
 \end{aligned}
 \tag{3.13}$$

16 Nilai $\delta_{pBest,0} = 0$ pada Persamaan 3.13 muncul karena nilai partikel 2 pada index ke-0 sama
17 dengan nilai *pBest*-nya pada index yang sama (keduanya bernilai 0), sesuai dengan definisi δ pada
18 Persamaan 3.10. Sebaliknya, $\delta_{nBest,0} = 1$ karena nilai partikel 2 pada index ke-0 (bernilai 0) berbeda
19 dengan nilai *nBest* pada index ke-0 (bernilai 3). Perhitungan serupa dilakukan untuk seluruh index,
20 menghasilkan vektor kecepatan baru pada Persamaan 3.14.

$$v_{i+1} = [0.682, 0.497, 0.606, 0.2728, 0.483, 0.579, 0.581] \tag{3.14}$$

21 Pembaruan Posisi

22 Selanjutnya, posisi partikel diperbarui berdasarkan kecepatan baru menggunakan Persamaan 3.12.
23 Untuk setiap index, bilangan acak $r \in [0, 1)$ dibangkitkan, dan apabila $r < v_{i+1}[j]$, nilai pada index
24 tersebut diadopsi dari *nBest*; apabila tidak, nilai tetap.

25 Sebagai contoh, untuk index ke-0 partikel 2, dibangkitkan bilangan acak $r = 0.25$. Karena
26 $r < v_{i+1}[0]$ (yaitu $0.25 < 0.682$), maka nilai pada index ke-0 mengadopsi nilai dari *nBest*, sehingga
27 $x_{i+1}[0] = 3$ (menggantikan nilai sebelumnya yang bernilai 0). Perhitungan serupa dilakukan untuk
28 seluruh index, menghasilkan posisi baru partikel 2 pada Persamaan 3.15.

$$x_{i+1} = [3, 3, 12, 2, 0, 2, 1] \tag{3.15}$$

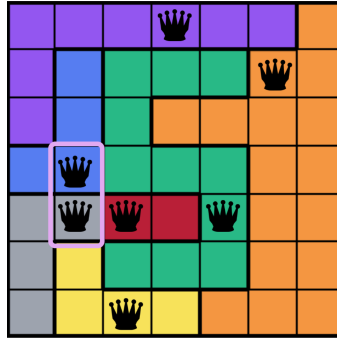
1 Evaluasi *Fitness* Partikel Baru

2 Setelah posisi diperbarui, nilai *fitness* partikel 2 dievaluasi ulang menggunakan Persamaan 3.8.
 3 Untuk contoh ini, ditetapkan $w_1 = 1$ dan $w_2 = 2$. Posisi baru partikel 2 pada Persamaan 3.15
 4 memetakan menteri ke sel-sel tertentu pada papan, dan pelanggaran kendala kemudian dihitung
 5 berdasarkan dua komponen, yaitu $V_{attacking}$ dan $V_{adjacency}$.

6 Dengan kelima pasangan tersebut, nilai $V_{attacking}$ adalah 5. Nilai *fitness* akhir partikel 2 setelah
 7 iterasi pertama dihitung berdasarkan Persamaan 3.8 dan menghasilkan nilai 11, sebagaimana
 8 ditunjukkan pada Persamaan 3.16.

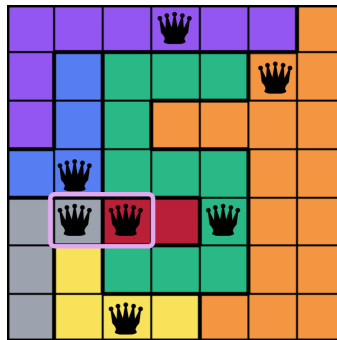
$$f(\vec{x}) = w_1 \cdot V_{attacking} + w_2 \cdot V_{adjacency} = (1 \cdot 5) + (2 \cdot 3) = 5 + 6 = 11 \quad (3.16)$$

9 **Pelanggaran *adjacency*.** Komponen ini menghitung pasangan menteri yang bertetangga
 10 secara langsung dalam delapan arah. Pada konfigurasi partikel 2, terdapat tiga pasangan *adjacency*.
 11 Pasangan pertama, sebagaimana disorot pada Gambar 3.20, terdiri atas menteri pada (3,1) dan
 12 menteri pada (4,1) yang bertetangga secara vertikal.



Gambar 3.20: Pasangan *adjacency* pertama: menteri pada (3,1) dan (4,1) bertetangga vertikal.

13 Pasangan kedua, sebagaimana disorot pada Gambar 3.21, terdiri atas menteri pada (4,1) dan
 14 menteri pada (4,2) yang bertetangga secara horizontal.

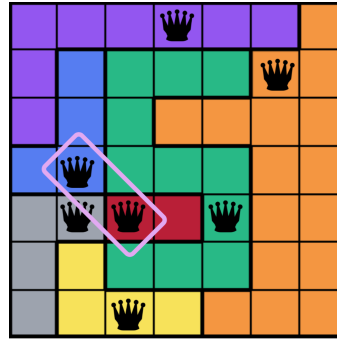


Gambar 3.21: Pasangan *adjacency* kedua: menteri pada (4,1) dan (4,2) bertetangga horizontal.

15 Pasangan ketiga, sebagaimana disorot pada Gambar 3.22, terdiri atas menteri pada (3,1) dan
 16 menteri pada (4,2) yang bertetangga secara diagonal.

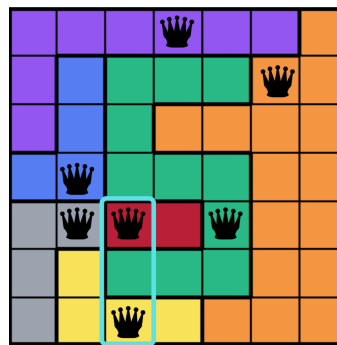
17 Dengan demikian, nilai *adjacencyViolation* adalah 3..

18 **Pelanggaran *attacking*.** Komponen ini menghitung pasangan menteri yang berada pada baris
 19 atau kolom yang sama. Pada konfigurasi partikel 2, Gambar 3.23 menyoroti pasangan pertama,



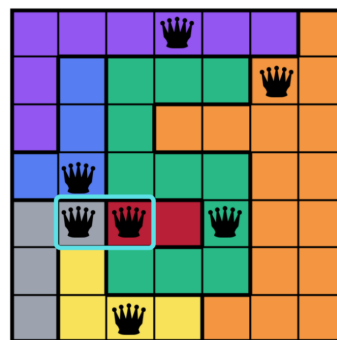
Gambar 3.22: Pasangan *adjacency* ketiga: menteri pada (3,1) dan (4,2) bertetangga diagonal.

- 1 yaitu menteri pada (4,2) dan menteri pada (6,2) yang berada pada kolom 2.



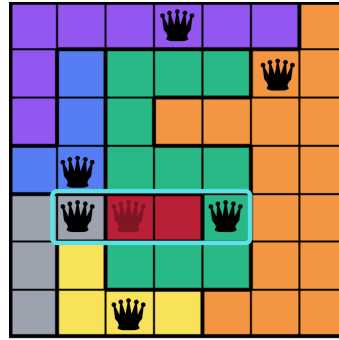
Gambar 3.23: Pasangan *attacking* pertama: menteri pada (4,2) dan (6,2) berada pada kolom yang sama.

- 2 Pasangan kedua, sebagaimana disorot pada Gambar 3.24, terdiri atas menteri pada (4,1) dan
- 3 menteri pada (4,2) yang berada pada baris 4. Perlu dicatat bahwa pasangan ini juga merupakan
- 4 pelanggaran *adjacency* sebagaimana telah disorot pada Gambar 3.21. Pasangan tersebut tetap
- 5 dihitung pada kedua komponen secara independen karena keduanya merepresentasikan jenis kendala
- 6 yang berbeda: *attacking* mengukur konflik berbasis baris dan kolom, sedangkan *adjacency* mengukur
- 7 kedekatan posisi.



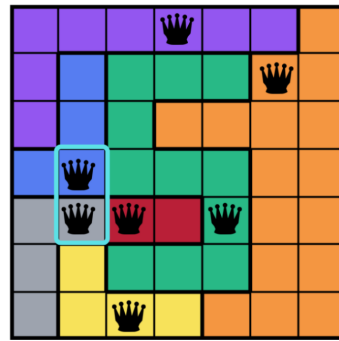
Gambar 3.24: Pasangan *attacking* kedua: menteri pada (4,1) dan (4,2) berada pada baris yang sama. Pasangan ini juga merupakan pelanggaran *adjacency*.

- 8 Pasangan ketiga, sebagaimana disorot pada Gambar 3.25, terdiri atas menteri pada (4,1) dan
- 9 menteri pada (4,4) yang berada pada baris 4. Berbeda dengan pasangan sebelumnya, pasangan ini
- 10 tidak bertetangga langsung sehingga hanya tergolong sebagai pelanggaran *attacking*.



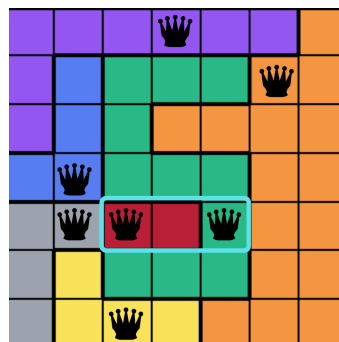
Gambar 3.25: Pasangan *attacking* ketiga: menteri pada (4,1) dan (4,4) berada pada baris yang sama.

- 1 Pasangan keempat, sebagaimana disorot pada Gambar 3.26, terdiri atas menteri pada (3,1) dan
- 2 menteri pada (4,1) yang berada pada kolom 1. Sama halnya dengan pasangan pada Gambar 3.24,
- 3 pasangan ini juga merupakan pelanggaran *adjacency* (telah disorot pada Gambar 3.20) dan dihitung
- 4 pada kedua komponen.



Gambar 3.26: Pasangan *attacking* keempat: menteri pada (3,1) dan (4,1) berada pada kolom yang sama. Pasangan ini juga merupakan pelanggaran *adjacency*.

- 5 Pasangan kelima, sebagaimana disorot pada Gambar 3.27, terdiri atas menteri pada (4,2) dan
- 6 menteri pada (4,4) yang berada pada baris 4. Pasangan ini tidak bertetangga langsung sehingga
- 7 hanya tergolong sebagai pelanggaran *attacking*.



Gambar 3.27: Pasangan *attacking* kelima: menteri pada (4,2) dan (4,4) berada pada baris yang sama.

- 8 Dengan kelima pasangan tersebut, nilai *attackingViolation* adalah 5. Nilai *fitness* akhir partikel 2
- 9 setelah iterasi pertama dihitung berdasarkan Persamaan 3.8 dan menghasilkan nilai 11, sebagaimana

1 ditunjukkan pada Persamaan 3.17.

$$fitness = w_1 \cdot attackingViolation + w_2 \cdot adjacencyViolation = (1 \cdot 5) + (2 \cdot 3) = 5 + 6 = 11 \quad (3.17)$$

2 Nilai *fitness* ini lebih rendah dari *fitness* awal partikel 2 (yang sebesar 20), sehingga *pBest*
3 partikel 2 diperbarui dengan posisi baru. Selain itu, karena nilai ini (11) juga lebih rendah daripada
4 *fitness nBest* sebelumnya (yaitu 14 dari partikel 1), maka *nBest neighborhood* diperbarui menjadi
5 partikel 2.

6 Proses pembaruan kecepatan, pembaruan posisi, dan evaluasi *fitness* ini berulang pada iterasi-
7 iterasi berikutnya. Apabila salah satu partikel mencapai *fitness* bernilai 0 (yaitu konfigurasi tanpa
8 pelanggaran), algoritma berhenti dan melaporkan partikel tersebut sebagai solusi valid. Apabila
9 tidak, pencarian dilanjutkan hingga batas iterasi atau kondisi stagnasi tercapai.

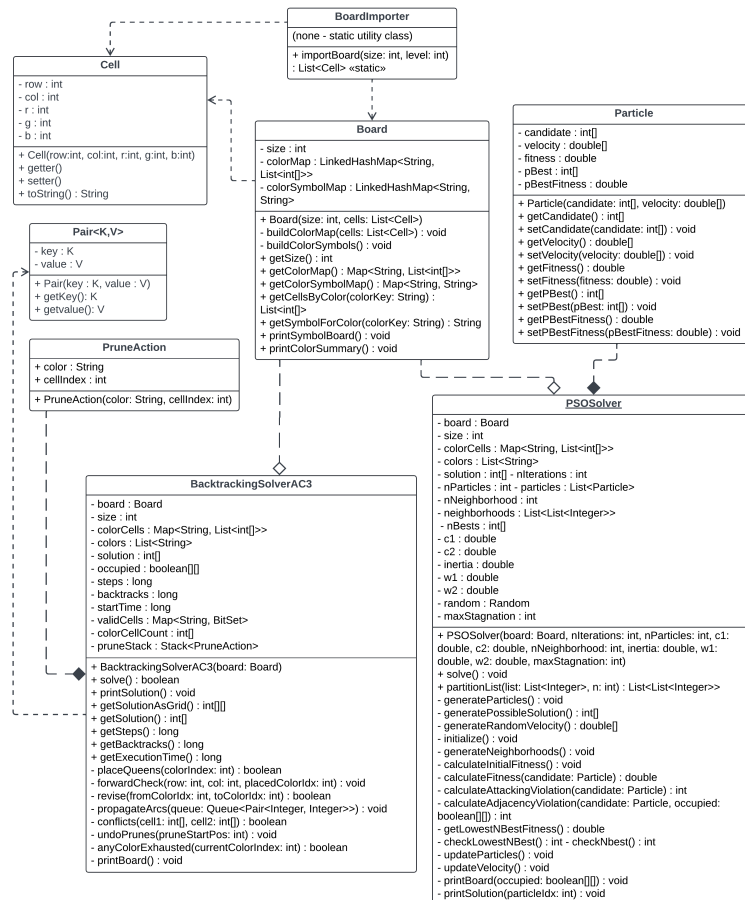
BAB 4

PERANCANGAN

4.1 Perancangan Perangkat Lunak

4.1.1 Desain Kelas

Sistem dibangun menggunakan arsitektur modular yang memisahkan tiga tanggung jawab utama, yaitu pengelolaan data papan, logika *solver*, dan antarmuka pengguna. Pemisahan ini memungkinkan kedua *solver* beroperasi secara independen terhadap representasi visual dan sebaliknya, sehingga perubahan pada salah satu bagian tidak berdampak pada bagian lainnya. Diagram kelas yang menunjukkan hubungan antarkeseluruhan kelas dalam sistem diilustrasikan pada Gambar 4.1.



Gambar 4.1: Diagram kelas sistem yang menunjukkan hubungan antara kelas data, kelas *solver*, dan kelas antarmuka.

Penjelasan rinci untuk setiap kelas, mencakup kegunaan, atribut, dan metode, dijabarkan pada bagian 4.1.1. Selain kelas-kelas tersebut, sistem juga memiliki dua titik masuk (*entry point*) yang terpisah. Titik masuk pertama adalah kelas **Main** yang menginisialisasi antarmuka pengguna dengan membuat *instance* dari **Homepage**, sehingga pengguna dapat memainkan *Colored Queens* melalui antarmuka grafis. Titik masuk kedua adalah kelas **BatchMain** yang digunakan secara khusus untuk keperluan eksperimen yang menjalankan kedua *solver* pada seluruh dataset puzzle secara otomatis dan paralel, kemudian mencatat metrik kinerja ke dalam berkas `.txt`. Kedua kelas ini tidak dijabarkan secara rinci karena hanya berfungsi sebagai pengelola yang memanggil kelas-kelas lain tanpa memiliki atribut atau relasi yang perlu dimodelkan. Rincian implementasi **BatchMain** dibahas pada Bagian 5.1.4.

11 **Kelas Cell**

Kelas **Cell** merepresentasikan satu sel pada papan permainan *Colored Queens*. Setiap objek **Cell** menyimpan posisi sel dalam koordinat baris dan kolom serta informasi warna dalam format RGB, sehingga kelas ini berfungsi sebagai satuan data terkecil yang digunakan untuk membangun struktur papan.

Atribut-atribut kelas **Cell** dirangkum pada Tabel 4.1.

Tabel 4.1: Atribut kelas **Cell**

Atribut	Tipe	Deskripsi
row	int	Koordinat baris sel pada papan (indeks berbasis nol)
col	int	Koordinat kolom sel pada papan (indeks berbasis nol)
r	int	Komponen <i>red</i> dari warna sel (0–255)
g	int	Komponen <i>green</i> dari warna sel (0–255)
b	int	Komponen <i>blue</i> dari warna sel (0–255)

Kelas ini menyediakan *constructor* yang menerima kelima atribut di atas, metode *getter* dan *setter* untuk setiap atribut, serta metode `toString()` yang mengembalikan representasi tekstual sel dalam format `Cell[row=_, col=_, rgb=(_,_,_)]` untuk keperluan *debugging*.

20 **Kelas Board**

Kelas **Board** merepresentasikan keseluruhan papan permainan *Colored Queens*. Kelas ini bertanggung jawab mengagregasi seluruh objek **Cell** ke dalam sebuah pemetaan yang mengelompokkan sel-sel berdasarkan warna, sehingga *solver* maupun antarmuka pengguna dapat mengakses struktur papan secara terstruktur.

Atribut-atribut kelas **Board** dirangkum pada Tabel 4.2.

Tabel 4.2: Atribut kelas Board

Atribut	Tipe	Deskripsi
size	int	Ukuran papan (n), menyatakan jumlah baris, kolom, dan sektor warna
colorMap	LinkedHashMap<String, List<int[]>>	Pemetaan dari kunci warna RGB (format "R,G,B") ke daftar koordinat [<i>baris</i> , <i>kolom</i>] seluruh sel yang memiliki warna tersebut. Penggunaan LinkedHashMap mempertahankan urutan insersi warna, yang berpengaruh pada urutan pemrosesan variabel oleh <i>solver</i>
colorSymbolMap	LinkedHashMap<String, String>	Pemetaan dari kunci warna RGB ke simbol huruf (A, B, C, dst.) untuk keperluan visualisasi tekstual papan

Pemilihan LinkedHashMap sebagai struktur data untuk colorMap dan colorSymbolMap, dan bukan HashMap biasa, didasarkan pada kebutuhan akan urutan iterasi yang deterministik. Kedua kelas tersebut memiliki kompleksitas akses (*lookup*) yang setara, yaitu $O(1)$ rata-rata. Perbedaannya terletak pada urutan iterasi, di mana HashMap mengiterasi entri-entri berdasarkan urutan internal *hash bucket* yang tidak terdefinisi dan dapat berubah antarsesi eksekusi, sedangkan LinkedHashMap mempertahankan urutan insersi entri. Pada konteks *solver*, urutan iterasi warna pada colorMap secara langsung menentukan urutan pemrosesan variabel oleh algoritma *backtracking*. Apabila urutan tersebut berubah antarsesi, perilaku *solver* (jumlah langkah, jumlah *backtrack*, dan cabang pencarian yang dieksplorasi) juga akan berubah, sehingga hasil eksperimen menjadi tidak *reproducible*. Dengan LinkedHashMap, urutan tersebut konsisten antarsesi, sehingga eksperimen dapat direplikasi secara persis dan perilaku *solver* dapat dianalisis dengan andal.

Metode-metode kelas Board dirangkum pada Tabel 4.3.

Tabel 4.3: Metode kelas Board

Metode	Deskripsi
Board(int, List<Cell>)	<i>Constructor</i> yang menerima ukuran papan dan daftar objek Cell, kemudian memanggil buildColorMap() dan buildColorSymbols() untuk menginisialisasi kedua peta internal
buildColorMap(List<Cell>)	Mengiterasi seluruh objek Cell dan mengelompokkan koordinat sel ke dalam colorMap berdasarkan kunci warna RGB

Metode	Deskripsi
<code>buildColorSymbols()</code>	Memetakan setiap kunci warna RGB yang unik ke sebuah simbol huruf berurutan (A, B, C, dst.) untuk representasi tekstual
<code>getSize()</code>	Mengembalikan ukuran papan
<code>getColorMap()</code>	Mengembalikan peta warna-ke-posisi
<code>getCellsByColor(String)</code>	Mengembalikan daftar koordinat seluruh sel yang memiliki warna tertentu
<code>getSymbolForColor(String)</code>	Mengembalikan simbol huruf yang merepresentasikan suatu kunci warna RGB
<code>printSymbolBoard()</code>	Mencetak papan ke konsol dengan setiap sel ditampilkan sebagai simbol huruf warnanya
<code>printColorSummary()</code>	Mencetak ringkasan distribusi warna papan, termasuk jumlah sel per warna dan total warna unik

1 Metode `buildColorMap()` merupakan metode inti yang dipanggil pada saat konstruksi objek.
2 Metode ini mengiterasi seluruh objek `Cell` yang diterima dari `BoardImporter`, membentuk kunci
3 warna berformat "R,G,B" dari komponen RGB setiap sel, dan memasukkan koordinat [*baris*, *kolom*]
4 sel tersebut ke dalam daftar yang bersesuaian pada `colorMap`. Hasil dari proses ini adalah sebuah
5 struktur pemetaan di mana setiap sektor warna terasosiasi dengan daftar lengkap posisi sel-selnya,
6 yang kemudian digunakan oleh kedua *solver* sebagai definisi domain variabel CSP.

7 Kelas `BoardImporter`

8 Kelas `BoardImporter` bertanggung jawab membaca dan mengurai berkas JSON papan permainan
9 menjadi daftar objek `Cell`. Kelas ini berfungsi sebagai jembatan antara data eksternal (berkas
10 JSON) dengan representasi internal sistem (objek `Cell` dan `Board`).

11 Kelas ini tidak memiliki atribut *instance* karena seluruh metodenya bersifat statis. Konstanta
12 `BOARD_FOLDER` menyimpan jalur direktori berkas papan. Metode-metode kelas `BoardImporter`
13 dirangkum pada Tabel 4.4.

Tabel 4.4: Metode kelas `BoardImporter`

Metode	Deskripsi
<code>importBoard(int, int)</code>	Menerima ukuran papan dan nomor level, membaca berkas JSON yang bersesuaian, mengurai setiap entri menjadi objek <code>Cell</code> , dan mengembalikan daftar seluruh sel

Metode	Deskripsi
<code>parseRgba(String)</code>	Mengurai <i>string</i> warna dalam format <code>rgba(R,G,B,A)</code> menjadi array komponen RGB bertipe integer
<code>printBoard(List<Cell>, int)</code>	Mencetak papan ke konsol dengan menampilkan nilai RGB setiap sel untuk keperluan <i>debugging</i>

1 Metode `importBoard()` merupakan metode utama kelas ini. Metode ini mengonstruksi nama
2 berkas berdasarkan format `{ukuran}x{ukuran}_level{nomor}.json`, membaca isi berkas sebagai
3 *string*, kemudian menguraikan *string* tersebut menjadi `JSONArray`. Untuk setiap objek JSON
4 dalam *array*, metode mengekstrak atribut `row`, `col`, dan `color`, memanggil `parseRgba()` untuk
5 mengonversi *string* warna menjadi komponen RGB, dan membentuk objek `Cell` yang ditambahkan
6 ke daftar hasil.

7 **Kelas BacktrackingSolverAC3**

8 Kelas `BacktrackingSolverAC3` mengimplementasikan *solver* berbasis *backtracking* yang diperkuat
9 dengan *forward checking* dan propagasi AC-3 untuk menyelesaikan permasalahan *Colored Queens*.
10 Kelas ini juga mendefinisikan dua kelas bantu internal, yaitu `PruneAction` yang mencatat infor-
11 masi pemangkasan domain untuk keperluan pemulihan, dan `Pair<K,V>` yang digunakan sebagai
12 representasi busur pada antrean AC-3.

13 Atribut-atribut kelas `BacktrackingSolverAC3` dirangkum pada Tabel 4.5.

Tabel 4.5: Atribut kelas `BacktrackingSolverAC3`

Atribut	Tipe	Deskripsi
<code>board</code>	<code>Board</code>	Objek papan yang menjadi masukan <i>solver</i>
<code>size</code>	<code>int</code>	Ukuran papan ($n \times n$)
<code>colorCells</code>	<code>Map<String, List<int[]>></code>	Pemetaan warna ke daftar koordinat sel, diperoleh dari <code>Board</code>
<code>colors</code>	<code>List<String></code>	Daftar kunci warna yang menentukan urutan pemrosesan variabel
<code>solution</code>	<code>int[]</code>	Array solusi di mana <code>solution[i]</code> menyimpan indeks sel terpilih dalam domain warna ke- <i>i</i> ; bernilai -1 apabila belum ditugaskan

Atribut	Tipe	Deskripsi
occupied	boolean[] []	Matriks yang menandai posisi sel yang telah ditempati menteri
validCells	Map<String, BitSet>	Peta dari kunci warna ke <code>BitSet</code> yang merepresentasikan status kevalidan setiap sel dalam domain; bit bernilai <code>true</code> apabila sel masih valid
colorCellCount	int[]	Pencacah ukuran domain efektif setiap variabel warna
pruneStack	Stack<PruneAction>	Tumpukan yang mencatat seluruh pemangkasan domain secara kronologis untuk mendukung pemulihan saat <i>backtrack</i>
steps	long	Pencacah jumlah penugasan yang berhasil dilanjutkan ke tingkat rekursi berikutnya
backtracks	long	Pencacah jumlah pembatalan penugasan
startTime	long	Waktu mulai eksekusi dalam milidetik

1 Metode-metode kelas `BacktrackingSolverAC3` dirangkum pada Tabel 4.6.

Tabel 4.6: Metode kelas `BacktrackingSolverAC3`

Metode	Deskripsi
<code>solve()</code>	Metode publik yang memulai proses pencarian dengan memanggil <code>placeQueens(0)</code> dan mencatat waktu eksekusi
<code>placeQueens(int)</code>	Prosedur rekursif utama yang menugaskan menteri untuk setiap variabel warna secara berurutan
<code>forwardCheck(int, int, int)</code>	Memangkas sel-sel yang berkonflik dengan menteri yang baru ditempatkan dari domain seluruh variabel yang belum ditugaskan
<code>revise(int, int)</code>	Memeriksa apakah setiap sel dalam domain variabel tujuan memiliki <i>support</i> yang konsisten di variabel sumber

Metode	Deskripsi
<code>propagateArcs(Queue)</code>	Menjalankan propagasi busur AC-3 untuk menegakkan konsistensi busur antara seluruh pasang variabel yang belum ditugaskan
<code>conflicts(int[], int[])</code>	Mengembalikan <code>true</code> apabila dua sel berkonflik (baris sama, kolom sama, atau bertetangga)
<code>undoPrunes(int)</code>	Memulihkan seluruh pemangkasan yang dicatat pada <code>pruneStack</code> sejak posisi <i>checkpoint</i> yang diberikan
<code>anyColorExhausted(int)</code>	Memeriksa apakah terdapat variabel yang belum ditugaskan dengan domain berukuran nol
<code>getSolutionAsGrid()</code>	Mengonversi solusi berbasis indeks menjadi matriks koordinat [<i>baris</i> , <i>kolom</i>] untuk digunakan oleh antarmuka
<code>getSteps(), getBacktracks()</code>	Mengembalikan metrik pencarian

Representasi domain setiap variabel pada *solver backtracking* menggunakan `BitSet`, yaitu struktur data Java yang menyimpan sekumpulan nilai biner (`true/false`) secara padat dalam bentuk *array of bits*. Setiap warna memiliki satu `BitSet` dengan panjang yang sama dengan jumlah sel pada domain awalnya, di mana bit pada indeks ke-*i* bernilai `true` apabila sel ke-*i* masih valid sebagai kandidat penempatan, dan `false` apabila sel tersebut telah dipangkas oleh *forward checking* atau AC-3. Pemilihan `BitSet` dibandingkan struktur alternatif seperti `List<Integer>` atau `boolean[]` didasarkan pada tiga keunggulan utama. Pertama, operasi pemangkasan dan pemulihan domain dapat dilakukan dalam waktu $O(1)$ melalui metode `clear(i)` dan `set(i)`; pendekatan berbasis `List` memerlukan penghapusan dari posisi tengah yang berbiaya $O(n)$ dan menyulitkan pemulihan saat *backtrack*. Kedua, `BitSet` menyediakan metode `nextSetBit(n)` yang secara efisien mencari indeks bit `true` berikutnya, sehingga iterasi atas sel-sel yang masih valid dapat dilakukan tanpa memeriksa sel-sel yang telah dipangkas. Pendekatan berbasis `boolean[]` memerlukan perulangan eksplisit yang memeriksa setiap elemen. Ketiga, `BitSet` menggunakan memori secara efisien dengan menyimpan 64 bit per *long*, dibandingkan dengan satu *byte* per elemen pada `boolean[]`. Mekanisme pencatatan dan pemulihan pemangkasan melalui `pruneStack` bekerja secara sinergis dengan `BitSet`, dengan cara setiap kali sebuah bit di-`clear` (pemangkasan), aksi tersebut dicatat pada *stack* sebagai objek `PruneAction`, dan saat *backtrack* dilakukan, aksi-aksi tersebut di-*undo* dengan memanggil `set` pada bit-bit yang dipangkas.

Berikutnya, Metode `placeQueens()` merupakan inti dari kelas ini. Metode ini bekerja secara rekursif *depth-first*, memproses variabel warna satu per satu sesuai urutan yang ditentukan oleh `colors`. Untuk setiap sel valid dalam domain variabel saat ini, metode ini menempatkan menteri, menjalankan *forward checking* dan propagasi AC-3, memeriksa apakah terdapat domain yang kosong, dan apabila tidak ada, melanjutkan pencarian ke variabel berikutnya secara rekursif. Apabila pencarian gagal, seluruh pemangkasan yang telah dilakukan dipulihkan melalui mekanisme

- 1 `pruneStack` dan sel berikutnya dicoba. Pseudocode metode ini ditunjukkan pada Algoritma 4.

Algorithm 4 Prosedur rekursif `placeQueens` pada kelas `BacktrackingSolverAC3`

```

1: function PLACEQUEENS(colorIndex)
2:   if colorIndex =  $|C|$  then
3:     return true                                     ▷ Seluruh variabel telah ditugaskan
4:   end if
5:   color  $\leftarrow C[\textit{colorIndex}]$ 
6:   if  $|D_{\textit{color}}| = 0$  then
7:     return false                                     ▷ Domain kosong, gagal
8:   end if
9:   for setiap sel s dalam  $D_{\textit{color}}$  yang masih valid do
10:    Tempatkan menteri pada sel s
11:    checkpoint  $\leftarrow$  posisi pruneStack saat ini
12:    FORWARDCHECK(s.row, s.col, colorIndex)
13:    PROPAGATEARCS(antrean busur variabel yang belum ditugaskan)
14:    if tidak ada domain variabel lain yang kosong then
15:      if PLACEQUEENS(colorIndex + 1) then
16:        return true
17:      end if
18:    end if
19:    UNDOPRUNES(checkpoint)
20:    Batalkan penempatan menteri pada sel s
21:  end for
22:  return false
23: end function

```

- 2 Metode `forwardCheck()` dipanggil setiap kali sebuah menteri ditempatkan. Metode ini mengi-
3 terasi seluruh variabel warna yang belum ditugaskan dan memeriksa setiap sel valid dalam domain
4 variabel tersebut terhadap posisi menteri yang baru ditempatkan. Sel yang berada pada baris atau
5 kolom yang sama, atau yang bertetangga langsung dalam delapan arah, dihapus dari domain dengan
6 menonaktifkan bit yang bersesuaian pada `BitSet`. Setiap pemangkasan dicatat pada `pruneStack`.
7 Pseudocode metode ini tertera pada Algoritma 5.

Algorithm 5 Prosedur `forwardCheck` pada kelas `BacktrackingSolverAC3`

```

1: function FORWARDCHECK(row, col, placedColorIdx)
2:   for colorIdx  $\leftarrow \textit{placedColorIdx} + 1$  hingga  $|C| - 1$  do
3:     valid  $\leftarrow$  BitSet domain warna colorIdx
4:     for setiap indeks i yang masih aktif dalam valid do
5:        $(r_2, k_2) \leftarrow$  koordinat sel ke-i dari warna colorIdx
6:       if  $\textit{row} = r_2 \vee \textit{col} = k_2 \vee (|\textit{row} - r_2| \leq 1 \wedge |\textit{col} - k_2| \leq 1)$  then
7:         Hapus bit i dari valid
8:         Kurangi colorCellCount[colorIdx]
9:         Catat (colorIdx, i) pada pruneStack
10:      end if
11:    end for
12:  end for
13: end function

```

- 8 Metode `revise()` merupakan komponen inti dari propagasi AC-3. Untuk setiap sel yang masih

1 valid dalam domain variabel tujuan, metode ini memeriksa apakah terdapat setidaknya satu sel
 2 dalam domain variabel sumber yang tidak berkonflik (*support*). Selain itu, sel tujuan juga diperiksa
 3 terhadap seluruh menteri yang telah ditempatkan. Apabila tidak ditemukan *support* yang valid,
 4 sel tersebut dihapus dari domain dan dicatat pada *pruneStack*. Pseudocode metode ini diuraikan
 5 pada Algoritma 6.

Algorithm 6 Fungsi *revise* pada kelas *BacktrackingSolverAC3*

```

1: function REVISE(fromColorIdx, toColorIdx)
2:   revised  $\leftarrow$  false
3:   for setiap indeks i yang masih aktif dalam  $D_{to}$  do
4:     supported  $\leftarrow$  false
5:     for setiap indeks j yang masih aktif dalam  $D_{from}$  do
6:       if  $\neg$  konflik( $sel_j^{from}$ ,  $sel_i^{to}$ ) then
7:         supported  $\leftarrow$  true
8:         break
9:       end if
10:    end for
11:    if supported then
12:      for setiap variabel p yang telah ditugaskan,  $p \neq toColorIdx$  do
13:        if konflik( $sel_p^{placed}$ ,  $sel_i^{to}$ ) then
14:          supported  $\leftarrow$  false
15:          break
16:        end if
17:      end for
18:    end if
19:    if  $\neg$  supported then
20:      Hapus bit i dari  $D_{to}$ 
21:      Kurangi colorCellCount[toColorIdx]
22:      Catat (toColorIdx, i) pada pruneStack
23:      revised  $\leftarrow$  true
24:    end if
25:  end for
26:  return revised
27: end function

```

6 Metode *propagateArcs()* menjalankan propagasi busur AC-3 secara iteratif. Antrean diinisi-
 7 alisasi dengan seluruh pasang busur (*i*, *j*) untuk $i \neq j$ di antara variabel yang belum ditugaskan.
 8 Untuk setiap busur yang diambil dari antrean, metode *revise()* dipanggil. Apabila domain variabel
 9 tujuan mengalami perubahan, busur-busur baru yang melibatkan variabel tersebut ditambahkan
 10 ke antrean untuk propagasi lebih lanjut. Propagasi berhenti apabila antrean kosong atau domain
 11 suatu variabel menjadi kosong. Pseudocode metode ini terdapat pada Algoritma 7.

12 Kelas Particle

13 Kelas *Particle* merepresentasikan satu partikel dalam populasi PSO. Setiap partikel menyimpan
 14 sebuah kandidat solusi lengkap, vektor kecepatan yang diinterpretasikan sebagai probabilitas
 15 perubahan per dimensi, serta informasi *personal best* yang dicapai selama proses optimasi.

16 Atribut-atribut kelas *Particle* dirangkum pada Tabel 4.7.

Algorithm 7 Prosedur `propagateArcs` pada kelas `BacktrackingSolverAC3`

```

1: function PROPAGATEARCS(queue)
2:   while queue tidak kosong do
3:     (from, to)  $\leftarrow$  ambil busur dari queue
4:     if REVISE(from, to) then
5:       if  $|D_{to}| = 0$  then
6:         return ▷ Domain kosong, hentikan propagasi
7:       end if
8:       for setiap variabel  $k$ ,  $k \neq to$ ,  $k \neq from$  do
9:         Tambahkan busur (to,  $k$ ) ke queue
10:      end for
11:    end if
12:  end while
13: end function

```

Tabel 4.7: Atribut kelas `Particle`

Atribut	Tipe	Deskripsi
<code>candidate</code>	<code>int[]</code>	Posisi saat ini; <code>candidate[i]</code> menyimpan indeks sel terpilih dalam domain warna ke- i
<code>velocity</code>	<code>double[]</code>	Vektor kecepatan; <code>velocity[j] $\in$ [0, 1]</code> menyatakan probabilitas perubahan posisi pada dimensi j
<code>fitness</code>	<code>double</code>	Nilai <i>fitness</i> saat ini, dihitung dari jumlah pelanggaran kendala berbobot
<code>pBest</code>	<code>int[]</code>	Posisi terbaik yang pernah dicapai oleh partikel ini
<code>pBestFitness</code>	<code>double</code>	Nilai <i>fitness</i> dari posisi <code>pBest</code>

1 Kelas ini menyediakan *constructor* yang menerima array posisi awal dan array kecepatan awal,
2 serta metode *getter* dan *setter* untuk seluruh atribut. Kelas `Particle` tidak mengandung logika
3 komputasi; seluruh perhitungan *fitness* dan pembaruan posisi dikelola oleh kelas `PSOSolver`.

4 Kelas PSOSolver

5 Kelas `PSOSolver` mengimplementasikan *solver* berbasis *Particle Swarm Optimization* diskrit. Ke-
6 las ini mengelola populasi partikel, topologi *neighborhood*, serta seluruh mekanisme pembaruan
7 kecepatan, posisi, dan *fitness*.

8 Atribut-atribut kelas `PSOSolver` dirangkum pada Tabel 4.8.

Tabel 4.8: Atribut kelas PSOSolver

Atribut	Tipe	Deskripsi
board	Board	Objek papan yang menjadi masukan <i>solver</i>
size	int	Ukuran papan
colorCells	Map<String, List<int[]>>	Pemetaan warna ke daftar koordinat sel
colors	List<String>	Daftar kunci warna
solution	int[]	Array solusi
particles	List<Particle>	Populasi partikel
neighborhoods	List<List<Integer>>	Partisi indeks partikel ke dalam kelompok <i>neighborhood</i>
nBests	int[]	Indeks partikel terbaik dalam setiap <i>neighborhood</i>
nIterations	int	Batas maksimum iterasi ($I_{\max}$)
nParticles	int	Jumlah partikel (P)
nNeighborhood	int	Jumlah kelompok <i>neighborhood</i> (N_{nh})
c1, c2	double	Koefisien akselerasi kognitif dan sosial
inertia	double	Koefisien inersia (w)
w1, w2	double	Bobot komponen <i>attacking</i> dan <i>adjacency</i> pada fungsi <i>fitness</i>
maxStagnation	int	Batas iterasi tanpa perbaikan sebelum terminasi dini ($S_{\max}$)
random	Random	Generator bilangan acak

1 Metode-metode kelas PSOSolver dirangkum pada Tabel 4.9.

Tabel 4.9: Metode kelas PSOSolver

Metode	Deskripsi
solve()	Metode publik utama yang menjalankan keseluruhan proses optimasi PSO

Metode	Deskripsi
<code>initialize()</code>	Menjalankan tiga tahap inisialisasi: pembangkitan partikel, pembentukan <i>neighborhood</i> , dan evaluasi <i>fitness</i> awal
<code>generateParticles()</code>	Membangkitkan P partikel dengan posisi dan kecepatan acak
<code>generateNeighborhoods()</code>	Mempartisi seluruh partikel ke dalam N_{nh} kelompok <i>neighborhood</i> dengan ukuran seimbang
<code>calculateInitialFitness()</code>	Mengevaluasi <i>fitness</i> awal seluruh partikel dan menentukan nBest setiap <i>neighborhood</i>
<code>calculateFitness(Particle)</code>	Menghitung nilai <i>fitness</i> partikel berdasarkan pelanggaran baris/kolom dan <i>adjacency</i> berbobot
<code>calculateAttackingViolation(Particle)</code>	Menghitung jumlah pasangan menteri yang berada pada baris atau kolom yang sama
<code>calculateAdjacencyViolation(Particle, boolean[][])</code>	Menghitung jumlah pasangan menteri yang bertetangga langsung dalam delapan arah
<code>updateVelocity()</code>	Memperbarui kecepatan seluruh partikel berdasarkan formula PSO diskrit
<code>updateParticles()</code>	Memperbarui posisi seluruh partikel berdasarkan kecepatan, mengevaluasi ulang <i>fitness</i> , dan memperbarui pBest serta nBest
<code>checkNbest()</code>	Memeriksa apakah salah satu nBest memiliki <i>fitness</i> bernilai nol (solusi valid ditemukan)
<code>checkLowestNBest()</code>	Mengembalikan indeks partikel dengan <i>fitness</i> terendah di antara seluruh nBest
<code>getLowestNBestFitness()</code>	Mengembalikan nilai <i>fitness</i> terendah di antara seluruh nBest untuk deteksi stagnasi

1 Metode `solve()` merupakan metode utama yang mengorkestrasi keseluruhan proses PSO. Setelah
2 inisialisasi populasi, metode ini menjalankan *loop* utama yang pada setiap iterasi memperbarui
3 kecepatan seluruh partikel, memperbarui posisi dan mengevaluasi ulang *fitness*, serta memeriksa tiga
4 kondisi terminasi, antar lain ditemukannya solusi valid, tercapainya batas iterasi, atau terdeteksinya
5 stagnasi. Pseudocode metode ini dapat dilihat pada Algoritma 8.

6 Metode `updateVelocity()` dan `updateParticles()` bersama-sama mengimplementasikan me-
7 kanisme eksplorasi PSO diskrit. Pada `updateVelocity()`, kecepatan baru setiap dimensi dihitung
8 menggunakan komponen inersia, kognitif, dan sosial dengan indikator perbedaan biner, kemudian

Algorithm 8 Prosedur `solve` pada kelas `PSOSolver`

```

1: function SOLVE
2:   Inisialisasi populasi, neighborhood, dan fitness awal
3:    $f_{globalBest} \leftarrow fitness$  terendah di antara seluruh nBest
4:   stagnasi  $\leftarrow 0$ 
5:   for iter  $\leftarrow 1$  hingga  $I_{max}$  do
6:     Perbarui kecepatan seluruh partikel
7:     Perbarui posisi dan evaluasi fitness seluruh partikel
8:     if terdapat nBest dengan fitness = 0 then
9:       return solusi valid ditemukan
10:    end if
11:     $f_{current} \leftarrow fitness$  terendah di antara seluruh nBest
12:    if  $f_{current} < f_{globalBest}$  then
13:       $f_{globalBest} \leftarrow f_{current}$ 
14:      stagnasi  $\leftarrow 0$ 
15:    else
16:      stagnasi  $\leftarrow stagnasi + 1$ 
17:    end if
18:    if stagnasi  $\geq S_{max}$  then
19:      return terminasi dini, laporkan solusi terbaik
20:    end if
21:  end for
22:  return batas iterasi tercapai, laporkan solusi terbaik
23: end function

```

1 di-*clamp* pada interval $[0, 1]$. Pada `updateParticles()`, kecepatan digunakan sebagai probabilitas
2 untuk mengadopsi posisi *neighborhood best* pada setiap dimensi, kemudian *fitness* dievaluasi ulang
3 dan **pBest** serta **nBest** diperbarui apabila ditemukan nilai yang lebih baik. Pseudocode gabungan
4 kedua metode ini disajikan pada Algoritma 9.

5 **Kelas Main**

6 Kelas **Main** berfungsi sebagai titik masuk aplikasi. Kelas ini hanya berisi satu metode `main()` yang
7 menginisialisasi objek **Homepage** pada *Event Dispatch Thread* melalui `SwingUtilities.invokeLater()`
8 untuk memastikan seluruh operasi antarmuka grafis berjalan pada *thread* yang tepat sesuai konvensi
9 Java Swing.

10 **Kelas Homepage**

11 Kelas **Homepage** merupakan subkelas dari **JFrame** yang merepresentasikan halaman utama aplikasi.
12 Halaman ini menampilkan judul permainan, ikon menteri, dan tombol *Start Game*. Kelas ini tidak
13 memuat logika permainan; fungsi utamanya adalah menyediakan titik masuk visual bagi pengguna.
14 Ketika tombol *Start Game* ditekan, kelas ini membuat *instance* baru dari **LevelSelectorWindow**
15 dan menutup jendela saat ini.

Algorithm 9 Prosedur `updateVelocity` dan `updateParticles` pada kelas `PSOSolver`

```

1: for setiap neighborhood  $N_i$  do
2:    $\mathbf{x}_{nBest} \leftarrow$  posisi nBest dari  $N_i$ 
3:   for setiap partikel  $p$  dalam  $N_i$  do
4:                                      $\triangleright$  Pembaruan kecepatan
5:     for  $j \leftarrow 0$  hingga  $n - 1$  do
6:        $r_1, r_2 \leftarrow$  bilangan acak  $\in [0, 1)$ 
7:        $\delta_{pBest} \leftarrow \mathbf{1}[x_j \neq pBest_j]$ 
8:        $\delta_{nBest} \leftarrow \mathbf{1}[x_j \neq nBest_j]$ 
9:        $v_j \leftarrow \text{clamp}(w \cdot v_j + c_1 \cdot r_1 \cdot \delta_{pBest} + c_2 \cdot r_2 \cdot \delta_{nBest}, 0, 1)$ 
10:    end for
11:                                      $\triangleright$  Pembaruan posisi
12:    for  $j \leftarrow 0$  hingga  $n - 1$  do
13:       $r \leftarrow$  bilangan acak  $\in [0, 1)$ 
14:      if  $r < v_j$  then
15:         $x_j \leftarrow nBest_j$ 
16:      end if
17:    end for
18:    Evaluasi ulang fitness partikel  $p$ 
19:    Perbarui pBest dan nBest jika diperlukan
20:  end for
21: end for

```

1 Kelas LevelSelectorWindow

2 Kelas `LevelSelectorWindow` merupakan subkelas dari `JFrame` yang menyajikan halaman pemilihan
3 level. Halaman ini menampilkan pilihan ukuran papan yang dikelompokkan berdasarkan tingkat
4 kesulitan (*Easy*, *Medium*, *Hard*, dan *Challenge*). Setiap tombol ukuran papan, ketika ditekan,
5 membuat *instance* baru dari `GameWindow` dengan ukuran dan level awal yang bersesuaian.

6 Kelas GameWindow

7 Kelas `GameWindow` merupakan subkelas dari `JFrame` yang merepresentasikan halaman permainan
8 utama. Kelas ini mengintegrasikan seluruh komponen permainan, yang termasuk *rendering* papan,
9 interaksi pengguna, eksekusi *solver* secara asinkron, pemeriksaan pelanggaran, fitur *hint*, dan
10 fitur *auto-solve*. Kelas ini juga mendefinisikan kelas dalam `BoardPanel` yang bertanggung jawab
11 menggambar papan permainan.

12 Atribut-atribut utama kelas `GameWindow` dirangkum pada Tabel 4.10.

Tabel 4.10: Atribut utama kelas `GameWindow`

Atribut	Tipe	Deskripsi
<code>currentSize</code>	<code>int</code>	Ukuran papan yang sedang diminta
<code>currentLevel</code>	<code>int</code>	Nomor level yang sedang diminta

Atribut	Tipe	Deskripsi
<code>board</code>	<code>Board</code>	Objek papan yang sedang aktif
<code>solution</code>	<code>int[][]</code>	Solusi yang dihitung oleh <i>solver</i> , digunakan oleh fitur <i>hint</i> dan <i>auto-solve</i>
<code>queenPlaced</code>	<code>boolean[][]</code>	Matriks yang menandai posisi sel tempat pengguna telah menempatkan menteri
<code>currentWorker</code>	<code>SwingWorker</code>	<i>Worker thread</i> yang menjalankan <i>solver</i> secara asinkron di latar belakang
<code>gameTimer</code>	<code>Timer</code>	Pencacah waktu permainan (<i>stopwatch</i>)
<code>boardPanel</code>	<code>BoardPanel</code>	Panel yang merender papan permainan

`SwingWorker` adalah kelas utilitas pada Java Swing yang dirancang untuk menjalankan tugas berdurasi panjang (*long-running task*) pada *thread* terpisah tanpa membekukan antarmuka pengguna. Pada arsitektur Swing, seluruh pembaruan antarmuka dilakukan pada satu *thread* khusus yang disebut *Event Dispatch Thread* (EDT). Apabila sebuah komputasi berat dijalankan langsung pada EDT, seluruh antarmuka akan menjadi tidak responsif selama komputasi tersebut berlangsung, semisal tombol tidak dapat ditekan, jendela tidak dapat di-*repaint*, dan input pengguna terabaikan. `SwingWorker` mengatasi permasalahan ini dengan menyediakan dua metode utama, yaitu `doInBackground()` yang dieksekusi pada *thread* latar belakang dan berisi komputasi berat, serta `done()` yang dieksekusi kembali pada EDT setelah `doInBackground()` selesai dan digunakan untuk memperbarui antarmuka berdasarkan hasil komputasi. Pada aplikasi yang dibangun, solver *backtracking* dijalankan di dalam `doInBackground()`, sehingga proses pencarian solusi tidak menghambat interaksi pengguna dengan papan permainan. Saat solusi ditemukan, `done()` dipanggil secara aman pada EDT untuk menyimpan solusi tersebut ke dalam variabel `solution` yang kemudian digunakan oleh fitur *hint* dan *auto-solve*.

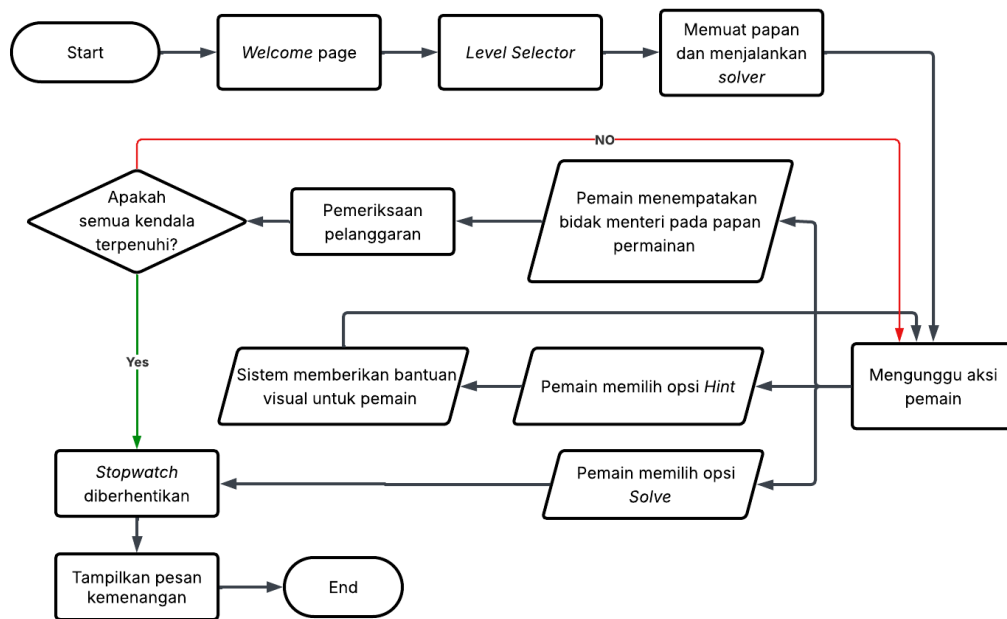
Metode-metode utama kelas `GameWindow` mencakup `loadBoard()` yang membaca berkas papan dan menjalankan *solver* secara asinkron, `checkViolations()` yang memeriksa seluruh pasangan menteri terhadap kendala dan menandai pelanggaran secara visual, serta `giveHint()` yang membandingkan penempatan pengguna dengan solusi dan memberikan petunjuk visual. Kelas dalam `BoardPanel` menggambar tiga lapisan secara berurutan pada metode `paintComponent()`, yang terdiri dari latar belakang berwarna setiap sel, garis grid pembatas, dan ikon menteri pada posisi yang telah ditempati.

1 Kelas `UITheme`

2 Kelas `UITheme` menyediakan konstanta visual dan komponen antarmuka yang digunakan secara
 3 konsisten di seluruh halaman aplikasi. Kelas ini mendefinisikan palet warna (seperti `CREAM`, `PLUM`,
 4 `ROSE`, `TAUPE`, `TEXT_DARK`, dan `TEXT_MUTED`), metode utilitas untuk membuat objek `Font` dengan
 5 tipografi yang konsisten, metode `roundedButton()` untuk membuat tombol dengan sudut membulat,
 6 serta kelas dalam `GradientPanel` yang merender latar belakang gradien pada setiap halaman.

7 4.1.2 Antarmuka Pengguna

8 Antarmuka pengguna terdiri atas tiga layar utama yang dinavigasi secara berurutan. Alur interaksi
 9 pengguna dengan aplikasi diilustrasikan pada Gambar 4.2.



Gambar 4.2: Alur interaksi pengguna pada aplikasi *Colored Queens*.

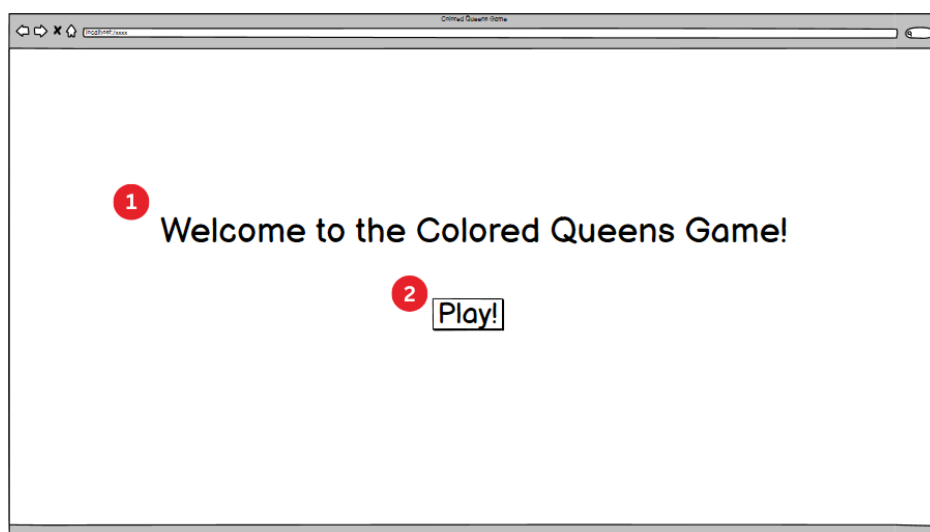
10 Berikut adalah penjelasan setiap proses pada Gambar 4.2:

- 11 1. **Welcome Page.** Layar pertama (`Homepage`) menampilkan judul permainan, ikon menteri,
 12 dan tombol *Start Game* yang mengarahkan pengguna ke halaman pemilihan level. Halaman
 13 ini berfungsi sebagai titik masuk visual dan tidak memuat logika permainan.
- 14 2. **Level Selector.** Layar kedua (`LevelSelectorWindow`) menyajikan pilihan ukuran papan
 15 yang dikelompokkan berdasarkan tingkat kesulitan, yaitu *Easy* (7×7), *Medium* (8×8 dan
 16 9×9), serta *Hard* (10×10 , 11×11 , dan 12×12). Selain itu, terdapat kategori *Challenge*
 17 yang menyediakan papan berukuran 20×20 dan 30×30 dengan masing-masing satu level.
 18 Pemilihan ukuran papan langsung membuka halaman permainan pada level pertama dari
 19 ukuran tersebut.
- 20 3. **Memuat papan dan menjalankan solver.** Setelah pengguna memilih level, kelas `GameWindow`
 21 membaca berkas JSON melalui `BoardImporter`, mengonstruksi objek `Board`, dan menjalank-
 22 an solver *backtracking* secara asinkron menggunakan `SwingWorker`. Solver berjalan di latar
 23 belakang agar antarmuka tetap responsif selama proses pencarian berlangsung seperti yang

sudah dijelaskan pada Bagian 4.1.1. Untuk papan berukuran 20×20 dan 30×30 , solusi disimpan secara *hardcoded* karena waktu komputasi solver pada ukuran tersebut terlalu lama untuk dieksekusi secara langsung. Solusi yang dihasilkan solver disimpan dan digunakan oleh fitur *hint* dan *auto-solve*.

4. **Menunggu aksi pemain.** Sistem memasuki kondisi menunggu, di mana papan ditampilkan sebagai grid berwarna di tengah jendela disertai panel kontrol di bagian bawah. Dari kondisi ini, pengguna dapat melakukan salah satu dari tiga aksi berikut.
5. **Pemain menempatkan bidak menteri pada papan permainan.** Pengguna menempatkan atau menghapus menteri dengan mengklik sel pada papan. Pencacah waktu mulai berjalan saat pengguna menempatkan menteri pertama.
6. **Pemeriksaan pelanggaran.** Setiap kali pengguna menempatkan atau menghapus menteri, sistem memeriksa seluruh pasangan menteri terhadap kendala baris, kolom, *adjacency*, dan kesamaan warna. Menteri yang terlibat dalam pelanggaran ditandai dengan ikon berwarna merah.
7. **Apakah semua kendala terpenuhi?** Sistem memeriksa apakah jumlah menteri yang ditempatkan sama dengan ukuran papan dan seluruh posisi sesuai dengan solusi. Apabila tidak, sistem kembali ke kondisi menunggu aksi pemain.
8. **Pemain memilih opsi *Hint*.** Sistem menampilkan petunjuk visual berupa sorotan hijau pada posisi yang benar dan sorotan oranye pada posisi yang salah untuk satu sektor warna. Setelah petunjuk ditampilkan, sistem kembali ke kondisi menunggu aksi pemain.
9. **Pemain memilih opsi *Solve*.** Solusi lengkap yang telah dihitung di latar belakang diterapkan secara otomatis pada papan.
10. ***Stopwatch* diberhentikan dan pesan kemenangan ditampilkan.** Pencacah waktu dihentikan dan dialog kemenangan ditampilkan kepada pengguna.

Rancangan antarmuka untuk setiap layar beserta elemen-elemen interaktifnya dapat dilihat pada Gambar 4.3 hingga Gambar 4.13.

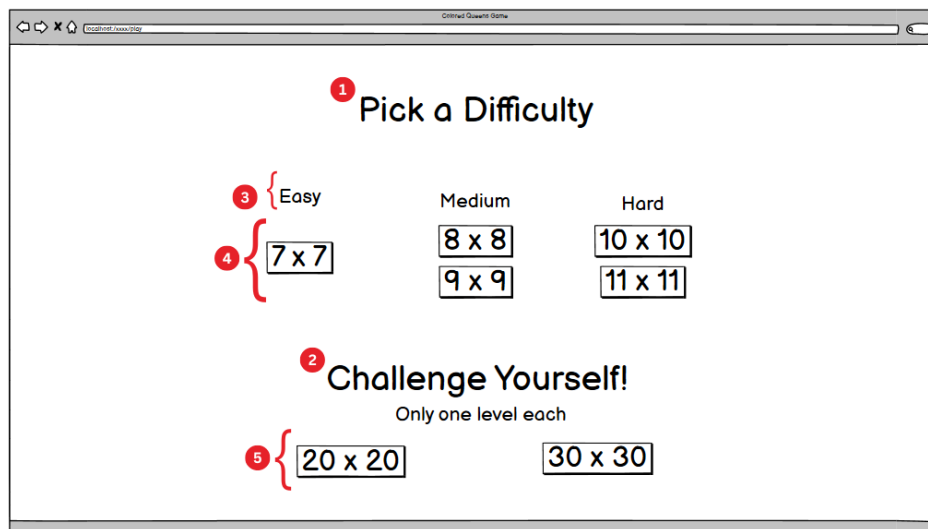


Gambar 4.3: Rancangan halaman utama (*Homepage*).

Penjelasan elemen pada Gambar 4.3:

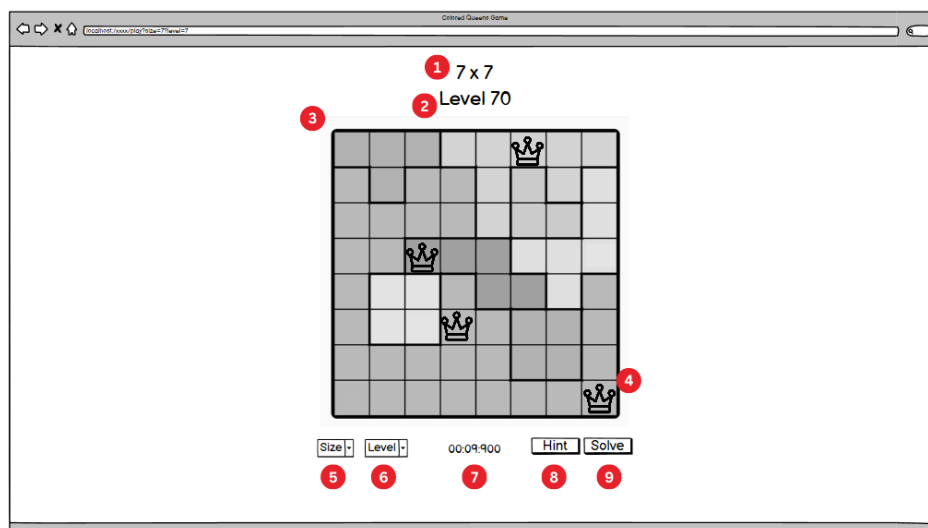
1. Judul aplikasi yang menampilkan nama permainan.

- 1 2. Tombol *Play* yang mengarahkan pengguna ke halaman pemilihan level.



Gambar 4.4: Rancangan halaman pemilihan level (*Level Selector*).

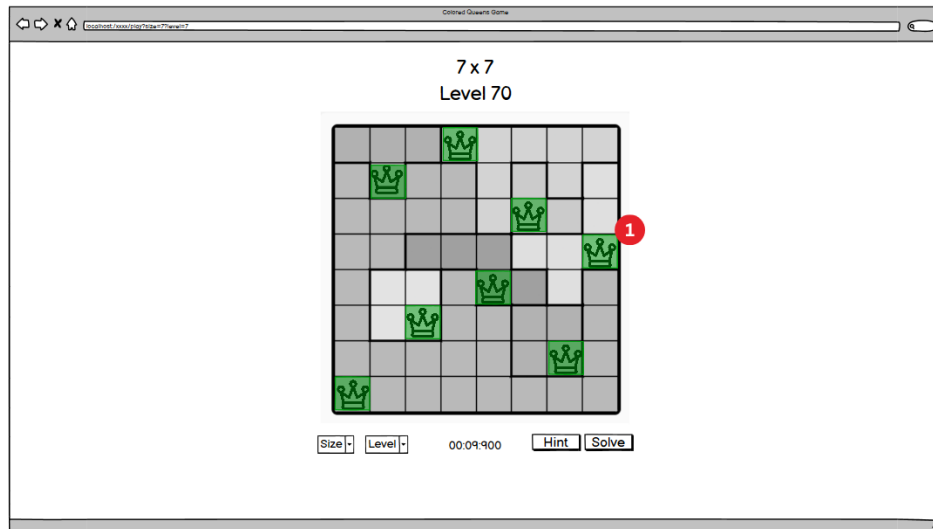
- 2 Penjelasan elemen pada Gambar 4.4:
- 3 1. Judul bagian pemilihan tingkat kesulitan.
- 4 2. Judul bagian tantangan (*Challenge*) untuk papan berukuran besar.
- 5 3. Label kategori kesulitan (*Easy, Medium, Hard*) yang mengelompokkan ukuran papan.
- 6 4. Tombol ukuran papan yang dapat dipilih oleh pengguna untuk memulai permainan.
- 7 5. Tombol ukuran papan pada kategori *Challenge* (20×20 dan 30×30).



Gambar 4.5: Rancangan halaman permainan (*Game Window*).

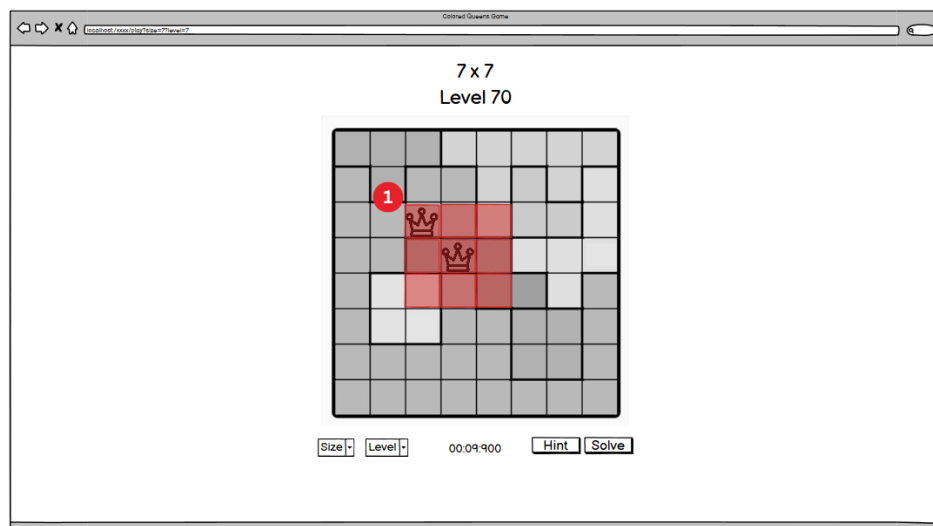
- 8 Penjelasan elemen pada Gambar 4.5:
- 9 1. Label ukuran papan yang sedang dimainkan.
- 10 2. Label nomor level yang sedang dimainkan.
- 11 3. Papan permainan berupa grid berwarna tempat pengguna menempatkan bidak menteri.
- 12 4. Bidak menteri yang telah ditempatkan pada papan. Ikon berwarna hitam digunakan pada
- 13 sel terang dan ikon berwarna putih pada sel gelap, dengan pemilihan berdasarkan luminansi

- 1 warna sel.
- 2 5. *Dropdown* ukuran papan untuk berpindah ke ukuran lain.
- 3 6. *Dropdown* level untuk berpindah ke level lain dalam ukuran yang sama.
- 4 7. Pencacah waktu (*stopwatch*) yang mulai berjalan saat pengguna menempatkan menteri pertama.
- 5 pertama.
- 6 8. Tombol *Hint* yang menampilkan petunjuk visual.
- 7 9. Tombol *Solve* yang menerapkan solusi lengkap secara otomatis.



Gambar 4.6: Tampilan papan setelah solver menerapkan solusi lengkap.

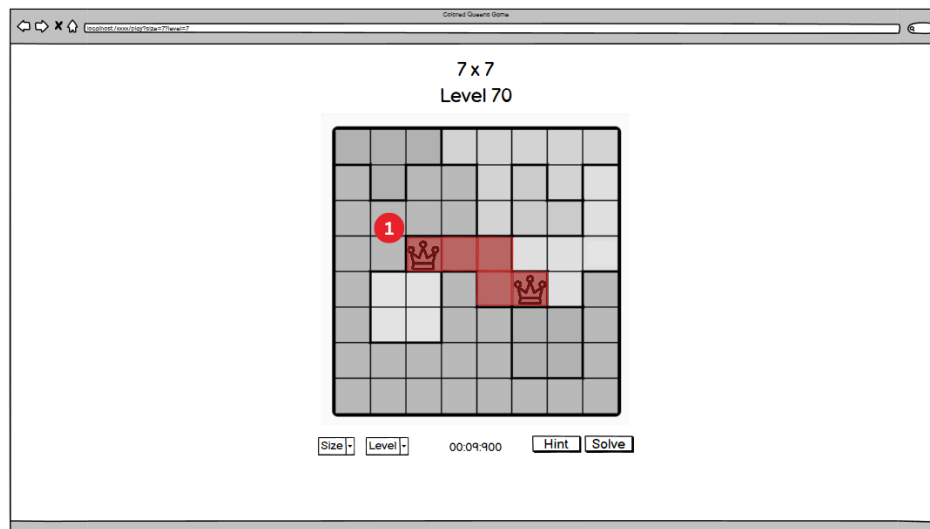
- 8 Penjelasan elemen pada Gambar 4.6:
- 9 1. Seluruh bidak menteri ditempatkan pada posisi yang benar oleh solver. Seluruh menteri
- 10 ditampilkan dengan warna hijau untuk menandakan solusi yang valid.



Gambar 4.7: Tampilan peringatan pelanggaran kendala *adjacency*.

- 11 Penjelasan elemen pada Gambar 4.7:
- 12 1. Dua bidak menteri yang bertetangga secara diagonal ditandai berwarna merah, menunjukkan
- 13 pelanggaran kendala *adjacency*. Sel-sel di sekitar kedua menteri yang berkonflik juga disorot

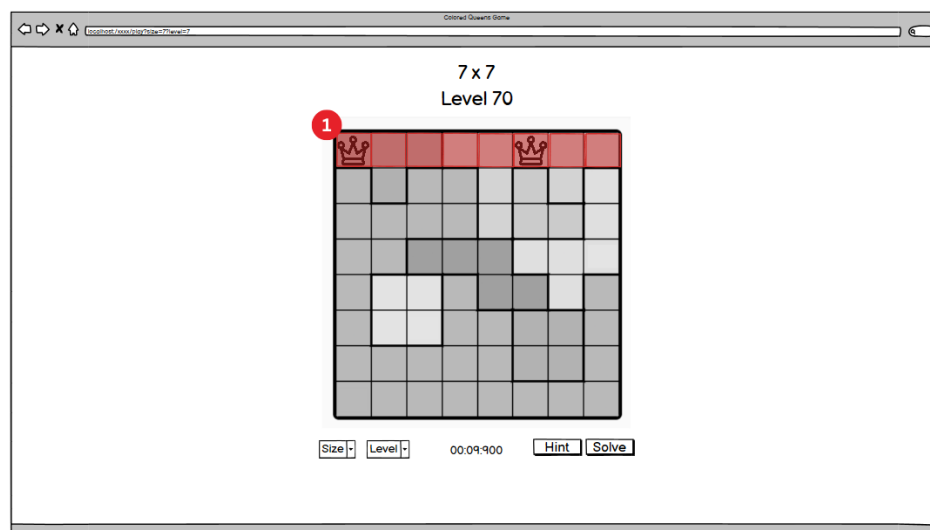
1 untuk memperjelas area pelanggaran.



Gambar 4.8: Tampilan peringatan pelanggaran kendala kesamaan warna.

2 Penjelasan elemen pada Gambar 4.8:

- 3 1. Dua bidak menteri yang ditempatkan pada sektor warna yang sama ditandai berwarna merah,
4 menunjukkan pelanggaran kendala *one queen per color*.



Gambar 4.9: Tampilan peringatan pelanggaran kendala baris.

5 Penjelasan elemen pada Gambar 4.9:

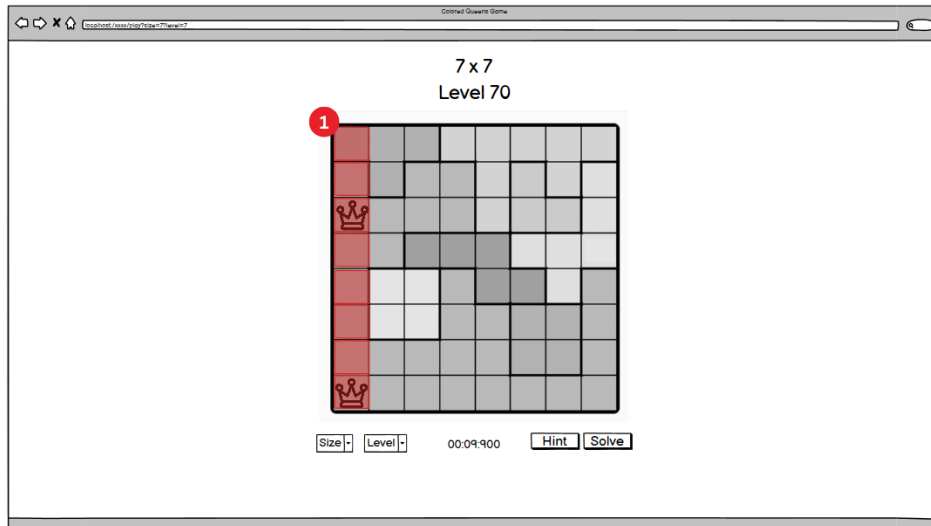
- 6 1. Dua bidak menteri yang berada pada baris yang sama ditandai berwarna merah. Seluruh sel
7 pada baris tersebut disorot untuk memperjelas area pelanggaran.

8 Penjelasan elemen pada Gambar 4.10:

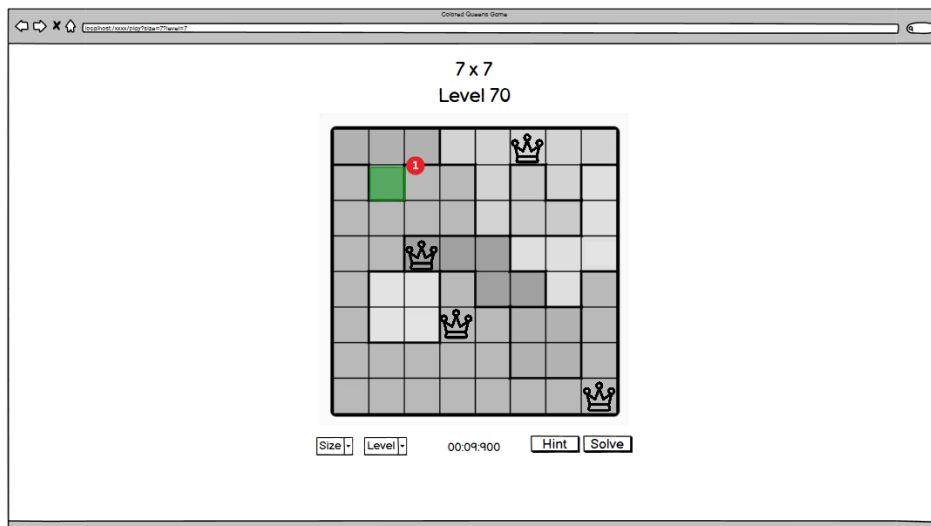
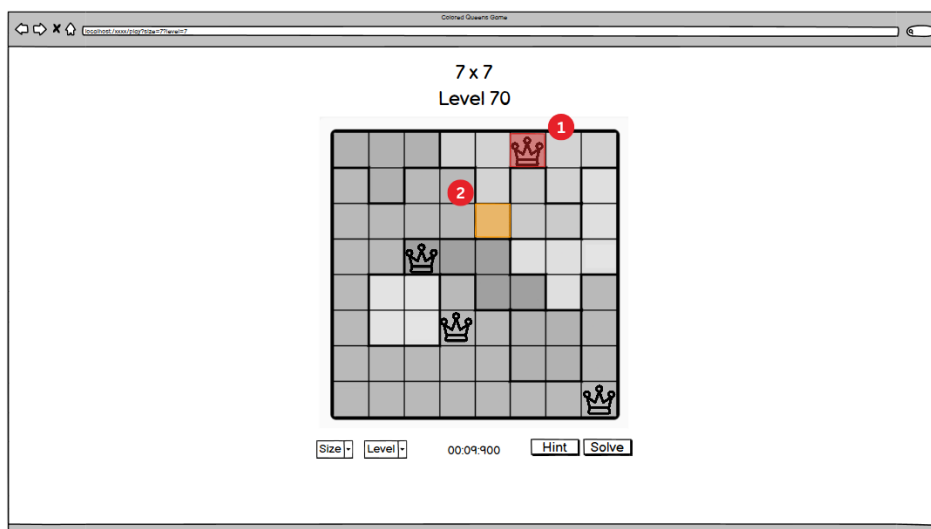
- 9 1. Dua bidak menteri yang berada pada kolom yang sama ditandai berwarna merah. Seluruh sel
10 pada kolom tersebut disorot untuk memperjelas area pelanggaran.

11 Penjelasan elemen pada Gambar 4.11:

- 12 1. Sorotan hijau pada sel yang merupakan posisi benar untuk sektor warna yang belum terisi,
13 menunjukkan di mana menteri seharusnya ditempatkan.

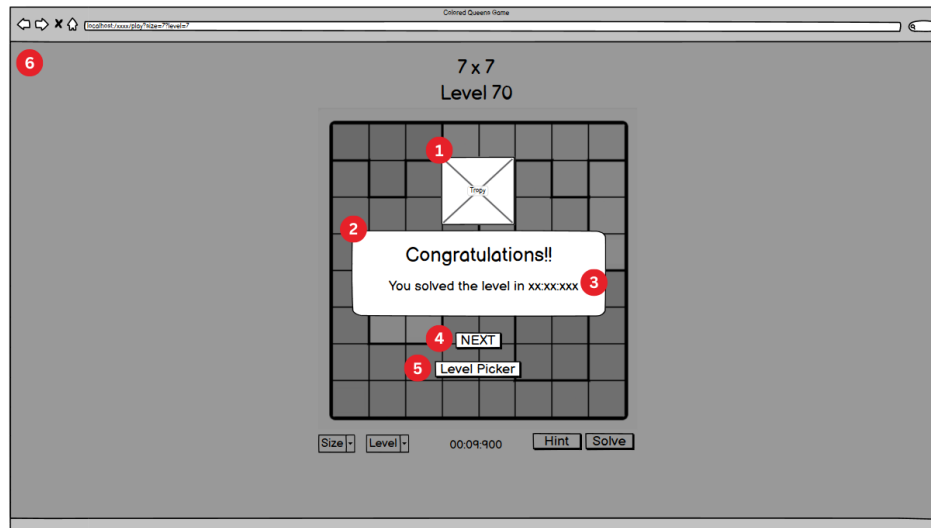


Gambar 4.10: Tampilan peringatan pelanggaran kendala kolom.

Gambar 4.11: Tampilan fitur *hint*: sorotan posisi yang benar.Gambar 4.12: Tampilan fitur *hint*: sorotan posisi salah dan posisi benar.

Penjelasan elemen pada Gambar 4.12:

1. Menteri yang ditempatkan pada posisi salah ditandai dengan sorotan merah, menunjukkan bahwa penempatan perlu dipindahkan.
2. Sorotan oranye pada sel yang merupakan posisi benar untuk sektor warna tersebut, menunjukkan ke mana menteri seharusnya dipindahkan.



Gambar 4.13: Rancangan tampilan kemenangan.

Penjelasan elemen pada Gambar 4.13:

1. Ikon trofi sebagai elemen visual perayaan kemenangan.
2. Pesan ucapan selamat kepada pemain.
3. Waktu penyelesaian yang dicatat oleh *stopwatch* selama permainan berlangsung.
4. Tombol *Next* untuk melanjutkan ke level berikutnya.
5. Tombol *Level Picker* untuk kembali ke halaman pemilihan level.
6. Latar belakang halaman permainan yang diredupkan (*dimmed*) untuk memberikan fokus pada dialog kemenangan.

4.1.3 Teknologi yang Digunakan

Tabel 4.11 merangkum teknologi dan pustaka yang digunakan dalam pengembangan sistem.

Tabel 4.11: Teknologi yang digunakan

Teknologi	Penggunaan
Java (JDK 23)	Bahasa pemrograman utama untuk seluruh modul
Java Swing / JFrame	<i>Framework</i> antarmuka grafis untuk rendering papan, navigasi, dan interaksi pengguna
org.json	Pustaka untuk penguraian berkas JSON papan
JSON	Format penyimpanan data papan (<i>NxN_levelM.json</i>)

1 Pemilihan Java Swing sebagai itframework antarmuka didasarkan pada kemudahan implementasi
2 dan ketersediaannya sebagai bagian dari *Java Standard Edition* tanpa memerlukan dependensi
3 eksternal tambahan. Meskipun Swing merupakan teknologi yang lebih lama dibandingkan alternatif
4 seperti JavaFX, kemampuannya untuk merender grid berwarna dan menangani interaksi klik
5 pengguna cukup memadai untuk kebutuhan visualisasi papan *Colored Queens*.

BAB 5

IMPLEMENTASI DAN PENGUJIAN

5.1 Implementasi

Bagian ini menjelaskan realisasi dari rancangan sistem yang telah dibahas pada Bab 3. Penjelasan difokuskan pada keputusan-keputusan teknis yang diambil selama proses pengembangan serta kesesuaian antara implementasi dengan rancangan. Kode sumber lengkap seluruh kelas disertakan pada Lampiran A; bagian ini hanya menyajikan potongan kode yang relevan untuk mengilustrasikan aspek-aspek implementasi yang penting.

5.1.1 Implementasi Solver *Backtracking* dengan *Forward Checking* dan AC-3

Solver *backtracking* diimplementasikan dalam kelas `BacktrackingSolverAC3` pada paket `solver.backtracking`. Kelas ini merealisasikan rancangan yang telah dijabarkan pada Bagian 4.1.1, mencakup alur rekursif *depth-first*, pemangkasan domain melalui *forward checking* dan AC-3, serta mekanisme *undo* berbasis *pruneStack*. Seluruh kode sumber kelas ini disajikan pada Lampiran A.2.1.

Struktur Data Domain

Keputusan implementasi utama pada solver ini adalah penggunaan `BitSet` dari pustaka standar Java untuk merepresentasikan domain setiap variabel. Setiap warna memiliki satu objek `BitSet` yang disimpan dalam `Map<String, BitSet> validCells`, di mana bit ke- i bernilai `true` apabila sel ke- i dalam daftar sel warna tersebut masih merupakan kandidat valid. Pendekatan ini dipilih karena dua alasan. Pertama, operasi pemangkasan (`clear`) dan pemulihan (`set`) pada satu bit memiliki kompleksitas $O(1)$, yang jauh lebih efisien dibandingkan operasi `add/remove` pada struktur `List` atau `Set`. Kedua, iterasi atas sel-sel yang masih valid dapat dilakukan secara efisien melalui `nextSetBit()`, yang melewati blok-blok bit bernilai nol tanpa memeriksa satu per satu.

Selain `BitSet`, sebuah array integer `colorCellCount[]` digunakan sebagai pencacah ukuran domain yang terpisah. Meskipun ukuran domain secara teoritis dapat diperoleh melalui `BitSet.cardinality()`, metode tersebut memiliki kompleksitas $O(n/64)$ karena harus menghitung jumlah bit pada setiap *word* internal. Pencacah terpisah memungkinkan pemeriksaan `anyColorExhausted()` berjalan dalam $O(n)$ terhadap jumlah warna, bukan $O(n \cdot d/64)$ di mana d adalah ukuran domain rata-rata.

1 Mekanisme *Undo* dengan *PruneStack*

2 Setiap operasi pemangkasan, baik yang berasal dari *forward checking* maupun AC-3, dicatat sebagai
 3 objek *PruneAction* yang menyimpan identitas warna dan indeks sel yang dipangkas. Objek-objek ini
 4 ditumpuk pada sebuah *Stack<PruneAction>* yang bersifat global sepanjang satu proses pencarian.
 5 Sebelum pemangkasan dimulai pada setiap tingkat rekursi, posisi *stack* saat ini dicatat sebagai
 6 *checkpoint* melalui variabel *pruneStartPos*. Integrasi mekanisme ini ke dalam alur rekursif utama
 7 ditunjukkan pada Kode 5.1.

Kode 5.1: Integrasi *checkpoint* dan *undo* pada alur rekursif utama

```

8
91  int pruneStartPos = pruneStack.size();
102
113  // Forward-check
124  forwardCheck(row, col, colorIndex);
135
146  // AC-3 arc propagation among future colors
157  Queue<Pair<Integer, Integer>> queue = new LinkedList<>();
168  for (int i = colorIndex + 1; i < colors.size(); i++) {
179      for (int j = colorIndex + 1; j < colors.size(); j++) {
180          if (i != j) queue.add(new Pair<>(i, j));
191      }
202  }
213  propagateArcs(queue);
224
235  if (anyColorExhausted(colorIndex)) {
246      undoPrunes(pruneStartPos);
257      occupied[row][col] = false;
268      solution[colorIndex] = -1;
279      backtrack++;
280      continue;
381  }

```

31 Apabila pencarian pada cabang saat ini gagal, fungsi *undoPrunes(pruneStartPos)* mengemba-
 32 likan seluruh bit yang telah dipangkas sejak *checkpoint*, memulihkan domain ke kondisi sebelum
 33 penugasan. Pendekatan berbasis *stack* ini menghindari kebutuhan untuk menyalin seluruh struktur
 34 *BitSet* pada setiap tingkat rekursi, yang akan memerlukan memori tambahan sebesar $O(n \cdot d)$ per
 35 tingkat.

36 Pemeriksaan Konflik *Inline*

37 Fungsi pemeriksaan konflik, sebagaimana dirancang pada Bagian 4.1.1, diimplementasikan da-
 38 lam dua bentuk. Bentuk pertama adalah metode *conflicts(int[] cell1, int[] cell2)* yang
 39 digunakan oleh prosedur *revise()* pada tahap AC-3. Bentuk kedua adalah pemeriksaan *inline*
 40 yang disisipkan langsung ke dalam badan perulangan *forwardCheck()* tanpa pemanggilan metode
 41 terpisah, sebagaimana ditunjukkan pada Kode 5.2.

Kode 5.2: Pemeriksaan konflik *inline* pada *forward checking*

```

42
431  boolean conflict = (row == r2 || col == c2 ||
442  Math.abs(row - r2) <= 1 && Math.abs(col - c2) <= 1);

```

46 Keputusan untuk menyisipkan pemeriksaan secara *inline* pada *forward checking* didasarkan
 47 pada pertimbangan bahwa *forward checking* dipanggil pada setiap tingkat rekursi dan mengiterasi
 48 seluruh sel valid dari setiap variabel yang belum ditugaskan. Penghematan *overhead* pemanggilan
 49 metode pada jalur kritis ini dapat berdampak signifikan pada waktu eksekusi keseluruhan, terutama
 50 pada papan berukuran besar.

1 Inisialisasi Antrean AC-3

2 Implementasi propagasi AC-3 menginisialisasi antrean busur dengan seluruh pasangan (i, j) untuk
 3 $i \neq j$ di antara variabel yang belum ditugaskan (indeks $> \text{colorIndex}$). Jumlah busur awal yang
 4 dimasukkan ke dalam antrean adalah $(n - k)(n - k - 1)$, di mana k adalah jumlah variabel yang
 5 telah ditugaskan. Pada tahap awal pencarian ($k = 0$) untuk papan 12×12 , jumlah ini mencapai
 6 $12 \times 11 = 132$ busur, yang berkurang secara kuadratik seiring bertambahnya variabel yang telah
 7 ditugaskan.

8 Apabila fungsi `revise()` berhasil memangkas domain suatu variabel, busur-busur yang meli-
 9 batkan variabel tersebut sebagai *supporter* ditambahkan kembali ke antrean untuk diperiksa ulang.
 10 Proses ini berlanjut hingga antrean kosong atau ditemukan *domain wipeout*. Implementasi ini
 11 sesuai dengan spesifikasi AC-3 standar sebagaimana dibahas pada Bagian 2.4.1, dengan adaptasi
 12 bahwa pemeriksaan *support* juga memperhitungkan posisi menteri yang telah ditempatkan pada
 13 variabel-variabel sebelumnya.

14 5.1.2 Implementasi Solver PSO Diskrit

15 Solver PSO diskrit diimplementasikan dalam dua kelas pada paket `solver.PSO`, yang mencakup
 16 kelas `Particle` yang merepresentasikan satu partikel, dan kelas `PSOSolver` yang mengelola populasi
 17 dan menjalankan proses optimasi. Implementasi ini merealisasikan rancangan yang telah dijabarkan
 18 pada Bagian 4.1.1. Seluruh kode sumber kedua kelas disajikan pada Lampiran A.2.2.

19 Struktur Kelas Particle

20 Setiap partikel dienkapsulasi dalam kelas `Particle` yang menyimpan empat atribut utama, yaitu
 21 `array integer candidate[]` sebagai posisi saat ini, `array double[] velocity` sebagai probabilitas per-
 22 ubahan per dimensi, `array integer pBest[]` sebagai posisi terbaik individual, dan nilai `pBestFitness`
 23 bertipe `double`. Pemisahan `Particle` sebagai kelas tersendiri dari `PSOSolver` dipilih untuk menjaga
 24 enkapsulasi data partikel dan memudahkan pengelolaan populasi melalui `List<Particle>`.

25 Fungsi Fitness

26 Fungsi *fitness* diimplementasikan dalam metode `calculateFitness()` yang memanggil dua sub-
 27 fungsi secara berurutan, yang pertama adalah `calculateAttackingViolation()` dan yang kedua
 28 adalah `calculateAdjacencyViolation()`. Penghitungan *fitness* keseluruhan ditunjukkan pada
 29 Kode 5.3.

Kode 5.3: Penghitungan fungsi *fitness*

```
30 1 int attackingViolations = calculateAttackingViolation(candidate);
32 2 int adjacencyViolations = calculateAdjacencyViolation(candidate, occupied);
33 3
35 4 double fitness = w1 * attackingViolations + w2 * adjacencyViolations;
```

36 Penghitungan *attacking violations* memeriksa setiap pasangan menteri dan menghitung jumlah
 37 pasangan yang berbagi baris atau kolom yang sama. Iterasi dilakukan dengan indeks $i < j$ untuk
 38 menghindari penghitungan ganda. Penghitungan *adjacency violations* menggunakan pendekatan ber-
 39 beda, di mana untuk setiap menteri, delapan sel tetangga diperiksa terhadap matriks `occupied[] []`.
 40 Karena pendekatan ini menghitung setiap pasangan bertetangga dua kali (sekali dari perspektif

- 1 masing-masing menteri), hasil akhir dibagi dua melalui pembagian integer, sebagaimana ditunjukkan
- 2 pada Kode 5.4.

Kode 5.4: Penghitungan *adjacency violations* dengan koreksi penghitungan ganda

```

3
41  int[][] dirs = {{-1,-1},{-1,0},{-1,1},{0,-1},{0,1},{1,-1},{1,0},{1,1}};
52
63  for (int i = 0; i < this.colors.size(); i++) {
74      String color = this.colors.get(i);
85      List<int[]> cells = this.colorCells.get(color);
96      int cellIdx = positions[i];
107     int[] cell = cells.get(cellIdx);
118
129     int row = cell[0];
130     int col = cell[1];
141
152     for (int[] d : dirs) {
163         int r = row + d[0];
174         int c = col + d[1];
185
196         if (r >= 0 && r < this.size && c >= 0 && c < this.size) {
207             if (occupied[r][c]) {
218                 count++;
229             }
230         }
241     }
252 }
263
274 return count/2;

```

- 29 Pembagian integer pada baris terakhir bersifat aman karena `count` selalu bernilai genap — setiap
- 30 pasangan menteri yang bertetangga berkontribusi tepat dua kali pada pencacah (masing-masing
- 31 satu dari perspektif setiap menteri dalam pasangan tersebut).

32 Partisi *Neighborhood*

- 33 Pembentukan topologi *local best* diimplementasikan dalam metode `generateNeighborhoods()`, yang
- 34 mengacak daftar indeks partikel menggunakan `Collections.shuffle()` kemudian mempartisinya
- 35 menjadi N_{nh} sublista dengan ukuran seimbang. Distribusi ukuran kelompok ditangani melalui
- 36 mekanisme *remainder*, di mana apabila P tidak habis dibagi N_{nh} , sisa partikel didistribusikan satu
- 37 per satu ke kelompok-kelompok pertama. Dengan demikian, perbedaan ukuran antarkelompok
- 38 tidak pernah melebihi satu partikel, sebagaimana ditunjukkan pada Kode 5.5.

Kode 5.5: Distribusi ukuran kelompok *neighborhood*

```

39
401  int baseSize = size / n;
412  int remainder = size % n;
423
434  for (int i = 0; i < n; i++) {
445      int groupSize = baseSize + (i < remainder ? 1 : 0);
456      // ...
467  }

```

- 48 Setiap *neighborhood* memelihara satu indeks `nBest` yang menunjuk pada partikel dengan *fitness*
- 49 terbaik dalam kelompok tersebut. Partisi bersifat tetap sepanjang eksekusi; tidak ada mekanisme
- 50 pengacakan ulang *neighborhood* pada saat stagnasi terdeteksi.

51 Pembaruan Kecepatan dan Posisi Diskrit

- 52 Pembaruan kecepatan dan posisi diimplementasikan dalam dua metode terpisah, yaitu `updateVelocity()`
- 53 dan `updateParticles()`. Pada setiap iterasi, seluruh kecepatan diperbarui terlebih dahulu, kemu-
- 54 dian posisi diperbarui berdasarkan kecepatan baru tersebut.

1 Pada pembaruan kecepatan, indikator perbedaan δ dihitung sebagai nilai biner (0 atau 1)
 2 berdasarkan kesamaan posisi saat ini dengan **pBest** dan **nBest** pada setiap dimensi. Hasil perhi-
 3 tungan di-*clamp* pada interval $[0, 1]$ untuk mempertahankan interpretasi probabilistik, sebagaimana
 4 dirancang pada Bagian 3.4.2 dan ditunjukkan pada Kode 5.6.

Kode 5.6: Pembaruan kecepatan diskrit dengan *clamping*

```

5
6 1   int pBestDiff = (currentPosition[j] != pBestPosition[j]) ? 1 : 0;
7 2   int nBestDiff = (currentPosition[j] != nBestPosition[j]) ? 1 : 0;
8 3
9 4   newVelocity[j] = this.inertia * velocity[j]
10 5   + this.c1 * r1 * pBestDiff
11 6   + this.c2 * r2 * nBestDiff;
12 7
13 8   if (newVelocity[j] < 0.0) newVelocity[j] = 0.0;
14 9   if (newVelocity[j] > 1.0) newVelocity[j] = 1.0;

```

16 Pada pembaruan posisi, kecepatan digunakan sebagai probabilitas untuk mengadopsi posisi
 17 *nBest*. Untuk setiap dimensi, bilangan acak $r \in [0, 1)$ dibangkitkan; apabila $r < v_j$, posisi pada
 18 dimensi tersebut diubah menjadi posisi *nBest*, sebagaimana ditunjukkan pada Kode 5.7.

Kode 5.7: Pembaruan posisi diskrit berbasis probabilitas

```

19
20 1   for (int j = 0; j < newPosition.length; j++) {
21 2       double rand = random.nextDouble(1.0);
22 3
23 4       if (rand < velocity[j]) {
24 5           newPosition[j] = nBestPosition[j];
25 6       }
26 7   }

```

28 Implementasi ini sesuai dengan rancangan pada Bagian 3.4.2, di mana *nBest* merupakan satu-
 29 satunya target perpindahan posisi, sementara pengaruh *pBest* masuk secara tidak langsung melalui
 30 komponen kognitif pada formula kecepatan.

31 Terminasi dan Deteksi Stagnasi

32 Tiga kondisi terminasi sebagaimana dirancang pada Bagian 2.4.2 diimplementasikan dalam *loop*
 33 utama metode **solve()**. Pertama, pada setiap akhir iterasi, metode **checkNbest()** memeriksa
 34 apakah salah satu *nBest* memiliki *fitness* bernilai nol. Kedua, *loop* utama dibatasi oleh **nIterations**
 35 ($I_{\max}$). Ketiga, sebuah pencacah stagnasi melacak jumlah iterasi berturut-turut tanpa perbaikan
 36 *fitness* terbaik global; apabila pencacah ini mencapai **maxStagnation** ($S_{\max}$), pencarian dihentikan
 37 secara dini.

38 Apabila tidak ada solusi valid yang ditemukan pada saat terminasi, solver melaporkan partikel
 39 dengan *fitness* terendah melalui metode **checkLowestNBest()** beserta nilai *fitness*-nya, yang menun-
 40 jukkan jumlah pelanggaran kendala yang tersisa. Dalam konteks pengujian, kasus-kasus tersebut
 41 dicatat sebagai kegagalan dan menjadi bagian dari metrik tingkat keberhasilan yang dianalisis pada
 42 Bagian 5.4.

43 5.1.3 Implementasi Antarmuka Pengguna

44 Antarmuka pengguna diimplementasikan menggunakan Java Swing dan JFrame sesuai dengan
 45 rancangan pada Bagian 4.1.2. Implementasi terdiri atas empat kelas dalam paket GUI, yaitu
 46 **Homepage**, **LevelSelectorWindow**, **GameWindow**, dan **UITheme**. Seluruh kode sumber kelas-kelas ini

- 1 disajikan pada Lampiran A.3. Bagian ini menjelaskan aspek-aspek implementasi yang relevan,
- 2 dengan fokus pada integrasi solver dan mekanisme interaksi pengguna.

3 Navigasi Antarlayar

4 Navigasi antar ketiga layar (**Homepage** → **LevelSelectorWindow** → **GameWindow**) diimplementasikan dengan mekanisme sederhana, di mana setiap tombol navigasi membuat *instance* baru dari kelas tujuan dan menutup jendela saat ini melalui `dispose()`. Pendekatan ini dipilih karena kesederhanaannya dan kesesuaiannya dengan alur aplikasi yang bersifat linier sebagaimana diilustrasikan pada Gambar 4.2.

9 Pada **GameWindow**, dua komponen `JComboBox` (untuk ukuran papan dan nomor level) memungkinkan pengguna berpindah level tanpa kembali ke halaman pemilihan. Setiap perubahan pada *dropdown* memicu pemanggilan metode `loadBoard()` yang memuat papan baru dan memulai ulang solver di latar belakang.

13 Integrasi Solver dengan Antarmuka

14 Keputusan implementasi terpenting pada modul antarmuka adalah eksekusi solver secara asinkron menggunakan `SwingWorker`. Ketika sebuah papan dimuat, solver *backtracking* dijalankan pada *thread* terpisah agar antarmuka tetap responsif selama proses pencarian berlangsung. Mekanisme ini ditunjukkan pada Kode 5.8.

Kode 5.8: Eksekusi solver secara asinkron menggunakan `SwingWorker`

```

18
19 1 currentWorker = new SwingWorker<>() {
20 2     @Override
21 3     protected int[][] doInBackground() {
22 4         BacktrackingSolverAC3 solver = new BacktrackingSolverAC3(board);
23 5         solver.solve();
24 6         return solver.getSolutionAsGrid();
25 7     }
26 8
27 9     @Override
28 0     protected void done() {
29 1         try {
30 2             if (!isCancelled()) {
31 3                 solution = get();
32 4             }
33 5         } catch (Exception e) {
34 6             System.err.println("Solver_error:_" + e.getMessage());
35 7         }
36 8     }
37 9 };
38 0 currentWorker.execute();

```

40 Solusi yang dihasilkan disimpan dalam variabel `solution` bertipe `int[][]` dan digunakan oleh fitur *Hint* dan *Solve*. Apabila pengguna mengakses salah satu fitur sebelum solver selesai, dialog peringatan ditampilkan. Untuk papan berukuran 20×20 dan 30×30 , solusi disimpan secara *hardcoded* dalam konstanta `PRELOADED_20x20` dan `PRELOADED_30x30` karena waktu komputasi solver pada ukuran tersebut tidak praktis untuk eksekusi secara *on-the-fly*.

45 Rendering Papan

46 Papan permainan dirender oleh kelas dalam `BoardPanel` yang merupakan subkelas dari `JPanel`. Metode `paintComponent()` menggambar tiga lapisan secara berurutan, yang pertama adalah latar belakang berwarna untuk setiap sel berdasarkan data warna RGB dari objek `Board`, yang kedua

1 adalah garis grid hitam sebagai pembatas antarsel, dan yang ketiga merupakan ikon menteri
2 pada sel-sel yang telah ditempati. Ukuran sel dihitung secara dinamis berdasarkan dimensi jendela
3 (`Math.min(getWidth(), getHeight()) / size`), sehingga papan tetap proporsional pada berbagai
4 ukuran jendela.

5 Pemilihan warna ikon menteri dilakukan secara adaptif berdasarkan luminansi latar belakang sel.
6 Fungsi `isDark()` menghitung luminansi menggunakan formula standar ($0,299R + 0,587G + 0,114B$);
7 apabila luminansi di bawah 0,5, ikon putih digunakan, dan sebaliknya ikon hitam digunakan.
8 Menteri yang terlibat dalam pelanggaran kendala ditampilkan dengan ikon merah.

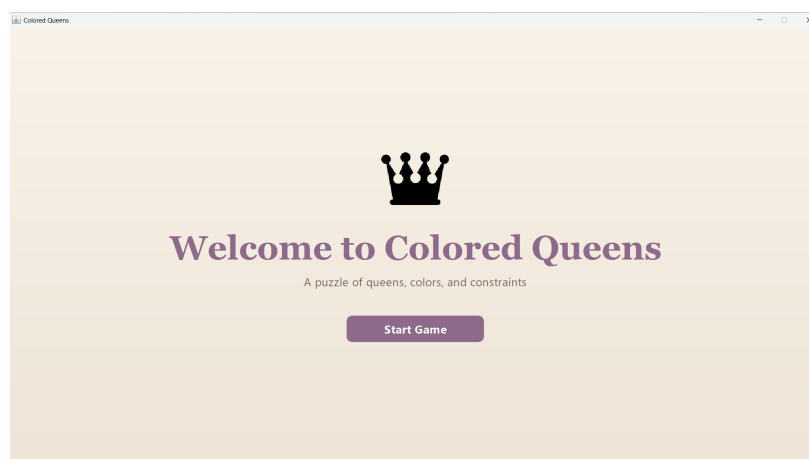
9 Pemeriksaan Pelanggaran dan Fitur *Hint*

10 Metode `checkViolations()` dipanggil setiap kali pengguna menempatkan atau menghapus menteri.
11 Metode ini memeriksa seluruh pasangan menteri yang telah ditempatkan terhadap empat jenis konflik,
12 yaitu baris yang sama, kolom yang sama, *adjacency*, dan kesamaan sektor warna. Pasangan yang
13 berkonflik ditandai melalui matriks `errorHighlight[][]` yang dirender sebagai overlay berwarna
14 merah semi-transparan pada sel-sel yang bersangkutan.

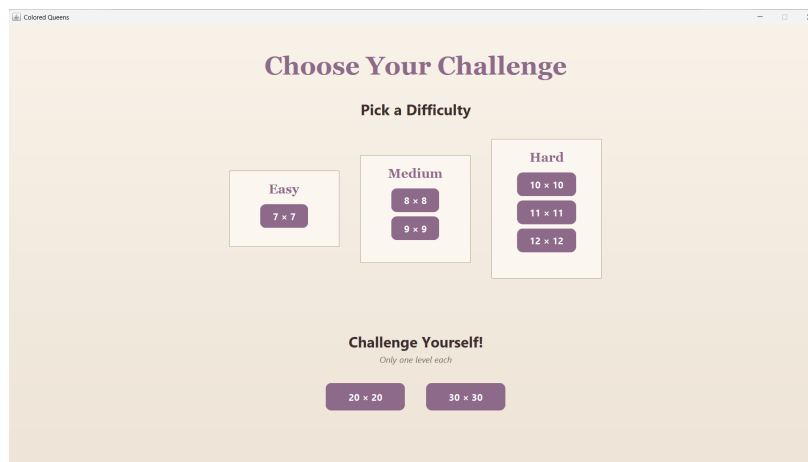
15 Fitur *hint* diimplementasikan dalam metode `giveHint()`, yang membandingkan penempatan
16 menteri oleh pengguna dengan solusi yang telah dihitung. Untuk sektor warna pertama yang belum
17 memiliki menteri pada posisi yang benar, sistem memberikan petunjuk visual berupa sorotan hijau
18 pada posisi yang benar. Apabila pengguna telah menempatkan menteri pada posisi yang salah
19 dalam sektor warna tersebut, sorotan oranye ditampilkan pada menteri yang salah sebagai indikator
20 korektif.

21 Kondisi kemenangan diperiksa setelah setiap penempatan dengan cara mengecek apabila jumlah
22 menteri yang ditempatkan sama dengan ukuran papan dan seluruh posisi sesuai dengan solusi,
23 pencacah waktu dihentikan dan dialog kemenangan ditampilkan.

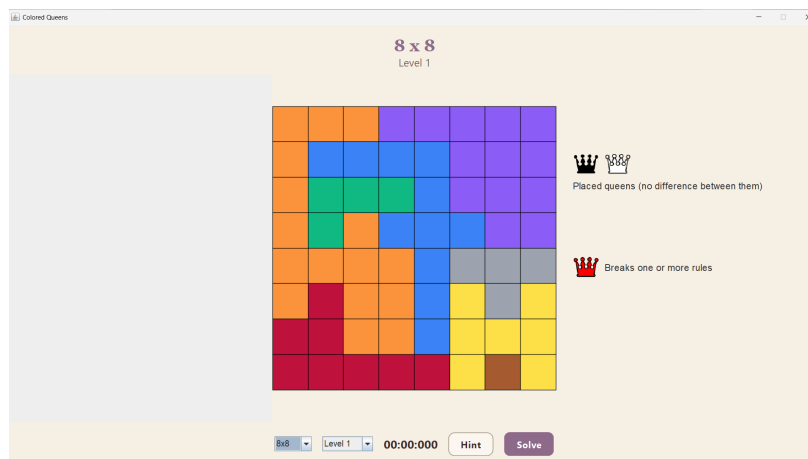
24 Tangkapan layar hasil implementasi antarmuka pada setiap layar diperlihatkan pada Gambar 5.1
25 hingga Gambar 5.10.



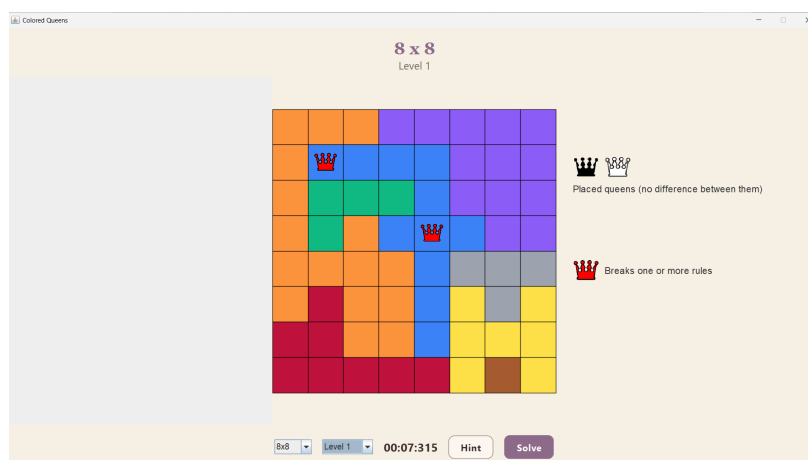
Gambar 5.1: Tangkapan layar halaman utama (*Homepage*).



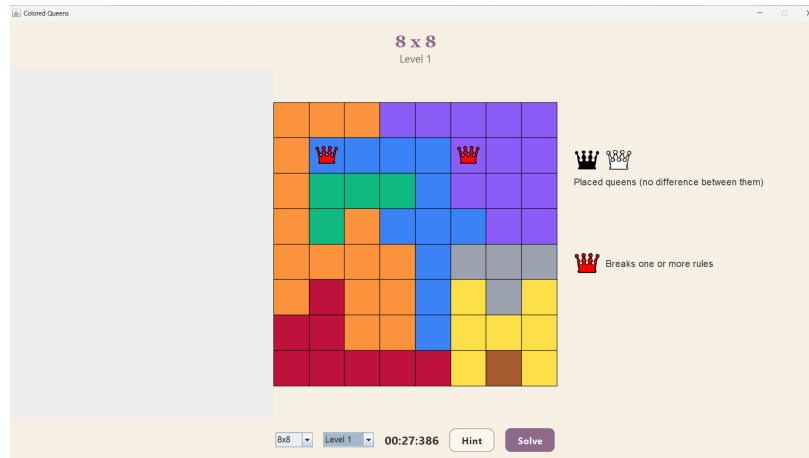
Gambar 5.2: Tangkapan layar halaman pemilihan level.



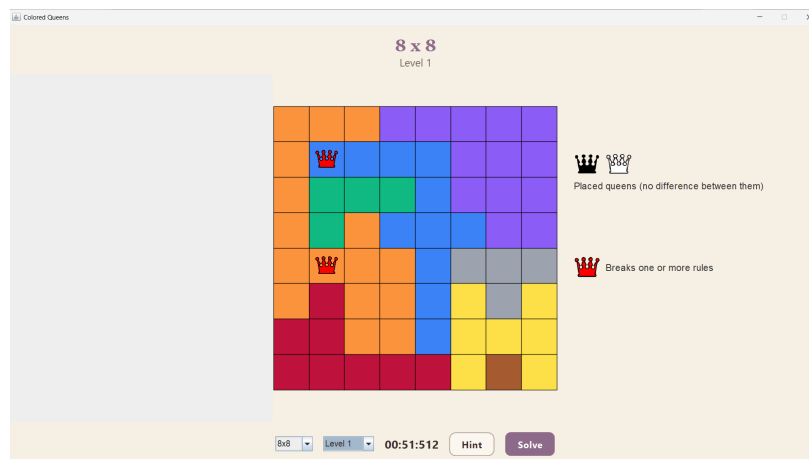
Gambar 5.3: Tangkapan layar halaman permainan utama.



Gambar 5.4: Tangkapan layar respons sistem terhadap pelanggaran penempatan dua bidak menteri pada warna yang sama.



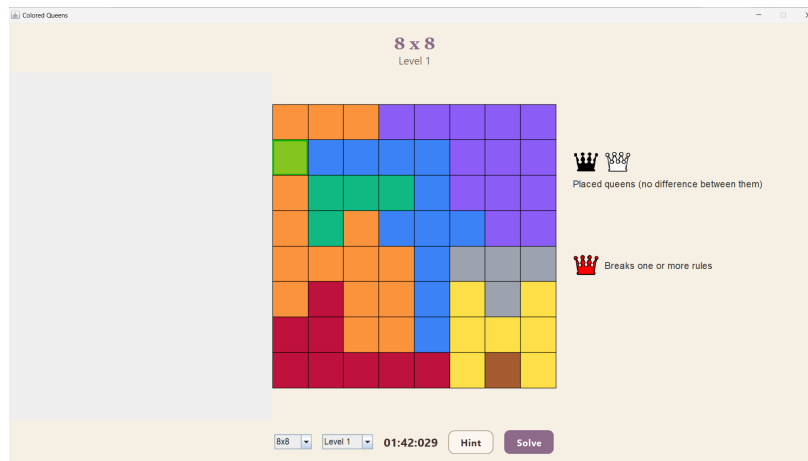
Gambar 5.5: Tangkapan layar respons sistem terhadap pelanggaran penempatan dua bidak menteri pada baris yang sama.



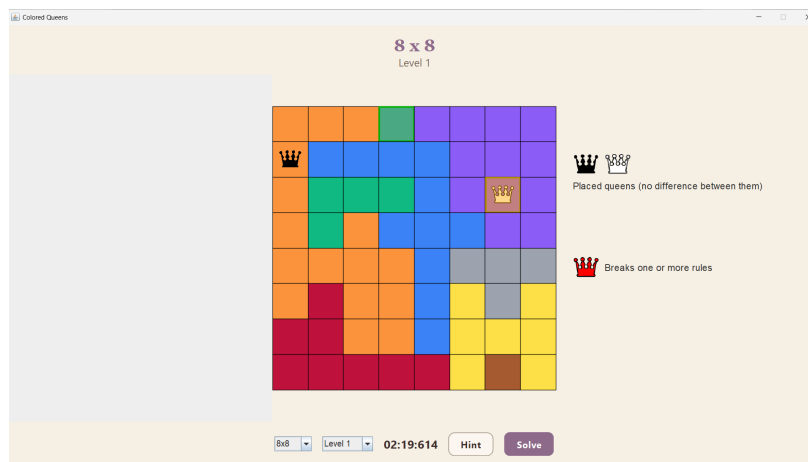
Gambar 5.6: Tangkapan layar respons sistem terhadap pelanggaran penempatan dua bidak menteri pada kolom yang sama.



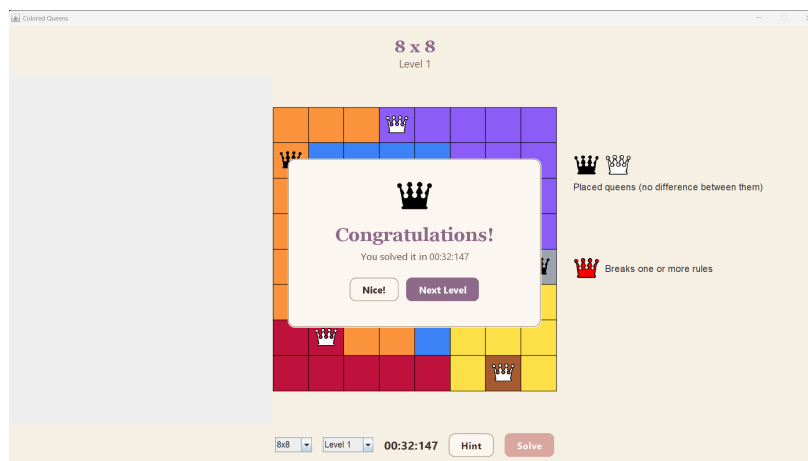
Gambar 5.7: Tangkapan layar respons sistem terhadap pelanggaran penempatan dua bidak menteri yang bersebelahan.



Gambar 5.8: Tangkapan layar bantuan dari sistem ketika belum ada bidak menteri yang ditempatkan pada sektor warna tersebut.



Gambar 5.9: Tangkapan layar bantuan dari sistem ketika ada bidak menteri yang ditempatkan pada sektor warna tersebut



Gambar 5.10: Tangkapan layar respons dari sistem ketika pemain berhasil menyelesaikan *puzzle*.

5.1.4 Implementasi Kerangka Pengujian

Selain aplikasi permainan dan kedua solver, sistem juga mencakup sebuah kerangka pengujian terpisah yang digunakan untuk menjalankan eksperimen secara otomatis pada seluruh dataset. Kerangka ini diimplementasikan dalam kelas `Main` pada paket `app` yang berbeda dari titik masuk aplikasi permainan. Seluruh kode sumber kelas ini disajikan pada Lampiran A.1.

Kerangka pengujian membaca seluruh berkas parameter PSO dari direktori `params/`, di mana setiap berkas mendefinisikan satu konfigurasi eksperimen (seperti `benchmark.txt`, `optimal.txt`, dan sebagainya). Untuk setiap konfigurasi, seluruh puzzle pada ukuran standar (7×7 hingga 12×12) dieksekusi dalam 5 *batch* yang masing-masing berisi 50 level. Pada setiap level dalam satu *batch*, kedua solver dijalankan secara berurutan, dengan solver *backtracking* terlebih dahulu, lalu solver PSO dengan parameter dari berkas konfigurasi. Untuk setiap eksekusi, kerangka mencatat waktu eksekusi, validitas solusi, posisi penempatan menteri, serta metrik tambahan berupa nilai *fitness* akhir, jumlah *adjacency violations*, dan jumlah *attacking violations* untuk PSO.

Eksekusi *batch* dilakukan secara paralel menggunakan `ExecutorService` dengan 6 *thread*. Setiap *batch* disubmit sebagai satu `Callable` yang independen, sehingga beberapa *batch* dari ukuran papan yang berbeda dapat berjalan secara bersamaan. Mekanisme paralelisasi ini ditunjukkan pada Kode 5.9.

Kode 5.9: Paralelisasi eksekusi *batch* menggunakan `ExecutorService`

```

18  int threads = 6;
19  ExecutorService executor = Executors.newFixedThreadPool(threads);
20
21
22  for (int size : sizes) {
23      for (int batch = 1; batch <= 5; batch++) {
24          int finalSize = size;
25          int finalBatch = batch;
26
27          futures.add(executor.submit(() ->
28              testBoardBatch(finalSize, finalBatch, params, paramDir, paramBaseName)
29              ));
30      }
31  }
32
33

```

Dengan 6 ukuran papan dan 5 *batch* per ukuran, total terdapat 30 *batch* yang dijadwalkan ke *thread pool* berukuran 6. Paralelisasi dilakukan pada tingkat *batch* (bukan pada tingkat puzzle individual) karena setiap eksekusi PSO memerlukan alokasi memori yang signifikan untuk populasi partikel, terutama pada konfigurasi dengan jumlah partikel besar ($P = 50000$).

Untuk ukuran papan besar (20×20 dan 30×30), pengujian dilakukan secara berbeda. Setiap ukuran hanya memiliki satu level yang dijalankan sebagai *batch* tunggal pada *thread* utama setelah seluruh *batch* paralel selesai. Pada ukuran ini, hanya solver PSO yang dijalankan melalui kerangka pengujian; solver *backtracking* tidak disertakan karena waktu komputasinya yang tidak praktis. Pengujian *backtracking* pada papan besar dilakukan secara terpisah sebagaimana akan dibahas pada Bagian 5.3.1.

Hasil setiap *batch* ditulis ke dalam berkas teks terpisah yang mencatat hasil per-level, kemudian diagregasi ke dalam berkas ringkasan per konfigurasi yang memuat rata-rata waktu eksekusi dan tingkat keberhasilan per ukuran papan dan per *batch*.

5.2 Skenario dan Metode Pengujian

5.2.1 Dataset Puzzle

Pengujian dilakukan pada delapan ukuran papan *Colored Queens*. Untuk ukuran standar (7×7 hingga 12×12), masing-masing memiliki 250 level puzzle yang diuji secara keseluruhan. Untuk ukuran besar (20×20 dan 30×30), masing-masing hanya memiliki satu level. Tabel 5.1 merangkum seluruh dataset yang digunakan.

Tabel 5.1: Dataset puzzle yang digunakan dalam pengujian

Ukuran Papan	Kategori	Jumlah Level
7×7	<i>Easy</i>	250
8×8	<i>Medium</i>	250
9×9	<i>Medium</i>	250
10×10	<i>Hard</i>	250
11×11	<i>Hard</i>	250
12×12	<i>Hard</i>	250
20×20	<i>Challenge</i>	1
30×30	<i>Challenge</i>	1

Total dataset untuk ukuran standar berjumlah 1.500 puzzle (6×250), ditambah 2 puzzle untuk ukuran besar, sehingga keseluruhan dataset mencakup 1.502 puzzle. Seluruh data papan bersumber dari permainan *LinkedIn Queens* dan disimpan dalam format JSON. Variasi ukuran papan dipilih untuk menguji skalabilitas kedua algoritma, di mana papan berukuran kecil (7×7 hingga 12×12) memiliki ruang pencarian yang relatif terbatas, sedangkan papan berukuran besar (20×20 dan 30×30) menghadirkan tantangan kombinatorik yang signifikan.

5.2.2 Parameter Pengujian

Kinerja kedua algoritma dievaluasi berdasarkan dua metrik utama, yaitu waktu eksekusi solver untuk menyelesaikan satu puzzle, diukur dalam milidetik menggunakan `System.nanoTime()` yang lalu diubah kembali ke bentuk milidetik. Metrik kedua adalah tingkat keberhasilan yang dihitung dengan cara mencari persentase eksekusi yang menghasilkan solusi valid (seluruh kendala terpenuhi). Untuk solver *backtracking*, metrik ini secara teoretis selalu bernilai 100% apabila solusi ada, karena sifat pencarian yang lengkap (*complete*). Untuk PSO, metrik ini bergantung pada konfigurasi parameter dan ukuran papan.

5.2.3 Skenario Pengujian

Pengujian dirancang dalam tiga kelompok skenario yang masing-masing menguji aspek yang berbeda.

Skenario 1: Pengujian *Backtracking*. Solver *backtracking* dengan *forward checking* dan AC-3 dijalankan pada seluruh 1.502 puzzle untuk mengukur waktu eksekusi, jumlah langkah, dan jumlah *backtracks*. Karena solver ini bersifat deterministik, setiap puzzle hanya perlu dijalankan satu kali.

Skenario 2: Pengujian variasi parameter PSO. Solver PSO dijalankan dengan 13 konfigurasi parameter sebagaimana dirangkum pada Tabel 5.2. Setiap konfigurasi dijalankan pada seluruh dataset. Untuk ukuran standar, 250 level per ukuran dieksekusi dalam 5 *batch* yang masing-masing berisi 50 level. Eksekusi *batch* dilakukan secara paralel menggunakan *ExecutorService* dengan 6 *thread* untuk mempercepat proses pengujian. Untuk ukuran besar (20×20 dan 30×30), hanya PSO yang dijalankan (tanpa *backtracking*) karena waktu komputasi solver deterministik pada ukuran tersebut tidak praktis. Hasil setiap konfigurasi diagregasi per *batch* dan per ukuran papan untuk menghasilkan metrik rata-rata waktu eksekusi dan tingkat keberhasilan.

Tabel 5.2: Konfigurasi pengujian PSO

Nama Konfigurasi	Parameter yang Diubah	Nilai
Benchmark	(baseline)	$P=5000$, $N_{nh}=100$, $c_1=1,0$, $c_2=1,0$, $w=0,7$, $w_1=1,0$, $w_2=1,0$
LParticles	Jumlah partikel	$P = 3000$
MParticles	Jumlah partikel	$P = 8000$
MMParticles	Jumlah partikel	$P = 50000$
LNeighborhoods	Jumlah <i>neighborhood</i>	$N_{nh} = 50$
MNeighborhoods	Jumlah <i>neighborhood</i>	$N_{nh} = 200$
LIInertia	Koefisien inersia	$w = 0,4$
MInertia	Koefisien inersia	$w = 0,9$
c1	Koefisien kognitif	$c_1 = 1,5$
c2	Koefisien sosial	$c_2 = 1,5$
w1	Bobot <i>attacking</i>	$w_1 = 2,0$
w2	Bobot <i>adjacency</i>	$w_2 = 2,0$
Optimal	Kombinasi terbaik	$P=50000$, $N_{nh}=50$, $c_1=1,5$, $w=0,4$

Skenario 3: Perbandingan antara *Backtracking* dan PSO. Hasil solver *backtracking* dan konfigurasi PSO terbaik (Optimal) dibandingkan secara langsung pada seluruh ukuran papan standar, dengan fokus pada waktu eksekusi dan tingkat keberhasilan. Perbandingan ini bertujuan untuk mengidentifikasi kelebihan dan keterbatasan masing-masing pendekatan pada skala permasalahan yang berbeda.

5.2.4 Lingkungan Pengujian

Seluruh pengujian dilakukan pada satu mesin dengan spesifikasi yang ditunjukkan pada Tabel 5.3. Penggunaan satu lingkungan pengujian yang konsisten memastikan bahwa perbedaan waktu eksekusi antarkonfigurasi tidak dipengaruhi oleh variasi perangkat keras.

Tabel 5.3: Spesifikasi lingkungan pengujian

Komponen	Spesifikasi
Sistem Operasi	Windows 11 Home
Prosesor	Intel i5-12400F
Memori (RAM)	32GB DDR4 @ 3200 MHz
Java Version	23
IDE	Visual Studio Code

5.3 Hasil Pengujian

Bagian ini menyajikan hasil pengujian berdasarkan ketiga skenario yang telah didefinisikan pada Bagian 5.2.3. Data disajikan dalam bentuk tabel dan grafik tanpa interpretasi mendalam; analisis dan pembahasan dilakukan pada Bagian 5.4.

5.3.1 Hasil Pengujian *Backtracking*

Solver *backtracking* dengan *forward checking* dan AC-3 dijalankan pada seluruh 1.500 puzzle ukuran standar (7×7 hingga 12×12) sebagai bagian dari kerangka pengujian yang dibahas pada Bagian 5.1.4. Karena solver bersifat deterministik, setiap puzzle hanya dijalankan satu kali. Seluruh puzzle berhasil diselesaikan, menghasilkan tingkat keberhasilan 100% pada semua ukuran papan standar. Tabel 5.4 merangkum hasil pengujian.

Tabel 5.4: Hasil pengujian solver *backtracking* dengan FC dan AC-3 pada ukuran standar

Ukuran Papan	Jumlah Puzzle	Tingkat Keberhasilan	Rata-rata Waktu (ms)
7×7	250	100%	0.032
8×8	250	100%	0.072
9×9	250	100%	0.08
10×10	250	100%	0.292
11×11	250	100%	0.584
12×12	250	100%	0.924

Hasil menunjukkan bahwa solver *backtracking* menyelesaikan seluruh puzzle pada ukuran standar dalam waktu yang sangat singkat dengan rata-rata kurang dari 1 ms untuk papan berukuran 7×7 hingga 12×12 .

1 Untuk papan berukuran besar, solver *backtracking* dengan *forward checking* dan AC-3 dijalankan
 2 secara terpisah di luar kerangka pengujian paralel. Tabel 5.5 merangkum hasilnya.

Tabel 5.5: Hasil pengujian solver *backtracking* dengan FC dan AC-3 pada ukuran besar

Ukuran	Waktu Eksekusi (ms)	Steps	Backtracks
20×20	54243969	778315514	2038321283
30×30	184859888	699374136	1900706094

3 5.3.2 Hasil Pengujian Variasi Parameter PSO

4 Solver PSO diskrit dijalankan dengan 13 konfigurasi parameter sebagaimana didefinisikan pada
 5 Tabel 5.2. Setiap konfigurasi dijalankan pada seluruh 1.500 puzzle ukuran standar ($5 \text{ batch} \times 50$
 6 $\text{level} \times 6$ ukuran) serta pada masing-masing satu puzzle untuk ukuran 20×20 dan 30×30 . Hasil
 7 diagregasi ke dalam dua metrik utama, yaitu tingkat keberhasilan (*success rate*) dan rata-rata
 8 waktu eksekusi.

9 Tingkat Keberhasilan

10 Tabel 5.6 menyajikan tingkat keberhasilan keseluruhan (*overall*) untuk setiap konfigurasi pada
 11 setiap ukuran papan standar. Nilai yang dicetak tebal menandakan tingkat keberhasilan tertinggi
 12 untuk ukuran papan yang bersangkutan.

Tabel 5.6: Tingkat keberhasilan PSO (%) per konfigurasi dan ukuran papan

Konfigurasi	7×7	8×8	9×9	10×10	11×11	12×12	20×20	30×30
Benchmark	67,20	22,80	5,60	1,20	0,80	0,00	0,00	0,00
LParticles	42,80	12,40	3,20	0,40	0,00	0,00	0,00	0,00
MParticles	87,60	42,00	10,80	1,20	0,00	0,00	0,00	0,00
MMParticles	100,00	94,40	61,20	26,80	8,00	0,40	0,00	0,00
LNeighborhoods	82,00	31,20	7,60	2,80	0,00	0,00	0,00	0,00
MNeighborhoods	55,20	19,20	0,80	0,40	0,00	0,00	0,00	0,00
LIInertia	80,00	35,60	8,80	0,40	0,40	0,00	0,00	0,00
MInertia	64,80	18,40	2,00	0,00	0,00	0,00	0,00	0,00
c1	66,00	29,60	6,00	0,40	0,00	0,00	0,00	0,00
c2	52,40	15,20	4,00	0,00	0,00	0,00	0,00	0,00
w1	74,80	25,20	2,80	1,20	0,00	0,00	0,00	0,00
w2	67,60	24,40	2,80	2,00	0,00	0,00	0,00	0,00
Optimal	99,20	94,40	62,00	37,60	13,60	5,20	0,00	0,00

13 Seluruh konfigurasi menunjukkan pola penurunan tingkat keberhasilan yang konsisten seiring
 14 bertambahnya ukuran papan. Pada papan 7×7 , mayoritas konfigurasi mencapai tingkat keberhasilan
 15 di atas 50%, dengan konfigurasi MMParticles dan Optimal mencapai 100% dan 99,20%. Namun,

1 pada papan 12×12 , hanya dua konfigurasi yang berhasil menemukan solusi, yaitu MMParticles
 2 (0,40%) dan Optimal (5,20%).

3 Untuk ukuran papan besar (20×20 dan 30×30), seluruh 13 konfigurasi mencatat tingkat
 4 keberhasilan 0%. Mengingat bahwa konfigurasi terbaik (Optimal) hanya mencapai 5,20% pada
 5 papan 12×12 dan bahkan pada papan 11×11 hanya 13,60%, kegagalan total pada ukuran yang
 6 jauh lebih besar merupakan hasil yang konsisten dengan tren penurunan yang diamati.

7 Rata-rata Waktu Eksekusi

8 Tabel 5.7 menyajikan rata-rata waktu eksekusi (dalam milidetik) untuk setiap konfigurasi. Waktu
 9 ini mencakup seluruh eksekusi, baik yang berhasil maupun yang gagal (terminasi karena batas
 10 iterasi atau stagnasi).

Tabel 5.7: Rata-rata waktu eksekusi PSO (ms) per konfigurasi dan ukuran papan

Konfigurasi	7×7	8×8	9×9	10×10	11×11	12×12
Benchmark	893	2.317	3.157	3.632	4.091	4.280
LParticles	903	1.561	1.940	2.204	2.471	2.585
MParticles	565	2.801	4.824	5.862	6.657	6.886
MMParticles	134	2.065	14.866	30.632	43.186	47.264
LNeighborhoods	492	2.063	3.075	3.562	4.099	4.270
MNeighborhoods	1.226	2.470	3.355	3.708	4.143	4.303
LIInertia	540	1.941	3.059	3.668	4.097	4.255
MInertia	947	2.434	3.276	3.676	4.121	4.293
c1	912	2.119	3.143	3.657	4.121	4.273
c2	1.282	2.538	3.194	3.667	4.116	4.294
w1	683	2.231	3.242	3.630	4.111	4.280
w2	869	2.259	3.239	3.601	4.109	4.253
Optimal	377	2.047	14.615	26.157	40.442	44.868

11 Terdapat korelasi terbalik antara waktu eksekusi dan tingkat keberhasilan pada konfigurasi
 12 dengan jumlah partikel besar: MMParticles dan Optimal memiliki tingkat keberhasilan tertinggi,
 13 namun juga memiliki waktu eksekusi yang jauh lebih lama pada papan berukuran 9×9 ke atas.
 14 Sebagai contoh, konfigurasi Optimal memerlukan rata-rata 44.868 ms (≈ 45 detik) pada papan
 15 12×12 , dibandingkan dengan 4.280 ms (≈ 4 detik) pada konfigurasi Benchmark. Sebaliknya,
 16 konfigurasi LParticles memiliki waktu eksekusi tercepat pada papan berukuran 8×8 ke atas, namun
 17 dengan tingkat keberhasilan yang paling rendah.

18 Ringkasan Pengaruh Per Parameter

19 Untuk memudahkan identifikasi pengaruh masing-masing parameter, Tabel 5.8 merangkum per-
 20bandingan antara nilai rendah dan tinggi setiap parameter terhadap konfigurasi Benchmark pada
 21 papan 7×7 dan 9×9 sebagai representasi ukuran kecil dan menengah.

Tabel 5.8: Ringkasan pengaruh parameter terhadap tingkat keberhasilan (%)

Parameter	Konfigurasi	Nilai	SR (%)		Waktu (ms)	
			7×7	9×9	7×7	9×9
Jumlah partikel (P)	LParticles	3.000	42,80	3,20	903	1.940
	Benchmark	5.000	67,20	5,60	893	3.157
	MMParticles	50.000	100,00	61,20	134	14.866
$Neighborhood$ (N_{nh})	LNeighborhoods	50	82,00	7,60	492	3.075
	Benchmark	100	67,20	5,60	893	3.157
	MNeighborhoods	200	55,20	0,80	1.226	3.355
Inersia (w)	LIInertia	0,4	80,00	8,80	540	3.059
	Benchmark	0,7	67,20	5,60	893	3.157
	MIInertia	0,9	64,80	2,00	947	3.276
Kognitif (c_1)	Benchmark	1,0	67,20	5,60	893	3.157
	c1	1,5	66,00	6,00	912	3.143
Sosial (c_2)	Benchmark	1,0	67,20	5,60	893	3.157
	c2	1,5	52,40	4,00	1.282	3.194
Bobot <i>attacking</i> (w_1)	Benchmark	1,0	67,20	5,60	893	3.157
	w1	2,0	74,80	2,80	683	3.242
Bobot <i>adjacency</i> (w_2)	Benchmark	1,0	67,20	5,60	893	3.157
	w2	2,0	67,60	2,80	869	3.239

Dari Tabel 5.8, parameter yang memberikan pengaruh paling signifikan terhadap tingkat keberhasilan adalah jumlah partikel (P), yaitu peningkatan dari 5.000 ke 50.000 menaikkan tingkat keberhasilan dari 5,60% menjadi 61,20% pada papan 9×9 . Jumlah *neighborhood* dan koefisien inersia juga menunjukkan pengaruh yang terukur, di mana nilai yang lebih kecil ($N_{nh} = 50$ dan $w = 0,4$) menghasilkan kinerja yang lebih baik. Sebaliknya, variasi koefisien kognitif (c_1), sosial (c_2), dan bobot *fitness* (w_1, w_2) memberikan pengaruh yang relatif kecil terhadap kinerja keseluruhan.

5.3.3 Hasil Perbandingan *Backtracking* dan PSO

Bagian ini membandingkan secara langsung hasil solver *backtracking* dengan *forward checking* dan AC-3 terhadap konfigurasi PSO terbaik (Optimal) pada seluruh ukuran papan. Konfigurasi Optimal dipilih sebagai representasi PSO karena menggabungkan nilai-nilai parameter terbaik yang teridentifikasi dari eksperimen individual pada Bagian 5.3.2, yaitu $P = 50000$, $N_{nh} = 50$, $c_1 = 1,5$, dan $w = 0,4$.

Tabel 5.9 menyajikan perbandingan tingkat keberhasilan kedua solver pada ukuran papan standar.

Tabel 5.9: Perbandingan tingkat keberhasilan (%) *Backtracking* dan PSO Optimal

Ukuran Papan	Backtracking (FC+AC-3)	PSO Optimal
7×7	100,00	99,20
8×8	100,00	94,40
9×9	100,00	62,00
10×10	100,00	37,60
11×11	100,00	13,60
12×12	100,00	5,20

Solver *backtracking* mempertahankan tingkat keberhasilan 100% pada seluruh ukuran papan, sesuai dengan sifat pencarian lengkap (*complete*) yang menjamin ditemukannya solusi apabila solusi tersebut ada. Konfigurasi PSO Optimal mencapai tingkat keberhasilan yang mendekati *backtracking* pada papan 7×7 (99,20%), namun menurun secara tajam seiring bertambahnya ukuran papan hingga hanya 5,20% pada papan 12×12 .

Tabel 5.10 menyajikan perbandingan rata-rata waktu eksekusi kedua solver.

Tabel 5.10: Perbandingan rata-rata waktu eksekusi (ms) *Backtracking* dan PSO Optimal

Ukuran Papan	Backtracking (FC+AC-3)	PSO Optimal
7×7	0.032	377
8×8	0.072	2.047
9×9	0.080	14.615
10×10	0.292	26.157
11×11	0.584	40.442
12×12	0.924	44.868

Perbedaan waktu eksekusi antara kedua solver sangat signifikan. Solver *backtracking* menyelesaikan seluruh puzzle dalam waktu kurang dari 1 ms, sedangkan konfigurasi PSO Optimal memerlukan waktu ratusan milidetik hingga puluhan detik. Pada papan 12×12 , rasio waktu eksekusi mencapai sekitar 1:44.868 — PSO memerlukan waktu sekitar 45.000 kali lebih lama dibandingkan *backtracking*, dengan tingkat keberhasilan yang jauh lebih rendah.

Perlu dicatat bahwa rata-rata waktu PSO mencakup seluruh eksekusi termasuk yang gagal (terminasi karena batas iterasi atau stagnasi). Eksekusi yang gagal cenderung memakan waktu lebih lama karena harus menunggu hingga seluruh iterasi selesai atau kondisi stagnasi tercapai, sehingga rata-rata waktu pada ukuran papan dengan tingkat keberhasilan rendah mencerminkan waktu terminasi maksimum lebih dari waktu pencarian solusi yang sesungguhnya.

5.4 Analisis dan Pembahasan

Bagian ini menginterpretasikan hasil pengujian yang telah disajikan pada Bagian 5.3 dan menghubungkannya dengan landasan teori pada Bab 2 serta rancangan pada Bab 3.

5.4.1 Analisis Skalabilitas *Backtracking*

Hasil pengujian pada Tabel 5.4 menunjukkan bahwa solver *backtracking* dengan *forward checking* dan AC-3 menyelesaikan seluruh 1.500 puzzle pada ukuran standar dengan tingkat keberhasilan 100% dan waktu eksekusi rata-rata di bawah 2 ms. Temuan ini mengonfirmasi efektivitas propagasi kendala pada permasalahan *Colored Queens* sebagaimana telah dibahas secara teoretis pada Bagian 2.4.1.

Kunci dari efisiensi ini terletak pada interaksi antara struktur kendala *Colored Queens* dan mekanisme propagasi domain. Sebagaimana dibahas pada Bagian 2.3.3, *constraint graph* pada *Colored Queens* bersifat *complete*, yang berarti setiap pasangan variabel (sektor warna) terhubung oleh setidaknya satu kendala (baris, kolom, atau *adjacency*). Implikasi dari graf kendala yang *complete* ini adalah bahwa setiap penugasan menteri pada satu sektor warna memicu propagasi ke *seluruh* sektor warna lainnya, bukan hanya ke tetangga tertentu. *Forward checking* menghapus posisi-posisi yang berkonflik langsung dengan penugasan baru, dan AC-3 melanjutkan dengan mendeteksi inkonsistensi tidak langsung yang muncul akibat penyempitan domain secara berantai.

Dampak propagasi ini dapat diilustrasikan secara kuantitatif. Pada papan $n \times n$, penempatan satu menteri pada posisi (r, k) langsung dua kelompok sel, yaitu seluruh posisi pada baris r dan kolom k dari domain variabel lain, serta hingga 8 posisi bertetangga dari domain setiap variabel yang memiliki sel pada area tersebut. Untuk papan 12×12 dengan rata-rata ukuran domain $\bar{d} \approx 12$ sel per warna, penugasan pertama dapat memangkas sebagian besar domain variabel lainnya, sehingga penugasan berikutnya beroperasi pada ruang pencarian yang sudah jauh menyempit. Efek kumulatif dari propagasi ini menjelaskan mengapa solver mampu menemukan seluruh solusi dalam waktu sub-milidetik.

5.4.2 Analisis Pengaruh Parameter PSO

Hasil pengujian pada Tabel 5.6 dan Tabel 5.7 memperlihatkan bahwa kinerja PSO pada permasalahan *Colored Queens* sangat sensitif terhadap konfigurasi parameter, dengan beberapa parameter memiliki pengaruh yang dominan dan parameter lainnya relatif marginal.

Jumlah Partikel (P)

Jumlah partikel merupakan parameter yang paling berpengaruh terhadap tingkat keberhasilan. Peningkatan dari $P = 3.000$ (LParticles) ke $P = 50.000$ (MMParticles) menaikkan tingkat keberhasilan dari 42,80% menjadi 100% pada papan 7×7 , dan dari 3,20% menjadi 61,20% pada papan 9×9 . Pada papan 12×12 , hanya konfigurasi dengan $P = 50.000$ yang berhasil menemukan solusi (0,40%), sementara seluruh konfigurasi dengan $P \leq 8.000$ mencatat kegagalan total.

Pengaruh jumlah partikel ini dapat dijelaskan dari perspektif eksplorasi ruang pencarian. PSO merupakan algoritma berbasis populasi di mana setiap partikel mewakili satu titik sampel dalam ruang solusi. Pada permasalahan CSP yang *tight* (rasio solusi terhadap ruang pencarian sangat kecil), probabilitas satu partikel diinisialisasi dekat dengan solusi valid berbanding lurus dengan jumlah partikel. Untuk papan 12×12 , ruang pencarian memiliki ukuran $\prod_{i=1}^{12} |D_{c_i}|$ yang bernilai sangat besar, sehingga diperlukan populasi yang masif untuk menjangkau area yang mengandung solusi.

Namun, peningkatan jumlah partikel memiliki *trade-off* yang signifikan terhadap waktu eksekusi. Setiap iterasi PSO memiliki kompleksitas $O(P \cdot n)$ untuk pembaruan kecepatan dan posisi, ditambah $O(P \cdot n^2)$ untuk evaluasi *fitness* (penghitungan pelanggaran antara seluruh pasangan menteri). Konfigurasi MMParticles memerlukan rata-rata 47.264 ms pada papan 12×12 , lebih dari 10 kali lipat dibandingkan konfigurasi Benchmark (4.280 ms), meskipun tingkat keberhasilan hanya naik dari 0% ke 0,40%. Hal ini mengindikasikan bahwa peningkatan jumlah partikel pada ukuran besar mulai mengalami *diminishing returns*, yang berarti biaya komputasi per iterasi meningkat secara linier terhadap P , namun probabilitas menemukan solusi tidak meningkat dengan laju yang sama.

Jumlah *Neighborhood* (N_{nh})

Pengurangan jumlah *neighborhood* dari 100 (Benchmark) ke 50 (LNeighborhoods) meningkatkan tingkat keberhasilan pada seluruh ukuran papan: dari 67,20% ke 82,00% pada 7×7 , dan dari 1,20% ke 2,80% pada 10×10 . Sebaliknya, peningkatan ke 200 (MNeighborhoods) menurunkan kinerja secara konsisten.

Temuan ini konsisten dengan teori topologi kawanan yang dibahas pada Bagian 2.4.2. Dengan jumlah *neighborhood* yang lebih kecil, setiap kelompok memiliki lebih banyak partikel, sehingga *nBest* dalam setiap kelompok cenderung memiliki *fitness* yang lebih baik. Hal ini meningkatkan kualitas informasi yang dibagikan antarpartikel dalam kelompok melalui komponen sosial pada formula kecepatan. Sebaliknya, *neighborhood* yang terlalu besar (200 kelompok dari 5.000 partikel = 25 partikel per kelompok) mengurangi keragaman informasi dalam setiap kelompok dan melemahkan daya dorong menuju area yang menjanjikan.

Di sisi lain, *neighborhood* yang lebih kecil juga berarti lebih sedikit *nBest* yang berbeda. Dengan 50 kelompok, hanya terdapat 50 *nBest* yang berfungsi sebagai *attractor*, sedangkan dengan 200 kelompok terdapat 200 *attractor*. Jumlah *attractor* yang lebih sedikit mengurangi fragmentasi pencarian dan memungkinkan partikel terkonsentrasi pada area-area yang lebih menjanjikan, namun dengan risiko konvergensi prematur yang lebih tinggi. Pada permasalahan *Colored Queens* yang memiliki sedikit solusi valid, konsentrasi pencarian terbukti lebih menguntungkan dibandingkan diversifikasi berlebihan.

Koefisien Inersia (w)

Penurunan koefisien inersia dari 0,7 (Benchmark) ke 0,4 (LIInertia) meningkatkan tingkat keberhasilan pada papan 7×7 dari 67,20% menjadi 80,00% dan pada papan 9×9 dari 5,60% menjadi 8,80%. Peningkatan ke 0,9 (MIInertia) berdampak negatif, menurunkan tingkat keberhasilan menjadi 64,80% pada 7×7 dan 2,00% pada 9×9 .

Koefisien inersia mengendalikan keseimbangan antara eksplorasi dan eksploitasi sebagaimana dibahas pada Bagian 2.4.2. Nilai inersia yang tinggi ($w = 0,9$) menyebabkan partikel mempertahankan kecepatan sebelumnya dengan bobot yang besar, sehingga cenderung bergerak melampaui area yang menjanjikan tanpa mengeksploitasinya secara memadai. Sebaliknya, inersia yang rendah ($w = 0,4$) memberikan bobot yang lebih besar pada komponen kognitif dan sosial, yang mengarahkan partikel secara lebih agresif menuju *pBest* dan *nBest*. Pada permasalahan CSP dengan *fitness landscape* yang memiliki banyak *plateau* (area luas dengan *fitness* serupa) dan *basin of attraction* yang sempit menuju solusi valid, eksploitasi yang lebih agresif terbukti lebih efektif.

1 Koefisien Kognitif (c_1) dan Sosial (c_2)

2 Variasi koefisien kognitif ($c_1 = 1,5$; $c_1 = 1,0$) memberikan pengaruh yang sangat kecil, yang dapat
 3 dilihat di tingkat keberhasilan pada 7×7 berubah dari 67,20% menjadi 66,00%, dan pada 9×9 dari
 4 5,60% menjadi 6,00%. Peningkatan koefisien sosial ($c_2 = 1,5$) justru menurunkan kinerja secara
 5 lebih terukur, dari 67,20% menjadi 52,40% pada 7×7 .

6 Asimetri pengaruh antara c_1 dan c_2 ini dapat ditelusuri dari mekanisme pembaruan posisi pada
 7 implementasi. Sebagaimana dibahas pada Bagian 3.4.2, posisi baru partikel hanya dipengaruhi
 8 oleh $nBest$ (bukan $pBest$ secara langsung) — kecepatan v_j digunakan sebagai probabilitas adopsi
 9 posisi $nBest$ per dimensi. Peningkatan c_2 memperbesar komponen sosial dalam formula kecepatan,
 10 yang pada gilirannya meningkatkan probabilitas adopsi $nBest$. Apabila $nBest$ belum mendekati
 11 solusi valid, adopsi yang terlalu agresif justru menghilangkan keragaman populasi dan menyebabkan
 12 konvergensi prematur menuju solusi yang kurang optimal. Sementara itu, peningkatan c_1 memiliki
 13 efek yang lebih lemah karena pengaruh $pBest$ hanya masuk secara tidak langsung melalui kecepatan,
 14 bukan melalui adopsi posisi.

15 Bobot *Fitness* (w_1 dan w_2)

16 Variasi bobot *attacking* ($w_1 = 2,0$) menaikkan tingkat keberhasilan pada 7×7 dari 67,20% menjadi
 17 74,80%, namun menurunkannya pada 9×9 dari 5,60% menjadi 2,80%. Variasi bobot *adjacency*
 18 ($w_2 = 2,0$) menunjukkan pola serupa, yaitu peningkatan kecil pada 7×7 (67,60%) namun penurunan
 19 pada 9×9 (2,80%).

20 Peningkatan salah satu bobot secara efektif memperbesar gradien *fitness* pada jenis pelanggaran
 21 yang bersangkutan, mendorong partikel untuk memprioritaskan penyelesaian jenis pelanggaran
 22 tersebut. Pada papan kecil, prioritas pelanggaran baris/kolom ($w_1 = 2,0$) sedikit membantu karena
 23 pelanggaran jenis ini lebih mudah diselesaikan dan memberikan sinyal arah yang jelas. Namun
 24 pada papan yang lebih besar, ketidakseimbangan bobot dapat mengarahkan pencarian ke area yang
 25 bebas dari satu jenis pelanggaran namun masih memiliki pelanggaran jenis lain, sehingga secara
 26 keseluruhan tidak membantu konvergensi menuju solusi valid. Hasil ini menunjukkan bahwa bobot
 27 seimbang ($w_1 = w_2 = 1,0$) merupakan pilihan yang cukup baik sebagai *baseline*.

28 5.4.3 Analisis Perbandingan Kedua Pendekatan

29 Hasil perbandingan pada Tabel 5.9 dan Tabel 5.10 menunjukkan dominasi solver *backtracking* yang
 30 sangat jelas atas PSO pada permasalahan *Colored Queens*. Dominasi ini bersifat absolut pada kedua
 31 dimensi evaluasi karena *backtracking* mencapai 100% tingkat keberhasilan pada seluruh ukuran
 32 dengan waktu sub-milidetik, sedangkan PSO Optimal hanya mendekati kinerja setara pada papan
 33 terkecil (7×7 : 99,20%) dan menurun tajam pada ukuran yang lebih besar.

34 Perbedaan kinerja yang sangat signifikan ini bukan merupakan artefak dari implementasi
 35 yang kurang optimal, melainkan mencerminkan perbedaan fundamental antara kedua paradigma
 36 pencarian ketika diterapkan pada struktur spesifik permasalahan *Colored Queens*. Tiga faktor
 37 utama menjelaskan dominasi *backtracking*.

38 **Pertama, pemanfaatan struktur kendala.** Solver *backtracking* dengan *forward checking* dan
 39 AC-3 secara langsung memanfaatkan hubungan antarkendala untuk memangkas ruang pencarian

secara proaktif. Setiap penugasan baru segera memicu propagasi yang mengeliminasi ketidak-konsistenan dari domain variabel-variabel yang belum ditugaskan. PSO, sebaliknya, memperlakukan masalah sebagai optimasi *black-box*, yang berarti fungsi *fitness* hanya memberikan umpan balik berupa *jumlah* pelanggaran tanpa informasi mengenai *sumber* atau *arah* perbaikan. Partikel tidak mengetahui kendala mana yang dilanggar atau bagaimana menggerakkan satu dimensi untuk mengurangi pelanggaran tanpa menimbulkan pelanggaran baru pada dimensi lain, sebagaimana dibahas pada Bagian 2.4.2.

Kedua, sifat ruang pencarian *Colored Queens*. Permasalahan *Colored Queens* memiliki karakteristik CSP yang *tight*, yang berarti jumlah solusi valid relatif sangat kecil dibandingkan ukuran ruang pencarian total. Hal ini menyulitkan pendekatan stokastik karena probabilitas sebuah partikel “mendarat” pada atau di dekat solusi valid sangat rendah. Lebih jauh, *fitness landscape* dari *Colored Queens* memiliki karakteristik yang merugikan PSO, yaitu perubahan pada satu dimensi (penempatan satu menteri) dapat secara drastis mengubah jumlah pelanggaran pada dimensi lain, menghasilkan *landscape* yang *rugged* dengan banyak *local optima*, sebagaimana dibahas pada Bagian 2.2.2.

Bukti empiris dari karakteristik ini terlihat pada pola terminasi eksekusi yang gagal, yaitu sebagian besar kegagalan pada seluruh ukuran papan berakhir dengan *fitness* = 1,0 yang terdiri dari 0 *adjacency violation* dan 1 *attacking violation*. Pola ini menunjukkan bahwa PSO mampu mengeliminasi pelanggaran *adjacency* secara efektif, namun terjebak dalam konfigurasi di mana sisa satu pelanggaran baris/kolom tidak dapat diselesaikan tanpa memperkenalkan pelanggaran baru. *Landscape* pada kondisi ini memiliki *basin of attraction* yang sangat sempit, yang berarti perbaikan pada satu dimensi memerlukan perubahan simultan yang tepat pada satu atau lebih dimensi lain, yang sangat tidak mungkin dicapai melalui pergerakan probabilistik partikel. Kondisi inilah yang secara operasional memicu stagnasi sebagaimana didefinisikan pada Bagian 2.4.2, yang berarti seluruh partikel telah berkumpul di sekitar konfigurasi yang sama, komponen kognitif dan sosial tidak lagi memberikan dorongan yang berarti, dan pencarian berakhir sebelum solusi valid ditemukan.

Ketiga, perbandingan efisiensi pencarian deterministik dan stokastik pada CSP. Algoritma *backtracking* membangun solusi secara inkremental, memvalidasi setiap langkah, dan segera mundur ketika inkonsistensi terdeteksi. Pendekatan ini memastikan bahwa setiap langkah pencarian menuju solusi parsial yang masih mungkin diperluas menjadi solusi lengkap. PSO, sebaliknya, bekerja dengan solusi lengkap yang seringkali melanggar banyak kendala secara simultan, dan bergantung pada proses stokastik untuk secara bertahap mengurangi pelanggaran. Pada CSP dengan kendala yang saling bergantung secara ketat, perbaikan inkremental melalui pergerakan stokastik sangat rentan terhadap terjadinya *cycling*, yaitu perbaikan pada satu dimensi memperburuk dimensi lain, sehingga *fitness* berosilasi tanpa konvergensi, yang pada akhirnya memicu stagnasi sebagaimana dijabarkan pada Bagian 2.4.2.

Meskipun demikian, hasil ini tidak meniadakan potensi PSO sebagai pendekatan penyelesaian CSP secara umum. Keunggulan PSO terletak pada kemampuannya untuk memberikan solusi *approximate* dalam waktu yang terkontrol oleh parameter, serta tidak memerlukan pemodelan eksplisit dari struktur kendala. Pada permasalahan di mana solusi sempurna tidak diperlukan (misalnya masalah optimasi dengan kendala lunak), atau di mana struktur kendala terlalu kompleks

-
- 1 untuk dieksploitasi secara efektif oleh propagasi, pendekatan *metaheuristic* dapat menjadi alternatif
 - 2 yang kompetitif sebagaimana diuraikan pada Bagian 2.4.2. Namun, untuk permasalahan *Colored*
 - 3 *Queens* yang merupakan CSP dengan kendala keras dan *constraint graph* yang *complete*, pendekatan
 - 4 berbasis propagasi kendala terbukti jauh lebih sesuai.

BAB 6

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Berdasarkan hasil implementasi, pengujian, dan analisis yang telah dilakukan terhadap dua pendekatan penyelesaian permasalahan *Colored Queens* pada tugas akhir ini, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Perangkat lunak permainan *Colored Queens* berhasil dibangun menggunakan bahasa pemrograman Java dengan antarmuka grafis berbasis Java Swing. Perangkat lunak ini terdiri atas tiga halaman utama, yaitu halaman beranda, halaman pemilihan level, dan halaman permainan, yang menyajikan papan permainan dengan ukuran beragam mulai dari 7×7 hingga 30×30 yang dimuat dari berkas berformat JSON.
2. Solusi permainan *Colored Queens* berhasil dibangun menggunakan dua pendekatan algoritmik yang berbeda. Pendekatan pertama adalah *backtracking* sistematis yang diperkuat dengan *Forward Checking* dan propagasi kendala AC-3, yang merepresentasikan permasalahan sebagai *Constraint Satisfaction Problem* (CSP) dengan variabel berupa sektor warna, domain berupa sel-sel pada sektor tersebut, dan kendala yang mencakup larangan baris, kolom, dan ketetanggaan dalam delapan arah. Pendekatan kedua adalah *Particle Swarm Optimization* (PSO) diskrit dengan representasi indeks sel per warna, fungsi *fitness* berbasis pelanggaran kendala, dan topologi *neighborhood* lokal (lBest) yang menerapkan adaptasi formula kecepatan dan posisi PSO untuk ruang pencarian diskrit.
3. Kedua solver berhasil diintegrasikan ke dalam perangkat lunak permainan sebagai kelas independen yang menerima objek papan sebagai masukan dan menghasilkan penugasan menteri sebagai keluaran. Integrasi diwujudkan melalui mekanisme eksekusi asinkron, sehingga solver dapat dijalankan di latar belakang untuk mendukung fitur *hint* dan *auto-solve* tanpa menghambat responsivitas antarmuka pengguna.
4. Berdasarkan hasil pengujian, kedua solver menunjukkan kinerja yang sangat berbeda dalam mencari solusi permainan *Colored Queens*. Algoritma *backtracking* dengan *Forward Checking* dan AC-3 mampu menyelesaikan seluruh instansi yang diujikan, mulai dari ukuran 7×7 hingga 30×30 , dengan tingkat keberhasilan 100% dan waktu penyelesaian yang cepat dan deterministik. Sebaliknya, algoritma PSO diskrit hanya mampu menemukan solusi yang valid pada papan berukuran kecil (7×7 hingga 9×9) dengan konfigurasi parameter tertentu, namun mengalami stagnasi dan kegagalan konvergensi yang signifikan pada papan berukuran 10×10 ke atas.
5. Hasil eksperimen mengonfirmasi bahwa kombinasi mekanisme propagasi kendala (*Forward*

Checking dan AC-3) secara signifikan mempersempit ruang pencarian melalui eliminasi nilai-nilai domain yang tidak konsisten sebelum penelusuran dilanjutkan. Pada PSO, eksperimen variasi parameter menunjukkan bahwa peningkatan jumlah partikel dan pengurangan jumlah *neighborhood* memberikan dampak positif terbesar pada kemampuan eksplorasi, namun bahkan kombinasi parameter optimal tidak mampu menjamin keberhasilan pada instansi berukuran besar. Dengan demikian, untuk permasalahan *Colored Queens* secara khusus dan permasalahan CSP dengan kendala keras secara umum, pendekatan berbasis propagasi kendala terbukti jauh lebih efektif dibandingkan pendekatan *metaheuristik* berbasis populasi.

6.2 Saran

Berdasarkan temuan dan keterbatasan yang diidentifikasi dalam penelitian ini, beberapa saran untuk pengembangan selanjutnya dapat dikemukakan.

Pertama, penyetelan parameter PSO dapat dilakukan secara lebih sistematis dan komprehensif. Eksperimen pada tugas akhir ini hanya mengeksplorasi 13 konfigurasi parameter dengan mengubah satu parameter pada satu waktu (*one-factor-at-a-time*), sehingga interaksi antarparameter belum tertangkap secara penuh. Penelitian selanjutnya dapat menerapkan metode penyetelan yang lebih sistematis seperti *grid search*, *random search*, atau *Bayesian optimization*, guna mengidentifikasi kombinasi parameter yang benar-benar optimal dan mengevaluasi potensi PSO pada permasalahan *Colored Queens* secara lebih adil.

Kedua, mekanisme rekonfigurasi *neighborhood* secara adaptif dapat ditambahkan ke dalam solver PSO untuk menangani stagnasi. Pada implementasi saat ini, struktur *neighborhood* bersifat statis sepanjang proses pencarian. Penelitian selanjutnya dapat mengeksplorasi mekanisme yang secara dinamis mengonfigurasi ulang *neighborhood* saat stagnasi terdeteksi, sehingga partikel memiliki kesempatan untuk keluar dari *local optima* dan mengeksplorasi area pencarian yang berbeda.

Ketiga, pendekatan *hybrid* yang menggabungkan propagasi kendala dengan *metaheuristik* dapat dieksplorasi sebagai arah penelitian lebih lanjut. Sebagai contoh, PSO dapat digunakan untuk menghasilkan solusi awal yang kemudian diperbaiki oleh *backtracking*, atau propagasi kendala dapat diterapkan sebagai mekanisme perbaikan *fitness* di dalam PSO. Pendekatan semacam ini berpotensi mengatasi kelemahan konvergensi PSO pada permasalahan dengan kendala keras.

Keempat, pengembangan heuristik pemilihan variabel yang lebih adaptif pada *backtracking* dapat menjadi arah penelitian yang menjanjikan. Implementasi *backtracking* dalam penelitian ini menggunakan pengurutan warna berdasarkan ukuran domain secara statik. Heuristik dinamis seperti *Minimum Remaining Values* (MRV) atau *Least Constraining Value* (LCV) yang memperbarui urutan pemilihan variabel secara adaptif selama proses pencarian berpotensi mengurangi jumlah *backtrack* lebih lanjut.

Kelima, mengingat keterbatasan PSO yang ditemukan, penelitian selanjutnya dapat membandingkan PSO dengan *metaheuristik* lain seperti *Simulated Annealing*, *Genetic Algorithm*, atau *Tabu Search*, agar dapat menilai apakah kelemahan yang ditemukan bersifat umum pada seluruh pendekatan *metaheuristik*, atau merupakan karakteristik spesifik PSO pada permasalahan ini.

Keenam, pengujian pada instansi yang lebih beragam perlu dilakukan. Penelitian ini dibatasi hingga ukuran 30×30 dengan satu instansi untuk ukuran besar, sehingga penelitian selanjutnya

-
- 1 sebaiknya mencakup lebih banyak instansi dengan variasi tata letak warna yang berbeda guna
 - 2 memperoleh gambaran statistik yang lebih komprehensif.

DAFTAR REFERENSI

- [1] Bell, J. dan Stevens, B. (2009) A survey of known results and research areas for n-queens. *Discrete mathematics*, **309**, 1–31. <https://www.sciencedirect.com/science/article/pii/S0012365X07010394#b20> Diakses: 2026-05-10.
- [2] Gent, I. P., Jefferson, C., dan Nightingale, P. (2017) Complexity of n-queens completion. *Journal of Artificial Intelligence Research*, **59**, 815–848. <https://www.jair.org/index.php/jair/article/view/11079/26262> Diakses: 2026-05-14.
- [3] LinkedIn (2024) Play queens game on linkedin. <https://www.linkedin.com/help/linkedin/answer/a6269510>. Diakses: 2026-05-14.
- [4] Snyder, T. (2013) Star battle rules and info. <https://www.gmpuzzles.com/blog/star-battle-rules-and-info/>. Diakses: 14 Mei 2026.
- [5] Ali, A. M. dan Mencia, E. (2026) A qubo formulation for the generalized linkedin queens and takuzu/tango game. <https://arxiv.org/abs/2410.06429>, Diakses: 28 Mei 2026.
- [6] Russell, S. J. dan Norvig, P. (2020) *Artificial Intelligence: A Modern Approach*, 4 edition. Pearson Education, Inc., New Jersey. Chapter 6. [http://repo.darmajaya.ac.id/5272/1/Artificial%20Intelligence-A%20Modern%20Approach%20\(3rd%20Edition\)%20\(%20PDFDrive%20\).pdf](http://repo.darmajaya.ac.id/5272/1/Artificial%20Intelligence-A%20Modern%20Approach%20(3rd%20Edition)%20(%20PDFDrive%20).pdf) Diakses: 2026-05-10.
- [7] Van Beek, P. (2006) Backtracking search algorithms. *Handbook of Constraint Programming*, pp. 85–134. Elsevier, Amsterdam. <https://cs.uwaterloo.ca/~vanbeek/Publications/survey06.pdf> Diakses: 17 Mei 2026.
- [8] Kennedy, J. dan Eberhart, R. (1995) Particle swarm optimization. *Proceedings of ICNN'95-international conference on neural networks*, Perth, pp. 1942–1948. iee. https://www.cs.tufts.edu/comp/150GA/homeworks/hw3/_reading6%201995%20particle%20swarming.pdf Diakses: 18 Mei 2026.
- [9] Glock, S., Munhá Correia, D., dan Sudakov, B. (2022) The n-queens completion problem. *Research in the Mathematical Sciences*, **9**, 41. <https://link.springer.com/article/10.1007/s40687-022-00335-1> Diakses: 2026-05-10.
- [10] Hoffman, E. J., Loessi, J., dan Moore, R. C. (1969) Constructions for the solution of the m queens problem. *Mathematics Magazine*, **42**, 66–72. <https://exp-blog.com/algorithm/poj/poj3239-solution-to-the-n-queens-puzzle/doc/Constructions%20for%20the%20Solution%20of%20the%20m%20Queens%20Problem.pdf> Diakses: 2026-05-14.
- [11] Arora, S. dan Barak, B. (2009) *Computational complexity: a modern approach*. Cambridge University Press, Cambridge. <https://theory.cs.princeton.edu/complexity/book.pdf> Diakses: 2026-06-07.

- [12] Montanari, U. (1974) Networks of constraints: Fundamental properties and applications to picture processing. *Information sciences*, **7**, 95–132. <http://cse.unl.edu/~choueiry/Documents/Montanari-74.pdf> Diakses 14 Mei 2026.
- [13] Kumar, V. (1998) Algorithms for constraint satisfaction problems: A survey. *A.I. Mag*, **13**. https://www.researchgate.net/publication/2513379_Algorithms_for_Constraint_Satisfaction_Problems_A_Survey Diakses: 17 Mei 2026.
- [14] Bessiere, C. (2006) Constraint propagation. *Handbook of Constraint Programming*, pp. 29–83. Elsevier, Amsterdam. https://www.dcs.gla.ac.uk/~pat/cpM/papers/CP_Handbook-20060315-final.pdf Diakses: 17 Mei 2026.
- [15] Mackworth, A. K. (1977) Consistency in networks of relations. *Artificial intelligence*, **8**, 99–118. <https://www.cs.ubc.ca/~mack/Publications/AI77.pdf> Diakses: 17 Mei 2026.
- [16] Mohr, R. dan Henderson, T. C. (1986) Arc and path consistency revisited. *Artificial intelligence*, **28**, 225–233. <https://sci-hub.mk/10.1016/0004-3702%2886%2990083-4> Diakses: 17 Mei 2026.
- [17] Blum, C. dan Roli, A. (2003) Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM computing surveys (CSUR)*, **35**, 268–308. <https://dl.acm.org/doi/epdf/10.1145/937503.937505> Diakses: 17 Mei 2026.
- [18] Hoos, H. H. dan Tsang, E. (2006) Local search methods. *Handbook of Constraint Programming*, pp. 245–268. Elsevier, Amsterdam. https://www.dcs.gla.ac.uk/~pat/cpM/papers/CP_Handbook-20060315-final.pdf Diakses: 17 Mei 2026.
- [19] Michalewicz, Z., Fogel, D. B., dan Michalewicz, A. (2000) *How to solve it: modern heuristics*. Springer, Berlin. [https://people.vts.su.ac.rs/~pmiki/AI/Michalewicz%20Z.,%20Fogel%20D.-How%20to%20solve%20it.%20Modern%20heuristics-Springer%20\(2004\).pdf](https://people.vts.su.ac.rs/~pmiki/AI/Michalewicz%20Z.,%20Fogel%20D.-How%20to%20solve%20it.%20Modern%20heuristics-Springer%20(2004).pdf) Diakses: 24 Mei 2026.
- [20] Engelbrecht, A. P. dkk. (2007) *Computational intelligence: an introduction*. Wiley Online Library, Chichester. <https://dai.fmph.uniba.sk/courses/ICI/engelbrecht.comp-intel-intro.07.pdf> Diakses: 24 Mei 2026.
- [21] Hu, X., Eberhart, R. C., dan Shi, Y. (2003) Swarm intelligence for permutation optimization: a case study of n-queens problem. *Proceedings of the 2003 IEEE Swarm Intelligence Symposium. SIS'03 (Cat. No. 03EX706)*, Indianapolis, pp. 243–246. IEEE. <http://www.swarmintelligence.org/papers/SIS2003Permutation.pdf> Diakses: 18 Mei 2026.
- [22] Kennedy, J. dan Eberhart, R. C. (1997) A discrete binary version of the particle swarm algorithm. *1997 IEEE International conference on systems, man, and cybernetics. Computational cybernetics and simulation*, Orlando, pp. 4104–4108. IEEE. https://www.ahmetcevahircinar.com.tr/wp-content/uploads/2017/02/A_discrete_binary_version_of_the_particle_swarm_algorithm.pdf Diakses: 18 Mei 2026.

LAMPIRAN A

KODE PROGRAM

A.1 Kelas Inti

Kode A.1: Kelas Main.java — Titik masuk aplikasi, meluncurkan antarmuka pengguna grafis

```
1 package app;
2
3 import javax.swing.SwingUtilities;
4 import GUI.Homepage;
5
6 public class Main {
7     public static void main(String[] args) {
8         SwingUtilities.invokeLater(() -> {
9             Homepage game = new Homepage();
10            game.setVisible(true);
11        });
12    }
13 }
```

Kode A.2: Kelas BatchMain.java — Program pengujian massal seluruh solver secara *multithreaded*

```
1 package com.ArloDante.coloredqueens.app;
2
3 import com.ArloDante.coloredqueens.objects.Board;
4 import com.ArloDante.coloredqueens.objects.Cell;
5 import com.ArloDante.coloredqueens.solver.PS0.PS0SolverTest;
6 import com.ArloDante.coloredqueens.solver.backtracking.BacktrackingSolverTest;
7 import com.ArloDante.coloredqueens.util.BoardImporter;
8
9 import java.io.*;
10 import java.util.*;
11 import java.util.concurrent.*;
12
13 public class Main {
14
15     static int[] sizes = {7, 8, 9, 10, 11, 12};
16     static int[] largeSizes = {20, 30};
17
18     public static void main(String[] args) throws Exception {
19
20         File paramsDir = new File(System.getProperty("user.dir") + "/params");
21         File[] paramFiles = paramsDir.listFiles((dir, name) -> name.endsWith(".txt"));
22
23         if (paramFiles == null || paramFiles.length == 0) {
24             System.err.println("No_parameter_files_found!");
25             return;
26         }
27
28         File resultsRoot = new File(System.getProperty("user.dir") + "/results");
29         if (!resultsRoot.exists()) resultsRoot.mkdir();
30
31         int threads = 6; // your CPU cores
32         ExecutorService executor = Executors.newFixedThreadPool(threads);
33
34         for (File paramFile : paramFiles) {
35             String paramFileName = paramFile.getName();
36             String paramBaseName = paramFileName.replace(".txt", "");
37
38             System.out.println("\n===_RUNNING_PARAM_SET:_ + paramBaseName + "_===");
39
40             File paramDir = new File(resultsRoot, paramBaseName);
41             if (!paramDir.exists()) paramDir.mkdir();
42
43             PS0Parameters params = readParameters(paramFileName);
44
45             List<Future<BatchResult>> futures = new ArrayList<>();
46
47             // ===== MULTITHREADED BATCHES =====
48             for (int size : sizes) {
49                 for (int batch = 1; batch <= 5; batch++) {
50
51                     int finalSize = size;
52                     int finalBatch = batch;
```

```

53         futures.add(executor.submit(() ->
54             testBoardBatch(finalSize, finalBatch, params, paramDir, paramBaseName)
55         ));
56     }
57 }
58
59 // ===== LARGE SIZES (single thread is fine) =====
60 SummaryStats stats = new SummaryStats();
61
62 for (Future<BatchResult> f : futures) {
63     BatchResult result = f.get();
64     stats.merge(result);
65 }
66
67 for (int size : largeSizes) {
68     BatchResult result = testSingleBoard(size, params, paramDir, paramBaseName);
69     stats.merge(result);
70 }
71
72 String summaryFile = paramDir.getPath() + "/summary_" + paramBaseName + ".txt";
73 generateSummary(summaryFile, stats);
74
75 System.out.println("Summary_saved_to_" + summaryFile);
76 }
77
78 executor.shutdown();
79 System.out.println("\nAll_experiments_completed!");
80 }
81
82 // ===== BATCH =====
83 private static BatchResult testBoardBatch(int size, int batch, PSOParameters params,
84     File paramDir, String paramBaseName) {
85
86     String outputFile = paramDir.getPath() +
87         "/results" + size + "x" + size + "_" + paramBaseName + "_batch" + batch + ".txt";
88
89     BatchResult result = new BatchResult(size, batch);
90
91     int start = (batch - 1) * 50 + 1;
92     int end = batch * 50;
93
94     try (PrintWriter writer = new PrintWriter(new FileWriter(outputFile))) {
95         writer.println("====_" + size + "x" + size + "_Batch_" + batch + "_====");
96
97         for (int level = start; level <= end; level++) {
98             writer.println("Level_" + level + ":");
99
100             try {
101                 List<Cell> cells = BoardImporter.importBoard(size, level);
102                 Board board = new Board(size, cells);
103
104                 // ===== BACKTRACKING =====
105                 BacktrackingSolverTest btSolver = new BacktrackingSolverTest(board);
106                 boolean btSolved = btSolver.solve();
107                 List<int[]> btCoords = btSolver.getSolutionCoordinates();
108                 long btTime = btSolver.getExecutionTime();
109
110                 writer.print("Backtracking:");
111                 writer.print(formatCoordinates(btCoords));
112                 writer.print(", " + btTime + "ms");
113                 writer.println(btSolved ? "valid" : "no_solution");
114
115                 result.addBT(btTime);
116
117                 // ===== PSO =====
118                 PSOSolverTest psoSolver = new PSOSolverTest(board,
119                     params.iterations, params.particles,
120                     params.c1, params.c2, params.neighborhoods,
121                     params.inertia, params.w1, params.w2,
122                     params.maxStagnation);
123
124                 psoSolver.solve();
125
126                 List<int[]> psoCoords = psoSolver.getSolutionCoordinates();
127                 long psoTime = psoSolver.getExecutionTime();
128                 boolean valid = psoSolver.isValid();
129                 double fitness = psoSolver.getFitness();
130                 int adj = psoSolver.getAdjacencyViolations();
131                 int att = psoSolver.getAttackingViolations();
132
133                 writer.print("PSO:");
134                 writer.print(formatCoordinates(psoCoords));
135                 writer.print(", " + psoTime + "ms");
136
137                 if (valid) {
138                     writer.println("valid");
139                 } else {
140                     writer.println("not_valid,fitness_" + fitness +
141                         ",adj_" + adj + ",att_" + att + "");
142                 }
143
144                 result.addPSO(valid, psoTime);
145             } catch (Exception e) {
146                 writer.println("Error:" + e.getMessage());
147             }
148         }
149     }
150 }
151

```

```

152         writer.println();
153         writer.flush();
154     }
155 }
156 } catch (IOException e) {
157     System.err.println("Error_writing_file:_" + e.getMessage());
158 }
159
160 return result;
161 }
162
163 // ===== SINGLE =====
164 private static BatchResult testSingleBoard(int size, PSOParameters params,
165                                           File paramDir, String paramBaseName) {
166
167     String outputFile = paramDir.getPath() +
168         "/results" + size + "x" + size + "_" + paramBaseName + ".txt";
169
170     BatchResult result = new BatchResult(size, 1);
171
172     try (PrintWriter writer = new PrintWriter(new FileWriter(outputFile))) {
173
174         writer.println("====_" + size + "x" + size + "_====");
175
176         List<Cell> cells = BoardImporter.importBoard(size, 1);
177         Board board = new Board(size, cells);
178
179         PSOSolverTest psoSolver = new PSOSolverTest(board,
180             params.iterations, params.particles,
181             params.c1, params.c2, params.neighborhoods,
182             params.inertia, params.w1, params.w2,
183             params.maxStagnation);
184
185         psoSolver.solve();
186
187         long psoTime = psoSolver.getExecutionTime();
188         boolean valid = psoSolver.isValid();
189
190         writer.println("PSO:_" + psoTime + "ms," + (valid ? "valid" : "not_valid"));
191
192         result.addPSO(valid, psoTime);
193
194     } catch (Exception e) {
195         System.err.println(e.getMessage());
196     }
197
198     return result;
199 }
200
201 // ===== SUMMARY =====
202 private static void generateSummary(String outputFile, SummaryStats stats) {
203
204     try (PrintWriter writer = new PrintWriter(new FileWriter(outputFile))) {
205
206         writer.println("===_SUMMARY_===\n");
207
208         for (int size : stats.getSizes()) {
209             writer.println(size + "x" + size + ":");
210
211             List<BatchStats> batches = stats.getBatches(size);
212
213             double totalValid = 0;
214             double totalRuns = 0;
215             long totalTime = 0;
216             long totalBTTime = 0;
217             int totalBTRuns = 0;
218
219             for (int i = 0; i < batches.size(); i++) {
220                 BatchStats b = batches.get(i);
221
222                 double rate = b.psoTotal > 0 ? (b.psoValid * 100.0 / b.psoTotal) : 0;
223
224                 writer.printf("_Batch_%d:_.2f%%,_PSO_Avg=_%dms,_BT_Avg=_%s\n",
225                     (i + 1),
226                     rate,
227                     b.getPSOAvg(),
228                     b.btTotal > 0 ? b.getBTAvg() + "ms" : "N/A"
229                 );
230
231                 totalValid += b.psoValid;
232                 totalRuns += b.psoTotal;
233                 totalTime += b.psoTime;
234
235                 totalBTTime += b.btTime;
236                 totalBTRuns += b.btTotal;
237             }
238
239             double meanBatchRate = batches.stream()
240                 .mapToDouble(b -> b.psoTotal > 0 ? (b.psoValid * 1.0 / b.psoTotal) : 0)
241                 .average().orElse(0) * 100;
242
243             long meanBatchTime = (long) batches.stream()
244                 .mapToLong(BatchStats::getPSOAvg)
245                 .average().orElse(0);
246
247             double overallRate = totalRuns > 0 ? (totalValid * 100.0 / totalRuns) : 0;
248             long overallTime = totalRuns > 0 ? totalTime / (long) totalRuns : 0;
249             long overallBTTime = totalBTRuns > 0 ? totalBTTime / totalBTRuns : 0;
250

```

```

251         writer.printf("Mean_(batch_avg):%.2f%%,%dms\n", meanBatchRate, meanBatchTime);
252         writer.printf("Overall_(all_runs):%.2f%%,%dms,BT_Avg_=%s\n",
253             overallRate,
254             overallTime,
255             totalBTRuns > 0 ? overallBTTime + "ms" : "N/A"
256         );
257     }
258 }
259 } catch (IOException e) {
260     System.err.println(e.getMessage());
261 }
262 }
263
264 // ===== UTILS =====
265 private static String formatCoordinates(List<int[]> coords) {
266     StringBuilder sb = new StringBuilder("{}");
267     for (int i = 0; i < coords.size(); i++) {
268         int[] c = coords.get(i);
269         sb.append("[").append(c[0]).append(",").append(c[1]).append("]");
270         if (i < coords.size() - 1) sb.append(",");
271     }
272     sb.append("}");
273     return sb.toString();
274 }
275
276 private static PSOParameters readParameters(String filename) {
277     PSOParameters params = new PSOParameters();
278
279     try (BufferedReader reader = new BufferedReader(
280         new FileReader(System.getProperty("user.dir") + "/params/" + filename))) {
281
282         String line;
283         while ((line = reader.readLine()) != null) {
284             String[] parts = line.split("=");
285             if (parts.length != 2) continue;
286
287             String key = parts[0].trim();
288             String value = parts[1].trim();
289
290             switch (key) {
291                 case "iterations": params.iterations = Integer.parseInt(value); break;
292                 case "particles": params.particles = Integer.parseInt(value); break;
293                 case "neighborhoods": params.neighborhoods = Integer.parseInt(value); break;
294                 case "c1": params.c1 = Double.parseDouble(value); break;
295                 case "c2": params.c2 = Double.parseDouble(value); break;
296                 case "inertia": params.inertia = Double.parseDouble(value); break;
297                 case "w1": params.w1 = Double.parseDouble(value); break;
298                 case "w2": params.w2 = Double.parseDouble(value); break;
299                 case "maxStagnation": params.maxStagnation = Integer.parseInt(value); break;
300             }
301         }
302     } catch (Exception e) {
303         System.err.println("Error_reading_parameters_file:" + e.getMessage());
304         System.exit(1);
305     }
306
307     return params;
308 }
309
310 // ===== DATA =====
311 static class BatchResult {
312     int size, batch;
313     int psoValid = 0, psoTotal = 0;
314     long psoTime = 0;
315     int btTotal = 0;
316     long btTime = 0;
317
318     BatchResult(int size, int batch) {
319         this.size = size;
320         this.batch = batch;
321     }
322
323     void addPSO(boolean valid, long time) {
324         psoTotal++;
325         if (valid) psoValid++;
326         psoTime += time;
327     }
328
329     void addBT(long time) {
330         btTotal++;
331         btTime += time;
332     }
333 }
334
335 static class SummaryStats {
336     Map<Integer, List<BatchStats>> data = new HashMap<>();
337
338     void merge(BatchResult r) {
339         data.computeIfAbsent(r.size, k -> new ArrayList<>());
340         List<BatchStats> list = data.get(r.size);
341
342         while (list.size() < r.batch) list.add(new BatchStats());
343
344         BatchStats b = list.get(r.batch - 1);
345
346         b.psoValid += r.psoValid;
347         b.psoTotal += r.psoTotal;
348         b.psoTime += r.psoTime;
349

```



```

350         b.btTotal += r.btTotal;
351         b.btTime += r.btTime;
352     }
353
354     List<Integer> getSizes() {
355         return new ArrayList<>(data.keySet());
356     }
357
358     List<BatchStats> getBatches(int size) {
359         return data.get(size);
360     }
361 }
362
363 static class BatchStats {
364     int psoValid = 0, psoTotal = 0;
365     long psoTime = 0;
366     int btTotal = 0;
367     long btTime = 0;
368
369     long getPSOAvg() {
370         return psoTotal > 0 ? psoTime / psoTotal : 0;
371     }
372
373     long getBTAvg() {
374         return btTotal > 0 ? btTime / btTotal : 0;
375     }
376 }
377
378 static class PSOParameters {
379     int iterations;
380     int particles;
381     int neighborhoods;
382     double c1, c2, inertia, w1, w2;
383     int maxStagnation;
384 }
385 }

```

Kode A.3: Kelas Board.java — Representasi papan permainan

```

1 package objects;
2
3 import java.util.*;
4
5 public class Board {
6     private int size;
7     //uses linkedhashmap to keep the order
8     private Map<String, List<int[]>> colorMap = new LinkedHashMap<>();
9     private Map<String, String> colorSymbolMap = new LinkedHashMap<>();
10
11     public Board(int size, List<Cell> cells) {
12         this.size = size;
13         buildColorMap(cells);
14         buildColorSymbols();
15     }
16
17     //memetakan setiap posisi warna ke dalam map
18     private void buildColorMap(List<Cell> cells) {
19         for (Cell cell : cells) {
20             String colorKey = String.format("%d,%d,%d", cell.getR(), cell.getG(), cell.getB());
21             colorMap.computeIfAbsent(colorKey, k -> new ArrayList<>())
22                 .add(new int[]{cell.getRow(), cell.getCol()});
23         }
24     }
25
26     //menggunakan huruf (A,B,C,D, dst.) untuk memodelkan warna RGB yang unik
27     private void buildColorSymbols() {
28         char symbol = 'A';
29         for (String color : colorMap.keySet()) {
30             if (symbol == 'Q') {
31                 symbol++;
32             }
33             colorSymbolMap.put(color, String.valueOf(symbol));
34             symbol++;
35         }
36     }
37
38     public int getSize() {
39         return size;
40     }
41
42     public Map<String, List<int[]>> getColorMap() {
43         return colorMap;
44     }
45
46     public Map<String, String> getColorSymbolMap() {
47         return colorSymbolMap;
48     }
49
50     //mengembalikan semua cell yang merupakan suatu warna
51     public List<int[]> getCellsByColor(String colorKey) {
52         return colorMap.getOrDefault(colorKey, Collections.emptyList());
53     }
54
55     //mengembalikan huruf yang merepresentasikan suatu key RGB
56     public String getSymbolForColor(String colorKey) {
57         return colorSymbolMap.getOrDefault(colorKey, "?");
58     }
59 }

```

```

59 //print dalam format huruf
60 public void printSymbolBoard() {
61     String[][] grid = new String[size][size];
62     for (Map.Entry<String, List<int[]>> entry : colorMap.entrySet()) {
63         String symbol = getSymbolForColor(entry.getKey());
64         for (int[] cell : entry.getValue()) {
65             grid[cell[0]][cell[1]] = symbol;
66         }
67     }
68 }
69
70 for (int r = 0; r < size; r++) {
71     for (int c = 0; c < size; c++) {
72         System.out.print((grid[r][c] != null ? grid[r][c] : ".") + " ");
73     }
74     System.out.println();
75 }
76 }
77
78 //logging
79 public void printColorSummary() {
80     System.out.println("Board_color_distribution:");
81     for (String color : colorMap.keySet()) {
82         String symbol = getSymbolForColor(color);
83         System.out.printf("Color_%s_-%s->%d_cells%n",
84             color, symbol, colorMap.get(color).size());
85     }
86     System.out.println("Board_size:_" + size);
87     System.out.println("Unique_colors:_" + colorMap.size());
88 }
89 }

```

Kode A.4: Kelas Cell.java — Representasi sel pada papan

```

1 package objects;
2
3 //Merepresentasikan 1 cell/kotak dalam permainan colored queens untuk memudahkan memasukkan data ke dalam Map di dalam object
  Board
4 public class Cell {
5     //menyimpan posisi dan warna dari kotak
6     private int row;
7     private int col;
8     private int r;
9     private int g;
10    private int b;
11
12    public Cell(int row, int col, int r, int g, int b) {
13        this.row = row;
14        this.col = col;
15        this.r = r;
16        this.g = g;
17        this.b = b;
18    }
19
20    // Getter dan setter
21    public int getRow() {
22        return row;
23    }
24
25    public void setRow(int row) {
26        this.row = row;
27    }
28
29    public int getCol() {
30        return col;
31    }
32
33    public void setCol(int col) {
34        this.col = col;
35    }
36
37    public int getR() {
38        return r;
39    }
40
41    public void setR(int r) {
42        this.r = r;
43    }
44
45    public int getG() {
46        return g;
47    }
48
49    public void setG(int g) {
50        this.g = g;
51    }
52
53    public int getB() {
54        return b;
55    }
56    public void setB(int b) {
57        this.b = b;
58    }
59
60    @Override
61    public String toString() {
62        return String.format("Cell[row=%d,col=%d,rgb=(%d,%d,%d)]", row, col, r, g, b);

```

```

63 }
64 }

```

Kode A.5: Kelas BoardImporter.java — Pemuat papan dari berkas JSON

```

1 package util;
2
3 import org.json.JSONArray;
4 import org.json.JSONObject;
5
6 import java.io.File;
7 import java.io.IOException;
8 import java.nio.file.Files;
9 import java.util.ArrayList;
10 import java.util.List;
11
12 import objects.Cell;
13
14 public class BoardImporter {
15
16     private static final String BOARD_FOLDER = "boards";
17
18     public static List<Cell> importBoard(int size, int level) throws IOException {
19         String filename = String.format("%s/%dx%d_level%d.json", BOARD_FOLDER, size, size, level);
20         File file = new File(filename);
21
22         if (!file.exists()) {
23             throw new IOException("Board_file_not_found:_" + filename);
24         }
25
26         String content = new String(Files.readAllBytes(file.toPath()));
27         JSONArray root = new JSONArray(content);
28         List<Cell> board = new ArrayList<>();
29
30         for (int i = 0; i < root.length(); i++) {
31             JSONObject node = root.getJSONObject(i);
32             int row = node.getInt("row");
33             int col = node.getInt("col");
34             String colorStr = node.getString("color");
35             int[] rgb = parseRgba(colorStr);
36
37             board.add(new Cell(row, col, rgb[0], rgb[1], rgb[2]));
38         }
39
40         return board;
41     }
42
43     private static int[] parseRgba(String rgbaString) {
44         rgbaString = rgbaString.replace("rgba(", "").replace(")", "");
45         String[] parts = rgbaString.split(",");
46         int r = Integer.parseInt(parts[0].trim());
47         int g = Integer.parseInt(parts[1].trim());
48         int b = Integer.parseInt(parts[2].trim());
49         return new int[]{r, g, b};
50     }
51
52     public static void printBoard(List<Cell> board, int size) {
53         String[][] display = new String[size][size];
54         for (Cell c : board) {
55             display[c.getRow()][c.getCol()] = String.format("(%d,%d,%d)", c.getR(), c.getG(), c.getB());
56         }
57
58         for (int r = 0; r < size; r++) {
59             for (int c = 0; c < size; c++) {
60                 System.out.print((display[r][c] != null ? display[r][c] : "._.") + "\t");
61             }
62             System.out.println();
63         }
64     }
65 }

```

A.2 Solver

A.2.1 Solver *Backtracking* dengan *Forward Checking* dan AC-3

Kode A.6: Kelas BacktrackingSolverAC3.java — Solver *backtracking* dengan *forward checking* dan AC-3

```

1 package solver.backtracking;
2
3 import java.util.*;
4
5 import objects.Board;
6
7 public class BacktrackingSolverAC3 {
8
9     private Board board;

```

```

10 private int size;
11 private Map<String, List<int[]>> colorCells;
12 private List<String> colors;
13 private int[] solution;
14 private boolean[][] occupied;
15 private long steps;
16 private long backtracks;
17 private long startTime;
18
19 // OPTIMIZATION: Use bitsets for O(1) operations
20 private Map<String, BitSet> validCells;
21 private int[] colorCellCount;
22 private Stack<PruneAction> pruneStack;
23
24 // Helper class to track what was pruned
25 private static class PruneAction {
26     String color;
27     int cellIndex;
28
29     PruneAction(String color, int cellIndex) {
30         this.color = color;
31         this.cellIndex = cellIndex;
32     }
33 }
34
35 // Simple Pair class for queue
36 private static class Pair<K, V> {
37     private K key;
38     private V value;
39
40     public Pair(K key, V value) {
41         this.key = key;
42         this.value = value;
43     }
44
45     public K getKey() { return key; }
46     public V getValue() { return value; }
47 }
48
49 public BacktrackingSolverAC3(Board board) {
50     this.board = board;
51     this.size = board.getSize();
52     this.colorCells = board.getColorMap();
53     this.colors = new ArrayList<>(colorCells.keySet());
54     this.solution = new int[colors.size()];
55     Arrays.fill(solution, -1);
56     this.occupied = new boolean[size][size];
57     this.steps = 0;
58     this.backtracks = 0;
59
60     // Initialize bitsets
61     this.validCells = new HashMap<>();
62     this.colorCellCount = new int[colors.size()];
63     for (int i = 0; i < colors.size(); i++) {
64         String color = colors.get(i);
65         int cellCount = colorCells.get(color).size();
66         BitSet bs = new BitSet(cellCount);
67         bs.set(0, cellCount);
68         validCells.put(color, bs);
69         colorCellCount[i] = cellCount;
70     }
71
72     this.pruneStack = new Stack<>();
73 }
74
75 public boolean solve() {
76     System.out.println("Starting_AC-3_solver_for_" + size + "x" + size + "_board_with_" + colors.size() + "_colors.");
77     startTime = System.currentTimeMillis();
78     boolean result = placeQueens(0);
79     long endTime = System.currentTimeMillis();
80
81     System.out.println("\nSolver_stats:");
82     System.out.println("Steps:_" + steps);
83     System.out.println("Backtracks:_" + backtracks);
84     System.out.println("Time:_" + (endTime - startTime) + "_ms");
85     System.out.println("Solution_found:_" + result);
86
87     return result;
88 }
89
90 private boolean placeQueens(int colorIndex) {
91     if (colorIndex == colors.size()) return true;
92
93     String color = colors.get(colorIndex);
94     List<int[]> cells = colorCells.get(color);
95     BitSet valid = validCells.get(color);
96
97     if (valid.cardinality() == 0) return false;
98
99     for (int cellIdx = valid.nextSetBit(0); cellIdx >= 0; cellIdx = valid.nextSetBit(cellIdx + 1)) {
100         int row = cells.get(cellIdx)[0];
101         int col = cells.get(cellIdx)[1];
102
103         solution[colorIndex] = cellIdx;
104         occupied[row][col] = true;
105
106         int pruneStartPos = pruneStack.size();
107
108         // Forward-check

```

```

109     forwardCheck(row, col, colorIndex);
110
111     // AC-3 arc propagation among future colors
112     Queue<Pair<Integer, Integer>> queue = new LinkedList<>();
113     for (int i = colorIndex + 1; i < colors.size(); i++) {
114         for (int j = colorIndex + 1; j < colors.size(); j++) {
115             if (i != j) queue.add(new Pair<>(i, j));
116         }
117     }
118     propagateArcs(queue);
119
120     if (anyColorExhausted(colorIndex)) {
121         undoPrunes(pruneStartPos);
122         occupied[row][col] = false;
123         solution[colorIndex] = -1;
124         backtracks++;
125         continue;
126     }
127
128     steps++;
129     if (placeQueens(colorIndex + 1)) return true;
130
131     undoPrunes(pruneStartPos);
132     occupied[row][col] = false;
133     solution[colorIndex] = -1;
134     backtracks++;
135 }
136
137 return false;
138 }
139
140 private void forwardCheck(int row, int col, int placedColorIdx) {
141     // Remove attacked cells from all future colors
142     for (int colorIdx = placedColorIdx + 1; colorIdx < colors.size(); colorIdx++) {
143         String color = colors.get(colorIdx);
144         // List<int[]> cells = colorCells.get(color); // Don't need this if we iterate bitset
145         BitSet valid = validCells.get(color);
146         List<int[]> currentCellList = colorCells.get(color);
147
148         for (int i = valid.nextSetBit(0); i >= 0; i = valid.nextSetBit(i + 1)) {
149             int[] target = currentCellList.get(i);
150             int r2 = target[0];
151             int c2 = target[1];
152
153             // INLINE CONFLICT CHECK (Replaces attackZone.contains)
154             boolean conflict = (row == r2 || col == c2 ||
155                               Math.abs(row - r2) <= 1 && Math.abs(col - c2) <= 1);
156
157             if (conflict) {
158                 valid.clear(i);
159                 colorCellCount[colorIdx]--;
160                 pruneStack.push(new PruneAction(color, i));
161             }
162         }
163     }
164 }
165
166 private boolean revise(int fromColorIdx, int toColorIdx) {
167     boolean revised = false;
168
169     String fromColor = colors.get(fromColorIdx);
170     String toColor = colors.get(toColorIdx);
171
172     List<int[]> fromCells = colorCells.get(fromColor);
173     List<int[]> toCells = colorCells.get(toColor);
174
175     BitSet fromValid = validCells.get(fromColor);
176     BitSet toValid = validCells.get(toColor);
177
178     for (int i = toValid.nextSetBit(0); i >= 0; i = toValid.nextSetBit(i + 1)) {
179         int[] toCell = toCells.get(i);
180         boolean supported = false;
181
182         // Check support in fromColor: at least one non-conflicting cell
183         for (int j = fromValid.nextSetBit(0); j >= 0; j = fromValid.nextSetBit(j + 1)) {
184             int[] fromCell = fromCells.get(j);
185             if (!conflicts(fromCell, toCell)) {
186                 supported = true;
187                 break;
188             }
189         }
190
191         // Also check already placed queens
192         if (supported) {
193             for (int placedIdx = 0; placedIdx < colors.size(); placedIdx++) {
194                 if (solution[placedIdx] != -1 && placedIdx != toColorIdx) {
195                     int[] placedCell = colorCells.get(colors.get(placedIdx)).get(solution[placedIdx]);
196                     if (conflicts(placedCell, toCell)) {
197                         supported = false;
198                         break;
199                     }
200                 }
201             }
202         }
203
204         // If no support, remove the value from toColor
205         if (!supported) {
206             toValid.clear(i);
207             colorCellCount[toColorIdx]--;

```

```

208         pruneStack.push(new PruneAction(toColor, i));
209         revised = true;
210     }
211 }
212
213     return revised;
214 }
215
216 private void propagateArcs(Queue<Pair<Integer, Integer>> queue) {
217     while (!queue.isEmpty()) {
218         Pair<Integer, Integer> arc = queue.poll();
219         int fromColorIdx = arc.getKey(); // The "Supporter"
220         int toColorIdx = arc.getValue(); // The "victim" (being pruned)
221
222         // revise checks if 'to' is supported by 'from'. If 'to' shrinks:
223         if (revise(fromColorIdx, toColorIdx)) {
224             if (colorCellCount[toColorIdx] == 0) return;
225
226             // We must notify neighbors of 'to' that 'to' has changed.
227             // We need to prune the neighbors ('k').
228             // So we add (to, k) so that revise(to, k) is called.
229             for (int k = 0; k < colors.size(); k++) {
230                 // Don't add the one we just came from, and don't add itself
231                 if (k != toColorIdx && k != fromColorIdx) {
232                     // CHANGED: Order swapped from (k, to) to (to, k)
233                     queue.add(new Pair<>(toColorIdx, k));
234                 }
235             }
236         }
237     }
238 }
239
240 // Check if two cells conflict (same row/col or adjacent)
241 private boolean conflicts(int[] cell1, int[] cell2) {
242     int r1 = cell1[0], c1 = cell1[1];
243     int r2 = cell2[0], c2 = cell2[1];
244
245     // Same row or column
246     if (r1 == r2 || c1 == c2) return true;
247
248     // Adjacent (including diagonal)
249     int dr = Math.abs(r1 - r2);
250     int dc = Math.abs(c1 - c2);
251     if (dr <= 1 && dc <= 1) return true;
252
253     return false;
254 }
255
256 private void undoPrunes(int pruneStartPos) {
257     while (pruneStack.size() > pruneStartPos) {
258         PruneAction action = pruneStack.pop();
259         String color = action.color;
260         int cellIdx = action.cellIndex;
261         validCells.get(color).set(cellIdx);
262         colorCellCount[colors.indexOf(color)]++;
263     }
264 }
265
266 // --- Check if any future color domain is empty ---
267 private boolean anyColorExhausted(int currentColorIndex) {
268     for (int i = currentColorIndex + 1; i < colors.size(); i++) {
269         if (colorCellCount[i] == 0) return true;
270     }
271     return false;
272 }
273
274 private void printBoard() {
275     System.out.println("Board state:");
276     String[][] grid = new String[size][size];
277
278     for (Map.Entry<String, List<int[]>> entry : colorCells.entrySet()) {
279         String symbol = board.getSymbolForColor(entry.getKey());
280         for (int[] cell : entry.getValue()) {
281             grid[cell[0]][cell[1]] = symbol;
282         }
283     }
284
285     for (int r = 0; r < size; r++) {
286         for (int c = 0; c < size; c++) {
287             if (occupied[r][c]) {
288                 grid[r][c] = "Q";
289             }
290         }
291     }
292
293     System.out.print("\n");
294     for (int c = 0; c < size; c++) {
295         System.out.print(c + " ");
296     }
297     System.out.println();
298
299     for (int r = 0; r < size; r++) {
300         System.out.print(r + " ");
301         for (int c = 0; c < size; c++) {
302             System.out.print((grid[r][c] != null ? grid[r][c] : ".") + " ");
303         }
304         System.out.println();
305     }
306     System.out.println();

```

```

307     }
308
309     public void printSolution() {
310         if (solution[0] == -1) {
311             System.out.println("No_solution_found!");
312             return;
313         }
314
315         System.out.println("Final_solution:");
316         for (int i = 0; i < colors.size(); i++) {
317             String color = colors.get(i);
318             String symbol = board.getSymbolForColor(color);
319             int[] cell = colorCells.get(color).get(solution[i]);
320             System.out.println("Color_" + symbol + "_at_" + cell[0] + "," + cell[1] + "");
321         }
322
323         printBoard();
324     }
325
326     public int[][] getSolutionAsGrid() {
327         int[][] result = new int[colors.size()][2];
328         for (int i = 0; i < colors.size(); i++) {
329             if (solution[i] == -1) return null;
330             int[] cell = colorCells.get(colors.get(i)).get(solution[i]);
331             result[i][0] = cell[0]; // row
332             result[i][1] = cell[1]; // col
333         }
334         return result;
335     }
336
337     public int[] getSolution() {
338         return solution.clone();
339     }
340
341     public long getSteps() {
342         return steps;
343     }
344
345     public long getBacktracks() {
346         return backtracks;
347     }
348
349     public long getExecutionTime() {
350         return System.currentTimeMillis() - startTime;
351     }
352 }

```

A.2.2 Solver PSO Diskrit

Kode A.7: Kelas PSOSolver.java — Solver *Particle Swarm Optimization* diskrit

```

1 package solver.PSO;
2
3 import java.util.*;
4 import java.util.stream.Collectors;
5 import java.util.stream.IntStream;
6
7 import objects.Board;
8
9 class Particle {
10     private int[] candidate;
11     private double[] velocity;
12     private double fitness;
13     private int[] pBest;
14     private double pBestFitness;
15
16     public Particle(int[] candidate, double[] velocity) {
17         this.candidate = candidate;
18         this.velocity = velocity;
19         this.pBest = candidate.clone();
20         this.pBestFitness = Double.MAX_VALUE;
21     }
22
23     public int[] getCandidate() {
24         return candidate;
25     }
26
27     public void setCandidate(int[] candidate) {
28         this.candidate = candidate;
29     }
30
31     public double[] getVelocity() {
32         return velocity;
33     }
34
35     public void setVelocity(double[] velocity) {
36         this.velocity = velocity;
37     }
38
39     public double getFitness() {
40         return fitness;
41     }
42
43     public void setFitness(double fitness) {

```

```

44     this.fitness = fitness;
45 }
46
47 public int[] getPBest() {
48     return pBest;
49 }
50
51 public void setPBest(int[] pBest) {
52     this.pBest = pBest;
53 }
54
55 public double getPBestFitness() {
56     return pBestFitness;
57 }
58
59 public void setPBestFitness(double pBestFitness) {
60     this.pBestFitness = pBestFitness;
61 }
62
63 }
64
65 public class PSOSolver {
66     private Board board;
67     private int size;
68     private Map<String, List<int[]>> colorCells;
69     private List<String> colors;
70     private int[] solution;
71
72     private int nIterations;
73     private int nParticles;
74     private List<Particle> particles;
75
76     private int nNeighborhood;
77     private List<List<Integer>> neighborhoods;
78     private int[] nBests;
79
80     private double c1;
81     private double c2;
82     private double inertia;
83
84     private double w1;
85     private double w2;
86
87     private Random random;
88
89     private int maxStagnation;
90
91     public PSOSolver(Board board, int nIterations, int nParticles, double c1, double c2, int nNeighborhood, double inertia, double
        w1, double w2, int maxStagnation) {
92         this.board = board;
93         this.size = this.board.getSize();
94
95         this.colorCells = this.board.getColorMap();
96         this.colors = new ArrayList<>(this.colorCells.keySet());
97
98         this.solution = new int[colors.size()];
99         Arrays.fill(solution, -1);
100
101         //main hyperparameters
102         this.nIterations = nIterations;
103         this.nParticles = nParticles;
104         this.nNeighborhood = nNeighborhood;
105
106         //create the initial
107         this.particles = new ArrayList<>();
108
109         //coeficients
110         this.c1 = c1;
111         this.c2 = c2;
112         this.inertia = inertia;
113
114         //fitness weights
115         this.w1 = w1;
116         this.w2 = w2;
117
118         this.random = new Random();
119
120         this.maxStagnation = maxStagnation;
121     }
122
123     private void generateParticles() {
124         for (int i = 0; i < nParticles; i++) {
125             int[] newSolution = generatePossibleSolution();
126             double[] newVelocity = generateRandomVelocity();
127
128             this.particles.add(new Particle(newSolution, newVelocity));
129         }
130     }
131
132     private int[] generatePossibleSolution() {
133         int[] newSolution = new int[this.colors.size()];
134
135         int idx = 0;
136         for (String s : this.colors) {
137             int domain = colorCells.get(s).size();
138
139             newSolution[idx] = random.nextInt(domain);
140             idx++;
141         }

```



```

142     return newSolution;
143 }
144
145 private double[] generateRandomVelocity() {
146     double[] newVelocity = new double[this.colors.size()];
147     for (int i = 0; i < this.colors.size(); i++) {
148         newVelocity[i] = random.nextDouble(1.0);
149     }
150     return newVelocity;
151 }
152
153 private void initialize() {
154     //1. generate the initial particles
155     this.generateParticles();
156     //2. separate into neighborhoods
157     this.generateNeighborhoods();
158     //3. initial fitness calculation
159     this.calculateInitialFitness();
160 }
161
162 private void generateNeighborhoods() {
163     this.neighborhoods = new ArrayList<List<Integer>>(nNeighborhood);
164     this.nBests = new int[this.nNeighborhood];
165     List<Integer> indexes = IntStream.range(0, nParticles).boxed().collect(Collectors.toList());
166     Collections.shuffle(indexes);
167     this.neighborhoods = partitionList(indexes, nNeighborhood);
168 }
169
170 public List<List<Integer>> partitionList(List<Integer> list, int n) {
171     //split the particles into neighborhoods
172     //set equal amounts of particles in each neighborhood
173     List<List<Integer>> result = new ArrayList<>(n);
174
175     int size = list.size();
176     int baseSize = size / n;
177     int remainder = size % n;
178
179     int index = 0;
180
181     for (int i = 0; i < n; i++) {
182         int groupSize = baseSize + (i < remainder ? 1:0);
183         List<Integer> neighborhood = new ArrayList<>();
184
185         for (int j = 0; j < groupSize && index < size; j++) {
186             neighborhood.add(list.get(index++));
187         }
188         result.add(neighborhood);
189     }
190     return result;
191 }
192
193 private void calculateInitialFitness() {
194     for (int i = 0; i < this.nNeighborhood; i++) {
195         int curNBestIdx = -1;
196         double curNBestFitness = Double.MAX_VALUE;
197
198         for (int index : this.neighborhoods.get(i)) {
199             Particle p = this.particles.get(index);
200             double fitness = calculateFitness(p);
201             p.setFitness(fitness);
202             p.setPBestFitness(fitness);
203             p.setPBest(p.getCandidate().clone());
204
205             if (fitness < curNBestFitness) {
206                 curNBestFitness = fitness;
207                 curNBestIdx = index;
208             }
209         }
210         this.nBests[i] = curNBestIdx;
211     }
212 }
213
214 private double calculateFitness(Particle candidate) {
215     boolean[][] occupied = new boolean[this.size][this.size];
216     int[] positions = candidate.getCandidate();
217
218     for (int i = 0; i < this.colors.size(); i++) {
219         String color = this.colors.get(i);
220         List<int[]> cells = this.colorCells.get(color);
221         int cellIdx = positions[i];
222
223         int[] cell = cells.get(cellIdx);
224         occupied[cell[0]][cell[1]] = true;
225     }
226 }

```

```

241     int attackingViolations = calculateAttackingViolation(candidate);
242     int adjacencyViolations = calculateAdjacencyViolation(candidate, occupied);
243
244     double fitness = w1 * attackingViolations + w2 * adjacencyViolations;
245
246     return fitness;
247 }
248
249 private int calculateAttackingViolation(Particle candidate) {
250     int count = 0;
251     int[] positions = candidate.getCandidate();
252
253     List<int[]> queenPositions = new ArrayList<>();
254
255     for (int i = 0; i < this.colors.size(); i++) {
256         String color = this.colors.get(i);
257         List<int[]> cells = this.colorCells.get(color);
258         int cellIdx = positions[i];
259         queenPositions.add(cells.get(cellIdx));
260     }
261
262     for (int i = 0; i < queenPositions.size(); i++) {
263         for (int j = i + 1; j < queenPositions.size(); j++) {
264             int[] pos1 = queenPositions.get(i);
265             int[] pos2 = queenPositions.get(j);
266
267             if (pos1[0] == pos2[0] || pos1[1] == pos2[1]) {
268                 count++;
269             }
270         }
271     }
272
273     return count;
274 }
275
276 private int calculateAdjacencyViolation(Particle candidate, boolean[][] occupied) {
277     int count = 0;
278     int[] positions = candidate.getCandidate();
279
280     int[][] dirs = {{-1,-1},{-1,0},{-1,1},{0,-1},{0,1},{1,-1},{1,0},{1,1}};
281
282     for (int i = 0; i < this.colors.size(); i++) {
283         String color = this.colors.get(i);
284         List<int[]> cells = this.colorCells.get(color);
285         int cellIdx = positions[i];
286         int[] cell = cells.get(cellIdx);
287
288         int row = cell[0];
289         int col = cell[1];
290
291         for (int[] d : dirs) {
292             int r = row + d[0];
293             int c = col + d[1];
294
295             if (r >= 0 && r < this.size && c >= 0 && c < this.size) {
296                 if (occupied[r][c]) {
297                     count++;
298                 }
299             }
300         }
301     }
302
303     return count/2;
304 }
305
306 public void solve() {
307     System.out.println("Starting_PS0_solver_for_" + size + "x" + size + "_board_with_" + colors.size() + "_colors.");
308     System.out.println("Parameters:_iterations=" + nIterations + ",_particles=" + nParticles +
309         ",_neighborhoods=" + nNeighborhood + ",_c1=" + c1 + ",_c2=" + c2 +
310         ",_inertia=" + inertia + ",_w1=" + w1 + ",_w2=" + w2 +
311         ",_maxStagnation=" + maxStagnation);
312
313     long startTime = System.currentTimeMillis();
314
315     //initialize the population
316     initialize();
317
318     //track the global best fitness across all neighborhoods for stagnation detection
319     double globalBestFitness = getLowestNBestFitness();
320     int stagnationCounter = 0;
321
322     for (int i = 1; i <= this.nIterations; i++) {
323         //update the velocity
324         updateVelocity();
325
326         //update particles and their fitness
327         updateParticles();
328
329         //check if solution exists
330         if (checkNbest() != -1) {
331             long endTime = System.currentTimeMillis();
332             System.out.println("\nSolution_found_at_iteration_" + i);
333             System.out.println("Time:_" + (endTime - startTime) + "_ms");
334             printSolution(checkNbest());
335             return;
336         }
337
338         //check for stagnation across all neighborhoods
339         double currentBestFitness = getLowestNBestFitness();

```

```

340     if (currentBestFitness < globalBestFitness) {
341         globalBestFitness = currentBestFitness;
342         stagnationCounter = 0;
343     } else {
344         stagnationCounter++;
345     }
346
347     if (stagnationCounter >= maxStagnation) {
348         long endTime = System.currentTimeMillis();
349         System.out.println("\nEarly_termination:_no_improvement_for_" + maxStagnation + "_iterations");
350         System.out.println("Time:_" + (endTime - startTime) + "_ms");
351         printSolution(checkLowestNBest());
352         return;
353     }
354 }
355
356 long endTime = System.currentTimeMillis();
357 System.out.println("\nNo_perfect_solution_found_after_" + nIterations + "_iterations");
358 System.out.println("Time:_" + (endTime - startTime) + "_ms");
359 printSolution(checkLowestNBest());
360 }
361
362 private double getLowestNBestFitness() {
363     double minFitness = Double.MAX_VALUE;
364
365     for (int n : this.nBests) {
366         double fitness = this.particles.get(n).getFitness();
367         if (fitness < minFitness) {
368             minFitness = fitness;
369         }
370     }
371     return minFitness;
372 }
373
374 private int checkLowestNBest() {
375     int solution = -1;
376     double minFitness = Double.MAX_VALUE;
377
378     for (int n : this.nBests) {
379         double fitness = this.particles.get(n).getFitness();
380         if (fitness < minFitness) {
381             minFitness = fitness;
382             solution = n;
383         }
384     }
385     return solution;
386 }
387
388 private int checkNbest() {
389     int solution = -1;
390
391     for (int n : this.nBests) {
392         if (this.particles.get(n).getFitness() == 0) {
393             solution = n;
394             break;
395         }
396     }
397     return solution;
398 }
399
400 private void updateParticles() {
401     for (int i = 0; i < this.nNeighborhood; i++) {
402         int nBestIdx = this.nBests[i];
403         int[] nBestPosition = this.particles.get(nBestIdx).getCandidate();
404
405         double curNBestFitness = this.particles.get(nBestIdx).getFitness();
406
407         for (int particleIdx : this.neighborhoods.get(i)) {
408             Particle p = this.particles.get(particleIdx);
409             int[] currentPosition = p.getCandidate();
410             double[] velocity = p.getVelocity();
411             int[] newPosition = currentPosition.clone();
412
413             for (int j = 0; j < newPosition.length; j++) {
414                 double rand = random.nextDouble(1.0);
415
416                 if (rand < velocity[j]) {
417                     newPosition[j] = nBestPosition[j];
418                 }
419             }
420
421             p.setCandidate(newPosition);
422
423             double newFitness = calculateFitness(p);
424             p.setFitness(newFitness);
425
426             if (newFitness < p.getPBestFitness()) {
427                 p.setPBestFitness(newFitness);
428                 p.setPBest(newPosition.clone());
429             }
430
431             if (newFitness < curNBestFitness) {
432                 curNBestFitness = newFitness;
433                 this.nBests[i] = particleIdx;
434             }
435         }
436     }
437 }
438

```

```

439 private void updateVelocity() {
440     for (int i = 0; i < this.nNeighborhood; i++) {
441         int nBestIdx = this.nBests[i];
442         int[] nBestPosition = this.particles.get(nBestIdx).getCandidate();
443
444         for (int particleIdx : this.neighborhoods.get(i)) {
445             Particle p = this.particles.get(particleIdx);
446             int[] currentPosition = p.getCandidate();
447             int[] pBestPosition = p.getPBest();
448             double[] velocity = p.getVelocity();
449             double[] newVelocity = new double[velocity.length];
450
451             for (int j = 0; j < velocity.length; j++) {
452                 double r1 = random.nextDouble(1.0);
453                 double r2 = random.nextDouble(1.0);
454
455                 int pBestDiff = (currentPosition[j] != pBestPosition[j]) ? 1 : 0;
456                 int nBestDiff = (currentPosition[j] != nBestPosition[j]) ? 1 : 0;
457
458                 newVelocity[j] = this.inertia * velocity[j] + this.c1 * r1 * pBestDiff + this.c2 * r2 * nBestDiff;
459
460                 if (newVelocity[j] < 0.0) newVelocity[j] = 0.0;
461                 if (newVelocity[j] > 1.0) newVelocity[j] = 1.0;
462             }
463
464             p.setVelocity(newVelocity);
465         }
466     }
467 }
468
469 private void printBoard(boolean[][] occupied) {
470     System.out.println("Board_state:");
471     String[][] grid = new String[size][size];
472
473     for (Map.Entry<String, List<int[]>> entry : colorCells.entrySet()) {
474         String symbol = board.getSymbolForColor(entry.getKey());
475         for (int[] cell : entry.getValue()) {
476             grid[cell[0]][cell[1]] = symbol;
477         }
478     }
479
480     for (int r = 0; r < size; r++) {
481         for (int c = 0; c < size; c++) {
482             if (occupied[r][c]) {
483                 grid[r][c] = "Q";
484             }
485         }
486     }
487
488     System.out.print("\n");
489     for (int c = 0; c < size; c++) {
490         System.out.print(c + " ");
491     }
492     System.out.println();
493
494     for (int r = 0; r < size; r++) {
495         System.out.print(r + " ");
496         for (int c = 0; c < size; c++) {
497             System.out.print((grid[r][c] != null ? grid[r][c] : ".") + " ");
498         }
499         System.out.println();
500     }
501     System.out.println();
502 }
503
504 private void printSolution(int particleIdx) {
505     if (particleIdx == -1) {
506         System.out.println("No_solution_found!");
507         return;
508     }
509
510     Particle p = this.particles.get(particleIdx);
511     int[] positions = p.getCandidate();
512     double fitness = p.getFitness();
513
514     boolean isValid = (fitness == 0.0);
515
516     if (isValid) {
517         System.out.println("\n===_VALID_SOLUTION_FOUND_===");
518     } else {
519         System.out.println("\n===_BEST_SOLUTION_(NOT_VALID)_===");
520         System.out.println("Fitness:_" + fitness);
521     }
522
523     System.out.println("Final_solution:");
524
525     boolean[][] occupied = new boolean[this.size][this.size];
526
527     for (int i = 0; i < colors.size(); i++) {
528         String color = colors.get(i);
529         String symbol = board.getSymbolForColor(color);
530         int[] cell = colorCells.get(color).get(positions[i]);
531         System.out.println("Color:_" + symbol + "_at_" + cell[0] + "," + cell[1] + " ");
532         occupied[cell[0]][cell[1]] = true;
533     }
534
535     printBoard(occupied);
536 }
537 }

```

A.3 Antarmuka Pengguna

Kode A.8: Kelas `UITheme.java` — Konstanta tema antarmuka pengguna

```

1 package GUI;
2
3 import javax.swing.*;
4 import javax.swing.border.*;
5 import java.awt.*;
6 import java.awt.event.*;
7 import java.awt.geom.RoundRectangle2D;
8
9 public class UITheme {
10     // palette
11     public static final Color CREAM      = new Color(245, 239, 228);
12     public static final Color CARD       = new Color(251, 247, 240);
13     public static final Color PLUM       = new Color(142, 106, 138);
14     public static final Color PLUM_DARK  = new Color(112, 82, 108);
15     public static final Color ROSE       = new Color(217, 166, 160);
16     public static final Color TAUPE      = new Color(168, 156, 138);
17     public static final Color TEXT_DARK  = new Color(62, 46, 46);
18     public static final Color TEXT_MUTED = new Color(120, 104, 94);
19
20     // fonts (Swing falls through to first installed)
21     public static Font font(int style, int size) {
22         return new Font("Segoe_UI", style, size);
23     }
24     public static Font serif(int style, int size) {
25         return new Font("Georgia", style, size);
26     }
27
28     // a reusable cream-gradient background panel
29     public static class GradientPanel extends JPanel {
30         public GradientPanel() {
31             setOpaque(false);
32         }
33         @Override
34         protected void paintComponent(Graphics g) {
35             Graphics2D g2 = (Graphics2D) g.create();
36             g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
37                                 RenderingHints.VALUE_ANTIALIAS_ON);
38             GradientPaint gp = new GradientPaint(
39                 0, 0, new Color(248, 242, 232),
40                 0, getHeight(), new Color(238, 228, 215));
41             g2.setPaint(gp);
42             g2.fillRect(0, 0, getWidth(), getHeight());
43             g2.dispose();
44             super.paintComponent(g);
45         }
46     }
47
48     // pretty rounded button, pass primary=true for plum, false for outline
49     public static JButton roundedButton(String text, boolean primary) {
50         JButton b = new JButton(text) {
51             @Override
52             protected void paintComponent(Graphics g) {
53                 Graphics2D g2 = (Graphics2D) g.create();
54                 g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
55                                     RenderingHints.VALUE_ANTIALIAS_ON);
56
57                 Color bg;
58                 if (primary) {
59                     bg = getModel().isPressed() ? PLUM_DARK
60                        : getModel().isRollover() ? ROSE
61                        : PLUM;
62                 } else {
63                     bg = getModel().isRollover() ? new Color(232, 220, 205)
64                        : CARD;
65                 }
66                 g2.setColor(bg);
67                 g2.fill(new RoundRectangle2D.Float(0, 0,
68                                                     getWidth(), getHeight(), 22, 22));
69
70                 if (!primary) {
71                     g2.setColor(TAUPE);
72                     g2.setStroke(new BasicStroke(1.5f));
73                     g2.draw(new RoundRectangle2D.Float(1, 1,
74                                                         getWidth() - 2, getHeight() - 2, 22, 22));
75                 }
76                 g2.dispose();
77                 super.paintComponent(g);
78             }
79             @Override public boolean isContentAreaFilled() { return false; }
80         };
81         b.setForeground(primary ? Color.WHITE : TEXT_DARK);
82         b.setFont(font(Font.BOLD, 18));
83         b.setFocusPainted(false);
84         b.setBorderPainted(false);
85         b.setContentAreaFilled(false);
86         b.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
87         b.setBorder(new EmptyBorder(10, 24, 10, 24));
88         return b;
89     }
90
91     // a soft card panel (for the level selector groups)
92     public static JPanel card() {

```

```

93     JPanel p = new JPanel();
94     p.setBackground(CARD);
95     p.setBorder(new CompoundBorder(
96         new LineBorder(TAUPE, 1, true),
97         new EmptyBorder(20, 28, 20, 28)
98     ));
99     return p;
100 }
101 }

```

Kode A.9: Kelas Homepage.java — Halaman utama aplikasi

```

1  package GUI;
2
3  import javax.imageio.ImageIO;
4  import javax.swing.*;
5  import java.awt.*;
6  import java.io.File;
7
8  public class Homepage extends JFrame {
9
10     public Homepage() {
11         setTitle("Colored Queens");
12         setSize(1920, 1080);
13         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
14         setLocationRelativeTo(null);
15         setResizable(false);
16
17         UITheme.GradientPanel panel = new UITheme.GradientPanel();
18         panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
19
20         // queen icon
21         JLabel queenIcon = new JLabel();
22         queenIcon.setAlignmentX(Component.CENTER_ALIGNMENT);
23         try {
24             Image img = ImageIO.read(new File("assets/queen_black.png"));
25             Image scaled = img.getScaledInstance(140, 140, Image.SCALE_SMOOTH);
26             queenIcon.setIcon(new ImageIcon(scaled));
27         } catch (Exception e) {
28             // fall through silently if asset missing
29         }
30
31         // main welcome title (big + bold, your main ask)
32         JLabel welcomeLabel = new JLabel("Welcome_to_Colored_Queens");
33         welcomeLabel.setFont(UITheme.serif(Font.BOLD, 64));
34         welcomeLabel.setForeground(UITheme.PLUM);
35         welcomeLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
36
37         // subtitle / tagline
38         JLabel subtitle = new JLabel("A_puzzle_of_queens,_colors,_and_constraints");
39         subtitle.setFont(UITheme.font(Font.PLAIN, 22));
40         subtitle.setForeground(UITheme.TEXT_MUTED);
41         subtitle.setAlignmentX(Component.CENTER_ALIGNMENT);
42
43         // start button
44         JButton startButton = UITheme.roundedButton("Start_Game", true);
45         startButton.setFont(UITheme.font(Font.BOLD, 22));
46         startButton.setAlignmentX(Component.CENTER_ALIGNMENT);
47         startButton.setMaximumSize(new Dimension(260, 60));
48         startButton.addActionListener(e -> {
49             new LevelSelectorWindow().setVisible(true);
50             dispose();
51         });
52
53         panel.add(Box.createVerticalGlue());
54         panel.add(queenIcon);
55         panel.add(Box.createRigidArea(new Dimension(0, 20)));
56         panel.add(welcomeLabel);
57         panel.add(Box.createRigidArea(new Dimension(0, 12)));
58         panel.add(subtitle);
59         panel.add(Box.createRigidArea(new Dimension(0, 50)));
60         panel.add(startButton);
61         panel.add(Box.createVerticalGlue());
62         panel.add(Box.createRigidArea(new Dimension(0, 30)));
63
64         setContentPane(panel);
65     }
66 }

```

Kode A.10: Kelas LevelSelectorWindow.java — Halaman pemilihan level

```

1  package GUI;
2
3  import javax.swing.*;
4  import java.awt.*;
5
6  public class LevelSelectorWindow extends JFrame {
7
8     public LevelSelectorWindow() {
9         setTitle("Colored Queens");
10         setSize(1920, 1080);
11         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
12         setLocationRelativeTo(null);
13         setResizable(false);

```

```

14
15     UIManager GradientPanel main = new UIManager.GradientPanel();
16     main.setLayout(new BorderLayout(main, BorderLayout.Y_AXIS));
17     main.setBorder(BorderFactory.createEmptyBorder(50, 80, 50, 80));
18
19     // page header
20     JLabel header = new JLabel("Choose_Your_Challenge");
21     header.setFont(UITheme.serif(Font.BOLD, 48));
22     header.setForeground(UITheme.PLUM);
23     header.setAlignmentX(Component.CENTER_ALIGNMENT);
24
25     // --- Section 1: Pick a Difficulty ---
26     JLabel diffLabel = new JLabel("Pick_a_Difficulty");
27     diffLabel.setFont(UITheme.font(Font.BOLD, 28));
28     diffLabel.setForeground(UITheme.TEXT_DARK);
29     diffLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
30
31     JPanel diffPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 40, 20));
32     diffPanel.setOpaque(false);
33     diffPanel.add(makeDifficultyGroup("Easy", new int[]{7}));
34     diffPanel.add(makeDifficultyGroup("Medium", new int[]{8, 9}));
35     diffPanel.add(makeDifficultyGroup("Hard", new int[]{10, 11, 12}));
36
37     // --- Section 2: Challenge Yourself ---
38     JLabel challengeLabel = new JLabel("Challenge_Yourself!");
39     challengeLabel.setFont(UITheme.font(Font.BOLD, 28));
40     challengeLabel.setForeground(UITheme.TEXT_DARK);
41     challengeLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
42
43     JLabel challengeSub = new JLabel("Only_one_level_each");
44     challengeSub.setFont(UITheme.font(Font.ITALIC, 16));
45     challengeSub.setForeground(UITheme.TEXT_MUTED);
46     challengeSub.setAlignmentX(Component.CENTER_ALIGNMENT);
47
48     JPanel challengePanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 40, 20));
49     challengePanel.setOpaque(false);
50     challengePanel.add(makeSizeButton(20));
51     challengePanel.add(makeSizeButton(30));
52
53     main.add(header);
54     main.add(Box.createRigidArea(new Dimension(0, 35)));
55     main.add(diffLabel);
56     main.add(Box.createRigidArea(new Dimension(0, 18)));
57     main.add(diffPanel);
58     main.add(Box.createRigidArea(new Dimension(0, 35)));
59     main.add(challengeLabel);
60     main.add(Box.createRigidArea(new Dimension(0, 4)));
61     main.add(challengeSub);
62     main.add(Box.createRigidArea(new Dimension(0, 14)));
63     main.add(challengePanel);
64
65     setContentPane(main);
66 }
67
68 private JPanel makeDifficultyGroup(String label, int[] sizes) {
69     JPanel group = UIManager.card();
70     group.setLayout(new BorderLayout(group, BorderLayout.Y_AXIS));
71
72     JLabel groupLabel = new JLabel(label);
73     groupLabel.setFont(UITheme.serif(Font.BOLD, 24));
74     groupLabel.setForeground(UITheme.PLUM);
75     groupLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
76     group.add(groupLabel);
77     group.add(Box.createRigidArea(new Dimension(0, 14)));
78
79     for (int size : sizes) {
80         JButton btn = makeSizeButton(size);
81         btn.setAlignmentX(Component.CENTER_ALIGNMENT);
82         group.add(btn);
83         group.add(Box.createRigidArea(new Dimension(0, 8)));
84     }
85
86     return group;
87 }
88
89 private JButton makeSizeButton(int size) {
90     JButton btn = UIManager.roundedButton(size + "x" + size, true);
91     btn.setPreferredSize(new Dimension(150, 52));
92     btn.addActionListener(e -> {
93         new GameWindow(size, 1).setVisible(true);
94         dispose();
95     });
96     return btn;
97 }
98 }

```

Kode A.11: Kelas GameWindow.java — Halaman permainan utama

```

1 package GUI;
2
3 import objects.Board;
4 import objects.Cell;
5 import solver.backtracking.BacktrackingSolverAC3;
6 import util.BoardImporter;
7
8 import javax.imageio.ImageIO;
9 import javax.swing.*;

```

```

10 import java.awt.*;
11 import java.awt.event.*;
12 import java.io.File;
13 import java.util.*;
14 import java.util.List;
15 import javax.swing.Timer;
16
17
18 public class GameWindow extends JFrame {
19
20     private SwingWorker<int[][], Void> currentWorker;
21     private int currentSize;
22     private int currentLevel;
23     private Board board;
24     private int[][] solution; // solution[i] = [row, col] for color i
25     private boolean[][] queenPlaced;
26     private boolean timerStarted = false;
27     private Timer gameTimer;
28     private JLabel timerLabel;
29     private JLabel sizeLabel;
30     private JLabel levelLabel;
31     private BoardPanel boardPanel;
32     private JComboBox<String> levelDropdown;
33     private boolean suppressDropdownEvents = false;
34     private JComboBox<String> sizeDropdown;
35
36     private long startTime;
37
38     private static final Map<Integer, Integer> MAX_LEVELS = new LinkedHashMap<>();
39     static {
40         MAX_LEVELS.put(7, 250);
41         MAX_LEVELS.put(8, 250);
42         MAX_LEVELS.put(9, 250);
43         MAX_LEVELS.put(10, 250);
44         MAX_LEVELS.put(11, 250);
45         MAX_LEVELS.put(12, 250);
46         MAX_LEVELS.put(20, 1);
47         MAX_LEVELS.put(30, 1);
48     }
49
50     // hardcoded solutions for 20x20 and 30x30
51     // each int[] is {row, col}, ordered by color symbol (A, B, C...)
52     private static final int[][] PRELOADED_20x20 = {
53         {15,14},{14,1},{17,16},{3,0},{0,12},{10,9},{1,7},{5,8},
54         {6,19},{2,3},{11,17},{16,10},{12,2},{18,11},{8,18},{7,13},
55         {4,15},{13,4},{19,5},{9,6}
56     };
57
58     private static final int[][] PRELOADED_30x30 = {
59         {21,22},{15,9},{6,15},{7,10},{26,4},{0,11},{9,3},{19,28},
60         {1,0},{27,7},{28,26},{18,23},{22,1},{29,13},{4,16},{11,25},
61         {10,21},{20,20},{23,12},{14,19},{3,24},{2,17},{25,2},{24,18},
62         {13,27},{12,14},{16,5},{17,29},{5,8},{8,6}
63     };
64
65     public GameWindow(int size, int level) {
66         this.currentSize = size;
67         this.currentLevel = level;
68
69         setTitle("Colored Queens");
70         setSize(1920, 1080);
71         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
72         setLocationRelativeTo(null);
73         setResizable(false);
74         setLayout(new BorderLayout());
75
76         // --- Top Panel ---
77         JPanel topPanel = new JPanel();
78         topPanel.setLayout(new BoxLayout(topPanel, BoxLayout.Y_AXIS));
79         topPanel.setBackground(UITheme.CREAM);
80         topPanel.setBorder(BorderFactory.createEmptyBorder(20, 0, 10, 0));
81
82         sizeLabel = new JLabel(size + "_X_" + size);
83         sizeLabel.setFont(UITheme.serif(Font.BOLD, 32));
84         sizeLabel.setForeground(UITheme.PLUM);
85         sizeLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
86
87         levelLabel = new JLabel("Level_" + level);
88         levelLabel.setFont(UITheme.font(Font.PLAIN, 20));
89         levelLabel.setForeground(UITheme.TEXT_MUTED);
90         levelLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
91
92         topPanel.add(sizeLabel);
93         topPanel.add(levelLabel);
94
95         // --- Bottom Panel ---
96         JPanel bottomPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 20, 10));
97         bottomPanel.setBackground(UITheme.CREAM);
98         bottomPanel.setBorder(BorderFactory.createEmptyBorder(10, 0, 20, 0));
99
100         // size dropdown
101         String[] sizeOptions = {"7x7", "8x8", "9x9", "10x10", "11x11", "12x12", "20x20", "30x30"};
102         sizeDropdown = new JComboBox<>(sizeOptions);
103         sizeDropdown.setSelectedItem(size + "x" + size);
104         sizeDropdown.setFont(new Font("Arial", Font.PLAIN, 16));
105         sizeDropdown.addActionListener(e -> {
106             if (suppressDropdownEvents) return;
107             String selected = (String) sizeDropdown.getSelectedItem();
108             int newSize = Integer.parseInt(selected.replace("x" + selected.split("x")[1], ""));

```



```

109         loadBoard(newSize, 1);
110     });
111
112     // level dropdown
113     levelDropdown = new JComboBox<>();
114     populateLevelDropdown(size);
115     levelDropdown.setSelectedItem("Level_␣" + level);
116     levelDropdown.setFont(new Font("Arial", Font.PLAIN, 16));
117     levelDropdown.addActionListener(e -> {
118         if (suppressDropdownEvents) return;
119         if (levelDropdown.getSelectedItem() == null) return;
120         String selected = (String) levelDropdown.getSelectedItem();
121         int newLevel = Integer.parseInt(selected.replace("Level_␣", ""));
122         if (newLevel != currentLevel) {
123             loadBoard(currentSize, newLevel);
124         }
125     });
126
127     // timer
128     timerLabel = new JLabel("00:00:000");
129     timerLabel.setFont(UITheme.font(Font.BOLD, 22));
130     timerLabel.setForeground(UITheme.TEXT_DARK);
131
132     gameTimer = new Timer(20, e -> {
133         long elapsed = System.currentTimeMillis() - startTime;
134         timerLabel.setText(formatTime((int) elapsed));
135     });
136
137     // hint button
138     JButton hintButton = UITHeme.roundedButton("Hint", false);
139     hintButton.addActionListener(e -> giveHint());
140
141     // solve button
142     JButton solveButton = UITHeme.roundedButton("Solve", true);
143     solveButton.addActionListener(e -> applySolution());
144
145     bottomPanel.add(sizeDropdown);
146     bottomPanel.add(levelDropdown);
147     bottomPanel.add(timerLabel);
148     bottomPanel.add(hintButton);
149     bottomPanel.add(solveButton);
150
151     // --- Board Panel (placeholder until board loads) ---
152     boardPanel = new BoardPanel();
153     boardPanel.setBackground(UITheme.CREAM);
154
155     JPanel legendPanel = createLegend();
156     legendPanel.setPreferredSize(new Dimension(500, 0));
157
158     JPanel leftSpacer = new JPanel();
159     leftSpacer.setPreferredSize(new Dimension(500, 0));
160     boardPanel.setBackground(UITheme.CREAM);
161
162     add(topPanel, BorderLayout.NORTH);
163     add(boardPanel, BorderLayout.CENTER);
164     add(legendPanel, BorderLayout.EAST);
165     add(leftSpacer, BorderLayout.WEST);
166     add(bottomPanel, BorderLayout.SOUTH);
167
168     // load the board
169     loadBoard(size, level);
170 }
171
172 private void loadBoard(int size, int level) {
173     this.currentSize = size;
174     this.currentLevel = level;
175
176     // stop timer
177     if (gameTimer.isRunning()) gameTimer.stop();
178     timerStarted = false;
179     startTime = 0;
180     timerLabel.setText("00:00:000");
181
182     // update labels
183     sizeLabel.setText(size + "x␣" + size);
184     levelLabel.setText("Level_␣" + level);
185
186     // update both dropdowns without triggering their listeners
187     ActionListener[] sizeListeners = sizeDropdown.getActionListeners();
188     ActionListener[] levelListeners = levelDropdown.getActionListeners();
189     for (ActionListener al : sizeListeners) sizeDropdown.removeActionListener(al);
190     for (ActionListener al : levelListeners) levelDropdown.removeActionListener(al);
191
192     sizeDropdown.setSelectedItem(size + "x␣" + size);
193     populateLevelDropdown(size);
194     levelDropdown.setSelectedItem("Level_␣" + level);
195
196     for (ActionListener al : sizeListeners) sizeDropdown.addActionListener(al);
197     for (ActionListener al : levelListeners) levelDropdown.addActionListener(al);
198
199     for (ActionListener al : levelListeners) levelDropdown.addActionListener(al);
200
201     try {
202         List<Cell> cells = BoardImporter.importBoard(size, level);
203         board = new Board(size, cells);
204         queenPlaced = new boolean[size][size];
205
206         if (currentWorker != null && !currentWorker.isDone()) {
207             currentWorker.cancel(true);

```

```

208     }
209
210     solution = null;
211
212     boardPanel.setBoard(board, queenPlaced);
213     boardPanel.setErrorHighlight(null);
214     boardPanel.revalidate();
215     boardPanel.repaint();
216
217     if (size == 20) {
218         solution = PRELOADED_20x20;
219     } else if (size == 30) {
220         solution = PRELOADED_30x30;
221     } else {
222         currentWorker = new SwingWorker<>() {
223             @Override
224             protected int[][] doInBackground() {
225                 BacktrackingSolverAC3 solver = new BacktrackingSolverAC3(board);
226                 solver.solve();
227                 return solver.getSolutionAsGrid();
228             }
229
230             @Override
231             protected void done() {
232                 try {
233                     if (!isCancelled()) {
234                         solution = get();
235                     }
236                 } catch (Exception e) {
237                     System.err.println("Solver_error:_" + e.getMessage());
238                 }
239             }
240         };
241         currentWorker.execute();
242     }
243
244     } catch (Exception e) {
245         System.err.println("Failed_to_load_board:_" + e.getMessage());
246     }
247 }
248
249 private void applySolution() {
250     if (solution == null) {
251         JOptionPane.showMessageDialog(this, "Solution_not_ready_yet,_please_wait.");
252         return;
253     }
254     // stop timer
255     if (gameTimer.isRunning()) gameTimer.stop();
256
257     // apply solution to board
258     queenPlaced = new boolean[currentSize][currentSize];
259     for (int[] pos : solution) {
260         queenPlaced[pos[0]][pos[1]] = true;
261     }
262     boardPanel.setQueens(queenPlaced);
263     boardPanel.repaint();
264     boardPanel.checkViolations();
265 }
266
267 private void giveHint() {
268     if (solution == null) {
269         JOptionPane.showMessageDialog(this, "Hint_not_ready_yet,_please_wait.");
270         return;
271     }
272
273     List<String> colors = new ArrayList<>(board.getColorMap().keySet());
274
275     for (int i = 0; i < solution.length; i++) {
276         int[] correctPos = solution[i];
277         String color = colors.get(i);
278         List<int[]> colorCells = board.getColorMap().get(color);
279
280         // skip if correct position already has a queen
281         boolean correctAlreadyPlaced = queenPlaced[correctPos[0]][correctPos[1]];
282         if (correctAlreadyPlaced) continue;
283
284         // check if player placed a wrong queen on this color region
285         int[] wrongPos = null;
286         for (int[] cell : colorCells) {
287             if (queenPlaced[cell[0]][cell[1]]) {
288                 wrongPos = cell;
289                 break;
290             }
291         }
292
293         // tell boardPanel to show the hint
294         boardPanel.setHint(correctPos, wrongPos);
295         boardPanel.repaint();
296         return;
297     }
298 }
299
300 private void populateLevelDropdown(int size) {
301     levelDropdown.removeAllItems();
302     int max = MAX_LEVELS.getOrDefault(size, 1);
303     for (int i = 1; i <= max; i++) {
304         levelDropdown.addItem("Level_" + i);
305     }
306 }

```

```

307
308 private String formatTime(int ms) {
309     int minutes = ms / 60000;
310     int seconds = (ms % 60000) / 1000;
311     int millis = ms % 1000;
312     return String.format("%02d:%02d:%03d", minutes, seconds, millis);
313 }
314
315 private JPanel createLegend() {
316     JPanel legend = new JPanel();
317     legend.setLayout(new BoxLayout(legend, BoxLayout.Y_AXIS));
318     legend.setBackground(UITheme.CREAM);
319     legend.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
320
321     JPanel placedRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 5));
322     placedRow.setBackground(UITheme.CREAM);
323     placedRow.add(createQueenLabel(boardPanel.queenImageBlack));
324     placedRow.add(createQueenLabel(boardPanel.queenImageWhite));
325     JLabel placedText = new JLabel("Placed_queens_(no_difference_between_them)");
326     placedText.setFont(new Font("Arial", Font.PLAIN, 18));
327     placedRow.add(placedText);
328
329     JPanel errorRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 5));
330     errorRow.setBackground(UITheme.CREAM);
331     errorRow.add(createQueenLabel(boardPanel.queenImageRed));
332     JLabel errorText = new JLabel("Breaks_one_or_more_rules");
333     errorText.setFont(new Font("Arial", Font.PLAIN, 18));
334     errorRow.add(errorText);
335
336     legend.add(Box.createVerticalGlue());
337     legend.add(placedRow);
338     legend.add(Box.createRigidArea(new Dimension(0, 15)));
339     legend.add(errorRow);
340     legend.add(Box.createVerticalGlue());
341
342     return legend;
343 }
344
345 private JLabel createQueenLabel(Image img) {
346     JLabel label = new JLabel();
347     if (img != null) {
348         label.setIcon(new ImageIcon(img.getScaledInstance(50, 50, Image.SCALE_SMOOTH)));
349     }
350     return label;
351 }
352
353 private void showWinDialog() {
354     JDialog dialog = new JDialog(this, true); // modal
355     dialog.setUndecorated(true);
356     dialog.setSize(480, 320);
357     dialog.setLocationRelativeTo(this);
358
359     // rounded card body
360     JPanel card = new JPanel() {
361         @Override
362         protected void paintComponent(Graphics g) {
363             Graphics2D g2 = (Graphics2D) g.create();
364             g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
365                 RenderingHints.VALUE_ANTIALIAS_ON);
366             g2.setColor(UITheme.CARD);
367             g2.fillRoundRect(0, 0, getWidth(), getHeight(), 28, 28);
368             g2.setColor(UITheme.TAUPE);
369             g2.setStroke(new BasicStroke(1.5f));
370             g2.drawRoundRect(1, 1, getWidth() - 3, getHeight() - 3, 28, 28);
371             g2.dispose();
372         }
373     };
374     card.setOpaque(false);
375     card.setLayout(new BoxLayout(card, BoxLayout.Y_AXIS));
376     card.setBorder(BorderFactory.createEmptyBorder(36, 40, 30, 40));
377
378     // queen icon (reuse the asset)
379     JLabel icon = new JLabel();
380     icon.setAlignmentX(Component.CENTER_ALIGNMENT);
381     if (boardPanel.queenImageBlack != null) {
382         icon.setIcon(new ImageIcon(boardPanel.queenImageBlack
383             .getScaledInstance(72, 72, Image.SCALE_SMOOTH)));
384     }
385
386     JLabel title = new JLabel("Congratulations!");
387     title.setFont(UITheme.serif(Font.BOLD, 34));
388     title.setForeground(UITheme.PLUM);
389     title.setAlignmentX(Component.CENTER_ALIGNMENT);
390
391     JLabel sub = new JLabel("You_solved_it_in_" + timerLabel.getText());
392     sub.setFont(UITheme.font(Font.PLAIN, 18));
393     sub.setForeground(UITheme.TEXT_MUTED);
394     sub.setAlignmentX(Component.CENTER_ALIGNMENT);
395
396     // figure out what comes next
397     int[] next = getNextSizeLevel(currentSize, currentLevel);
398
399     JButton ok = UITheme.roundedButton("Nice!", false);
400     ok.setMaximumSize(new Dimension(150, 48));
401     ok.addActionListener(e -> dialog.dispose());
402
403     JPanel buttonRow = new JPanel(new FlowLayout(FlowLayout.CENTER, 14, 0));
404     buttonRow.setOpaque(false);
405     buttonRow.setAlignmentX(Component.CENTER_ALIGNMENT);

```

```

406     buttonRow.setMaximumSize(new Dimension(400, 56));
407
408     if (next != null) {
409         JButton nextButton = UITheme.roundedButton("Next_Level", true);
410         nextButton.setMaximumSize(new Dimension(170, 48));
411         nextButton.addActionListener(e -> {
412             dialog.dispose();
413             loadBoard(next[0], next[1]);
414         });
415         buttonRow.add(ok);
416         buttonRow.add(nextButton);
417     } else {
418         ok.setText("You_finished_everything!");
419         ok.setMaximumSize(new Dimension(280, 48));
420         buttonRow.add(ok);
421     }
422
423     // assemble, order matters
424     card.add(icon);
425     card.add(Box.createRigidArea(new Dimension(0, 16)));
426     card.add(title);
427     card.add(Box.createRigidArea(new Dimension(0, 8)));
428     card.add(sub);
429     card.add(Box.createRigidArea(new Dimension(0, 28)));
430     card.add(buttonRow);
431
432     dialog.setContentPane(card);
433     dialog.setBackground(new Color(0, 0, 0, 0));
434     dialog.setVisible(true);
435 }
436
437 // returns {nextSize, nextLevel}, or null if there is no next board
438 private int[] getNextSizeLevel(int size, int level) {
439     int max = MAX_LEVELS.getOrDefault(size, 1);
440
441     // still levels left in the current size
442     if (level < max) {
443         return new int[]{size, level + 1};
444     }
445
446     // finished this size, advance to the next size in order
447     List<Integer> sizeOrder = new ArrayList<>(MAX_LEVELS.keySet());
448     int idx = sizeOrder.indexOf(size);
449     if (idx >= 0 && idx < sizeOrder.size() - 1) {
450         int nextSize = sizeOrder.get(idx + 1);
451         return new int[]{nextSize, 1};
452     }
453
454     // was the very last size at its last level, nothing left
455     return null;
456 }
457
458 // --- Board Panel ---
459 private class BoardPanel extends JPanel {
460
461     private Color[][] errorHighlight;
462
463     Image queenImageBlack;
464     Image queenImageWhite;
465     Image queenImageRed;
466
467     private Board board;
468     private Color[][] colorGrid;
469     private boolean[][] queens;
470     private int size;
471
472     private int[] hintCorrectCell = null;
473     private int[] hintWrongCell = null;
474
475     public BoardPanel() {
476         try {
477             queenImageBlack = ImageIO.read(new File("assets/queen_black.png"));
478             queenImageWhite = ImageIO.read(new File("assets/queen_white.png"));
479             queenImageRed = ImageIO.read(new File("assets/queen_red.png"));
480             System.out.println("Queen_image_loaded:_" + queenImageBlack);
481         } catch (Exception e) {
482             System.out.println("Queen_image_failed:_" + e.getMessage());
483             queenImageBlack = null;
484             queenImageWhite = null;
485             queenImageRed = null;
486         }
487     }
488
489     public void setBoard(Board board, boolean[][] queens) {
490         this.board = board;
491         this.size = board.getSize();
492         this.queens = queens;
493         buildColorGrid();
494
495         // remove old listeners first
496         for (MouseListener ml : getMouseListeners()) {
497             removeMouseListener(ml);
498         }
499
500         addMouseListener(new MouseAdapter() {
501             @Override
502             public void mouseClicked(MouseEvent e) {
503
504                 if (getWidth() == 0 || getHeight() == 0) return;

```

```

505
506         int cellSize = Math.min(getWidth(), getHeight()) / size;
507         int totalSize = cellSize * size;
508         int xOffset = (getWidth() - totalSize) / 2;
509         int yOffset = (getHeight() - totalSize) / 2;
510
511         int col = (e.getX() - xOffset) / cellSize;
512         int row = (e.getY() - yOffset) / cellSize;
513
514         if (row < 0 || row >= size || col < 0 || col >= size) return;
515
516         // toggle queen
517         queens[row][col] = !queens[row][col];
518
519         // start timer on first placement
520         if (!timerStarted && queens[row][col]) {
521             timerStarted = true;
522             startTime = System.currentTimeMillis();
523             gameTimer.start();
524         }
525
526         if (hintCorrectCell != null && hintCorrectCell[0] == row && hintCorrectCell[1] == col) {
527             clearHint();
528         } else if (hintWrongCell != null && hintWrongCell[0] == row && hintWrongCell[1] == col) {
529             clearHint();
530         }
531
532         repaint();
533         checkViolations();
534     }
535 }
536
537 }
538
539 public void setQueens(boolean[][] queens) {
540     this.queens = queens;
541 }
542
543 private void buildColorGrid() {
544     colorGrid = new Color[size][size];
545     for (Map.Entry<String, List<int>>> entry : board.getColorMap().entrySet()) {
546         String[] rgb = entry.getKey().split(",");
547         Color color = new Color(
548             Integer.parseInt(rgb[0].trim()),
549             Integer.parseInt(rgb[1].trim()),
550             Integer.parseInt(rgb[2].trim())
551         );
552         for (int[] cell : entry.getValue()) {
553             colorGrid[cell[0]][cell[1]] = color;
554         }
555     }
556 }
557
558 @Override
559 protected void paintComponent(Graphics g) {
560     super.paintComponent(g);
561     if (board == null) return;
562
563     Graphics2D g2 = (Graphics2D) g;
564     g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
565
566     int cellSize = Math.min(getWidth(), getHeight()) / size;
567     int totalSize = cellSize * size;
568
569     // center the board
570     int xOffset = (getWidth() - totalSize) / 2;
571     int yOffset = (getHeight() - totalSize) / 2;
572
573     for (int row = 0; row < size; row++) {
574         for (int col = 0; col < size; col++) {
575             int x = xOffset + col * cellSize;
576             int y = yOffset + row * cellSize;
577
578             // draw cell color
579             Color color = colorGrid[row][col];
580             if (color != null) {
581                 g2.setColor(color);
582                 g2.fillRect(x, y, cellSize, cellSize);
583             }
584
585             // draw grid lines
586             g2.setColor(Color.BLACK);
587             g2.drawRect(x, y, cellSize, cellSize);
588
589             // draw queen
590             if (queens != null && queens[row][col]) {
591                 boolean isError = errorHighlight != null && errorHighlight[row][col] != null;
592                 Image img;
593                 if (isError) {
594                     img = queenImageRed;
595                 } else {
596                     img = isDark(colorGrid[row][col]) ? queenImageWhite : queenImageBlack;
597                 }
598                 if (img != null) {
599                     int padding = cellSize / 6;
600                     g2.drawImage(img, x + padding, y + padding,
601                         cellSize - padding * 2, cellSize - padding * 2, null);
602                 }
603             }

```

```

604
605         if (hintCorrectCell != null && hintCorrectCell[0] == row && hintCorrectCell[1] == col) {
606             g2.setColor(new Color(0, 255, 0, 120));
607             g2.fillRect(x, y, cellSize, cellSize);
608             g2.setColor(Color.GREEN.darker());
609             g2.setStroke(new BasicStroke(3));
610             g2.drawRect(x + 1, y + 1, cellSize - 2, cellSize - 2);
611             g2.setStroke(new BasicStroke(1));
612         }
613         if (hintWrongCell != null && hintWrongCell[0] == row && hintWrongCell[1] == col) {
614             g2.setColor(new Color(255, 165, 0, 120));
615             g2.fillRect(x, y, cellSize, cellSize);
616             g2.setColor(Color.ORANGE.darker());
617             g2.setStroke(new BasicStroke(3));
618             g2.drawRect(x + 1, y + 1, cellSize - 2, cellSize - 2);
619             g2.setStroke(new BasicStroke(1));
620         }
621     }
622 }
623
624
625 }
626
627 public void checkViolations() {
628     errorHighlight = new Color[currentSize][currentSize];
629
630     // collect all placed queens
631     List<int[]> placed = new ArrayList<>();
632     for (int r = 0; r < currentSize; r++) {
633         for (int c = 0; c < currentSize; c++) {
634             if (queenPlaced[r][c]) placed.add(new int[]{r, c});
635         }
636     }
637
638     // check each pair for conflicts
639     for (int i = 0; i < placed.size(); i++) {
640         for (int j = i + 1; j < placed.size(); j++) {
641             int r1 = placed.get(i)[0], c1 = placed.get(i)[1];
642             int r2 = placed.get(j)[0], c2 = placed.get(j)[1];
643
644             Color color1 = colorGrid[r1][c1];
645             Color color2 = colorGrid[r2][c2];
646             boolean sameColor = color1 != null && color1.equals(color2);
647
648             boolean conflict = (r1 == r2 || c1 == c2 ||
649                 (Math.abs(r1 - r2) <= 1 && Math.abs(c1 - c2) <= 1) ||
650                 sameColor);
651
652             if (conflict) {
653                 errorHighlight[r1][c1] = new Color(255, 0, 0, 100);
654                 errorHighlight[r2][c2] = new Color(255, 0, 0, 100);
655             }
656         }
657     }
658
659     boardPanel.setErrorHighlight(errorHighlight);
660
661     // check win condition
662     if (solution != null && placed.size() == currentSize) {
663         boolean matchesSolution = true;
664         for (int[] pos : solution) {
665             if (!queenPlaced[pos[0]][pos[1]]) {
666                 matchesSolution = false;
667                 break;
668             }
669         }
670         if (matchesSolution) {
671             gameTimer.stop();
672             SwingUtilities.invokeLater(GameWindow.this::showWinDialog);
673         }
674     }
675 }
676
677 public void setErrorHighlight(Color[][] errorHighlight) {
678     this.errorHighlight = errorHighlight;
679 }
680
681 private boolean isDark(Color c) {
682     // standard luminance formula
683     double luminance = (0.299 * c.getRed() + 0.587 * c.getGreen() + 0.114 * c.getBlue()) / 255;
684     return luminance < 0.5;
685 }
686
687 @Override
688 public Dimension getPreferredSize() {
689     int available = Math.min(
690         GameWindow.this.getHeight() - 150, // subtract top and bottom panels
691         GameWindow.this.getWidth() - 300 // subtract legend space
692     );
693     return new Dimension(available, available);
694 }
695
696
697 public void setHint(int[] correctCell, int[] wrongCell) {
698     this.hintCorrectCell = correctCell;
699     this.hintWrongCell = wrongCell;
700 }
701
702 public void clearHint() {

```

```
703|         this.hintCorrectCell = null;  
704|         this.hintWrongCell = null;  
705|     }  
706|  
707| }  
708|  
709|  
710| }
```